pyXRT: code reference

Functions, arguments, hypotheses and the physics behind them

Samuel Tardif

CRG-IF BM32, European Synchrotron Radiation Facility, Grenoble, France

Listings taken from commit 69b8de1 (2026-10-21) – document compiled October 21, 2026

Abstract

pyXRT gathers the Python tools used on the CRG-IF BM32 beamline of the ESRF to read BLISS data files, reduce series of Laue diffraction images, and analyse X-ray fluorescence (XRF) spectra recorded under white-beam (Laue) or monochromatic excitation. This document is the reference of every module of the repository: for each class and function it states what the code does, the arguments it takes and what it returns, the physical model and the hypotheses it rests on, and the scientific literature behind it, and it shows the code itself next to the explanation, with the line numbers of the source file. The fluorescence modules form a complete fundamental-parameter chain—incident spectrum of a bending magnet through mirrors and filters, flux-weighted fluorescence cross-sections, matrix absorption and secondary fluorescence in layered samples, a constrained spectrum model with detector response, escape and pile-up peaks and scatter peaks, a linear per-pixel map solve, and the energy calibration—and are described in the most detail. The older scripts (SPEC reader, phase-retarder tools, webcam Fourier transform, first fluorescence fitter) are documented too, with their limits, including those that keep some of them from running on current Python installations.

Contents

1 Introduction **6**
- 1.1 What the repository contains . . . 6
- 1.2 Dependencies . . . 6
- 1.3 Conventions . . . 7
- 1.4 How to read the function entries . . . 8

2 `bliss_hdf5_reader`: reading BLISS scans **8**
- 2.1 Overview . . . 8
- 2.2 Module header and helpers . . . 9
  - 2.2.1 `_open` . . . 10
  - 2.2.2 `_to_str`, `_read_scalar`, `_is_number` . . . 10
  - 2.2.3 `_parse_key` . . . 11
  - 2.2.4 `list_scans` . . . 11
  - 2.2.5 `ScanMotor` . . . 11
  - 2.2.6 `FrameSource` . . . 12
- 2.3 Title parsers . . . 12
  - 2.3.1 `_parse_step_scan` . . . 12
  - 2.3.2 `_parse_mesh` . . . 13
  - 2.3.3 `_parse_fscan_nd` . . . 13
  - 2.3.4 `_parse_fnscan` . . . 13
  - 2.3.5 `_parse_time_scan`, `_parse_single_point` . . . 14
  - 2.3.6 `_parse_title` . . . 14
- 2.4 `BlissScan` . . . 15

2.4.1 `BlissScan` . . . 15
2.4.2 `BlissScan._read_metadata` . . . 16
2.4.3 `BlissScan._find_positioners` . . . 17
2.4.4 `BlissScan._count_acquired_points` . . . 17
2.4.5 `BlissScan._infer_geometry` . . . 18
2.4.6 `BlissScan._row_length` . . . 18
2.4.7 `BlissScan._infer_mesh` . . . 19
2.4.8 `BlissScan._detect_snake` . . . 19
2.4.9 Properties: `ndim`, `is_complete`, `motor_names`, `positioner_names`; `motors_of_dim` 20
2.4.10 `BlissScan.index_grid` . . . 20
2.4.11 `get_available_counters`, `get`, `__getitem__`, `positions` . . . 21
2.4.12 `BlissScan.to_grid` . . . 21
2.4.13 `BlissScan.read_data` . . . 22
2.5 Detector frames . . . 23
2.5.1 `image_detectors`, `spectrum_detectors` ; `_pick_detector` . . . 23
2.5.2 `BlissScan.frame_sources` . . . 23
2.5.3 `read_frame`, `iter_frames` . . . 24
2.5.4 `BlissScan._process_images` . . . 24
2.6 Other methods, the mesh subclass and the command line . . . 25
2.6.1 `positioners_start`, `get_relative_position_grid`, `summary`, `get_summary`, `__repr__` . . . 25
2.6.2 `BlissMeshScan` . . . 26
2.6.3 `main` . . . 26

3 `xrf_beamline`: the incident spectrum **27**
3.1 Overview . . . 27
3.2 Source . . . 29
3.2.1 `BendingMagnetSource` . . . 29
3.2.2 `critical_energy_kev`, `gamma` ; `_g1` . . . 30
3.2.3 `spectral_flux` . . . 31
3.2.4 `angular_flux` . . . 31
3.2.5 `accepted_flux` . . . 31
3.3 Optical elements . . . 32
3.3.1 `Mirror` . . . 32
3.3.2 `focal_length_m`, `aperture_m` . . . 33
3.3.3 `Mirror.transfer`, `Mirror._reflectivity` . . . 33
3.3.4 `Filter`; `transfer` . . . 34
3.3.5 `Slit`; `transfer` . . . 35
3.4 Monochromatic sources and units . . . 36
3.4.1 `tube_lines` . . . 36
3.4.2 `monochromatic_spectrum` . . . 36
3.4.3 `per_unit_energy` . . . 37
3.5 Air . . . 37
3.5.1 `AIR_FORMULA`, `air_density` . . . 37
3.5.2 `air_path` . . . 38
3.6 The assembled beamline . . . 39
3.6.1 `Beamline` . . . 39
3.6.2 `_positioned`, `trace` . . . 40
3.6.3 `acceptance` . . . 40
3.6.4 `effective_vertical_acceptance` . . . 41
3.6.5 `resolved_optics` . . . 41
3.6.6 `describe` . . . 42

3.6.7 `spectrum`, `spectral_density`, `band_limits` . . . 42
3.7 BM32 . . . 43
3.7.1 `BM32_LAYOUT`, `BM32_WHITE_BEAM` . . . 43
3.7.2 `bm32_from_h5` . . . 45

4 `xrf_matrix`: matrix effects in layered samples **46**
4.1 Overview and physics . . . 46
4.2 Cached cross-sections . . . 49
4.2.1 `_grid_key`, `_cs_total`, `_cs_fluor`, `_cs_fluor_scalar` . . . 49
4.3 Geometry and sample . . . 49
4.3.1 `Geometry` . . . 49
4.3.2 `Layer` . . . 50
4.3.3 `Stack` . . . 51
4.4 Depth integrals . . . 52
4.4.1 `_F`, `_dF` . . . 52
4.4.2 `_G` . . . 53
4.4.3 `_D` . . . 53
4.5 `MatrixModel` . . . 53
4.5.1 `_Exciter` . . . 53
4.5.2 `MatrixModel` . . . 53
4.5.3 `layer`, `with_layer`, `with_stack` . . . 55
4.5.4 `exciters` . . . 56
4.5.5 `_k`, `_b`, `_tout` . . . 56
4.5.6 `primary` . . . 57
4.5.7 `secondary` . . . 57
4.5.8 `sensitivity`, `emission`, `enhancement`, `absorption_factor` . . . 58
4.6 Quantification . . . 59
4.6.1 `_line` . . . 59
4.6.2 `quantify_layer` . . . 59
4.6.3 `atom_ratio` . . . 61

5 `xrf_fit`: the spectrum model and the fits **62**
5.1 Overview: the constrained model . . . 62
5.2 Energy calibration . . . 65
5.2.1 `Calibration` . . . 65
5.3 The detector . . . 66
5.3.1 `Detector` . . . 66
5.3.2 `pair_energy_ev` . . . 68
5.3.3 `Detector.efficiency` . . . 68
5.3.4 `Detector.escape_lines` . . . 69
5.3.5 `sigma_ev`, `fwhm_ev`, `from_fwhm` . . . 70
5.4 Fluorescence lines and line groups . . . 70
5.4.1 `Line`, `LineGroup` . . . 70
5.4.2 `_family_of`, `_all_line_macros`, `_is_iupac_line` . . . 71
5.4.3 `_line_energy_ev` . . . 72
5.4.4 `_line_strength` . . . 72
5.4.5 `_line_strength_spectral` . . . 73
5.4.6 `element_line_groups` . . . 74
5.4.7 `_merge_close_lines`, `_collapse` . . . 77
5.5 Normalisation, averaging and counting statistics . . . 77
5.5.1 `normalise_spectra` . . . 77
5.5.2 `_pixel_rows` . . . 78
5.5.3 `average_spectrum` . . . 79

5.5.4 `counting_error` . . . 79
5.6 Pile-up . . . 80
5.6.1 `pileup_spectra` . . . 80
5.6.2 `average_pileup_template` . . . 82
5.7 Background estimate . . . 82
5.7.1 `snip_background` . . . 82
5.8 Line shape . . . 83
5.8.1 `peak_shape` . . . 83
5.9 Scatter peaks under monochromatic excitation . . . 84
5.9.1 `_angle_grid` . . . 84
5.9.2 `ScatterSource` . . . 85
5.10 `XRFModel` . . . 86
5.10.1 `XRFModel` . . . 86
5.10.2 `_unpack`, `calibration_of`, `air_path_of`, `pileup_tau_us`, `pileup_profile`, `tails_of` . . . 90
5.10.3 `background`, `_shape` . . . 91
5.10.4 `group_profile` . . . 91
5.10.5 `scatter_profile`, `_compton_kernel`, `_compton_ia` . . . 92
5.10.6 `__call__` . . . 93
5.10.7 `initial_guess` . . . 94
5.11 Fitting a spectrum . . . 95
5.11.1 `FitResult` . . . 95
5.11.2 `fit_average_spectrum` . . . 96
5.12 Maps: the per-pixel linear solve . . . 98
5.12.1 `MapResult` . . . 98
5.12.2 `number_ratio` . . . 100
5.12.3 `design_matrix` . . . 101
5.12.4 `fit_map` . . . 102
5.13 Resolution stability during a scan . . . 105
5.13.1 `width_window` . . . 105
5.13.2 `peak_width_profile` . . . 106
5.13.3 `detect_steps` . . . 106
5.13.4 `segment_labels`, `fit_segments` . . . 107
5.14 Diagnostics and plots . . . 108
5.14.1 `identify_peaks` . . . 108
5.14.2 `pixel_parameters`, `select_pixels` . . . 109
5.14.3 `plot_pixel_fits` . . . 110
5.14.4 `plot_fit` . . . 112

6 `xrf_calibration`: energy calibration 113
6.1 Overview . . . 113
6.2 Reference samples and results . . . 114
6.2.1 `Calibrant`; `elements`, `atom_ratio` . . . 114
6.2.2 `CALIBRANTS`, `LineCheck` . . . 115
6.2.3 `CalibrationResult` . . . 116
6.3 Procedure . . . 118
6.3.1 `coarse_calibration` . . . 118
6.3.2 `calibration_from_peaks` . . . 119
6.3.3 `_line_checks` . . . 120
6.3.4 `_intensity_model` . . . 121
6.3.5 `calibrate` . . . 121
6.3.6 `plot_calibration` . . . 123

7 `xrf_region`: surface region of a map **124**
7.1 Overview . . . 124
7.2 Polygons and coordinates . . . 125
7.2.1 `polygon_area`, `points_in_polygon` . . . 125
7.2.2 `scan_coordinates` . . . 126
7.3 The region . . . 127
7.3.1 `SurfaceRegion` . . . 127
7.3.2 constructors . . . 128
7.3.3 `mask`, `scan_mask` . . . 128
7.3.4 `plot` . . . 129
7.3.5 input/output . . . 130
7.4 Drawing a region . . . 130
7.4.1 `map_image` . . . 130
7.4.2 `RegionSelector` . . . 131

8 Diffraction-image scripts **134**
8.1 `reduce_image_series`: minimum, maximum and mean images . . . 134
8.1.1 `parse_args` . . . 135
8.1.2 `_read_frame_array` . . . 136
8.1.3 `_worker_process_files` . . . 136
8.1.4 `compute_scan_reductions` . . . 137
8.1.5 `save_reduced_hdf5`, `main` . . . 138
8.2 `make_dynamic_mosaic`: region-of-interest maps . . . 139
8.2.1 `parse_args` . . . 140
8.2.2 `read_image`, `roi_slices`, `get_image_roi` . . . 140
8.2.3 `_init_worker`, `_read_roi_batch`, `read_rois` . . . 141
8.2.4 `fill_missing_positions`, `to_scan_grid` . . . 141
8.2.5 `compute_pipeline_data` . . . 142
8.2.6 `save_metrics_to_hdf5`, `plot_and_save_figure` . . . 143
8.2.7 `main` . . . 144

9 Older and superseded tools **146**
9.1 `fluo_map_fit`: the first fluorescence map fitter . . . 146
9.1.1 `coupled_double_gaussian`, `spectrum_model` . . . 147
9.1.2 `fit_single_spectrum`, `process_and_plot` . . . 147
9.2 `spec_reader`: SPEC data files . . . 149
9.2.1 `SpecFile` . . . 149
9.2.2 Line handlers . . . 151
9.2.3 `Scan` . . . 153
9.3 `xray_tools`: wavelengths, Bragg angles and flipping ratios . . . 156
9.3.1 `E2L`, `L2E` . . . 157
9.3.2 `tth` . . . 157
9.3.3 `uncertainty_fr` . . . 157
9.3.4 `xprplot_th` . . . 158
9.3.5 `XprScan`, `XprScan.fr` . . . 159
9.4 `LiveFourierTransform`: webcam Fourier transform . . . 160

10 Notebooks **162**

References **165**

1 Introduction

1.1 What the repository contains

The repository is a flat collection of Python modules (no package structure: each module is imported by its file name from the repository folder) and of Jupyter notebooks that chain them into analyses. Table 1 lists the modules, their size, their role and what they import from each other; Fig. 1 shows the same dependencies as a graph.

Table 1: Modules of pyXRT, in the order of this document. "Lines" counts every line of the file, docstrings included.

Module	Lines	Role	Uses
`bliss_hdf5_reader`	852	Reads any scan of a BLISS HDF5 master file: metadata, scan geometry, counters, MCA spectra, detector frames.	–
`xrf_beamline`	820	Incident spectrum of a bending magnet through mirrors, windows, slits and air; BM32 model; monochromatic sources.	–
`xrf_matrix`	703	Primary and secondary fluorescence in a stack of layers; self-consistent quantification of a layer.	`xrf_beamline`
`xrf_fit`	2494	Spectrum model (line groups, detector response, escape, pile-up, scatter), fits of single spectra and of maps, diagnostics.	`xrf_beamline`, `xrf_matrix`
`xrf_calibration`	559	Energy calibration on reference samples (AXO thin films) or on known peaks.	`xrf_fit`, `xrf_beamline`
`xrf_region`	492	Surface region of a map (facet of a crystal): polygons, pixel masks, interactive drawing in Jupyter.	–
`reduce_image_series`	266	Pixel-wise minimum, maximum and mean of the detector frames of a scan (command line).	`bliss_hdf5_reader`
`make_dynamic_mosaic`	358	Region of interest of every frame of a mesh: mosaic, entropy, peak-to-mean and centre-of-mass maps (command line).	`bliss_hdf5_reader`
`fluo_map_fit`	169	First per-pixel fluorescence fitter (two constrained doublets); superseded by `xrf_fit`.	–
`spec_reader`	383	Reader of SPEC data files (2016).	–
`xray_tools`	155	Wavelength/energy, Bragg angle, flipping ratio of X-ray phase retarder scans (SPEC era).	`spec_reader`
`LiveFourierTransform`	154	Live Fourier transform of a webcam image (2011, Python 2).	–

The physics of the fluorescence chain is explained, with figures from real BM32 data, in the technical note `docs/xrf_quantification/xrf_quantification.pdf`; this document is its companion on the code side and repeats the equations where a function implements them.

1.2 Dependencies

`numpy, scipy` arrays, special functions (Bessel K_ν, erfc, erfcx, Voigt), quadrature, and the bounded non-linear least-squares solver of `scipy.optimize.curve_fit` (trust-region reflective algorithm [1, 2]).

`xraylib` atomic and X-ray data [3, 4]: line energies, radiative rates, fluorescence yields, partial photo-ionisation cross-sections, the "Kissel" partial fluorescence cross-sections that include Coster–Kronig transfer, attenuation coefficients of elements and compounds, refractive indices, and Hartree–Fock Compton profiles [5]. Tested with xraylib 4.3.

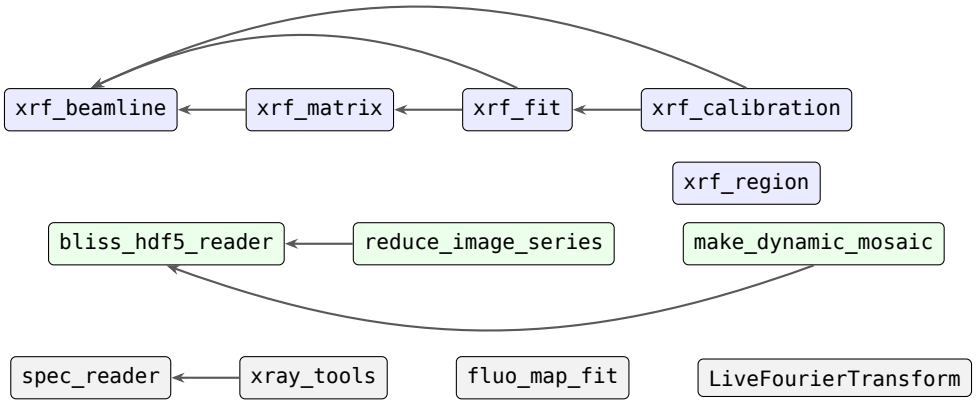


Figure 1: Imports between the modules (an arrow points to the module that is imported). Blue: fluorescence analysis; green: BLISS data and diffraction images; grey: older or superseded tools. The notebooks (Section 10) sit on top of the blue modules and of `bliss_hdf5_reader`.

`h5py` HDF5 files [6], including virtual datasets.

`matplotlib` plots (imported inside the plotting functions only).

`fabio, tqdm` TIFF images and progress bars in the image scripts.

The analysis environment used on the beamline runs Python 3.12, NumPy 2.5, SciPy 1.18, xraylib 4.3 and h5py 3.16.

1.3 Conventions

Units. The units are part of the argument names wherever a quantity has one (`energy_ev`, `thickness_um`, `air_path_cm`, `grazing_angle_mrad` …). The fluorescence fit works in **eV** (line energies, widths, calibration), while xraylib and the beam and matrix modules work in **keV**; conversions are explicit in the code. Mass attenuation coefficients and cross-sections are in cm^2/g, densities in g/cm^3, mass thicknesses in g/cm^2. Angles of the sample geometry are in degrees from the sample *surface*; beamline angles are in mrad (mirrors) or µrad (source acceptance).

Spectral flux. Synchrotron fluxes are given, as is customary, per 0.1 % bandwidth: $N_{0.1\%}(E)$ photons in the energy interval $\mathrm{d}E = 10^{-3}E$. Integrals over energy need the density per unit energy,

$$\frac{\mathrm{d}N}{\mathrm{d}E} = \frac{N_{0.1\%}(E)}{10^{-3}\,E}, \tag{1}$$

which `xrf_beamline.per_unit_energy` implements and every function that integrates over the incident spectrum applies internally. Forgetting this Jacobian over-weights high energies by a factor E.

Channels and calibration. MCA channels are numbered from 0. The energy calibration is linear, $E = G\,c + O$ with gain G (eV/channel) and offset O (eV). Channel ranges are half-open, $[c_{\mathrm{lo}}, c_{\mathrm{hi}})$, as Python slices.

Uncertainties. All quoted errors are one standard deviation. Fit uncertainties come from the covariance matrix of the linearised least-squares problem (Section 5.11).

Line names. Fluorescence lines are named with IUPAC notation (initial vacancy, final vacancy): `KL3` is Kα_1, `L3M5` is Lα_1, `L2M4` Lβ_1, `L2N4` Lγ_1, `M5N7` Mα_1. Table 4 in Section 6 gives the correspondence used for labels.

1.4 How to read the function entries

Each documented class or function has an entry with, in this order: what it does; its arguments (name, type, default and meaning) and what it returns; the method, with the equations it evaluates; its hypotheses and limits (blue boxes); known problems (red boxes); the references; and the code. The listings are copied from the source files by `make_listings.py`, so the numbers in their margin are the line numbers of the file at commit 69b8de1 (2026-10-21). Methods are shown without their class indentation. A class listing contains the class header, its docstring and its fields; each method follows separately. Private helpers (names starting with an underscore) are documented like public functions, since they hold much of the physics.

2 `bliss_hdf5_reader`: reading BLISS scans

2.1 Overview

BLISS, the ESRF beamline control system, writes one HDF5 *master file* per dataset, organised along NeXus conventions [7]. Every scan is an entry named `"<number>.<subscan>"` (`"12.1"`) containing:

- `title`: the command as typed (`"amesh xech 0 1 50 yech 0 1 40 0.1"`), `start_time`, `end_time`, `end_reason` (`SUCCESS`, `USER_ABORT`, …);
- `measurement/`: one dataset per recorded quantity, one entry per scan point along the first axis (scalar counters and motor positions: shape (n); MCA spectra: (n, n_{ch}); detector images: (n, h, w)). These are links to the `instrument/` group;
- `instrument/positioners_start`: every motor position at the start of the scan; `instrument/<name>` groups with an `NX_class` attribute (`NXpositioner` for motors);
- `scan_parameters/`: `npoints`, `count_time` and other parameters, when recorded;
- `writer/status`, `sample/name`.

Detector images are written by Lima in separate files of a `scanNNNN/` folder next to the master file; the master file exposes them as an HDF5 *virtual dataset*, which maps ranges of the virtual array onto datasets of the source files.

The module turns this into a `BlissScan` object that knows the *geometry* of the scan (its shape in acquisition order, the motor driving each dimension, back-and-forth rows, aborted points) and reads the data on demand. The geometry comes from parsing the command in the title when it is recognised (Table 2), and otherwise from the recorded motor positions, so that scans run through user macros with custom titles are still read correctly.

Table 2: Scan commands recognised in titles, and the resulting shape (slowest axis first).

Command	Title grammar	Shape
`ascan dscan a2scan d3scan lineup`	(motor start stop) $\times N$, intervals, count time	$(\text{intervals}+1,)$
`amesh dmesh`	fast motor start stop intervals, slow motor start stop intervals, time (the *first* motor is the fast one)	$(n_{\mathrm{slow}}, n_{\mathrm{fast}})$
`fscan fscan2d fscan3d`	(motor start step npoints) $\times d$, acquisition time [period] (the *last* motor is the fast one)	$(n_0, \ldots, n_{d-1})$
`f2scan f3scan …`	(motor start step) $\times N$, npoints, time	$(n,)$
`loopscan`	npoints, time	$(n,)$
`timescan ftimescan`	time [npoints] (0 or absent: until stopped)	$(n,)$
`ct sct`	[time]	$()$

Hypotheses and limits

- Data are stored in acquisition order, one entry per point along the first axis. Meshes are row-major with the slow motor first.
- The number of recorded points is the most common first-axis length of the measurement datasets (fast scans add "`*_raw`" channels with $2n+1$ entries, which are ignored).
- A mesh scanned back and forth (snake) is recognised when the second row of the fast motor matches the reversed first row better than the first row itself.
- An aborted 1D scan keeps its recorded length; an aborted mesh keeps its planned shape and missing points are NaN in gridded data.
- Frames are read through the master file's virtual dataset; when there is none, the image files are found by listing the scan folder (top level only).

2.2 Module header and helpers

bliss_hdf5_reader.py, lines 1–62

```
"""
Module: bliss_hdf5_reader.py

Read any scan from a BLISS HDF5 master file (ESRF).

    from bliss_hdf5_reader import BlissScan, list_scans

    list_scans("sample_0001.h5")                # what is in the file
    scan = BlissScan("sample_0001.h5", 12)      # any scan type
    print(scan.summary())
    mon  = scan.get("mon")                      # raw 1D counter (one value per point)
    img  = scan.get("mon", grid=True)           # same, shaped like the scan
    pos  = scan.positions("xech", grid=True)    # measured motor positions
    frm  = scan.read_frame(0)                   # first detector image

Scan geometry
-------------
``scan.shape`` follows the order of acquisition, slowest axis first:

    ct / sct / single point        ()
    ascan, a2scan, dscan, fscan,   (n,)
    loopscan, timescan, ...
    amesh / dmesh / fscan2d        (n_slow, n_fast)
    fscan3d                        (n_slowest, n_mid, n_fast)

The geometry is taken from the scan title when the command is recognised, and
otherwise inferred from the measured motor positions, so scans with a custom
title are still read correctly. ``scan.motors`` lists the scanned motors with
the dimension (``dim``) each one drives; several motors can share a dimension
(a2scan, a3scan, ...).

Aborted scans: ``scan.npoints`` is the number of points actually recorded and
``scan.npoints_planned`` the requested one. For 1D scans the shape is
``(npoints,)``; for meshes it stays the planned grid and missing points are
NaN in gridded data. Meshes acquired back-and-forth (snake) are detected from
the fast-motor positions and un-snaked in gridded data.

``BlissMeshScan`` is kept for existing scripts: a ``BlissScan`` that must be
two-dimensional and exposes the former ``fast_*`` / ``slow_*`` attributes.
"""

from __future__ import annotations

import logging
import re
from collections import Counter
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Dict, Iterator, List, Optional, Sequence, Tuple, Union

import h5py
import numpy as np
```

```
53
54 logger = logging.getLogger(__name__)
55
56 PathLike = Union[str, Path]
57
58 # Datasets that are written as positioners but are time axes, not motors.
59 TIME_AXES = ("elapsed_time", "epoch", "epoch_trig", "timer_trig")
60 # Commands without a motor: one dimension indexed by time / point number.
61 TIME_SCANS = ("loopscan", "timescan", "ftimescan", "fscanloop", "limatake")
62 SINGLE_POINT_SCANS = ("ct", "sct")
```

The constants list the positioner names that are time axes, not motors (`TIME_AXES`), the commands without a motor (`TIME_SCANS`) and the single-point commands.

2.2.1 `_open`

`_open(path)`

Opens a file read-only with HDF5 file locking disabled (`locking=False`), so that a file that BLISS is still writing can be read at the beamline; falls back to the default for h5py older than 3.5, which has no `locking` argument. Returns an `h5py.File`, to be used as a context manager.

bliss_hdf5_reader.py, lines 65–74

```
65 # ---------------------------------------------------------------------------- #
66 # helpers
67 # ---------------------------------------------------------------------------- #
68 def _open(path: PathLike) -> h5py.File:
69     """Open read-only without HDF5 file locking, so files that BLISS is still
70     writing can be read at the beamline."""
71     try:
72         return h5py.File(path, "r", locking=False)
73     except TypeError:  # h5py < 3.5 has no `locking` argument
74         return h5py.File(path, "r")
```

2.2.2 `_to_str`, `_read_scalar`, `_is_number`

`_to_str(value)`, `_read_scalar(group, name)`, `_is_number(token)`

`_to_str` decodes bytes (UTF-8, with replacement of invalid sequences) and zero-dimensional arrays to a Python string. `_read_scalar` returns `group[name]` as a Python scalar or string, or `None` if the dataset is missing. `_is_number` tells whether a title token parses as a float.

bliss_hdf5_reader.py, lines 77–82

```
77 def _to_str(value: Any) -> str:
78     if isinstance(value, bytes):
79         return value.decode("utf-8", errors="replace")
80     if isinstance(value, np.ndarray) and value.shape == ():
81         return _to_str(value[()])
82     return str(value)
```

bliss_hdf5_reader.py, lines 85–93

```
85 def _read_scalar(group: h5py.Group, name: str) -> Any:
86     if name not in group:
87         return None
88     value = group[name][()]
89     if isinstance(value, (bytes, str)):
90         return _to_str(value)
91     if isinstance(value, np.generic):
92         return value.item()
93     return value
```

bliss_hdf5_reader.py, lines 96–101

```
96 def _is_number(token: str) -> bool:
97     try:
98         float(token)
```

```
99          return True
100     except ValueError:
101         return False
```

2.2.3 `_parse_key`

```
_parse_key(scan, subscan)
```

Accepts a scan given as an integer (`12`), a string (`"12"`) or a full entry key (`"12.2"`). Returns `(number, subscan, key)`; the subscan of a full key takes precedence over the `subscan` argument.

bliss_hdf5_reader.py, lines 104–109

```
104 def _parse_key(scan: Union[int, str], subscan: int) -> Tuple[int, int, str]:
105     """Accept 12, "12" or "12.1"."""
106     if isinstance(scan, str) and "." in scan:
107         number, sub = scan.split(".", 1)
108         return int(number), int(sub), scan
109     return int(scan), subscan, f"{int(scan)}.{subscan}"
```

2.2.4 `list_scans`

```
list_scans(h5_filepath)
```

Lists the scans of a master file without reading any data.

Returns a list of dictionaries, one per top-level group, with keys `key`, `number` (integer part of the key, or `None`), `title`, `npoints` (planned, from `scan_parameters`, if recorded), `end_reason` and `start_time`, sorted numerically by scan then subscan (so `"10.1"` comes after `"9.1"`).

bliss_hdf5_reader.py, lines 112–135

```
112 def list_scans(h5_filepath: PathLike) -> List[Dict[str, Any]]:
113     """
114     One dict per scan entry in a master file, in scan order:
115     key, number, title, npoints (planned, if recorded), end_reason, start_time.
116     """
117     rows = []
118     with _open(h5_filepath) as f:
119         for key in f:
120             group = f[key]
121             if not isinstance(group, h5py.Group):
122                 continue
123             params = group.get("scan_parameters")
124             rows.append({
125                 "key": key,
126                 "number": int(key.split(".")[0]) if key.split(".")[0].isdigit() else None,
127                 "title": _read_scalar(group, "title"),
128                 "npoints": _read_scalar(params, "npoints") if params is not None else None,
129                 "end_reason": _read_scalar(group, "end_reason"),
130                 "start_time": _read_scalar(group, "start_time"),
131             })
132     def order(row):
133         parts = row["key"].split(".")
134         return tuple(int(p) if p.isdigit() else 0 for p in parts)
135     return sorted(rows, key=order)
```

2.2.5 `ScanMotor`

A motor driven by the scan: `name`, `dim` (the scan dimension it drives, 0 = slowest), `npoints` along that dimension, `start` and `stop` as written in the title (relative to the current position for d-scans, flagged by `relative`). Its `step` property is $(\text{stop} - \text{start})/(n-1)$, or `None` when undefined. Several motors can share a dimension (`a2scan`, the slow motors of an inferred mesh).

bliss_hdf5_reader.py, lines 138–148

```python
@dataclass
class ScanMotor:
    """A motor driven by the scan. `start`/`stop` are as written in the title
    (relative to the start position for d-scans); `dim` is the scan dimension
    it moves along (0 = slowest)."""
    name: str
    dim: int
    npoints: int
    start: Optional[float] = None
    stop: Optional[float] = None
    relative: bool = False
```

bliss_hdf5_reader.py, lines 150–154

```python
    @property
    def step(self) -> Optional[float]:
        if self.start is None or self.stop is None or self.npoints < 2:
            return None
        return (self.stop - self.start) / (self.npoints - 1)
```

2.2.6 `FrameSource`

Location of one detector frame on disk: the source `file`, the internal `dataset` path, and the frame `index` in that dataset. Produced by `BlissScan.frame_sources`.

bliss_hdf5_reader.py, lines 157–162

```python
@dataclass
class FrameSource:
    """Where one detector frame lives on disk."""
    file: Path
    dataset: str
    index: int
```

2.3 Title parsers

Each parser receives the command and the list of title tokens (commas are treated as spaces) and returns `(shape, motors, count_time)`, or `None` if the tokens do not follow the expected grammar—in which case the geometry is inferred from the data.

2.3.1 `_parse_step_scan`

```
_parse_step_scan(cmd, args)
```

Step scans with N motors: $3N+2$ tokens, each triple a motor name and two numbers, then an integer number of intervals and a count time. The number of points is intervals $+1$. Commands starting with `d`, and `lineup`, are relative.

bliss_hdf5_reader.py, lines 165–181

```python
# ---------------------------------------------------------------------------- #
# title parsers: command -> (shape, motors, count_time) or None if not matched
# ---------------------------------------------------------------------------- #
def _parse_step_scan(cmd: str, args: List[str]):
    """ascan/dscan/a2scan/d3scan/lineup:  (motor start stop)*N intervals time"""
    if len(args) < 5 or (len(args) - 2) % 3:
        return None
    n_motors = (len(args) - 2) // 3
    triples = [args[3 * i: 3 * i + 3] for i in range(n_motors)]
    if not all(_is_number(s) and _is_number(e) for _, s, e in triples):
        return None
    if not (args[-2].lstrip("-").isdigit() and _is_number(args[-1])):
        return None
    npts = int(args[-2]) + 1
    relative = cmd.startswith("d") or cmd == "lineup"
    motors = [ScanMotor(m, 0, npts, float(s), float(e), relative) for m, s, e in triples]
    return (npts,), motors, float(args[-1])
```

2.3.2 `_parse_mesh`

```
_parse_mesh(cmd, args)
```

`amesh`/`dmesh`: the first motor is the fast one, but the returned shape and motor list put the slow motor first, so that every 2D scan has shape $(n_{\text{slow}}, n_{\text{fast}})$ whatever the command.

bliss_hdf5_reader.py, lines 184–198

```
184 def _parse_mesh(cmd: str, args: List[str]):
185     """amesh/dmesh: fast start stop intervals  slow start stop intervals  time.
186     The FIRST motor is the fast one."""
187     if len(args) < 9:
188         return None
189     try:
190         fast = (args[0], float(args[1]), float(args[2]), int(args[3]) + 1)
191         slow = (args[4], float(args[5]), float(args[6]), int(args[7]) + 1)
192         count_time = float(args[8])
193     except ValueError:
194         return None
195     relative = cmd == "dmesh"
196     motors = [ScanMotor(slow[0], 0, slow[3], slow[1], slow[2], relative),
197               ScanMotor(fast[0], 1, fast[3], fast[1], fast[2], relative)]
198     return (slow[3], fast[3]), motors, count_time
```

2.3.3 `_parse_fscan_nd`

```
_parse_fscan_nd(cmd, args)
```

Continuous scans `fscan`, `fscan2d`, `fscan3d`: d groups of (motor, start, step, npoints), the *last* motor being the fast one; the stop position is start + step $(n-1)$.

bliss_hdf5_reader.py, lines 201–217

```
201 def _parse_fscan_nd(cmd: str, args: List[str]):
202     """fscan / fscan2d / fscan3d:  (motor start step npoints)*N acq_time [period].
203     The LAST motor is the fast one (opposite of amesh)."""
204     ndim = {"fscan": 1, "fscan2d": 2, "fscan3d": 3}[cmd]
205     if len(args) < 4 * ndim + 1:
206         return None
207     motors, shape = [], []
208     try:
209         for d in range(ndim):
210             name, start, step, npts = args[4 * d: 4 * d + 4]
211             start, step, npts = float(start), float(step), int(npts)
212             motors.append(ScanMotor(name, d, npts, start, start + step * (npts - 1)))
213             shape.append(npts)
214         count_time = float(args[4 * ndim])
215     except ValueError:
216         return None
217     return tuple(shape), motors, count_time
```

2.3.4 `_parse_fnscan`

```
_parse_fnscan(cmd, args)
```

Continuous scans of several motors along one dimension (`f2scan`, `f3scan`): the number of motors is read from the command name.

bliss_hdf5_reader.py, lines 220–235

```
220 def _parse_fnscan(cmd: str, args: List[str]):
221     """f2scan/f3scan:  (motor start step)*N npoints acq_time [period]"""
222     n_motors = int(cmd[1:-4])
223     if len(args) < 3 * n_motors + 2:
224         return None
225     try:
226         npts = int(args[3 * n_motors])
```

```
        motors = []
        for i in range(n_motors):
            name, start, step = args[3 * i: 3 * i + 3]
            start, step = float(start), float(step)
            motors.append(ScanMotor(name, 0, npts, start, start + step * (npts - 1)))
        count_time = float(args[3 * n_motors + 1])
    except ValueError:
        return None
    return (npts,), motors, count_time
```

2.3.5 `_parse_time_scan`, `_parse_single_point`

`_parse_time_scan(cmd, args)`, `_parse_single_point(cmd, args)`

Scans without motors. A `timescan` without a number of points runs until it is stopped: its planned size is unknown (shape $(0,)$ at this stage, replaced by the recorded length). `ct`/`sct` have shape $()$.

bliss_hdf5_reader.py, lines 238–249

```
def _parse_time_scan(cmd: str, args: List[str]):
    """loopscan npoints time | timescan time [npoints] | ftimescan time npoints [period]"""
    nums = [a for a in args if _is_number(a)]
    try:
        if cmd == "loopscan" and len(nums) >= 2:
            return (int(nums[0]),), [], float(nums[1])
        if cmd in ("timescan", "ftimescan") and nums:
            npts = int(float(nums[1])) if len(nums) > 1 else 0
            return (npts,), [], float(nums[0])   # npoints 0 = run until stopped
    except ValueError:
        return None
    return None
```

bliss_hdf5_reader.py, lines 252–254

```
def _parse_single_point(cmd: str, args: List[str]):
    count_time = float(args[0]) if args and _is_number(args[0]) else None
    return (), [], count_time
```

2.3.6 `_parse_title`

`_parse_title(title)`

Dispatches on the first token with regular expressions (`[ad]\d*scan|lineup`, `f\d+scan`) and the command tables. Returns `(command, parsed)` with `parsed` `None` for unknown commands.

bliss_hdf5_reader.py, lines 257–274

```
def _parse_title(title: str):
    tokens = title.replace(",", " ").split()
    if not tokens:
        return "", None
    cmd, args = tokens[0], tokens[1:]
    if re.fullmatch(r"[ad]\d*scan|lineup", cmd):
        return cmd, _parse_step_scan(cmd, args)
    if cmd in ("amesh", "dmesh"):
        return cmd, _parse_mesh(cmd, args)
    if cmd in ("fscan", "fscan2d", "fscan3d"):
        return cmd, _parse_fscan_nd(cmd, args)
    if re.fullmatch(r"f\d+scan", cmd):
        return cmd, _parse_fnscan(cmd, args)
    if cmd in TIME_SCANS:
        return cmd, _parse_time_scan(cmd, args)
    if cmd in SINGLE_POINT_SCANS:
        return cmd, _parse_single_point(cmd, args)
    return cmd, None
```

2.4 `BlissScan`

2.4.1 `BlissScan`

```
BlissScan(h5_filepath, scan_number, subscan=1, image_prefix="eiger", image_suffix=".h5",
verbose=False)
```

One scan of a master file, of any type. Instantiation reads only metadata; data are read on demand.

`h5_filepath`
: path of the master file.

`scan_number`
: integer, or full key `"12.1"`.

`subscan`
: used when `scan_number` is an integer (default 1).

`image_prefix`
: preferred image detector: the first image dataset whose name starts with it (default `"eiger"`); `None` takes the first image detector.

`image_suffix`
: extension of image files when they have to be found by listing the scan folder.

`verbose`
: print the summary after `read_data()`.

Main attributes `title`, `command` (and its former name `scan_type`), `shape`, `motors` (list of `ScanMotor`), `npoints` (recorded), `npoints_planned`, `count_time`, `snake`, `geometry_source` (`"title"` or `"data"`), `start_time`, `end_time`, `end_reason`, `writer_status`, `sample_name`, `scan_folder`; after `read_data()`: `counters` (raw arrays), `grids` (scalar counters on the scan grid), `image_path_list`, `nb_images`, `nb_frames`.

bliss_hdf5_reader.py, lines 277–296

```
277 # ------------------------------------------------------------------------------ #
278 # main class
279 # ------------------------------------------------------------------------------ #
280 class BlissScan:
281     """
282     One scan of a BLISS HDF5 master file, of any type.
283 
284     Instantiation only reads metadata. Data are read on demand with `get()`,
285     `positions()` and `read_frame()`, or in bulk with `read_data()`.
286 
287     Args:
288         h5_filepath: BLISS master file (<dataset>.h5).
289         scan_number: scan number (12), or full entry key ("12.1").
290         subscan: sub-scan index, used when `scan_number` is an integer.
291         image_prefix: preferred image detector, matched against the start of
292             the detector name (e.g. "eiger"); None = first image detector.
293         image_suffix: file extension used when images have to be found by
294             listing the scan folder (no virtual dataset in the master file).
295         verbose: print the summary after `read_data()`.
296     """
```

bliss_hdf5_reader.py, lines 298–320

```
298     def __init__(
299         self,
300         h5_filepath: PathLike,
301         scan_number: Union[int, str],
302         subscan: int = 1,
303         image_prefix: Optional[str] = "eiger",
304         image_suffix: str = ".h5",
305         verbose: bool = False,
306     ):
```

```
307         self.h5_filepath = Path(h5_filepath)
308         self.scan_number, self.subscan, self.scan_key = _parse_key(scan_number, subscan)
309         self.image_prefix = image_prefix
310         self.image_suffix = image_suffix
311         self.verbose = verbose
312 
313         self.counters: Dict[str, np.ndarray] = {}   # raw data, as stored
314         self.grids: Dict[str, np.ndarray] = {}      # scalar counters shaped like the scan
315         self.image_path_list: List[Path] = []
316         self.nb_images: int = 0                      # image files
317         self.nb_frames: int = 0                      # detector frames
318         self.scan_folder = self.h5_filepath.parent / f"scan{self.scan_number:04d}"
319 
320         self._read_metadata()
```

2.4.2 `BlissScan._read_metadata`

```
BlissScan._read_metadata()
```

The core of the reader. It opens the file once and reads the entry's scalars, the shapes and types of every measurement dataset (not the data), the positioner names (`_find_positioners`), the number of recorded points (`_count_acquired_points`) and the positions of every positioner recorded once per point (kept in a cache for geometry inference and snake detection). It then:

1. parses the title; if that fails, infers the geometry from the motor positions;
2. takes the count time from `scan_parameters`, else from the title;
3. determines the planned number of points: the product of the title shape, overridden by `scan_parameters/npoints` if they disagree (with a warning);
4. for 0D and 1D scans, replaces the shape by what was recorded;
5. detects back-and-forth meshes.

Raises `FileNotFoundError` or `KeyError` if the file or the scan does not exist.

bliss_hdf5_reader.py, lines 323–383

```
323     def _read_metadata(self) -> None:
324         if not self.h5_filepath.exists():
325             raise FileNotFoundError(f"HDF5 file not found: {self.h5_filepath}")
326 
327         with _open(self.h5_filepath) as f:
328             if self.scan_key not in f:
329                 raise KeyError(f"Scan {self.scan_key} not found in {self.h5_filepath.name}")
330             entry = f[self.scan_key]
331             self.title: str = _read_scalar(entry, "title") or ""
332             self.start_time: Optional[str] = _read_scalar(entry, "start_time")
333             self.end_time: Optional[str] = _read_scalar(entry, "end_time")
334             self.end_reason: Optional[str] = _read_scalar(entry, "end_reason")
335             self.writer_status = (_read_scalar(entry["writer"], "status")
336                                   if "writer" in entry else None)
337             self.sample_name = (_read_scalar(entry["sample"], "name")
338                                 if "sample" in entry else None)
339 
340             params = entry.get("scan_parameters")
341             npts_param = _read_scalar(params, "npoints") if params is not None else None
342             time_param = _read_scalar(params, "count_time") if params is not None else None
343 
344             meas = entry["measurement"]
345             self._datasets = {k: (d.shape, d.dtype) for k, d in meas.items()
346                               if isinstance(d, h5py.Dataset)}
347             self._positioner_names = self._find_positioners(entry, meas)
348             self.npoints = self._count_acquired_points(npts_param)
349             self._positions_cache = {
350                 name: np.asarray(meas[name][()], dtype=float)
351                 for name in self._positioner_names
352                 if self._datasets[name][0][:1] == (self.npoints,) and len(self._datasets[name][0]) == 1
353             }
354 
355         self.command, parsed = _parse_title(self.title)
356         self.scan_type = self.command          # former attribute name
357         if parsed is not None:
```

```
358             self.shape, self.motors, title_time = parsed
359             self.geometry_source = "title"
360         else:
361             self.shape, self.motors = self._infer_geometry(npts_param)
362             title_time = None
363             self.geometry_source = "data"
364             logger.info("Scan %s: title '%s' not recognised; geometry inferred from the "
365                         "motor positions: shape %s", self.scan_key, self.title, self.shape)
366 
367         count_time = time_param if time_param is not None else title_time
368         self.count_time: Optional[float] = float(count_time) if count_time is not None else None
369         planned = int(np.prod(self.shape)) if self.shape else 1
370         if self.shape and planned == 0:          # timescan without npoints
371             planned = None
372         if self.geometry_source == "data":
373             planned = npts_param or self.npoints
374         elif npts_param and planned and npts_param != planned:
375             logger.warning("Scan %s: title gives %d points but scan_parameters %d; using the latter.",
376                            self.scan_key, planned, npts_param)
377             planned = npts_param
378         self.npoints_planned: Optional[int] = planned
379 
380         # 0D/1D: the shape is what was actually recorded.
381         if len(self.shape) <= 1:
382             self.shape = () if (self.shape == () and self.npoints <= 1) else (self.npoints,)
383         self.snake = self._detect_snake()
```

2.4.3 `BlissScan._find_positioners`

`BlissScan._find_positioners(entry, meas)` (static)

Motors among the measurement datasets: names present in `positioners_start`, and datasets whose link target is an `instrument/` group of class `NXpositioner`. The second rule catches the fast motor of continuous scans, recorded as an encoder channel. Time axes are excluded.

bliss_hdf5_reader.py, lines 385–408

```
385     @staticmethod
386     def _find_positioners(entry: h5py.Group, meas: h5py.Group) -> List[str]:
387         """Measurement datasets that are motors, time axes excluded: NXpositioner
388         entries, plus encoder channels named after a motor (the fast motor of
389         fscans is recorded as a detector channel)."""
390         snapshot = entry.get("instrument/positioners_start")
391         motor_names = set(snapshot.keys()) if isinstance(snapshot, h5py.Group) else set()
392         names = []
393         for name in meas:
394             if name in motor_names and name not in TIME_AXES:
395                 names.append(name)
396                 continue
397             link = meas.get(name, getlink=True)
398             target = getattr(link, "path", "")
399             if "/instrument/" not in target:
400                 continue
401             inst_name = target.split("/instrument/", 1)[1].split("/", 1)[0]
402             try:
403                 nx_class = _to_str(entry["instrument"][inst_name].attrs.get("NX_class", ""))
404             except KeyError:
405                 continue
406             if nx_class == "NXpositioner" and name not in TIME_AXES:
407                 names.append(name)
408         return names
```

2.4.4 `BlissScan._count_acquired_points`

`BlissScan._count_acquired_points(npts_param)`

The number of recorded points is the most common first-dimension length among the measurement datasets (ties broken towards the larger value). Lengths above the planned number of points

are discarded first, which removes the $2n+1$ "`*_raw`" channels of fast scans.

bliss_hdf5_reader.py, lines 410–420

```
410     def _count_acquired_points(self, npts_param: Optional[int]) -> int:
411         """Points actually recorded: the most common first-dimension length of the
412         measurement datasets, ignoring the 2n+1 '*_raw' channels of fast scans."""
413         lengths = [shape[0] for shape in (s for s, _ in self._datasets.values()) if len(shape) >= 1]
414         if not lengths:
415             return 0
416         if npts_param:
417             lengths = [n for n in lengths if n <= npts_param] or lengths
418         counts = Counter(lengths)
419         top = max(counts.values())
420         return max(n for n, c in counts.items() if c == top)
```

2.4.5 `BlissScan._infer_geometry`

`BlissScan._infer_geometry(npts_planned=None)`

Geometry for unrecognised titles. Motors that do not move (peak-to-peak range below 10^{-9} of their magnitude) are ignored. If the moving motors form a mesh (`_infer_mesh`), the shape is $(n_{\text{slow}}, n_{\text{fast}})$, extended to the planned number of rows for an aborted mesh; otherwise the scan is 1D and every moving motor drives dimension 0.

bliss_hdf5_reader.py, lines 422–446

```
422     def _infer_geometry(self, npts_planned: Optional[int] = None
423                         ) -> Tuple[Tuple[int, ...], List[ScanMotor]]:
424         """Geometry for titles that are not recognised: find the motors that move,
425         and check whether they form a mesh (one motor stepping once per row)."""
426         n = self.npoints
427         if n <= 1:
428             return (), []
429         moving = {k: v for k, v in self._positions_cache.items()
430                   if np.ptp(v) > 1e-9 * max(1.0, np.abs(v).max())}   # ignore rounding noise
431         if not moving:
432             return (n,), []
433 
434         mesh = self._infer_mesh(moving)
435         if mesh is not None:
436             (n_slow, n_fast), motors = mesh
437             if npts_planned and npts_planned % n_fast == 0 and npts_planned // n_fast > n_slow:
438                 n_slow = npts_planned // n_fast          # aborted: keep the planned rows
439                 for m in motors:
440                     if m.dim == 0:
441                         m.npoints = n_slow
442             return (n_slow, n_fast), motors
443 
444         # 1D: every motor that moves
445         motors = [ScanMotor(k, 0, n, float(v[0]), float(v[-1])) for k, v in moving.items()]
446         return (n,), motors
```

2.4.6 `BlissScan._row_length`

`BlissScan._row_length(pos)` (static)

Tests whether a position series is the slow motor of a mesh: it must jump at regular intervals (one jump per row, all rows of equal length, at least two full rows) and stay constant within a row (range within a row below a tenth of the median jump). Returns the row length or `None`.

bliss_hdf5_reader.py, lines 448–463

```
448     @staticmethod
449     def _row_length(pos: np.ndarray) -> Optional[int]:
450         """Row length if `pos` is constant along rows and steps between rows
451         (the slow motor of a mesh), else None."""
452         n = pos.size
```

```
453         d = np.abs(np.diff(pos))
454         jumps = np.flatnonzero(d > 0.5 * d.max()) + 1
455         row = int(jumps[0])
456         # one jump per row boundary, evenly spaced, and at least two full rows
457         if row < 2 or n < 2 * row or len(jumps) != (n - 1) // row \
458                 or np.any(np.diff(jumps) != row):
459             return None
460         full = pos[: (n // row) * row].reshape(-1, row)
461         if np.ptp(full, axis=1).max() > 0.1 * np.median(d[jumps - 1]):
462             return None                                  # moves along the row: not a slow motor
463         return row
```

2.4.7 `BlissScan._infer_mesh`

`BlissScan._infer_mesh(moving)` (class method)

For each candidate row length (shortest first), the slow motors are those with that row length, and the fast motors those that repeat the same positions on the second row as on the first, in the same or the reversed order (mismatch below a tenth of their range). Returns `((n_slow, n_fast), motors)` or `None`.

bliss_hdf5_reader.py, lines 465–493

```
465     @classmethod
466     def _infer_mesh(cls, moving: Dict[str, np.ndarray]):
467         """Return ((n_slow, n_fast), motors) if some motors change only every n_fast
468         points while others repeat the same positions on every row, else None."""
469         rows = {name: cls._row_length(pos) for name, pos in moving.items()}
470         for row in sorted({r for r in rows.values() if r}):
471             slow_names = [k for k, r in rows.items() if r == row]
472             n = next(iter(moving.values())).size
473             n_slow = -(-n // row)
474             fast_candidates = []
475             for fast_name, fast in moving.items():
476                 if fast_name in slow_names:
477                     continue
478                 row0, row1 = fast[:row], fast[row:2 * row]
479                 span = np.ptp(row0)
480                 if span == 0:
481                     continue
482                 # the fast motor repeats the same positions on every row (or reversed)
483                 mismatch = min(np.abs(row1 - row0).max(), np.abs(row1 - row0[::-1]).max())
484                 if mismatch < 0.1 * span:
485                     fast_candidates.append(fast_name)
486             if not fast_candidates:
487                 continue
488             motors = [ScanMotor(name, 0, n_slow, float(moving[name][0]), float(moving[name][-1]))
489                       for name in slow_names]
490             motors += [ScanMotor(name, 1, row, float(moving[name][0]), float(moving[name][row - 1]))
491                        for name in fast_candidates]
492             return (n_slow, row), motors
493         return None
```

2.4.8 `BlissScan._detect_snake`

`BlissScan._detect_snake()`

A 2D scan is back-and-forth if the second row of the fast motor is closer to the reversed first row than to the first row itself.

bliss_hdf5_reader.py, lines 495–507

```
495     def _detect_snake(self) -> bool:
496         """True if a 2D scan was acquired back-and-forth (odd rows reversed)."""
497         if len(self.shape) != 2 or self.shape[0] < 2 or self.npoints < 2 * self.shape[1]:
498             return False
499         fast = [m for m in self.motors if m.dim == 1]
500         if not fast or fast[0].name not in self._positions_cache:
```

```
501             return False
502         pos = self._positions_cache[fast[0].name]
503         n_fast = self.shape[1]
504         row0, row1 = pos[:n_fast], pos[n_fast:2 * n_fast]
505         if np.ptp(row0) == 0:
506             return False
507         return bool(np.abs(row1 - row0[::-1]).max() < np.abs(row1 - row0).max())
```

2.4.9 Properties: `ndim`, `is_complete`, `motor_names`, `positioner_names`; `motors_of_dim`

Properties: `ndim`, `is_complete`, `motor_names`, `positioner_names`; `motors_of_dim(dim)`

`is_complete` is `end_reason == "SUCCESS"` when the end reason is recorded, otherwise a comparison of recorded and planned points. `positioner_names` lists every motor recorded at each point, scanned or not.

bliss_hdf5_reader.py, lines 510–512

```
510     @property
511     def ndim(self) -> int:
512         return len(self.shape)
```

bliss_hdf5_reader.py, lines 514–518

```
514     @property
515     def is_complete(self) -> bool:
516         if self.end_reason is not None:
517             return self.end_reason == "SUCCESS"
518         return self.npoints_planned is None or self.npoints >= self.npoints_planned
```

bliss_hdf5_reader.py, lines 520–522

```
520     @property
521     def motor_names(self) -> List[str]:
522         return [m.name for m in self.motors]
```

bliss_hdf5_reader.py, lines 524–527

```
524     @property
525     def positioner_names(self) -> List[str]:
526         """Every motor recorded at each point (scanned and dependent motors)."""
527         return list(self._positioner_names)
```

bliss_hdf5_reader.py, lines 529–530

```
529     def motors_of_dim(self, dim: int) -> List[ScanMotor]:
530         return [m for m in self.motors if m.dim == dim]
```

2.4.10 `BlissScan.index_grid`

`BlissScan.index_grid` (property)

The flat point index at each grid position, -1 where no point was recorded; odd rows are reversed for a back-and-forth mesh. It is the single object that places per-point data on the grid, used by `to_grid` and by the image scripts.

bliss_hdf5_reader.py, lines 532–543

```
532     @property
533     def index_grid(self) -> np.ndarray:
534         """Flat point index at each grid position (-1 where no point was recorded).
535         Use it to place any per-point quantity on the scan grid."""
536         planned = int(np.prod(self.shape)) if self.shape else 1
537         idx = np.full(planned, -1, dtype=np.int64)
538         n = min(self.npoints, planned)
539         idx[:n] = np.arange(n)
```

```
        idx = idx.reshape(self.shape)
        if self.snake:
            idx[1::2] = idx[1::2, ::-1]
        return idx
```

2.4.11 `get_available_counters`, `get`, `__getitem__`, `positions`

`get_available_counters()` (alias `list_counters`), `get(name, grid=False)`, `__getitem__(name)`, `positions(motor, grid=False)`

`get_available_counters` returns `{name: {"shape", "dtype"}}` for every measurement dataset without reading data. `get` reads one dataset (or returns it from `counters` if `read_data` loaded it), warns when it would load more than 2 GB, and with `grid=True` reshapes the point axis to the scan shape: an MCA dataset of a mesh becomes $(n_{\mathrm{slow}}, n_{\mathrm{fast}}, n_{\mathrm{ch}})$. `scan["name"]` is `get(name, grid=True)`; `positions` returns floats.

bliss_hdf5_reader.py, lines 546–549

```
    def get_available_counters(self) -> Dict[str, Dict[str, Any]]:
        """Name -> {'shape', 'dtype'} for every dataset of the measurement group,
        without reading any data."""
        return {k: {"shape": s, "dtype": str(t)} for k, (s, t) in self._datasets.items()}
```

bliss_hdf5_reader.py, lines 553–574

```
    def get(self, name: str, grid: bool = False) -> np.ndarray:
        """
        Read one dataset of the measurement group.

        With grid=True the per-point axis is reshaped to `self.shape`
        (2D spectra become (n_slow, n_fast, n_channels), etc.).
        Counters already loaded by `read_data()` are not read again.
        """
        if name in self.counters:
            data = self.counters[name]
        else:
            if name not in self._datasets:
                raise KeyError(f"'{name}' not in scan {self.scan_key}. "
                               f"Available: {sorted(self._datasets)}")
            shape, _ = self._datasets[name]
            nbytes = int(np.prod(shape)) * self._datasets[name][1].itemsize
            if nbytes > 2 ** 31:
                logger.warning("Reading '%s' %s loads %.1f GB into memory; "
                               "use read_frame() for images.", name, shape, nbytes / 2 ** 30)
            with _open(self.h5_filepath) as f:
                data = f[f"{self.scan_key}/measurement/{name}"][()]
        return self.to_grid(data, name) if grid else data
```

bliss_hdf5_reader.py, lines 576–577

```
    def __getitem__(self, name: str) -> np.ndarray:
        return self.get(name, grid=True)
```

bliss_hdf5_reader.py, lines 579–581

```
    def positions(self, motor: str, grid: bool = False) -> np.ndarray:
        """Measured positions of a motor (motor units)."""
        return self.get(motor, grid=grid).astype(float)
```

2.4.12 `BlissScan.to_grid`

`BlissScan.to_grid(data, name="array")`

Places per-point data on the scan grid. Complete, non-snake data are simply reshaped. With missing points (aborted mesh) the output is filled with NaN (integer data are promoted to floating

point for that) and the recorded points are placed through `index_grid`, which also un-snakes back-and-forth rows. A counter with one value more than the grid (BLISS sometimes records one point ahead on abort) is truncated. Raises `ValueError` when the first axis is not one value per point.

bliss_hdf5_reader.py, lines 583–614

```
583     def to_grid(self, data: np.ndarray, name: str = "array") -> np.ndarray:
584         """
585         Place per-point data (first axis = point index) on the scan grid.
586 
587         Missing points of an aborted mesh are NaN (integer data are converted
588         to float for that), back-and-forth rows are put back in order.
589         Data whose first axis is not the number of points (e.g. the 2n+1
590         '*_raw' channels of fast scans) cannot be gridded: ValueError.
591         """
592         data = np.asarray(data)
593         if data.ndim == 0:
594             return data
595         n = data.shape[0]
596         size = int(np.prod(self.shape)) if self.shape else 1
597         limit = max(size, self.npoints_planned or 0)
598         if n > limit:
599             raise ValueError(f"'{name}' has {n} values for a scan of {limit} points; "
600                              "it is not one value per point.")
601         if n > size:            # aborted 1D scan: a counter one point ahead of the others
602             data, n = data[:size], size
603         if n == size and not self.snake:
604             return data.reshape(self.shape + data.shape[1:])
605 
606         if n < size:
607             logger.debug("'%s': %d of %d points recorded, missing points set to NaN.",
608                          name, n, size)
609         idx = self.index_grid
610         out_dtype = data.dtype if n == size else np.result_type(data.dtype, np.float32)
611         out = np.full(idx.shape + data.shape[1:], np.nan if n < size else 0, dtype=out_dtype)
612         valid = (idx >= 0) & (idx < n)
613         out[valid] = data[idx[valid]]
614         return out
```

2.4.13 `BlissScan.read_data`

```
BlissScan.read_data(extra_counters=None)
```

Loads the scanned motors and the requested counters into `counters`, grids every 1D counter into `grids`, and locates the image files (`_process_images`). Missing names produce warnings, not errors.

bliss_hdf5_reader.py, lines 616–646

```
616     def read_data(self, extra_counters: Optional[Sequence[str]] = None) -> None:
617         """
618         Load the scanned motors and the requested counters into `self.counters`
619         (raw, as stored), put scalar counters on the scan grid in `self.grids`,
620         and locate the detector image files.
621         """
622         names = [m.name for m in self.motors if m.name in self._datasets]
623         for m in self.motors:
624             if m.name not in self._datasets:
625                 logger.warning("Scanned motor '%s' not found in the measurement group.", m.name)
626         missing = [c for c in (extra_counters or []) if c not in self._datasets]
627         for c in missing:
628             logger.warning("Counter '%s' not found in the measurement group.", c)
629         names += [c for c in (extra_counters or []) if c in self._datasets and c not in names]
630 
631         with _open(self.h5_filepath) as f:
632             meas = f[f"{self.scan_key}/measurement"]
633             for name in names:
634                 if name not in self.counters:
635                     self.counters[name] = meas[name][()]
636 
637         for name, data in self.counters.items():
```

```
638             if data.ndim == 1:
639                 try:
640                     self.grids[name] = self.to_grid(data, name)
641                 except ValueError as err:
642                     logger.warning("Not gridded: %s", err)
643 
644         self._process_images()
645         if self.verbose:
646             print(self.summary())
```

2.5 Detector frames

2.5.1 `image_detectors`, `spectrum_detectors` ; `_pick_detector`

`image_detectors`, `spectrum_detectors` (properties); `_pick_detector(detector)`

Datasets with one 2D image per point (3D datasets) and with one spectrum per point (2D datasets). `_pick_detector` returns the named detector, or the first image detector matching `image_prefix`, or the first image detector.

bliss_hdf5_reader.py, lines 649–652

```
649     @property
650     def image_detectors(self) -> List[str]:
651         """Measurement datasets holding one 2D image per point."""
652         return [k for k, (s, _) in self._datasets.items() if len(s) == 3]
```

bliss_hdf5_reader.py, lines 654–657

```
654     @property
655     def spectrum_detectors(self) -> List[str]:
656         """Measurement datasets holding one 1D spectrum (MCA, ...) per point."""
657         return [k for k, (s, _) in self._datasets.items() if len(s) == 2]
```

bliss_hdf5_reader.py, lines 659–668

```
659     def _pick_detector(self, detector: Optional[str]) -> Optional[str]:
660         if detector is not None:
661             if detector not in self._datasets:
662                 raise KeyError(f"Detector '{detector}' not in scan {self.scan_key}.")
663             return detector
664         dets = self.image_detectors
665         if self.image_prefix:
666             preferred = [d for d in dets if d.startswith(self.image_prefix)]
667             dets = preferred or dets
668         return dets[0] if dets else None
```

2.5.2 `BlissScan.frame_sources`

`BlissScan.frame_sources(detector=None)`

Reads the mapping of the virtual dataset: for each virtual source, the selection bounds in the virtual and source spaces give the range of frames it holds; relative file names are resolved against the master file's folder. Returns one `FrameSource` per frame, or an empty list when the detector is not a virtual dataset.

bliss_hdf5_reader.py, lines 670–692

```
670     def frame_sources(self, detector: Optional[str] = None) -> List[FrameSource]:
671         """
672         File, internal dataset and index of every frame of an image detector,
673         read from the virtual dataset of the master file. Empty list if the
674         detector is not stored as a virtual dataset.
675         """
676         detector = self._pick_detector(detector)
677         if detector is None:
678             return []
```

```
        with _open(self.h5_filepath) as f:
            dset = f[f"{self.scan_key}/measurement/{detector}"]
            if not dset.is_virtual:
                return []
            sources = []
            for vs in dset.virtual_sources():
                (v0, *_), (v1, *_) = vs.vspace.get_select_bounds()
                (s0, *_), _ = vs.src_space.get_select_bounds()
                path = Path(vs.file_name)
                if not path.is_absolute():
                    path = self.h5_filepath.parent / path
                for k in range(v1 - v0 + 1):
                    sources.append(FrameSource(path, vs.dset_name, s0 + k))
        return sources
```

2.5.3 `read_frame`, `iter_frames`

`read_frame(index, detector=None, roi=None)`, `iter_frames(detector=None, indices=None, roi=None)`

One frame by point index, read through the master file (HDF5 resolves the virtual mapping), optionally restricted to `roi = (row slice, column slice)` so that only that part is read from disk. `iter_frames` yields frames while keeping the file open, which is much faster than repeated `read_frame` calls.

bliss_hdf5_reader.py, lines 694–703

```
    def read_frame(self, index: int, detector: Optional[str] = None,
                   roi: Optional[Tuple[slice, slice]] = None) -> np.ndarray:
        """One detector frame by point index (through the master file).
        `roi` = (row slice, column slice) returns only that part of the frame."""
        detector = self._pick_detector(detector)
        if detector is None:
            raise KeyError(f"No image detector in scan {self.scan_key}.")
        with _open(self.h5_filepath) as f:
            dset = f[f"{self.scan_key}/measurement/{detector}"]
            return dset[(index,) + tuple(roi)] if roi is not None else dset[index]
```

bliss_hdf5_reader.py, lines 705–716

```
    def iter_frames(self, detector: Optional[str] = None,
                    indices: Optional[Sequence[int]] = None,
                    roi: Optional[Tuple[slice, slice]] = None) -> Iterator[np.ndarray]:
        """Frames one at a time, keeping the file open (faster than repeated
        `read_frame` calls). `roi` as in `read_frame`."""
        detector = self._pick_detector(detector)
        if detector is None:
            return
        with _open(self.h5_filepath) as f:
            dset = f[f"{self.scan_key}/measurement/{detector}"]
            for i in (range(dset.shape[0]) if indices is None else indices):
                yield dset[(i,) + tuple(roi)] if roi is not None else dset[i]
```

2.5.4 `BlissScan._process_images`

`BlissScan._process_images()`

Fills `image_path_list` with the distinct image files in acquisition order (from the virtual dataset, or by listing `scanNNNN/<prefix>*<suffix>`), `nb_images` (files) and `nb_frames` (frames).

bliss_hdf5_reader.py, lines 718–736

```
    def _process_images(self) -> None:
        """Fill `image_path_list` (image files in acquisition order) and
        `nb_images`, `nb_frames`."""
        sources = self.frame_sources()
        if sources:
            self.image_path_list = list(dict.fromkeys(s.file for s in sources))
            self.nb_frames = len(sources)
```

```
        elif self.scan_folder.is_dir():
            # No virtual dataset: list the scan folder (top level only, so that
            # processed files saved in sub-folders are not picked up).
            pattern = f"{self.image_prefix or ''}*{self.image_suffix}"
            self.image_path_list = sorted(self.scan_folder.glob(pattern))
            self.nb_frames = len(self.image_path_list)
        else:
            self.image_path_list = []
            self.nb_frames = 0
            if self.image_detectors:
                logger.warning("Image folder not found: %s", self.scan_folder)
        self.nb_images = len(self.image_path_list)
```

2.6 Other methods, the mesh subclass and the command line

2.6.1 `positioners_start`, `get_relative_position_grid`, `summary`, `get_summary`, `__repr__`

`positioners_start()`, `get_relative_position_grid(motor_name, scale=1000.0)`, `summary()`, `get_summary()`, `__repr__()`

`positioners_start` returns the snapshot of every motor at the start of the scan. `get_relative_position_grid` returns the gridded positions minus that of the first recorded point, times `scale` (mm to µm by default). `summary` is a human-readable description: shape and its source, points recorded and planned, completion, snake, count time, motors with their ranges and steps, detectors, loaded counters.

bliss_hdf5_reader.py, lines 739–750

```
    def positioners_start(self) -> Dict[str, Any]:
        """Snapshot of every motor position at the start of the scan."""
        with _open(self.h5_filepath) as f:
            grp = f.get(f"{self.scan_key}/instrument/positioners_start")
            if grp is None:
                return {}
            out = {}
            for k, v in grp.items():
                if isinstance(v, h5py.Dataset):
                    value = v[()]
                    out[k] = value.item() if isinstance(value, np.generic) else value
            return out
```

bliss_hdf5_reader.py, lines 752–758

```
    def get_relative_position_grid(self, motor_name: str, scale: float = 1000.0) -> np.ndarray:
        """Gridded positions relative to the first recorded point, times `scale`
        (default 1000: mm -> µm)."""
        grid = self.to_grid(self.positions(motor_name), motor_name)
        flat = grid.ravel()
        origin = flat[self.index_grid.ravel() == 0][0] if grid.ndim else grid
        return (grid - origin) * scale
```

bliss_hdf5_reader.py, lines 760–783

```
    def summary(self) -> str:
        lines = [f"Scan {self.scan_key} of {self.h5_filepath.name}: {self.title}"]
        status = "complete" if self.is_complete else f"INCOMPLETE ({self.end_reason})"
        planned = f" of {self.npoints_planned}" if self.npoints_planned else ""
        lines.append(f"  shape {self.shape} [{self.geometry_source}], {self.npoints}{planned} "
                     f"points, {status}" + (", back-and-forth" if self.snake else ""))
        if self.count_time is not None:
            lines.append(f"  count time {self.count_time:g} s")
        for m in sorted(self.motors, key=lambda m: m.dim):
            rng = ""
            if m.start is not None and m.stop is not None:
                rel = " (relative)" if m.relative else ""
                step = f", step {m.step:.6g}" if m.step is not None else ""
                rng = f": {m.start:g} -> {m.stop:g}{rel}, {m.npoints} pts{step}"
            lines.append(f"  dim {m.dim}  {m.name}{rng}")
        if self.image_detectors:
```

```
776             lines.append(f"  image detectors: {self.image_detectors}")
777         if self.spectrum_detectors:
778             lines.append(f"  spectrum detectors: {self.spectrum_detectors}")
779         if self.nb_images:
780             lines.append(f"  image files found: {self.nb_images}")
781         if self.counters:
782             lines.append(f"  loaded: {list(self.counters)}")
783         return "\n".join(lines)
```

bliss_hdf5_reader.py, lines 785–786

```
785     def get_summary(self) -> str:
786         return self.summary()
```

bliss_hdf5_reader.py, lines 788–790

```
788     def __repr__(self) -> str:
789         return (f"<{type(self).__name__} {self.h5_filepath.name}:{self.scan_key} "
790                 f"'{self.title}' shape={self.shape}>")
```

2.6.2 `BlissMeshScan`

A `BlissScan` that must be two-dimensional (`ValueError` otherwise), kept for scripts written for the former mesh-only reader. It adds the attributes `slow_/fast_motor_name`, `_start`, `_stop`, `_steps`, `_intervals`, `_step_size`, `scan_shape` and `expected_points`.

bliss_hdf5_reader.py, lines 793–803

```
793 class BlissMeshScan(BlissScan):
794     """
795     A two-dimensional scan (amesh, dmesh, fscan2d, or a mesh with a custom title),
796     with the attributes of the former mesh-only reader:
797 
798         fast_motor_name, fast_start, fast_stop, fast_intervals, fast_steps,
799         fast_step_size (same for slow_*), scan_shape = (slow, fast),
800         expected_points, count_time, scan_type.
801 
802     Grids are (slow, fast): rows follow the slow motor, whatever its name.
803     """
```

bliss_hdf5_reader.py, lines 805–819

```
805     def _read_metadata(self) -> None:
806         super()._read_metadata()
807         if self.ndim != 2:
808             raise ValueError(f"Scan {self.scan_key} is not a 2D mesh scan "
809                              f"(shape {self.shape}). Title: {self.title}. Use BlissScan.")
810         for label, dim in (("slow", 0), ("fast", 1)):
811             m = self.motors_of_dim(dim)[0]
812             setattr(self, f"{label}_motor_name", m.name)
813             setattr(self, f"{label}_start", m.start)
814             setattr(self, f"{label}_stop", m.stop)
815             setattr(self, f"{label}_steps", self.shape[dim])
816             setattr(self, f"{label}_intervals", self.shape[dim] - 1)
817             setattr(self, f"{label}_step_size", m.step or 0.0)
818         self.scan_shape = self.shape
819         self.expected_points = int(np.prod(self.shape))
```

2.6.3 `main`

```
main()
```

Command line: `python bliss_hdf5_reader.py file.h5` lists the scans (key, planned points, end reason, title); with scan keys after the file name it prints their summaries and dataset shapes.

bliss_hdf5_reader.py, lines 822–848

```
822 # ---------------------------------------------------------------------------- #
```

```
def main() -> None:
    """Command line: list the scans of a file, or summarise some of them.

        python bliss_hdf5_reader.py file.h5           # list scans
        python bliss_hdf5_reader.py file.h5 3 7 12.2   # summary of scans 3, 7, 12.2
    """
    import argparse
    parser = argparse.ArgumentParser(description=main.__doc__,
                                     formatter_class=argparse.RawDescriptionHelpFormatter)
    parser.add_argument("h5_file", type=Path)
    parser.add_argument("scans", nargs="*")
    args = parser.parse_args()
    logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")

    if not args.scans:
        for row in list_scans(args.h5_file):
            npts = row["npoints"] if row["npoints"] is not None else "-"
            print(f"{row['key']:>7}  {str(npts):>7}  {str(row['end_reason'] or ''):<11} {row['title']}")
        return
    for key in args.scans:
        scan = BlissScan(args.h5_file, key)
        print(scan.summary())
        counters = scan.get_available_counters()
        print(f"  {len(counters)} datasets: " + ", ".join(
            f"{k}{list(v['shape'][1:]) if len(v['shape']) > 1 else ''}" for k, v in counters.items()))
        print()
```

3 `xrf_beamline`: the incident spectrum

3.1 Overview

Under white-beam (Laue) illumination every element is excited by the whole incident spectrum $\Phi(E)$, and the relative intensities of its fluorescence lines—not only its total signal—depend on the shape of that spectrum: each L or M subshell is ionised only by photons above its own edge, and those edges can be kilo-electronvolts apart (Ir: L_3 at 11.22 keV, L_1 at 13.42 keV). The module computes $\Phi(E)$ at the sample from first principles: the synchrotron radiation of a bending magnet [8, 9], accepted through the apertures of the beamline, and filtered by grazing-incidence mirrors [10, 11], beryllium windows and the air path. It also provides narrow "monochromatic" spectra for tube or monochromator excitation, and the air model used by the detector-efficiency calculation of `xrf_fit`.

The quantities returned by `Beamline.spectrum` are fluxes per mrad of horizontal angle and per 0.1 % bandwidth; only their *shape* matters for line ratios and for the matrix corrections, but the absolute scale is correct (photons/s) for a given ring current.

Hypotheses and limits

- Bending-magnet radiation of a filament electron beam (no emittance, no energy spread); the horizontal fan is uniform, so horizontal apertures scale the flux but not the spectrum.
- Mirrors: specular s-polarised Fresnel reflectivity of a homogeneous coating of nominal density, with an optional Névot–Croce roughness factor; no slope errors, no thermal deformation, no contamination layer. Bent mirrors act as thin cylindrical lenses in the paraxial approximation.
- Windows and air: Beer–Lambert attenuation with total cross-sections; dry air, ideal gas.
- The BM32 defaults (magnetic field 0.86 T, Ir coatings, 2 × 500 µm Be, 3.3 m of air, element positions) were confirmed in October 2026; mirror roughness is unknown (0) and the KB grazing angles are not recorded in the scan files (3 mrad).

xrf_beamline.py, lines 1–110

```
"""
Module: xrf_beamline.py

Incident white-beam spectrum of a bending-magnet beamline (BM32, ESRF).
```

```
5
6   Computes the Schwinger spectral flux of the source and propagates it through a
7   list of optical elements (grazing-incidence mirrors, transmission windows and
8   filters) to give Phi(E) at the sample.  That spectrum is what weights the
9   fluorescence cross-sections in :mod:`xrf_fit` when the illumination is
10  polychromatic.
11
12  Why this matters for the line ratios
13  ------------------------------------
14  With a monochromatic beam every subshell of an element sees the same photon
15  energy, and the relative intensity of its lines is a property of the element
16  alone.  Under white-beam illumination each subshell is excited by the part of
17  the spectrum lying above *its own* edge, and those edges can be far apart
18  compared with the width of the band:
19
20  =======  ========================  ====================================
21  element  L3 / L2 / L1 edge (keV)     fraction of a 5-28 keV band above each
22  =======  ========================  ====================================
23  Nd       6.21 / 6.72 / 7.13          nearly all of it, for every subshell
24  Ir       11.22 / 12.82 / 13.42       noticeably different per subshell
25  Pb       13.04 / 15.20 / 15.86       strongly different per subshell
26  =======  ========================  ====================================
27
28  So the heavier the element, the more a single effective energy misrepresents
29  the Lb/La and Lg/La ratios.  Flux weighting removes that error.
30
31  Conventions and caveats
32  -----------------------
33  * ``spectral_flux`` returns photons/s/mrad(horizontal)/0.1 % bandwidth
34    integrated over the full vertical opening angle,
35    ``2.457e13 * E[GeV] * I[A] * G1(y)``; ``angular_flux`` returns the angular
36    density photons/s/mrad^2/0.1 % BW, ``1.327e13 * E[GeV]^2 * I[A] * F(y, psi)``,
37    whose on-axis value is the familiar ``H2(y)`` form.  **The current is in
38    amperes.**  (Before Oct 2026 ``spectral_flux`` multiplied ``G1`` by the
39    angular-density constant ``1.327e13 E^2 I``, a constant factor E/1.85 too
40    high; irrelevant for ratios, wrong for absolute flux.)
41  * **Vertical acceptance.**  The photon beam of a bending magnet is
42    ~1/gamma = 85 urad wide vertically at ESRF-EBS (FWHM 110-270 urad between 30
43    and 5 keV), and softer photons spread wider.  When the beamline apertures
44    accept much less than that, as on BM32 where slit s3 passes a few urad, the
45    accepted spectrum is the *on-axis* spectrum, noticeably harder than the
46    vertically integrated ``G1`` shape.  :class:`Beamline` therefore traces the
47    apertures (:class:`Slit`, mirror footprints) back to the source, through the
48    bent mirrors, and integrates :meth:`BendingMagnetSource.angular_flux` over
49    the accepted vertical window.  On the 2026 BM32 maps this changes the Ga/As
50    sensitivity ratio by ~4.5 %.  Horizontal apertures only scale the flux (the
51    fan is uniform in the horizontal angle) and do not change the spectrum.
52  * **KB mirrors.**  Along an elliptical focusing mirror the grazing angle
53    varies, because rays from a distant source arrive almost parallel but leave
54    converging on the focus: over an illuminated beam of size ``s`` at image
55    distance ``q`` the angle spans ``theta0 +/- s/(4q)``.  This smooths the
56    mirror's high-energy cut-off and is applied to mirrors given an
57    ``image_distance_m``.
58  * **Per-bandwidth vs per-energy.**  A 0.1 % bandwidth is ``dE = 1e-3 * E``, so
59    the flux per unit energy is ``dN/dE = N_0.1%BW / (1e-3 * E)``.  Any integral
60    over ``dE`` (e.g. a flux-weighted cross-section ``int sigma(E) Phi(E) dE``)
61    must use the per-energy density, otherwise high energies are over-weighted
62    by a factor E.  Use :func:`per_unit_energy` (or
63    :meth:`Beamline.spectral_density`) for the conversion; ``xrf_fit`` applies
64    it internally to the per-0.1 %BW spectrum returned by
65    :meth:`Beamline.spectrum`.
66  * Mirror reflectivity is s-polarised Fresnel with an optional Nevot-Croce
67    roughness factor.  Synchrotron radiation is predominantly horizontally
68    polarised and the mirrors here deflect vertically, so s-polarisation is the
69    relevant case; at grazing incidence the two are indistinguishable anyway.
70  * The default BM32 configuration (0.86 T bending magnet, four Ir-coated
71    mirrors, two 500 um Be windows, 3.3 m measured incident air path, element
72    positions in :data:`BM32_LAYOUT`) was confirmed in Oct 2026.  Mirror
73    roughness is not known and is left at 0; the KB grazing angles are not
74    recorded in the scan files and default to 3 mrad.  Use :func:`bm32_from_h5`
75    to build the beamline with the slit gaps, mirror angles, bending radii and
76    ring current of a given scan.
77
78  Authors: Samuel Tardif, with Claude (Anthropic).
79  """
```

```
80
81  from __future__ import annotations
82
83  import logging
84  import math
85  from dataclasses import dataclass, field, replace
86  from pathlib import Path
87  from typing import Dict, List, Optional, Sequence, Tuple, Union
88
89  import numpy as np
90  import xraylib
91  from scipy.integrate import quad, trapezoid
92  from scipy.special import kv
93
94  logger = logging.getLogger(__name__)
95  xraylib.SetErrorMessages(0)
96
97  __all__ = [
98      "BendingMagnetSource",
99      "Mirror",
100     "Filter",
101     "Slit",
102     "Beamline",
103     "BM32_LAYOUT",
104     "BM32_WHITE_BEAM",
105     "bm32_from_h5",
106     "air_path",
107     "air_density",
108     "AIR_FORMULA",
109     "per_unit_energy",
110 ]
```

3.2 Source

3.2.1 `BendingMagnetSource`

```
BendingMagnetSource(ring_energy_gev=6.04, magnetic_field_t=0.86, current_a=0.2)
```

Synchrotron radiation of a bending magnet. The defaults describe the BM32 source on ESRF-EBS.

`ring_energy_gev`
: electron energy E_e (6.04 GeV for EBS).

`magnetic_field_t`
: dipole field B; sets the critical energy, to which the high-energy end of the spectrum is exponentially sensitive.

`current_a`
: ring current I, in amperes.

Physics The critical energy and the Lorentz factor are

$$E_c\,[\mathrm{keV}] = 0.665\,E_e^2[\mathrm{GeV}]\,B[\mathrm{T}], \qquad \gamma = \frac{E_e}{m_e c^2} \approx 11\,820, \tag{2}$$

giving $E_c = 20.9\,\mathrm{keV}$ for BM32. With $y = E/E_c$, the flux integrated over the vertical opening angle, per mrad of horizontal angle and per 0.1 % bandwidth, is [9]

$$\frac{\mathrm{d}N}{\mathrm{d}\theta} = 2.457\times10^{13}\,E_e[\mathrm{GeV}]\,I[\mathrm{A}]\,G_1(y), \qquad G_1(y) = y\int_y^\infty K_{5/3}(x)\,\mathrm{d}x, \tag{3}$$

and the angular density at vertical angle ψ, per mrad^2, with $X = \gamma\psi$ and $\xi = \tfrac{y}{2}(1+X^2)^{3/2}$, is

$$\frac{\mathrm{d}^2N}{\mathrm{d}\theta\,\mathrm{d}\psi} = 1.327\times10^{13}\,E_e^2\,I\,y^2(1+X^2)^2\left[K_{2/3}^2(\xi) + \frac{X^2}{1+X^2}K_{1/3}^2(\xi)\right], \tag{4}$$

where the first term is the horizontally (σ) polarised part and the second the vertically (π) polarised part. On axis it reduces to $1.327\times10^{13}E_e^2 I\,H_2(y)$ with $H_2(y)=y^2K_{2/3}^2(y/2)$. The vertical opening angle is of order $1/\gamma = 85\,\mu\text{rad}$, and wider for soft photons, so an aperture that accepts only a few µrad selects the harder on-axis spectrum.

xrf_beamline.py, lines 113–136

```
113 # ============================================================================
114 # Source
115 # ============================================================================
116
117 @dataclass
118 class BendingMagnetSource:
119     """Synchrotron radiation of a bending-magnet (or short wiggler) source.
120
121     Parameters
122     ----------
123     ring_energy_gev : float
124         Storage-ring energy.  6.04 GeV for ESRF-EBS.
125     magnetic_field_t : float
126         Dipole field.  Sets the critical energy, to which the high-energy
127         spectrum is exponentially sensitive — confirm it for your source
128         (an EBS two-pole wiggler is not the same as a standard EBS dipole).
129         Default 0.86 T: the BM32 bending magnet (confirmed Oct 2026).
130     current_a : float
131         Ring current in **amperes** (0.2 A = 200 mA).
132     """
133
134     ring_energy_gev: float = 6.04
135     magnetic_field_t: float = 0.86
136     current_a: float = 0.2
```

3.2.2 `critical_energy_kev`, `gamma` ; `_g1`

`critical_energy_kev`, `gamma` (properties); `_g1(y)` (static)

E_c and γ as above. `_g1` evaluates $G_1(y)$ by adaptive quadrature of $K_{5/3}$ (`scipy.special.kv`) from y to infinity; it returns 0 for $y \le 0$ and for $y > 50$, where $G_1 < 10^{-20}$.

xrf_beamline.py, lines 138–141

```
138     @property
139     def critical_energy_kev(self) -> float:
140         """``Ec = 0.665 * E[GeV]^2 * B[T]`` in keV."""
141         return 0.665 * self.ring_energy_gev ** 2 * self.magnetic_field_t
```

xrf_beamline.py, lines 143–146

```
143     @property
144     def gamma(self) -> float:
145         """Lorentz factor of the electrons."""
146         return self.ring_energy_gev * 1e3 / 0.51099895
```

xrf_beamline.py, lines 148–156

```
148     @staticmethod
149     def _g1(y: float) -> float:
150         """``G1(y) = y * integral_y^inf K_{5/3}(x) dx`` — vertically integrated shape."""
151         if y <= 0:
152             return 0.0
153         if y > 50.0:                        # underflows; the tail is negligible here
154             return 0.0
155         val, _ = quad(lambda x: kv(5.0 / 3.0, x), y, np.inf, epsabs=1e-12, limit=200)
156         return y * val
```

3.2.3 `spectral_flux`

```
spectral_flux(energies_kev)
```

Eq. (3) on an energy grid (keV): photons/s/mrad(horizontal)/0.1 % BW, integrated over the vertical angle. Returns an array of the grid's size.

xrf_beamline.py, lines 158–164

```
158     def spectral_flux(self, energies_kev: np.ndarray) -> np.ndarray:
159         """Flux in photons/s/mrad(horizontal)/0.1 % bandwidth, integrated over
160         the full vertical opening angle: ``2.457e13 E[GeV] I[A] G1(E/Ec)``."""
161         e = np.atleast_1d(np.asarray(energies_kev, dtype=float))
162         y = e / self.critical_energy_kev
163         shape = np.array([self._g1(float(yy)) for yy in y])
164         return 2.457e13 * self.ring_energy_gev * self.current_a * shape
```

3.2.4 `angular_flux`

```
angular_flux(energies_kev, psi_urad)
```

Eq. (4): photons/s/mrad^2/0.1 % BW, both polarisations, on a grid of energies and vertical angles (µrad). Returns an array of shape (n_E, n_ψ). Bessel functions that overflow or underflow far from the axis are set to zero.

xrf_beamline.py, lines 166–184

```
166     def angular_flux(self, energies_kev: np.ndarray, psi_urad: np.ndarray) -> np.ndarray:
167         """Angular density, photons/s/mrad^2/0.1 % BW, both polarisations.
168
169         ``1.327e13 E^2 I y^2 (1+X^2)^2 [K_{2/3}^2(xi) + X^2/(1+X^2) K_{1/3}^2(xi)]``
170         with ``y = E/Ec``, ``X = gamma psi`` and ``xi = (y/2)(1+X^2)^{3/2}``; the
171         first term is the horizontally (sigma) polarised part.  On axis it
172         reduces to ``1.327e13 E^2 I H2(y)``; integrated over ``psi`` (in mrad)
173         it gives :meth:`spectral_flux`.
174
175         Returns an array of shape ``(n_energies, n_angles)``.
176         """
177         e = np.atleast_1d(np.asarray(energies_kev, dtype=float))
178         y = (e / self.critical_energy_kev)[:, None]
179         x2 = (self.gamma * np.atleast_1d(np.asarray(psi_urad, dtype=float)) * 1e-6)[None, :] ** 2
180         xi = 0.5 * y * (1.0 + x2) ** 1.5
181         with np.errstate(over="ignore", under="ignore", invalid="ignore"):
182             core = kv(2.0 / 3.0, xi) ** 2 + x2 / (1.0 + x2) * kv(1.0 / 3.0, xi) ** 2
183         core = np.nan_to_num(core, nan=0.0, posinf=0.0)
184         return 1.327e13 * self.ring_energy_gev ** 2 * self.current_a * y ** 2 * (1.0 + x2) ** 2 * core
```

3.2.5 `accepted_flux`

```
accepted_flux(energies_kev, vertical_acceptance_urad=None, vertical_offset_urad=0.0,
n_angles=41)
```

Flux per mrad (horizontal) per 0.1 % BW inside the vertical window $\psi_0 \pm \Delta\psi/2$: the trapezoidal integral of Eq. (4) over `n_angles` angles, converted from µrad to mrad. `None` (or a non-finite value) returns the full integral, Eq. (3). For $\Delta\psi \ll 1/\gamma$ the result is the on-axis spectrum times $\Delta\psi$.

xrf_beamline.py, lines 186–199

```
186     def accepted_flux(self, energies_kev: np.ndarray, vertical_acceptance_urad: Optional[float] = None,
187                       vertical_offset_urad: float = 0.0, n_angles: int = 41) -> np.ndarray:
188         """Flux per mrad (horizontal) per 0.1 % BW inside a vertical window.
189
190         The window is ``vertical_offset_urad +/- vertical_acceptance_urad / 2``
191         in the source vertical angle.  ``None`` means the full opening angle,
192         i.e. :meth:`spectral_flux`.  For an acceptance much smaller than
193         1/gamma the result is the on-axis spectrum times the acceptance.
194         """
```

```
        if vertical_acceptance_urad is None or not np.isfinite(vertical_acceptance_urad):
            return self.spectral_flux(energies_kev)
        half = 0.5 * float(vertical_acceptance_urad)
        psi = np.linspace(vertical_offset_urad - half, vertical_offset_urad + half, n_angles)
        return trapezoid(self.angular_flux(energies_kev, psi), psi * 1e-3, axis=1)
```

3.3 Optical elements

3.3.1 `Mirror`

```
Mirror(material, density_g_cm3, grazing_angle_mrad, roughness_nm=0.0, label="",
distance_m=None, length_m=None, plane="vertical", bending_radius_m=inf, image_distance_m=None,
angle_spread_mrad=0.0)
```

A grazing-incidence mirror. Its spectral effect is the reflectivity; if it is positioned (`distance_m` and `length_m`), its footprint also acts as an aperture; if it is bent, it focuses the beam divergence; if it is a focusing (KB) mirror, its grazing angle varies along the footprint.

`material, density_g_cm3`
: coating, e.g. `"Ir"`, 22.56.

`grazing_angle_mrad`
: angle between the beam and the surface, θ.

`roughness_nm`
: r.m.s. roughness σ (Névot–Croce factor; 0 disables it).

`distance_m, length_m`
: distance from the source and optical length.

`plane`
: `"vertical"` or `"horizontal"`: deflection plane.

`bending_radius_m`
: meridional radius R (`inf` for a flat mirror).

`image_distance_m`
: distance q to the focus of a focusing mirror.

`angle_spread_mrad`
: half-range of grazing angles over the illuminated footprint; filled in by `Beamline.resolved_optics` from `image_distance_m`.

xrf_beamline.py, lines 202–254

```
# ============================================================================
# Optics
# ============================================================================

@dataclass
class Mirror:
    """Grazing-incidence mirror, s-polarised Fresnel reflectivity.

    Parameters
    ----------
    material : str
        Compound formula of the reflecting surface, e.g. ``"Ir"``, ``"Si"``.
    density_g_cm3 : float
        Density of that material.
    grazing_angle_mrad : float
        Angle between the incident beam and the mirror surface.
    roughness_nm : float
        RMS surface roughness; applied as a Nevot-Croce factor.  Zero disables it.
    label : str
        Free-text name for plots.
    distance_m : float, optional
        Distance from the source.  Needed, with ``length_m``, for the mirror
```

```
        footprint to act as an aperture in :meth:`Beamline.acceptance`.
    length_m : float, optional
        Optical length.  The mirror accepts a beam of size
        ``length * sin(theta)`` in its deflection plane.
    plane : {"vertical", "horizontal"}
        Deflection plane.
    bending_radius_m : float
        Meridional radius of curvature (``inf`` for flat).  A concave mirror
        of radius ``R`` acts as a lens of focal length ``R sin(theta) / 2`` on
        the beam divergence, which changes how downstream apertures map back
        to the source.
    image_distance_m : float, optional
        For a focusing mirror (KB): distance to the focus.  The grazing angle
        then varies across the illuminated footprint by ``+/- s / (4 q)`` for a
        beam of size ``s``; :class:`Beamline` sets ``angle_spread_mrad`` from it.
    angle_spread_mrad : float
        Half-range of grazing angles over the illuminated footprint; the
        reflectivity is averaged uniformly over ``theta +/- spread``.
    """

    material: str
    density_g_cm3: float
    grazing_angle_mrad: float
    roughness_nm: float = 0.0
    label: str = ""
    distance_m: Optional[float] = None
    length_m: Optional[float] = None
    plane: str = "vertical"
    bending_radius_m: float = math.inf
    image_distance_m: Optional[float] = None
    angle_spread_mrad: float = 0.0
```

xrf_beamline.py, lines 256–259

```
    def __post_init__(self):
        self.label = self.label or f"{self.material} mirror @ {self.grazing_angle_mrad:.3f} mrad"
        if self.plane not in ("vertical", "horizontal"):
            raise ValueError(f"plane must be 'vertical' or 'horizontal', got {self.plane!r}")
```

3.3.2 `focal_length_m`, `aperture_m`

`focal_length_m`, `aperture_m` (properties)

A cylindrically bent mirror of radius R at grazing angle θ focuses in its deflection plane with focal length $f = R\sin\theta/2$ (the meridional Coddington equation at grazing incidence). The footprint of length L accepts a beam of size $L\sin\theta$.

xrf_beamline.py, lines 261–266

```
    @property
    def focal_length_m(self) -> float:
        """``R sin(theta) / 2`` (``inf`` for a flat mirror)."""
        if not np.isfinite(self.bending_radius_m):
            return math.inf
        return self.bending_radius_m * math.sin(self.grazing_angle_mrad * 1e-3) / 2.0
```

xrf_beamline.py, lines 268–273

```
    @property
    def aperture_m(self) -> Optional[float]:
        """Beam size the footprint accepts, ``length sin(theta)``."""
        if self.length_m is None:
            return None
        return self.length_m * math.sin(self.grazing_angle_mrad * 1e-3)
```

3.3.3 `Mirror.transfer`, `Mirror._reflectivity`

`Mirror.transfer(energies_kev)`, `Mirror._reflectivity(energies_kev, angle_mrad)`

Physics With the complex refractive index $n = 1 - \delta + i\beta$ of the coating (xraylib `Refractive_Index_Re` and `_Im`), the s-polarised Fresnel amplitude at grazing angle θ is [10, 12]

$$r_s = \frac{\sin\theta - q}{\sin\theta + q}, \qquad q = \sqrt{n^2 - \cos^2\theta}, \qquad R = |r_s|^2. \tag{5}$$

Total external reflection holds below the critical angle $\theta_c \approx \sqrt{2\delta}$; since $\delta \propto E^{-2}$, a mirror at fixed angle cuts off above $E_c^{\mathrm{mirror}} \propto 1/\theta$ (for Ir, $\delta = 3.4 \times 10^{-5}$ at 10 keV, so $\theta_c = 8.3$ mrad at 10 keV and the edge of reflection at 3 mrad lies near 27 keV). Roughness multiplies the *amplitude* by $\exp(-2k_{z0}k_{z1}\sigma^2)$ [11], with $k_{z0} = k_0 \sin\theta$ and $k_{z1} = k_0 q$, so the intensity factor is $\exp(-4k_0^2 \sin\theta\, \Re(q)\, \sigma^2)$; below the critical angle $\Re q \ll \sin\theta$ and the factor tends to 1, while well above it the Debye–Waller form $\exp(-4k_{z0}^2\sigma^2)$ is recovered. Synchrotron radiation is polarised in the orbit plane and the mirrors of BM32 deflect vertically, so s-polarisation is the relevant case (at mrad angles both polarisations give the same result anyway).

`transfer` returns $R(E)$, averaged uniformly over 21 angles in $\theta \pm$ `angle_spread_mrad` if a spread is set. Energies where xraylib has no data get $n = 1$.

xrf_beamline.py, lines 275–281

```
275     def transfer(self, energies_kev: np.ndarray) -> np.ndarray:
276         """Reflectivity in [0, 1], averaged over ``angle_spread_mrad`` if set."""
277         if self.angle_spread_mrad > 0:
278             angles = np.linspace(self.grazing_angle_mrad - self.angle_spread_mrad,
279                                  self.grazing_angle_mrad + self.angle_spread_mrad, 21)
280             return np.mean([self._reflectivity(energies_kev, a) for a in angles], axis=0)
281         return self._reflectivity(energies_kev, self.grazing_angle_mrad)
```

xrf_beamline.py, lines 283–311

```
283     def _reflectivity(self, energies_kev: np.ndarray, angle_mrad: float) -> np.ndarray:
284         e = np.atleast_1d(np.asarray(energies_kev, dtype=float))
285         theta = angle_mrad * 1e-3
286         sin_t, cos_t = np.sin(theta), np.cos(theta)
287 
288         n = np.ones_like(e, dtype=complex)
289         for i, ek in enumerate(e):
290             try:
291                 n[i] = (xraylib.Refractive_Index_Re(self.material, ek, self.density_g_cm3)
292                         + 1j * xraylib.Refractive_Index_Im(self.material, ek, self.density_g_cm3))
293             except ValueError:
294                 n[i] = 1.0
295 
296         # q = sqrt(n^2 - cos^2(theta));  r_s = (sin - q) / (sin + q)
297         q = np.sqrt(n ** 2 - cos_t ** 2)
298         r = np.abs((sin_t - q) / (sin_t + q)) ** 2
299 
300         if self.roughness_nm > 0:
301             # Nevot-Croce acts on the reflection *amplitude*:
302             #   r_s -> r_s * exp(-2 kz0 kz1 sigma^2),
303             # so the intensity factor is exp(-4 kz0 Re(kz1) sigma^2), with
304             # kz0 = k0 sin(theta) and kz1 = k0 q.  Below the critical angle
305             # Re(q) << sin(theta) (evanescent wave) and the factor -> 1; the
306             # Debye-Waller limit exp(-4 kz0^2 sigma^2) holds only well above it.
307             lam_nm = 1.23984193 / e                      # keV -> nm
308             k0 = 2.0 * np.pi / lam_nm
309             r = r * np.exp(-4.0 * k0 ** 2 * sin_t * np.real(q) * self.roughness_nm ** 2)
310 
311         return np.clip(r.real, 0.0, 1.0)
```

3.3.4 `Filter`; `transfer`

```
Filter(material, density_g_cm3, thickness_um, label=""); transfer(energies_kev)
```

A slab (window, filter, air path) with Beer–Lambert transmission $T(E) = \exp[-(\mu/\rho)(E)\,\rho\,t]$, using the total mass attenuation coefficient of the compound (`CS_Total_CP`: photoabsorption plus

coherent and incoherent scattering, since any interaction removes the photon from the beam). Energies without data are fully absorbed.

xrf_beamline.py, lines 314–331

```
@dataclass
class Filter:
    """Transmission through a slab of material (window, filter, air path).

    Parameters
    ----------
    material : str
        Compound formula, e.g. ``"Be"``, ``"Al"``, ``"C"``, ``"Kapton"`` as
        ``"C22H10N2O5"``.
    density_g_cm3 : float
    thickness_um : float
    label : str
    """

    material: str
    density_g_cm3: float
    thickness_um: float
    label: str = ""
```

xrf_beamline.py, lines 333–334

```
    def __post_init__(self):
        self.label = self.label or f"{self.thickness_um:g} um {self.material}"
```

xrf_beamline.py, lines 336–346

```
    def transfer(self, energies_kev: np.ndarray) -> np.ndarray:
        """Transmission in [0, 1], from the Beer-Lambert law."""
        e = np.atleast_1d(np.asarray(energies_kev, dtype=float))
        t_cm = self.thickness_um * 1e-4
        mu = np.zeros_like(e)
        for i, ek in enumerate(e):
            try:
                mu[i] = xraylib.CS_Total_CP(self.material, ek) * self.density_g_cm3
            except ValueError:
                mu[i] = np.inf
        return np.exp(-mu * t_cm)
```

3.3.5 `Slit`; `transfer`

`Slit(distance_m, vertical_gap_mm=None, horizontal_gap_mm=None, label="")`; `transfer`

A pair of slits: an aperture without spectral effect (`transfer` returns ones). A gap of `None` is open.

xrf_beamline.py, lines 349–365

```
@dataclass
class Slit:
    """A pair of slits: an aperture without spectral effect.

    Parameters
    ----------
    distance_m : float
        Distance from the source.
    vertical_gap_mm, horizontal_gap_mm : float, optional
        Gaps; ``None`` means open (no limit) in that plane.
    label : str
    """

    distance_m: float
    vertical_gap_mm: Optional[float] = None
    horizontal_gap_mm: Optional[float] = None
    label: str = ""
```

xrf_beamline.py, lines 367–370

```
367     def __post_init__(self):
368         v = "open" if self.vertical_gap_mm is None else f"{self.vertical_gap_mm:g}"
369         h = "open" if self.horizontal_gap_mm is None else f"{self.horizontal_gap_mm:g}"
370         self.label = self.label or f"slit @ {self.distance_m:g} m ({v} x {h} mm)"
```

xrf_beamline.py, lines 372–373

```
372     def transfer(self, energies_kev: np.ndarray) -> np.ndarray:
373         return np.ones_like(np.atleast_1d(np.asarray(energies_kev, dtype=float)))
```

3.4 Monochromatic sources and units

3.4.1 `tube_lines`

```
tube_lines(anode, lines=("KL3", "KL2"))
```

Energies (eV) and relative intensities of X-ray tube anode lines, by default the Kα_1/ Kα_2 doublet that a crystal or multilayer monochromator passes (`tube_lines("Mo")`: 17 479 and 17 374 eV). The intensities are the radiative rates (`RadRate`), normalised to sum 1; this is correct for lines that share the same initial vacancy, such as Kα_1 and Kα_2. The constant `OpticalElement`, the union type of the optics, precedes it in the file.

Hypotheses and limits

The monochromator passes both lines with the same efficiency and suppresses Kβ and the bremsstrahlung completely.

xrf_beamline.py, lines 376–389

```
376 OpticalElement = Union[Mirror, Filter, Slit]
377
378
379 def tube_lines(anode: str, lines: Sequence[str] = ("KL3", "KL2")) -> Tuple[np.ndarray, np.ndarray]:
380     """Energies (eV) and relative intensities of X-ray tube anode lines.
381
382     The default is the K-alpha1/K-alpha2 doublet that a crystal or multilayer
383     monochromator passes from a tube (e.g. ``tube_lines("Mo")``: 17479 and
384     17374 eV).  Intensities are the radiative rates (they share the K vacancy).
385     """
386     z = xraylib.SymbolToAtomicNumber(anode)
387     e = np.array([xraylib.LineEnergy(z, getattr(xraylib, f"{n}_LINE")) * 1000.0 for n in lines])
388     w = np.array([xraylib.RadRate(z, getattr(xraylib, f"{n}_LINE")) for n in lines])
389     return e, w / w.sum()
```

3.4.2 `monochromatic_spectrum`

```
monochromatic_spectrum(energies_ev, weights=(1.0,), width_ev=5.0, step_ev=1.0)
```

Builds a narrow incident spectrum from Gaussian lines of standard deviation `width_ev`, for the functions that expect an incident spectrum (`element_line_groups`, `MatrixModel`). The density per eV, normalised so that each line has area w_i, is converted to the per-0.1 % bandwidth convention, $N_{0.1\%} = 10^{-3} E\,\mathrm{d}N/\mathrm{d}E$.

Returns `(energy_kev, flux_per_0.1%BW)` on a grid from 8σ below the lowest to 8σ above the highest line.

Hypotheses and limits

The width only has to be resolved by the grid; its value does not matter as long as the cross-sections vary slowly across it (true away from absorption edges). With `MatrixModel`, pass `sf_step_kev = step_ev/1000`: its secondary-fluorescence grid is decimated to 40 eV by default, which would miss a 5 eV-wide line.

xrf_beamline.py, lines 392–411

```
392 def monochromatic_spectrum(energies_ev: Sequence[float], weights: Sequence[float] = (1.0,),
393                           width_ev: float = 5.0, step_ev: float = 1.0
394                           ) -> Tuple[np.ndarray, np.ndarray]:
395     """A narrow incident spectrum ``(energy_kev, flux per 0.1 % BW)`` made of
396     Gaussian lines, for the functions that take an incident spectrum
397     (:func:`xrf_fit.element_line_groups`, :class:`xrf_matrix.MatrixModel`).
398
399     ``width_ev`` is the Gaussian sigma of each line; it only needs to be
400     resolved by the grid (``step_ev``), its value does not matter for
401     cross-sections that vary slowly across it.  With ``MatrixModel``, pass
402     ``sf_step_kev=step_ev / 1000`` (its secondary-fluorescence grid is
403     decimated to 40 eV by default, which would miss a narrow line).
404     """
405     e = np.asarray(energies_ev, dtype=float)
406     w = np.broadcast_to(np.asarray(weights, dtype=float), e.shape)
407     grid = np.arange(e.min() - 8 * width_ev, e.max() + 8 * width_ev + step_ev / 2, step_ev)
408     density = sum(wi * np.exp(-0.5 * ((grid - ei) / width_ev) ** 2) for ei, wi in zip(e, w))
409     density = density / (np.sqrt(2 * np.pi) * width_ev)          # photons per eV
410     # per 0.1 % bandwidth = per (1e-3 E): the convention of Beamline.spectrum
411     return grid / 1000.0, density * 1e-3 * grid
```

3.4.3 `per_unit_energy`

```
per_unit_energy(energies_kev, flux_01bw)
```

Eq. (1): per-0.1 % bandwidth flux to a density per keV. Integrals over dE (flux-weighted cross-sections) must use this form.

xrf_beamline.py, lines 414–421

```
414 def per_unit_energy(energies_kev: np.ndarray, flux_01bw: np.ndarray) -> np.ndarray:
415     """Convert a per-0.1 %-bandwidth flux into a per-keV spectral density.
416
417     ``dN/dE = N_0.1%BW / (1e-3 * E)``: a fixed *relative* bandwidth spans an
418     energy interval proportional to E.  Integrals over ``dE`` need this form.
419     """
420     e = np.asarray(energies_kev, dtype=float)
421     return np.asarray(flux_01bw, dtype=float) / (1e-3 * np.maximum(e, 1e-12))
```

3.5 Air

3.5.1 `AIR_FORMULA`, `air_density`

```
AIR_FORMULA, air_density(temperature_c=20.0, pressure_hpa=1013.25)
```

Dry air is written as an atom-count formula from its volume composition (N_2 78.084 %, O_2 20.946 %, Ar 0.934 %, CO_2 0.0417 %): $N_{156.168}O_{41.975}Ar_{0.934}C_{0.0417}$ per 100 molecules, which reproduces the mass fractions N 75.5 %, O 23.2 %, Ar 1.29 %. Its density follows the ideal-gas law,

$$\rho = \frac{P\,M}{R\,T}, \qquad M = 28.9647 \text{ g/mol}, \tag{6}$$

returned in g/cm^3 (1.204 mg/cm^3 at 20 °C and 1013.25 hPa).

Hypotheses and limits

Humidity is neglected. Moist air is slightly lighter than dry air, so at 50 % relative humidity and 20 °C this overestimates density and absorption by a few parts per thousand [13]. The pressure is the local value: about 990 hPa at the ESRF, 2.5 % below the sea-level standard.

xrf_beamline.py, lines 424–454

```
424 # ============================================================================
425 # Air
426 # ============================================================================
427
428 #: Dry air as an atom-count formula, from the standard volume composition
429 #: (N2 78.084 %, O2 20.946 %, Ar 0.934 %, CO2 0.0417 %) converted to atoms per
430 #: 100 molecules.  Reproduces the accepted mass fractions
431 #: N 75.5 %, O 23.2 %, Ar 1.29 %, C 0.017 %.
432 AIR_FORMULA = "N156.168O41.975Ar0.934C0.0417"
433
434 #: Molar mass of dry air, g/mol.
435 AIR_MOLAR_MASS = 28.9647
436
437 #: Molar gas constant, J/(mol K).
438 GAS_CONSTANT = 8.31446261815324
439
440
441 def air_density(temperature_c: float = 20.0, pressure_hpa: float = 1013.25) -> float:
442     """Dry-air density in g/cm^3 from the ideal gas law.
443
444     ``rho = P M / (R T)``.  Humidity is neglected: moist air is slightly *less*
445     dense than dry air (water is lighter than N2/O2), so at 50 % relative
446     humidity and 20 C this overestimates the density, and hence the absorption,
447     by a few parts per thousand.
448
449     Note that pressure is the *local* value, not sea-level corrected: at the
450     ESRF (~210 m) the ambient pressure is nearer 990 hPa than 1013 hPa, which is
451     a 2.5 % change in density and therefore in optical depth.
452     """
453     return (pressure_hpa * 100.0) * (AIR_MOLAR_MASS * 1e-3) / (
454         GAS_CONSTANT * (temperature_c + 273.15)) * 1e-3
```

3.5.2 `air_path`

```
air_path(length_m, temperature_c=20.0, pressure_hpa=1013.25)
```

A `Filter` of air of the given length. A long air path is the strongest low-energy filter of a white-beam line: 3 m transmit about 10^{-7} at 5 keV, 1.7 % at 8 keV and 12 % at 10 keV.

xrf_beamline.py, lines 457–476

```
457 def air_path(length_m: float, temperature_c: float = 20.0,
458              pressure_hpa: float = 1013.25) -> Filter:
459     """A :class:`Filter` representing a path through ambient air.
460
461     A long air path is a strong low-energy filter and cannot be neglected on a
462     white-beam line: 3 m of air at ambient conditions transmits ~1e-7 at 5 keV,
463     1.7 % at 8 keV and 12 % at 10 keV, so it sets the practical lower bound of
464     the usable band.
465
466     Parameters
467     ----------
468     length_m : float
469         Path length in metres.
470     temperature_c, pressure_hpa : float
471         Ambient conditions, used to compute the density.
472     """
473     rho = air_density(temperature_c, pressure_hpa)
474     return Filter(material=AIR_FORMULA, density_g_cm3=rho,
475                   thickness_um=length_m * 1e6,
476                   label=f"{length_m:g} m air ({temperature_c:g} C, {pressure_hpa:g} hPa)")
```

3.6 The assembled beamline

3.6.1 `Beamline`

```
Beamline(source, optics=[], sample_distance_m=None, vertical_acceptance="auto",
vertical_offset_urad=0.0, settings={})
```

A source and a list of optical elements. The elements with a position act as apertures (slits through their gaps, mirrors through their footprint); `acceptance` maps each of them back to an angle at the source, and the smallest vertical one sets the window of source angles whose spectrum is used.

`source`
: a `BendingMagnetSource`.

`optics`
: mirrors, filters and slits. Positions come from `distance_m`; the order of the list only sets the order of labels and transfers.

`sample_distance_m`
: source–sample distance (for `trace` and `describe`).

`vertical_acceptance`
: `"auto"` (from the apertures), `"full"` (vertically integrated spectrum, Eq. (3)) or a number (full acceptance in µrad).

`vertical_offset_urad`
: centre of the accepted window relative to the orbit plane.

`settings`
: free-form provenance (motor positions read from a scan).

xrf_beamline.py, lines 479–520

```
479 # ============================================================================
480 # Assembled beamline
481 # ============================================================================
482
483 @dataclass
484 class Beamline:
485     """A source plus an ordered list of optical elements.
486
487     Elements with a position (:class:`Slit`, :class:`Mirror` with
488     ``distance_m``) also act as apertures.  :meth:`acceptance` maps each of them
489     back to an angle at the source, through the focusing action of the bent
490     mirrors upstream (paraxial optics), and the smallest one in the vertical
491     plane sets the window of source angles whose spectrum is used.
492
493     Parameters
494     ----------
495     source : BendingMagnetSource
496     optics : list
497         Mirrors, filters and slits.  The order matters only for labelling;
498         positions are taken from ``distance_m``.
499     sample_distance_m : float, optional
500         Source-to-sample distance (for :meth:`trace` and :meth:`describe`).
501     vertical_acceptance : "auto", "full" or float
502         ``"auto"``: from the apertures (full opening angle if there are none);
503         ``"full"``: vertically integrated spectrum (``G1``), the pre-2026
504         behaviour; a number: full vertical acceptance in urad.
505     vertical_offset_urad : float
506         Centre of the accepted window relative to the orbit plane (0 when the
507         apertures are aligned on the beam).
508     settings : dict
509         Free-form provenance (e.g. the motor positions read by
510         :func:`bm32_from_h5`).
511     """
512
513     source: BendingMagnetSource
```

```
514     optics: List[OpticalElement] = field(default_factory=list)
515     sample_distance_m: Optional[float] = None
516     vertical_acceptance: Union[str, float] = "auto"
517     vertical_offset_urad: float = 0.0
518     settings: Dict[str, object] = field(default_factory=dict)
519
520     # -- geometry ------------------------------------------------------------
```

3.6.2 `_positioned`, `trace`

`_positioned()`, `trace(plane="vertical")`

Paraxial ray tracing. A ray leaves the source on axis with unit angle; at each positioned element its height h (per radian of source angle, in m/rad) is accumulated, and every mirror bent in `plane` changes the ray angle by $-h/f$ (thin concave lens). Returns `[(element, h), ...]` in order of distance, plus `(None, h_sample)` if the sample distance is set. The height per unit source angle is what converts an aperture into an angular acceptance at the source.

xrf_beamline.py, lines 521–523

```
521     def _positioned(self) -> List[OpticalElement]:
522         return sorted((o for o in self.optics if getattr(o, "distance_m", None) is not None),
523                       key=lambda o: o.distance_m)
```

xrf_beamline.py, lines 525–543

```
525     def trace(self, plane: str = "vertical") -> List[Tuple[Optional[OpticalElement], float]]:
526         """Paraxial height per unit source angle (m/rad) at each positioned element.
527
528         A ray leaving the source on axis with unit angle is propagated; every
529         mirror deflecting in ``plane`` with a finite bending radius reduces the
530         angle by ``h / f`` (concave mirror, ``f = R sin(theta) / 2``).  The last
531         entry, labelled ``None``, is the sample if ``sample_distance_m`` is set.
532         """
533         h, a, z = 0.0, 1.0, 0.0
534         out: List[Tuple[Optional[OpticalElement], float]] = []
535         for o in self._positioned():
536             h += a * (o.distance_m - z)
537             z = o.distance_m
538             out.append((o, h))
539             if isinstance(o, Mirror) and o.plane == plane and np.isfinite(o.focal_length_m):
540                 a -= h / o.focal_length_m
541         if self.sample_distance_m is not None:
542             out.append((None, h + a * (self.sample_distance_m - z)))
543         return out
```

3.6.3 `acceptance`

`acceptance()`

For each plane and each aperture, the full angular acceptance at the source in µrad, $\Delta\psi = s/h$ with s the gap or the mirror's $L\sin\theta$. Returns `{"vertical": {limits, acceptance_urad, limited_by}, "horizontal": {...}}`, with infinite acceptance when nothing limits a plane.

xrf_beamline.py, lines 545–573

```
545     def acceptance(self) -> Dict[str, dict]:
546         """Full angular acceptance at the source (urad) of every aperture, per plane.
547
548         Returns ``{"vertical": {...}, "horizontal": {...}}`` with ``limits``
549         ({label: urad}), ``acceptance_urad`` (the smallest) and ``limited_by``.
550         Slits act through their gaps, mirrors through ``length sin(theta)`` in
551         their deflection plane.
552         """
553         res = {}
554         for plane in ("vertical", "horizontal"):
555             lim: Dict[str, float] = {}
```

```
            for o, m in self.trace(plane):
                if o is None or m <= 0:
                    continue
                size = None
                if isinstance(o, Slit):
                    gap = o.vertical_gap_mm if plane == "vertical" else o.horizontal_gap_mm
                    size = None if gap is None else gap * 1e-3
                elif isinstance(o, Mirror) and o.plane == plane:
                    size = o.aperture_m
                if size is not None:
                    key = o.label if o.label not in lim else f"{o.label} @ {o.distance_m:g} m"
                    lim[key] = size / m * 1e6
            if lim:
                k = min(lim, key=lim.get)
                res[plane] = dict(limits=lim, acceptance_urad=lim[k], limited_by=k)
            else:
                res[plane] = dict(limits={}, acceptance_urad=math.inf, limited_by=None)
        return res
```

3.6.4 `effective_vertical_acceptance`

```
effective_vertical_acceptance()
```

The acceptance actually used for the spectrum: `None` for `"full"` or when no aperture limits the beam, the computed value for `"auto"`, or the number given.

xrf_beamline.py, lines 575–584

```
    def effective_vertical_acceptance(self) -> Optional[float]:
        """Full vertical acceptance used for the spectrum (urad), ``None`` = full."""
        if isinstance(self.vertical_acceptance, str):
            if self.vertical_acceptance == "full":
                return None
            if self.vertical_acceptance != "auto":
                raise ValueError("vertical_acceptance must be 'auto', 'full' or a number")
            a = self.acceptance()["vertical"]["acceptance_urad"]
            return None if not np.isfinite(a) else float(a)
        return float(self.vertical_acceptance)
```

3.6.5 `resolved_optics`

```
resolved_optics()
```

Returns the optics with the grazing-angle spread of each focusing mirror filled in. An elliptical mirror receives nearly parallel rays from the distant source and sends them converging to the focus at distance q; across an illuminated beam of size s the outgoing direction varies by s/q, so the grazing angle varies by

$$\Delta\theta = \pm\frac{s}{4q} \tag{7}$$

[14]. s is the accepted angle in the mirror's plane times the height per angle at the mirror, capped by the mirror's own aperture. The reflectivity is then averaged over this range, which smooths the high-energy cut-off.

xrf_beamline.py, lines 586–608

```
    def resolved_optics(self) -> List[OpticalElement]:
        """The optics with the KB angle spreads filled in.

        For a positioned mirror with ``image_distance_m`` and no explicit
        ``angle_spread_mrad``, the
        illuminated beam size ``s`` is the accepted angle in the mirror's plane
        times the height-per-angle at the mirror, or the mirror aperture if
        nothing upstream limits it, and the spread is ``s / (4 q)``.
        """
        acc = self.acceptance()
        traces = {p: {id(o): m for o, m in self.trace(p) if o is not None} for p in ("vertical", "horizontal")}
```

```
        out = []
        for o in self.optics:
            if isinstance(o, Mirror) and o.angle_spread_mrad == 0.0 and o.distance_m is not None:
                q = o.image_distance_m            # only mirrors declared as focusing get a spread
                if q is not None and q > 0:
                    a = acc[o.plane]["acceptance_urad"]
                    size = a * 1e-6 * traces[o.plane][id(o)] if np.isfinite(a) else (o.aperture_m or 0.0)
                    if o.aperture_m is not None:
                        size = min(size, o.aperture_m)
                    o = replace(o, angle_spread_mrad=1e3 * size / (4.0 * q))
            out.append(o)
        return out
```

3.6.6 `describe`

`describe()`

A dictionary summarising the geometry: vertical and horizontal acceptances and the aperture limiting each, every aperture's limit, the KB angle spreads, the height per µrad at each element, the acceptance and offset used.

xrf_beamline.py, lines 610–625

```
    def describe(self) -> Dict[str, object]:
        """Summary: acceptances, limiting apertures, KB spreads, apparent source."""
        acc = self.acceptance()
        tr = self.trace("vertical")
        spreads = {o.label: o.angle_spread_mrad for o in self.resolved_optics()
                   if isinstance(o, Mirror) and o.angle_spread_mrad > 0}
        return dict(vertical_acceptance_urad=acc["vertical"]["acceptance_urad"],
                    vertical_limited_by=acc["vertical"]["limited_by"],
                    horizontal_acceptance_urad=acc["horizontal"]["acceptance_urad"],
                    horizontal_limited_by=acc["horizontal"]["limited_by"],
                    vertical_limits_urad=acc["vertical"]["limits"],
                    horizontal_limits_urad=acc["horizontal"]["limits"],
                    kb_angle_spread_mrad=spreads,
                    vertical_height_per_urad_um={(o.label if o is not None else "sample"): m for o, m in tr},
                    used_vertical_acceptance_urad=self.effective_vertical_acceptance(),
                    vertical_offset_urad=self.vertical_offset_urad)
```

3.6.7 `spectrum`, `spectral_density`, `band_limits`

`spectrum(energies_kev)`, `spectral_density(energies_kev)`, `band_limits(energies_kev, frac=0.01)`

`spectrum` returns `(raw, transmitted, transfers)`: the accepted source flux, the flux at the sample (the product of every mirror and filter transfer), and the list of `(label, transfer)`, all per mrad and per 0.1 % BW. `spectral_density` converts the transmitted flux to photons/s/mrad/keV. `band_limits` gives the energies where the transmitted flux first and last exceeds `frac` of its maximum—a compact statement of the usable band (about 5–25 keV on BM32).

xrf_beamline.py, lines 628–652

```
    def spectrum(self, energies_kev: np.ndarray) -> Tuple[np.ndarray, np.ndarray, List[Tuple[str, np.ndarray]]]:
        """Propagate the accepted source spectrum through the optics.

        Returns
        -------
        raw : (n,) array
            Source flux inside the accepted vertical window, per mrad
            (horizontal) per 0.1 % BW, before any optic.
        transmitted : (n,) array
            Flux at the sample.
        transfers : list of (label, array)
            Each mirror's or filter's transfer function, in order (slits
            have none).
        """
        e = np.asarray(energies_kev, dtype=float)
        raw = self.source.accepted_flux(e, self.effective_vertical_acceptance(), self.vertical_offset_urad)
```

```
644         transmitted = raw.copy()
645         transfers = []
646         for optic in self.resolved_optics():
647             if isinstance(optic, Slit):
648                 continue
649             t = optic.transfer(e)
650             transfers.append((optic.label, t))
651             transmitted = transmitted * t
652         return raw, transmitted, transfers
```

xrf_beamline.py, lines 654–661

```
654     def spectral_density(self, energies_kev: np.ndarray) -> np.ndarray:
655         """Flux at the sample per unit energy, photons/s/mrad/keV.
656 
657         The quantity to integrate over ``dE``; see :func:`per_unit_energy`.
658         """
659         e = np.asarray(energies_kev, dtype=float)
660         _, transmitted, _ = self.spectrum(e)
661         return per_unit_energy(e, transmitted)
```

xrf_beamline.py, lines 663–673

```
663     def band_limits(self, energies_kev: np.ndarray, frac: float = 0.01) -> Tuple[float, float]:
664         """Energies where the transmitted flux first and last exceed ``frac`` of its peak.
665 
666         A compact way to state the usable white-beam band.
667         """
668         e = np.asarray(energies_kev, dtype=float)
669         _, phi, _ = self.spectrum(e)
670         above = np.flatnonzero(phi >= frac * phi.max())
671         if above.size == 0:
672             raise ValueError("Transmitted flux never reaches the requested fraction of its peak.")
673         return float(e[above[0]]), float(e[above[-1]])
```

3.7 BM32

3.7.1 `BM32_LAYOUT, BM32_WHITE_BEAM`

```
BM32_LAYOUT, BM32_WHITE_BEAM(air_path_m=3.3, temperature_c=20.0, pressure_hpa=1013.25,
current_a=0.2, magnetic_field_t=0.86, *, vg1=0.3, hg1=0.2, vg2=None, hg2=None, vg3=0.2,
hg3=0.2, m1_angle_mrad=3.062, m2_angle_mrad=2.551, m1_radius_km=71.4, m2_radius_km=145.7,
kbv_angle_mrad=3.0, kbh_angle_mrad=3.0, kb_angle_spread=True, vertical_acceptance="auto",
vertical_offset_urad=0.0)
```

The BM32 white-beam model. `BM32_LAYOUT` holds the distances from the source (m) of the primary slits s1 (26.368), the bent Ir mirrors m1 (28.309) and m2 (31.732), the guard slits s2 (35.370), the slits s3 that define the KB illumination (44.485), the KB mirrors (vertical 44.645, horizontal 44.880) and the sample (45.0), and the mirror lengths. The function builds: s1; m1 and m2 (Ir, 22.56 g/cm^3, bent); two 500 µm Be windows; s2; s3; KB-V and KB-H (Ir, focusing, with angle spread if `kb_angle_spread`); and the incident air path.

`air_path_m`
: incident air path (3.3 m measured on BM32); 0 removes it. Not to be confused with the sample–detector path of the detector model.

`vg1 … hg3`
: slit gaps (mm), motors of the same names; `None` = open.

`m1_angle_mrad, m2_angle_mrad, m1_radius_km, m2_radius_km`
: mirror angles (motors `ma1`, `ma2`) and bending radii (`mc1`, `mc2`).

`kbv_angle_mrad, kbh_angle_mrad`
: KB grazing angles, not recorded in the scan files.

`others`

as in `Beamline` and `BendingMagnetSource`.

With the default slits the vertical acceptance is a few µrad, set by s3: the sample sees the on-axis spectrum, which raised the Ga/As sensitivity ratio by 4.5 % on the 2026 maps compared with the vertically integrated spectrum.

xrf_beamline.py, lines 676–772

```
676 # ============================================================================
677 # BM32
678 # ============================================================================
679
680 #: BM32 layout: distances from the source (m) and optical lengths (m), as
681 #: given by the beamline (Oct 2026).  s1 primary slits; m1, m2 bent Ir mirrors
682 #: deflecting vertically; s2 guard slits against scattered radiation; s3 slits
683 #: defining the KB illumination; KB-V (vertical) and KB-H (horizontal)
684 #: elliptical Ir mirrors; sample.
685 BM32_LAYOUT: Dict[str, float] = dict(
686     s1=26.368, m1=28.309, m2=31.732, s2=35.370, s3=44.485, kbv=44.645, kbh=44.880, sample=45.0,
687     m1_length=1.2, m2_length=1.2, kbv_length=0.30, kbh_length=0.15,
688 )
689
690
691 def BM32_WHITE_BEAM(air_path_m: float = 3.3, temperature_c: float = 20.0,
692                     pressure_hpa: float = 1013.25, current_a: float = 0.2,
693                     magnetic_field_t: float = 0.86, *,
694                     vg1: Optional[float] = 0.3, hg1: Optional[float] = 0.2,
695                     vg2: Optional[float] = None, hg2: Optional[float] = None,
696                     vg3: Optional[float] = 0.2, hg3: Optional[float] = 0.2,
697                     m1_angle_mrad: float = 3.062, m2_angle_mrad: float = 2.551,
698                     m1_radius_km: float = 71.4, m2_radius_km: float = 145.7,
699                     kbv_angle_mrad: float = 3.0, kbh_angle_mrad: float = 3.0,
700                     kb_angle_spread: bool = True,
701                     vertical_acceptance: Union[str, float] = "auto",
702                     vertical_offset_urad: float = 0.0) -> Beamline:
703     """BM32 white-beam model (see module caveats).
704
705     The defaults are representative of the 2026 Laue maps; use
706     :func:`bm32_from_h5` to take the slit gaps, mirror angles, bending radii
707     and ring current of a given scan.  The spectrum is insensitive to the exact
708     gaps as long as the vertical acceptance stays well below ~30 urad.
709
710     Parameters
711     ----------
712     air_path_m : float
713         Length of the **incident** air path, in metres (3.3 m measured on
714         BM32).  Set to 0 to remove it.
715         This is the dominant low-energy filter on the line and it is what sets
716         the ~5 keV lower bound of the usable band; the 1 mm of Be alone would
717         pass energies well below that.
718
719         Do not confuse it with the **detection** air path (sample to detector
720         window), which attenuates the *emitted* lines instead and belongs to a
721         separate detector-response model.
722     temperature_c, pressure_hpa : float
723         Ambient conditions for the air density.
724     current_a : float
725         Ring current in amperes (scales the absolute flux only).
726     magnetic_field_t : float
727         Source field.  Sets the critical energy.
728     vg1, hg1, vg2, hg2, vg3, hg3 : float or None
729         Vertical / horizontal gaps of slits s1, s2, s3 in mm (motors of the
730         same names); ``None`` = open.
731     m1_angle_mrad, m2_angle_mrad : float
732         Grazing angles of m1, m2 (motors ``ma1``, ``ma2``).
733     m1_radius_km, m2_radius_km : float
734         Bending radii of m1, m2 (motors ``mc1``, ``mc2``); ``inf`` for flat.
735     kbv_angle_mrad, kbh_angle_mrad : float
736         KB grazing angles (not recorded in the scan files; 3 mrad assumed).
737     kb_angle_spread : bool
738         Average the KB reflectivities over the grazing-angle spread of their
739         illuminated footprints.
740     vertical_acceptance, vertical_offset_urad
```

```
        See :class:`Beamline`.  ``"full"`` restores the vertically integrated
        (``G1``) spectrum.
    """
    L = BM32_LAYOUT
    q_v = L["sample"] - L["kbv"] if kb_angle_spread else None
    q_h = L["sample"] - L["kbh"] if kb_angle_spread else None
    optics: List[OpticalElement] = [
        Slit(L["s1"], vg1, hg1, label="s1"),
        Mirror("Ir", 22.56, m1_angle_mrad, label=f"m1 Ir @ {m1_angle_mrad:.3f} mrad", distance_m=L["m1"],
               length_m=L["m1_length"], bending_radius_m=m1_radius_km * 1e3),
        Mirror("Ir", 22.56, m2_angle_mrad, label=f"m2 Ir @ {m2_angle_mrad:.3f} mrad", distance_m=L["m2"],
               length_m=L["m2_length"], bending_radius_m=m2_radius_km * 1e3),
        Filter("Be", 1.85, 500.0),
        Filter("Be", 1.85, 500.0),
        Slit(L["s2"], vg2, hg2, label="s2"),
        Slit(L["s3"], vg3, hg3, label="s3"),
        Mirror("Ir", 22.56, kbv_angle_mrad, label=f"KB-V Ir @ {kbv_angle_mrad:.3f} mrad", distance_m=L["kbv"],
               length_m=L["kbv_length"], plane="vertical", image_distance_m=q_v),
        Mirror("Ir", 22.56, kbh_angle_mrad, label=f"KB-H Ir @ {kbh_angle_mrad:.3f} mrad", distance_m=L["kbh"],
               length_m=L["kbh_length"], plane="horizontal", image_distance_m=q_h),
    ]
    if air_path_m > 0:
        optics.append(air_path(air_path_m, temperature_c, pressure_hpa))
    return Beamline(
        source=BendingMagnetSource(ring_energy_gev=6.04, magnetic_field_t=magnetic_field_t, current_a=current_a),
        optics=optics, sample_distance_m=L["sample"],
        vertical_acceptance=vertical_acceptance, vertical_offset_urad=vertical_offset_urad,
        settings=dict(vg1=vg1, hg1=hg1, vg2=vg2, hg2=hg2, vg3=vg3, hg3=hg3, ma1=m1_angle_mrad,
                      ma2=m2_angle_mrad, mc1_km=m1_radius_km, mc2_km=m2_radius_km,
                      kbv_angle_mrad=kbv_angle_mrad, kbh_angle_mrad=kbh_angle_mrad,
                      current_a=current_a, air_path_m=air_path_m),
    )
```

3.7.2 `bm32_from_h5`

```
bm32_from_h5(h5_path, scan=None, **overrides)
```

The BM32 beamline with the settings recorded in a BLISS scan: slit gaps (`vg1 hg1 vg2 hg2 vg3 hg3`), mirror angles (`ma1 ma2`) and radii (`mc1 mc2`) from `<scan>/instrument/positioners`, and the ring current from `<scan>/instrument/machine/current` (mA, converted to A). Missing or non-numeric values (BLISS writes `ERR` for disconnected motors) keep their defaults with a warning; keyword arguments override anything. The source file and scan are stored in `settings`.

xrf_beamline.py, lines 775–820

```
#: BLISS positioner names read by :func:`bm32_from_h5`, with the keyword they set.
_BM32_MOTORS = dict(vg1="vg1", hg1="hg1", vg2="vg2", hg2="hg2", vg3="vg3", hg3="hg3",
                    ma1="m1_angle_mrad", ma2="m2_angle_mrad", mc1="m1_radius_km", mc2="m2_radius_km")


def bm32_from_h5(h5_path: Union[str, Path], scan: Optional[str] = None, **overrides) -> Beamline:
    """BM32 beamline with the settings recorded in a BLISS scan file.

    Reads the slit gaps (``vg1 hg1 vg2 hg2 vg3 hg3``, mm), the m1/m2 grazing
    angles (``ma1 ma2``, mrad) and bending radii (``mc1 mc2``, km) from
    ``<scan>/instrument/positioners`` and the ring current from
    ``<scan>/instrument/machine/current`` (mA).  Values that are missing or not
    numeric (BLISS writes ``ERR`` for disconnected motors) keep their defaults,
    with a warning.  Keyword arguments override anything, e.g. ``air_path_m``,
    ``kbv_angle_mrad`` or ``vertical_acceptance``.

    Parameters
    ----------
    h5_path : path
    scan : str, optional
        Scan entry, e.g. ``"1.1"``; the first entry by default.
    """
    import h5py

    kw: Dict[str, object] = {}
    with h5py.File(h5_path, "r") as f:
```

```
        entry = scan or sorted(f.keys())[0]
        ins = f[entry]["instrument"]
        pos = ins["positioners"] if "positioners" in ins else {}
        for motor, key in _BM32_MOTORS.items():
            if motor not in pos:
                logger.warning("bm32_from_h5: positioner %r not found; default kept.", motor)
                continue
            try:
                kw[key] = float(np.atleast_1d(pos[motor][()])[0])
            except (TypeError, ValueError):
                logger.warning("bm32_from_h5: positioner %r is not numeric; default kept.", motor)
        try:
            kw["current_a"] = float(ins["machine"]["current"][()]) * 1e-3
        except (KeyError, TypeError, ValueError):
            logger.warning("bm32_from_h5: ring current not found; default kept.")
    kw.update(overrides)
    bl = BM32_WHITE_BEAM(**kw)
    bl.settings["source_file"] = str(h5_path)
    bl.settings["scan"] = entry
    return bl
```

4 xrf_matrix: matrix effects in layered samples

4.1 Overview and physics

The fluorescence intensity of an element is proportional to its concentration only in an infinitely thin sample. In a real sample the incident beam is attenuated on its way in, the fluorescence on its way out (*primary absorption*), and the fluorescence of one element can excite another (*secondary fluorescence*). The module computes both for a stack of homogeneous layers—cap, film, substrate—of arbitrary thickness, the last one possibly infinitely thick, under polychromatic excitation. It follows the fundamental-parameter approach of Sherman [15] and Shiraiwa and Fujino [16], extended to multilayers [17, 18]; see also the handbook of Van Grieken and Markowicz [19].

Notation. Layers are numbered from the surface, $k = 0, 1, \ldots$; layer k has density ρ_k, thickness t_k and mass thickness $M_k = \rho_k t_k$ (infinite for a bulk last layer). The beam meets the surface at the angle ψ_{in}, the detector sees it at the take-off angle ψ_{out}, both measured from the surface. $\mu_k(E)$ is the total mass attenuation coefficient of layer k, $Q_{il}(E)$ the partial fluorescence cross-section of line l of element i (xraylib `CS_FluorLine_Kissel`, cm^2/g: photo-ionisation of the subshell, Coster–Kronig transfers, fluorescence yield and radiative rate), w_i^k the mass fraction of i in layer k, and $\mathrm{d}\Phi/\mathrm{d}E$ the incident density per keV. Write

$$a_k(E) = \frac{\mu_k(E)}{\sin\psi_{\mathrm{in}}}, \qquad b_k = \frac{\mu_k(E_l)}{\sin\psi_{\mathrm{out}}}, \qquad F(c;M) = \int_0^M \mathrm{e}^{-cm}\mathrm{d}m = \frac{1-\mathrm{e}^{-cM}}{c}, \qquad G(x,y;M) = \int_0^M \mathrm{e}^{-xm}\mathrm{e}^{-y(M} \tag{8}$$

Primary fluorescence. A photon of energy E reaching mass depth m of layer k has crossed $m/\sin\psi_{\mathrm{in}}$ of it; the probability that it produces line l of element i in the slab $\mathrm{d}m$ is $w_i^k Q_{il}(E)\,\mathrm{d}m/\sin\psi_{\mathrm{in}}$, and the fluorescence photon reaching the detector has crossed $m/\sin\psi_{\mathrm{out}}$. Integrating over the depth and over the spectrum, and leaving out the solid angle and the detection efficiency (common to all lines and applied by `xrf_fit.Detector.efficiency`), the sensitivity per unit mass fraction is

$$P_{il}^k = T_{\mathrm{out}}^k \int \frac{\mathrm{d}\Phi}{\mathrm{d}E}\,\frac{Q_{il}(E)}{\sin\psi_{\mathrm{in}}}\,T_{\mathrm{in}}^k(E)\,F(a_k(E)+b_k;\,M_k)\,\mathrm{d}E, \tag{9}$$

with the overlayer transmissions $T_{\mathrm{in}}^k(E) = \exp(-\sum_{c<k} a_c M_c)$ and $T_{\mathrm{out}}^k = \exp(-\sum_{c<k} b_c M_c)$. In the thin limit $F \to M_k$ (no absorption); for a bulk layer $F = 1/(a_k + b_k)$, Sherman's equation.

Secondary fluorescence. Line j (energy E_j) of an element of layer a, produced by the beam at depth m', is emitted isotropically. From a plane source the fraction absorbed at a distance x (in mass thickness) per unit thickness is $\frac{1}{2}\int_0^1 \mu\, \mathrm{e}^{-\mu x/u}\, \mathrm{d}u/u$ with u the direction cosine; replacing μ by the partial cross-section $w_i^k Q_{il}(E_j)$ gives the rate of secondary emission of line l. With $\kappa_c = \mu_c(E_j)/u$ the depth integrals are closed-form for every u:

$$S_{il}^k = T_{\mathrm{out}}^k \sum_{a,j} \tfrac{1}{2}\, Q_{il}(E_j)\, w_j^a \int \frac{\mathrm{d}\Phi}{\mathrm{d}E}\, \frac{Q_j(E)}{\sin\psi_{\mathrm{in}}}\, T_{\mathrm{in}}^a(E) \int_0^1 K_{ak}(E,u)\, \frac{\mathrm{d}u}{u}\, \mathrm{d}E, \tag{10}$$

$$a = k: \quad K = D(a_k, b_k, \kappa_k; M_k) = \int_0^{M_k}\!\!\int_0^{M_k} \mathrm{e}^{-a_k m' - b_k m - \kappa_k |m-m'|}\, \mathrm{d}m\, \mathrm{d}m',$$

$$a < k: \quad K = G(a_a, \kappa_a; M_a)\, \mathrm{e}^{-\sum_{a<c<k} \kappa_c M_c}\, F(b_k + \kappa_k; M_k),$$

$$a > k: \quad K = F(a_a + \kappa_a; M_a)\, \mathrm{e}^{-\sum_{k<c<a} \kappa_c M_c}\, G(b_k, \kappa_k; M_k).$$

The same-layer double integral has the closed form

$$D(a,b,\kappa;M) = \frac{F(a+\kappa) - F(a+b)}{b-\kappa} + \frac{F(b+\kappa) - F(a+b)}{a-\kappa}, \tag{11}$$

(both terms written with $F(\cdot) = F(\cdot\,; M)$), obtained by splitting the domain at $m = m'$; when $b \to \kappa$ or $a \to \kappa$ the corresponding quotient becomes $-F'(a+\kappa)$ or $-F'(b+\kappa)$. For a single bulk layer Eq. (10) reduces to the classical $[\ln(1+a/\mu_j)/a + \ln(1+b/\mu_j)/b]\,/[2(a+b)]$ [16]; for a thin layer it gives the logarithmic thin-film enhancement. The integral over u is computed on a logarithmic grid ($\int_0^1 f\,\mathrm{d}u/u = \int f\,\mathrm{d}(\ln u)$), which resolves the grazing paths that dominate in thin films.

The measured intensity of line l of element i is then

$$I_{il} \propto \epsilon(E_l) \sum_k w_i^k \left(P_{il}^k + S_{il}^k\right). \tag{12}$$

Hypotheses and limits

- Homogeneous layers with flat, parallel, sharp interfaces and infinite lateral extent; no roughness, interdiffusion or lateral inhomogeneity.
- A single incidence and take-off angle (point-like beam footprint seen from a distant detector).
- No refraction and no total external reflection: at incidence angles comparable to the critical angle (a few mrad) the penetration of the beam is not the geometrical one assumed here.
- Secondary fluorescence from the lines of the stack's own elements only (contaminants do not excite), lines above 1 keV, and lines contributing at least `exciter_min_rel` of their element's strongest line. No tertiary fluorescence and no excitation by scattered radiation or by photo- or Auger electrons.
- The secondary term is smooth in E and is evaluated on a decimated energy grid.

xrf_matrix.py, lines 1–92

```
"""
Module: xrf_matrix.py

Matrix effects in X-ray fluorescence under polychromatic (white-beam)
excitation: primary absorption and secondary fluorescence in a stack of
homogeneous layers (e.g. cap / film / substrate), each of arbitrary thickness,
the last one possibly bulk.

Geometry and notation
---------------------
Layers are numbered from the surface, ``k = 0, 1, ...``; layer ``k`` has
density ``rho_k``, thickness ``t_k`` and mass thickness ``M_k = rho_k t_k``
(``inf`` for a bulk last layer).  The incident beam meets the surface at
``psi_in`` and the detector sees it at the take-off angle ``psi_out`` (both from
```

```
the surface).  ``m`` is mass depth *within* a layer, ``mu_k(E)`` the total mass
attenuation coefficient of layer ``k``, ``Q_il(E)`` the partial fluorescence
cross-section of line ``l`` of element ``i`` (``xraylib.CS_FluorLine_Kissel``,
cm^2/g, including subshell ionisation, Coster-Kronig transfer and fluorescence
yield) and ``dPhi/dE`` the incident spectral density per keV.  Write

    a_k(E) = mu_k(E) / sin(psi_in),      b_k = mu_k(E_l) / sin(psi_out),
    F(c; M) = (1 - exp(-c M)) / c,       G(x, y; M) = (exp(-y M) - exp(-x M)) / (x - y)

Primary fluorescence of line ``l`` from layer ``k``, per unit mass fraction of
``i`` in that layer::

    P_il^k = T_out^k * int dE dPhi/dE / sin(psi_in) * Q_il(E) * T_in^k(E) * F(a_k + b_k; M_k)

with the overlayer transmissions ``T_in^k = exp(-sum_{c<k} a_c M_c)`` and
``T_out^k = exp(-sum_{c<k} b_c M_c)``.

Secondary fluorescence: line ``j`` (energy ``E_j``) produced by the beam in
layer ``a`` is emitted isotropically and absorbed in layer ``k`` by element
``i``, which emits line ``l``.  With the isotropic point-source kernel
``(1/2) int_0^1 exp(-mu x / u) du/u`` (``u = cos theta``, ``k_c = mu_c(E_j)/u``)
the depth integrals are closed-form for every ``u``::

    S_il^k = T_out^k * sum_{a, j} (1/2) Q_il(E_j) w_j^a
             int dE dPhi/dE / sin(psi_in) * Q_j(E) * T_in^a(E) * int_0^1 K_ak du/u

    a == k :  K = D(a_k, b_k, k_k; M_k)
              = [F(a_k+k_k) - F(a_k+b_k)] / (b_k - k_k) + [F(b_k+k_k) - F(a_k+b_k)] / (a_k - k_k)
    a <  k :  K = G(a_a, k_a; M_a) * exp(-sum_{a<c<k} k_c M_c) * F(b_k + k_k; M_k)
    a >  k :  K = F(a_a + k_a; M_a) * exp(-sum_{k<c<a} k_c M_c) * G(b_k, k_k; M_k)

For a single bulk layer this reduces to the classical
``[ln(1 + a/mu_j)/a + ln(1 + b/mu_j)/b] / (2 (a + b))``; for a thin layer it
gives the logarithmic thin-film enhancement.  The ``u`` integral is done on a
logarithmic grid, which resolves the grazing paths that dominate in thin films.

Both terms share the solid-angle factor and the detection efficiency (applied
in ``xrf_fit.Detector.efficiency``), so the measured intensity of line ``l`` is

    I_il  ∝  eff(E_l) * sum_k w_i^k (P_il^k + S_il^k).

Not included: tertiary fluorescence, excitation by scattered radiation,
interface roughness or interdiffusion, lateral inhomogeneity.

Authors: Samuel Tardif, with Claude (Anthropic).
"""

from __future__ import annotations

import logging
import re
from dataclasses import dataclass
from typing import Dict, Iterator, List, Mapping, Optional, Sequence, Union

import numpy as np
import xraylib
from scipy.integrate import trapezoid

from xrf_beamline import per_unit_energy

logger = logging.getLogger(__name__)
xraylib.SetErrorMessages(0)

__all__ = [
    "Geometry",
    "Layer",
    "Stack",
    "MatrixModel",
    "quantify_layer",
    "atom_ratio",
]


# ============================================================================
# Cached xraylib look-ups on an energy grid
```

```
# ============================================================================

_CACHE: Dict[tuple, np.ndarray] = {}
```

4.2 Cached cross-sections

4.2.1 _grid_key, _cs_total, _cs_fluor, _cs_fluor_scalar

`_grid_key(e)`, `_cs_total(z, e_kev)`, `_cs_fluor(z, macro, e_kev)`, `_cs_fluor_scalar(z, macro, e_kev)`

xraylib is called one energy at a time, so the cross-sections on the spectrum's energy grid are computed once and cached, keyed by the element, the line and a hash of the grid. `_cs_fluor` returns zero below the edge (where xraylib raises `ValueError`) and clips negative values. `_IUPAC_LINES` holds the IUPAC line macros only (see Section 5.4 for why the Siegbahn names are excluded).

xrf_matrix.py, lines 95–96

```
def _grid_key(e: np.ndarray) -> tuple:
    return (e.size, hash(e.tobytes()))
```

xrf_matrix.py, lines 99–103

```
def _cs_total(z: int, e_kev: np.ndarray) -> np.ndarray:
    key = ("tot", z, _grid_key(e_kev))
    if key not in _CACHE:
        _CACHE[key] = np.array([xraylib.CS_Total(z, float(e)) for e in e_kev])
    return _CACHE[key]
```

xrf_matrix.py, lines 106–116

```
def _cs_fluor(z: int, macro: int, e_kev: np.ndarray) -> np.ndarray:
    key = ("fl", z, macro, _grid_key(e_kev))
    if key not in _CACHE:
        out = np.zeros(e_kev.size)
        for n, e in enumerate(e_kev):
            try:
                out[n] = max(xraylib.CS_FluorLine_Kissel(z, macro, float(e)), 0.0)
            except ValueError:
                pass
        _CACHE[key] = out
    return _CACHE[key]
```

xrf_matrix.py, lines 119–123

```
def _cs_fluor_scalar(z: int, macro: int, e_kev: float) -> float:
    try:
        return max(xraylib.CS_FluorLine_Kissel(z, macro, float(e_kev)), 0.0)
    except ValueError:
        return 0.0
```

4.3 Geometry and sample

4.3.1 Geometry

`Geometry(incidence_deg=40.0, takeoff_deg=40.0)`

Incidence and take-off angles in degrees from the surface, and their sines (properties `sin_in`, `sin_out`). Both default to 40°; set them to the geometry of the measurement, since the path lengths scale as $1/\sin\psi$.

xrf_matrix.py, lines 126–140

```
#: IUPAC line macros only (see xrf_fit for why Siegbahn names are excluded).
_IUPAC_RE = re.compile(r"^(K|L[1-3]|M[1-5])(K|L[1-3]|M[1-5]|N[1-7]|O[1-7]|P[1-5])_LINE$")
_IUPAC_LINES = {n: getattr(xraylib, n) for n in dir(xraylib) if _IUPAC_RE.match(n)}

```

```
130
131 # ============================================================================
132 # Geometry and sample
133 # ============================================================================
134
135 @dataclass(frozen=True)
136 class Geometry:
137     """Incidence and take-off angles, in degrees from the sample **surface**."""
138
139     incidence_deg: float = 40.0
140     takeoff_deg: float = 40.0
```

xrf_matrix.py, lines 142–144

```
142     @property
143     def sin_in(self) -> float:
144         return float(np.sin(np.radians(self.incidence_deg)))
```

xrf_matrix.py, lines 146–148

```
146     @property
147     def sin_out(self) -> float:
148         return float(np.sin(np.radians(self.takeoff_deg)))
```

4.3.2 `Layer`

```
Layer(composition, density_g_cm3, thickness_um=None, name="")
```

A homogeneous layer.

`composition`
: a formula (`"In0.5Ga0.5As"`) or `{symbol: atoms}`; only atomic proportions matter.

`density_g_cm3`
: density.

`thickness_um`
: thickness; `None` or `inf` for a bulk layer.

`name`
: label.

Attributes `formula`, `mass_fractions` (`{Z: w}`, from xraylib's `CompoundParser`), `atoms` (`{symbol: n}`), `density_g_cm3`, `thickness_um`. `mass_thickness` is ρt in g/cm^2 (infinite for bulk); `mu(e_kev)` is $\mu(E) = \sum_i w_i (\mu/\rho)_i(E)$, cm^2/g, with the cache for grids of more than 8 points; `with_composition` and `with_thickness` return modified copies.

xrf_matrix.py, lines 151–165

```
151 class Layer:
152     """A homogeneous layer.
153
154     Parameters
155     ----------
156     composition : str or mapping
157         A chemical formula (``"In0.5Ga0.5As"``) or ``{symbol: atoms}``
158         (``{"In": 0.5, "Ga": 0.5, "As": 1}``); only atomic *proportions* matter.
159     density_g_cm3 : float
160     thickness_um : float or None
161         ``None`` (or ``inf``) for a bulk sample; only the last layer of a
162         :class:`Stack` may be bulk.
163     name : str
164         Optional label ("cap", "film", "substrate").
165     """
```

xrf_matrix.py, lines 167–184

```
167     def __init__(self, composition: Union[str, Mapping[str, float]],
168                  density_g_cm3: float, thickness_um: Optional[float] = None,
```

```
                  name: str = ""):
        if isinstance(composition, str):
            formula = composition
        else:
            formula = "".join(f"{el}{n:.6g}" for el, n in composition.items() if n > 0)
        parsed = xraylib.CompoundParser(formula)
        self.formula = formula
        self.mass_fractions: Dict[int, float] = dict(zip(parsed["Elements"],
                                                         parsed["massFractions"]))
        atoms = dict(zip(parsed["Elements"], parsed["nAtoms"]))
        self.atoms: Dict[str, float] = {xraylib.AtomicNumberToSymbol(z): float(n)
                                        for z, n in atoms.items()}
        self.density_g_cm3 = float(density_g_cm3)
        self.thickness_um = (None if thickness_um is None or not np.isfinite(thickness_um)
                             else float(thickness_um))
        self.name = name
```

xrf_matrix.py, lines 186–191

```
    @property
    def mass_thickness(self) -> float:
        """``rho t`` in g/cm^2 (``inf`` for bulk)."""
        if self.thickness_um is None:
            return np.inf
        return self.density_g_cm3 * self.thickness_um * 1e-4
```

xrf_matrix.py, lines 193–201

```
    def mu(self, e_kev: np.ndarray | float) -> np.ndarray | float:
        """Total mass attenuation coefficient, cm^2/g."""
        scalar = np.ndim(e_kev) == 0
        e = np.atleast_1d(np.asarray(e_kev, dtype=float))
        out = np.zeros(e.size)
        for z, w in self.mass_fractions.items():
            out += w * (_cs_total(z, e) if e.size > 8 else
                        np.array([xraylib.CS_Total(z, float(x)) for x in e]))
        return float(out[0]) if scalar else out
```

xrf_matrix.py, lines 203–204

```
    def with_composition(self, composition: Union[str, Mapping[str, float]]) -> "Layer":
        return Layer(composition, self.density_g_cm3, self.thickness_um, self.name)
```

xrf_matrix.py, lines 206–207

```
    def with_thickness(self, thickness_um: Optional[float]) -> "Layer":
        return Layer(self.atoms, self.density_g_cm3, thickness_um, self.name)
```

xrf_matrix.py, lines 209–212

```
    def __repr__(self) -> str:
        t = "bulk" if self.thickness_um is None else f"{self.thickness_um:g} um"
        nm = f"{self.name}: " if self.name else ""
        return f"Layer({nm}{self.formula!r}, rho={self.density_g_cm3:g} g/cm3, {t})"
```

4.3.3 `Stack`

`Stack(layers)`

Ordered layers, surface first; only the last may be bulk (`ValueError` otherwise). Supports `len`, indexing, iteration, and `replace(index, layer)`, which returns a new stack.

xrf_matrix.py, lines 215–216

```
class Stack:
    """Ordered layers, **surface first**; only the last may be bulk."""
```

xrf_matrix.py, lines 218–225

```
    def __init__(self, layers: Sequence[Layer]):
```

```
        layers = list(layers)
        if not layers:
            raise ValueError("A stack needs at least one layer.")
        for lay in layers[:-1]:
            if lay.thickness_um is None:
                raise ValueError(f"{lay}: only the last layer of a stack may be bulk.")
        self.layers = layers
```

xrf_matrix.py, lines 227–228

```
    def __len__(self) -> int:
        return len(self.layers)
```

xrf_matrix.py, lines 230–231

```
    def __getitem__(self, k: int) -> Layer:
        return self.layers[k]
```

xrf_matrix.py, lines 233–234

```
    def __iter__(self) -> Iterator[Layer]:
        return iter(self.layers)
```

xrf_matrix.py, lines 236–239

```
    def replace(self, index: int, layer: Layer) -> "Stack":
        new = list(self.layers)
        new[index] = layer
        return Stack(new)
```

xrf_matrix.py, lines 241–242

```
    def __repr__(self) -> str:
        return "Stack(" + " / ".join(repr(l) for l in self.layers) + ")"
```

4.4 Depth integrals

4.4.1 `_F`, `_dF`

`_F(c, M)`, `_dF(c, M)`

$F(c;M) = (1-\mathrm{e}^{-cM})/c$ computed as `-expm1(-cM)/c`, which keeps full precision for $cM \ll 1$ (thin layers), and $1/c$ for $M=\infty$; `_dF` is its derivative with respect to c, $[cM\mathrm{e}^{-cM} - (1-\mathrm{e}^{-cM})]/c^2$ ($-1/c^2$ for bulk), used in the limits of Eq. (11).

xrf_matrix.py, lines 245–253

```
# ============================================================================
# Depth integrals
# ============================================================================

def _F(c: np.ndarray, M: float) -> np.ndarray:
    """``int_0^M exp(-c m) dm``  (c > 0)."""
    if not np.isfinite(M):
        return 1.0 / c
    return -np.expm1(-c * M) / c
```

xrf_matrix.py, lines 256–261

```
def _dF(c: np.ndarray, M: float) -> np.ndarray:
    """``d F / d c``."""
    if not np.isfinite(M):
        return -1.0 / c ** 2
    e = np.exp(-c * M)
    return (c * M * e - (1.0 - e)) / c ** 2
```

4.4.2 _G

```
_G(x, y, M)
```

$G(x, y; M) = \mathrm{e}^{-\min(x,y)M}\,(1 - \mathrm{e}^{-|x-y|M})/|x-y|$, a form that never evaluates the exponential of a large positive number, with the limit $M\,\mathrm{e}^{-xM}$ for $x = y$.

xrf_matrix.py, lines 264–271

```
264 def _G(x, y, M: float):
265     """``int_0^M exp(-x m) exp(-y (M - m)) dm`` for finite M (stable form)."""
266     x, y = np.broadcast_arrays(np.asarray(x, float), np.asarray(y, float))
267     lo = np.minimum(x, y)
268     d = np.abs(x - y)
269     with np.errstate(divide="ignore", invalid="ignore"):
270         core = np.where(d * M > 1e-12, -np.expm1(-d * M) / np.where(d > 0, d, 1.0), M)
271     return np.exp(-lo * M) * core
```

4.4.3 _D

```
_D(a, b, k, M, rtol=1e-4)
```

Eq. (11), broadcasting over arrays (energies × direction cosines), switching to the derivative form when $|b-\kappa|$ or $|a-\kappa|$ is within `rtol` of their sum.

xrf_matrix.py, lines 274–283

```
274 def _D(a, b, k, M: float, rtol: float = 1e-4):
275     """``int_0^M int_0^M exp(-a m - b m' - k|m - m'|) dm dm'`` (closed form)."""
276     a, b, k = np.broadcast_arrays(np.asarray(a, float), np.asarray(b, float), np.asarray(k, float))
277     f_ab = _F(a + b, M)
278     with np.errstate(divide="ignore", invalid="ignore"):
279         near_b = np.abs(b - k) <= rtol * (b + k)
280         t1 = np.where(near_b, -_dF(a + k, M), (_F(a + k, M) - f_ab) / (b - k))
281         near_a = np.abs(a - k) <= rtol * (a + k)
282         t2 = np.where(near_a, -_dF(b + k, M), (_F(b + k, M) - f_ab) / (a - k))
283     return t1 + t2
```

4.5 MatrixModel

4.5.1 _Exciter

An exciting line: layer index, element, line name, energy (keV), mass fraction, its cross-section $Q_j(E)$ on the secondary-fluorescence grid, and $\mu_c(E_j)$ in every layer.

xrf_matrix.py, lines 286–298

```
286 # ============================================================================
287 # The model
288 # ============================================================================
289
290 @dataclass
291 class _Exciter:
292     layer: int
293     symbol: str
294     name: str
295     energy_kev: float
296     mass_fraction: float
297     cs: np.ndarray          # Q_j(E) on the SF grid
298     mu: np.ndarray          # mu_c(E_j) for every layer c
```

4.5.2 MatrixModel

```
MatrixModel(sample, geometry, energy_kev, flux_01bw, secondary=True, target=None,
sf_step_kev=0.04, n_u=160, exciter_min_rel=1e-3)
```

Primary and secondary sensitivities of every line in a layer stack, for one incident spectrum and geometry.

`sample`
: a `Layer` (one-layer stack) or a `Stack`.

`geometry`
: a `Geometry`.

`energy_kev, flux_01bw`
: incident spectrum per 0.1 % bandwidth, as returned by `Beamline.spectrum` or `monochromatic_spectrum`; converted to per keV internally. A uniform grid with ∼10 eV steps is appropriate for a white beam.

`secondary`
: include secondary fluorescence.

`target`
: index of the layer being quantified (required for a multilayer); per-layer quantities default to it, and elements absent from the stack are treated as if in it.

`sf_step_kev`
: step of the decimated grid for the secondary term (default 40 eV).

`n_u`
: points of the logarithmic direction-cosine grid (u from 10^{-7} to 1).

`exciter_min_rel`
: relative threshold for exciting lines.

The constructor precomputes $\mu_k(E)$ of every layer on the grid, the decimated grids and the incident transmissions $T_{\mathrm{in}}^{k}(E)$.

xrf_matrix.py, lines 301–327

```
301 class MatrixModel:
302     """Primary and secondary fluorescence sensitivities for a layer stack.
303 
304     Parameters
305     ----------
306     sample : Layer or Stack
307         A single :class:`Layer` is treated as a one-layer stack.
308     geometry : Geometry
309     energy_kev, flux_01bw : (n,) arrays
310         Incident spectrum, **per 0.1 % bandwidth** as returned by
311         ``xrf_beamline.Beamline.spectrum``; converted to per keV internally.
312         A uniform grid with ~10 eV steps is appropriate.
313     secondary : bool
314         Include secondary fluorescence.
315     target : int, optional
316         Index of the layer being quantified (required for a multi-layer
317         stack).  Per-layer quantities default to it, and elements that are
318         not part of the stack (contaminants) are treated as if in it.
319     sf_step_kev : float
320         The secondary term is smooth in E and is evaluated on a grid decimated
321         to about this step (default 40 eV).
322     n_u : int
323         Points of the logarithmic direction-cosine grid for the SF kernel.
324     exciter_min_rel : float
325         Exciting lines weaker than this fraction of their element's strongest
326         line (primary production in its layer) are ignored.
327     """
```

xrf_matrix.py, lines 329–365

```
329     def __init__(self, sample: Union[Layer, Stack], geometry: Geometry,
330                  energy_kev: np.ndarray, flux_01bw: np.ndarray,
331                  secondary: bool = True, target: Optional[int] = None,
332                  sf_step_kev: float = 0.04, n_u: int = 160,
333                  exciter_min_rel: float = 1e-3):
```

```
        self.stack = sample if isinstance(sample, Stack) else Stack([sample])
        if target is None:
            if len(self.stack) > 1:
                raise ValueError("A multi-layer stack needs the index of the target layer.")
            target = 0
        if not 0 <= target < len(self.stack):
            raise ValueError(f"target {target} outside the stack of {len(self.stack)} layers.")
        self.target = int(target)
        self.geometry = geometry
        self.energy = np.asarray(energy_kev, dtype=float)
        self.flux_01bw = np.asarray(flux_01bw, dtype=float)
        self.density = per_unit_energy(self.energy, self.flux_01bw)
        self.secondary_enabled = bool(secondary)
        self.sf_step_kev = sf_step_kev
        self.n_u = int(n_u)
        self.exciter_min_rel = exciter_min_rel

        g = geometry
        self._M = np.array([lay.mass_thickness for lay in self.stack])
        self._mu_E = [lay.mu(self.energy) for lay in self.stack]
        step = float(np.median(np.diff(self.energy)))
        self._dec = max(1, int(round(sf_step_kev / step)))
        self._E_sf = self.energy[::self._dec]
        self._phi_sf = self.density[::self._dec]
        self._mu_sf = [m[::self._dec] for m in self._mu_E]
        # incident-beam transmission through the layers above each layer
        self._tin = [np.exp(-sum(self._mu_E[c] * self._M[c] for c in range(k)) / g.sin_in)
                     if k else np.ones_like(self.energy) for k in range(len(self.stack))]
        self._tin_sf = [t[::self._dec] for t in self._tin]
        s = np.linspace(np.log(1e-7), 0.0, self.n_u)
        self._s, self._u = s, np.exp(s)
        self._exciters: Optional[List[_Exciter]] = None
```

4.5.3 `layer`, `with_layer`, `with_stack`

`layer` (property), `with_layer(layer, index=None)`, `with_stack(stack, target=None)`

The target layer, and copies of the model with one layer or the whole stack replaced (same beam, geometry, numerics and target). Used by the self-consistent quantification.

xrf_matrix.py, lines 369–372

```
    @property
    def layer(self) -> Layer:
        """The target layer."""
        return self.stack[self.target]
```

xrf_matrix.py, lines 374–381

```
    def with_layer(self, layer: Layer, index: Optional[int] = None) -> "MatrixModel":
        """Same beam, geometry, numerics and target, one layer replaced
        (the target layer by default)."""
        idx = self.target if index is None else index
        return MatrixModel(self.stack.replace(idx, layer), self.geometry, self.energy,
                           self.flux_01bw, secondary=self.secondary_enabled, target=self.target,
                           sf_step_kev=self.sf_step_kev, n_u=self.n_u,
                           exciter_min_rel=self.exciter_min_rel)
```

xrf_matrix.py, lines 383–388

```
    def with_stack(self, stack: Union[Layer, Stack], target: Optional[int] = None) -> "MatrixModel":
        return MatrixModel(stack, self.geometry, self.energy, self.flux_01bw,
                           secondary=self.secondary_enabled,
                           target=self.target if target is None else target,
                           sf_step_kev=self.sf_step_kev, n_u=self.n_u,
                           exciter_min_rel=self.exciter_min_rel)
```

4.5.4 exciters

`exciters` (property)

The lines of the stack's elements that can drive secondary fluorescence, computed once: for each element of each layer, every line above 1 keV, ranked by its primary production at the top of its layer, $\int \Phi\, Q_j\, T^a_{\rm in}\, \mathrm{d}E / \sin\psi_{\rm in}$; lines below `exciter_min_rel` of the strongest line of that element are dropped.

xrf_matrix.py, lines 390–421

```
390     @property
391     def exciters(self) -> List[_Exciter]:
392         """Lines of the stack's own elements that can drive secondary fluorescence."""
393         if self._exciters is None:
394             ex: List[_Exciter] = []
395             sin_in = self.geometry.sin_in
396             for a, lay in enumerate(self.stack):
397                 for z, w in lay.mass_fractions.items():
398                     sym = xraylib.AtomicNumberToSymbol(z)
399                     cands = []
400                     for name, macro in _IUPAC_LINES.items():
401                         try:
402                             e_j = xraylib.LineEnergy(z, macro)
403                         except ValueError:
404                             continue
405                         if e_j < 1.0:
406                             continue
407                         cs = _cs_fluor(z, macro, self._E_sf)
408                         if not cs.any():
409                             continue
410                         strength = trapezoid(self._phi_sf / sin_in * cs * self._tin_sf[a], self._E_sf)
411                         cands.append((name[:-5], e_j, cs, strength))
412                     if not cands:
413                         continue
414                     smax = max(c[3] for c in cands)
415                     for name, e_j, cs, st in cands:
416                         if st >= self.exciter_min_rel * smax:
417                             mu = np.array([l.mu(e_j) for l in self.stack])
418                             ex.append(_Exciter(a, sym, name, e_j, w, cs, mu))
419             self._exciters = ex
420             logger.info("%d exciting lines in %s", len(ex), self.stack)
421         return self._exciters
```

4.5.5 _k, _b, _tout

`_k(layer)`, `_b(line_energy_kev)`, `_tout(b, k)`

Helpers: the layer index (target by default); the exit attenuation $b_c = \mu_c(E_l)/\sin\psi_{\rm out}$ of every layer; and $T^k_{\rm out}$.

xrf_matrix.py, lines 423–424

```
423     def _k(self, layer: Optional[int]) -> int:
424         return self.target if layer is None else int(layer)
```

xrf_matrix.py, lines 426–427

```
426     def _b(self, line_energy_kev: float) -> np.ndarray:
427         return np.array([lay.mu(line_energy_kev) for lay in self.stack]) / self.geometry.sin_out
```

xrf_matrix.py, lines 429–430

```
429     def _tout(self, b: np.ndarray, k: int) -> float:
430         return float(np.exp(-np.sum(b[:k] * self._M[:k]))) if k else 1.0
```

4.5.6 primary

```
primary(z, macro, line_energy_kev, layer=None)
```

Eq. (9), integrated with the trapezoidal rule on the full grid. Returns 0 if the line cannot be excited by the spectrum.

xrf_matrix.py, lines 434–447

```
434     def primary(self, z: int, macro: int, line_energy_kev: float,
435                 layer: Optional[int] = None) -> float:
436         """``P_il^k`` — primary sensitivity per unit mass fraction in layer ``k``
437         (the target layer by default), overlayer attenuation included."""
438         k = self._k(layer)
439         g = self.geometry
440         q = _cs_fluor(z, macro, self.energy)
441         if not q.any():
442             return 0.0
443         b = self._b(line_energy_kev)
444         a_k = self._mu_E[k] / g.sin_in
445         val = trapezoid(self.density / g.sin_in * q * self._tin[k] * _F(a_k + b[k], self._M[k]),
446                         self.energy)
447         return float(val) * self._tout(b, k)
```

4.5.7 secondary

```
secondary(z, macro, line_energy_kev, layer=None, by_exciter=False)
```

Eq. (10): for each exciter able to excite the line ($Q_{il}(E_j) > 0$), the kernel is computed on the (energy, u) grid, integrated over $\ln u$, then over energy. With `by_exciter=True` it also returns the contribution of each exciter, keyed `"<layer>:<element> <line>"`—useful to see which element drives the enhancement.

xrf_matrix.py, lines 449–487

```
449     def secondary(self, z: int, macro: int, line_energy_kev: float,
450                   layer: Optional[int] = None, by_exciter: bool = False):
451         """``S_il^k`` — secondary sensitivity per unit mass fraction in layer
452         ``k`` (target by default), summed over exciting lines in every layer.
453
454         With ``by_exciter=True`` also returns ``{"<layer>:<element> <line>": value}``.
455         """
456         if not self.secondary_enabled:
457             return (0.0, {}) if by_exciter else 0.0
458         k = self._k(layer)
459         g = self.geometry
460         M = self._M
461         b = self._b(line_energy_kev)
462         u, s = self._u, self._s
463         total, parts = 0.0, {}
464         for ex in self.exciters:
465             q_ij = _cs_fluor_scalar(z, macro, ex.energy_kev)
466             if q_ij <= 0.0:
467                 continue
468             a = ex.layer
469             kc = ex.mu[:, None] / u[None, :]                    # (n_layers, n_u)
470             A = (self._mu_sf[a] / g.sin_in)[:, None]            # (n_E, 1)
471             if a == k:
472                 kern = trapezoid(_D(A, b[k], kc[k][None, :], M[k]), s, axis=1)
473             elif a < k:
474                 between = np.exp(-sum(kc[c] * M[c] for c in range(a + 1, k))) if k - a > 1 else 1.0
475                 tail = between * _F(b[k] + kc[k], M[k])          # (n_u,)
476                 kern = trapezoid(_G(A, kc[a][None, :], M[a]) * tail[None, :], s, axis=1)
477             else:
478                 between = np.exp(-sum(kc[c] * M[c] for c in range(k + 1, a))) if a - k > 1 else 1.0
479                 tail = between * _G(b[k], kc[k], M[k])           # (n_u,)
480                 kern = trapezoid(_F(A + kc[a][None, :], M[a]) * tail[None, :], s, axis=1)
481             val = 0.5 * q_ij * ex.mass_fraction * float(trapezoid(
482                 self._phi_sf / g.sin_in * ex.cs * self._tin_sf[a] * kern, self._E_sf))
483             val *= self._tout(b, k)
```

```
484             total += val
485             key = f"{a}:{ex.symbol} {ex.name}"
486             parts[key] = parts.get(key, 0.0) + val
487         return (total, parts) if by_exciter else total
```

4.5.8 `sensitivity`, `emission`, `enhancement`, `absorption_factor`

`sensitivity` is $P^k + S^k$ per unit mass fraction in layer k. `emission` is the intensity from the whole stack, $\sum_k w_i^k (P^k[+S^k])$; it is what sets relative intensities *within* a line group in `xrf_fit` (elements absent from the stack are evaluated per unit mass fraction in the target). `enhancement` is S/P. `absorption_factor` is P^k divided by the absorption-free value $M_k \int \Phi Q \, \mathrm{d}E / \sin\psi_{\mathrm{in}}$ (thin film, no overlayer): 1 means no absorption; NaN for a bulk layer.

xrf_matrix.py, lines 489–495

```
489     def sensitivity(self, z: int, macro: int, line_energy_kev: float,
490                     layer: Optional[int] = None) -> float:
491         """``P_il^k + S_il^k`` (per unit mass fraction in layer ``k``)."""
492         p = self.primary(z, macro, line_energy_kev, layer)
493         if p <= 0.0:
494             return 0.0
495         return p + self.secondary(z, macro, line_energy_kev, layer)
```

xrf_matrix.py, lines 497–515

```
497     def emission(self, z: int, macro: int, line_energy_kev: float,
498                  secondary: bool = True) -> float:
499         """Line intensity from the whole stack, ``sum_k w_i^k (P^k [+ S^k])``.
500
501         Elements absent from the stack (contaminants) are evaluated per unit
502         mass fraction in the target layer.  This is the weighting that sets
503         relative intensities *within* a line group.
504         """
505         layers = [(k, lay.mass_fractions[z]) for k, lay in enumerate(self.stack)
506                   if z in lay.mass_fractions]
507         if not layers:
508             layers = [(self.target, 1.0)]
509         tot = 0.0
510         for k, w in layers:
511             p = self.primary(z, macro, line_energy_kev, k)
512             if p > 0 and secondary:
513                 p += self.secondary(z, macro, line_energy_kev, k)
514             tot += w * p
515         return tot
```

xrf_matrix.py, lines 517–521

```
517     def enhancement(self, z: int, macro: int, line_energy_kev: float,
518                     layer: Optional[int] = None) -> float:
519         """Secondary / primary, for layer ``k`` (target by default)."""
520         p = self.primary(z, macro, line_energy_kev, layer)
521         return self.secondary(z, macro, line_energy_kev, layer) / p if p > 0 else 0.0
```

xrf_matrix.py, lines 523–534

```
523     def absorption_factor(self, z: int, macro: int, line_energy_kev: float,
524                           layer: Optional[int] = None) -> float:
525         """Primary intensity of layer ``k`` relative to the same layer free of
526         absorption (thin-film limit, no overlayers): 1 means no absorption.
527         Includes the overlayers.  NaN for a bulk layer."""
528         k = self._k(layer)
529         if not np.isfinite(self._M[k]):
530             return float("nan")
531         g = self.geometry
532         q = _cs_fluor(z, macro, self.energy)
533         thin = trapezoid(self.density / g.sin_in * q * self._M[k], self.energy)
534         return self.primary(z, macro, line_energy_kev, k) / thin if thin > 0 else float("nan")
```

4.6 Quantification

4.6.1 `_line`

```
_line(group)
```

`(Z, macro, energy in keV)` of a group's reference line; merged lines (`"KL3+1"`) are refused, since their cross-section is not defined.

xrf_matrix.py, lines 537–548

```
537 # ============================================================================
538 # Quantification by iteration on the composition of the target layer
539 # ============================================================================
540
541 def _line(group):
542     """(Z, macro, energy keV) of a group's reference line."""
543     z = xraylib.SymbolToAtomicNumber(group.element)
544     name = group.reference.name
545     if "+" in name:
546         raise ValueError(f"{group.label}: merged reference line {name!r} is not supported "
547                          "for matrix quantification; build the groups with merge_within_ev=0.")
548     return z, getattr(xraylib, name + "_LINE"), group.reference.energy_ev / 1000.0
```

4.6.2 `quantify_layer`

```
quantify_layer(areas, groups, matrix, detector, reference, analytes, fixed_atoms=None,
max_iter=50, tol=1e-6)
```

Self-consistent composition of the target layer from fitted reference-line areas.

`areas`
: `{label: reference-line area}`, e.g. `FitResult.areas`.

`groups`
: `{label: LineGroup}`.

`matrix`
: a `MatrixModel`; its target layer gives the starting composition.

`detector`
: an `xrf_fit.Detector` (detection efficiency).

`reference`
: label of the reference group (`"As K"`).

`analytes`
: labels of the groups to quantify.

`fixed_atoms`
: `{symbol: atoms per reference atom}` for unmeasured elements of the layer (e.g. oxygen).

`max_iter, tol`
: iteration limit and convergence threshold on the atom ratios.

Method For each group g the measured area is modelled as

$$A_g \propto w_g^t s_g^t + C_g, \qquad s_g^k = (P^k + S^k)\,\epsilon, \qquad C_g = \sum_{k \neq t} w_g^k s_g^k, \tag{13}$$

where t is the target layer and C_g the known contribution of the other layers (caps, substrate). Relative to the reference group r,

$$w_i^t = \frac{(A_i/A_r)\,(w_r^t s_r^t + C_r) - C_i}{s_i^t}. \tag{14}$$

The atom ratios $n_i/n_r = (w_i^t/W_i)/(w_r^t/W_r)$ (W: atomic weights) rebuild the target composition, the sensitivities are recomputed, and the loop stops when no ratio changes by more than `tol`. Density and thicknesses are held fixed. Elements of the target that are neither analytes, reference nor fixed are dropped, because the absolute mass fraction of the reference matters once other layers contribute. A negative w_i (the other layers explain more than the measured intensity) is clipped to 0 with a warning. For a one-layer sample $C = 0$ and the result is $n_i/n_r = (A_i/A_r)(s_r W_r)/(s_i W_i)$, the formula of `xrf_fit.number_ratio` with matrix-corrected sensitivities.

Returns a dictionary: `ratios` (atoms per reference atom), the converged `layer` and `matrix`, `k` (the factors $(n_i/n_r)/(A_i/A_r)$), `enhancement` and `absorption` of the reference lines, `other_layers` (fraction of each modelled intensity coming from the other layers), `terms` (s^t, C), `iterations`, `history`, `reference`.

xrf_matrix.py, lines 551–660

```
551 def quantify_layer(areas: Mapping[str, float], groups: Mapping[str, object],
552                   matrix: MatrixModel, detector, reference: str,
553                   analytes: Sequence[str], fixed_atoms: Optional[Mapping[str, float]] = None,
554                   max_iter: int = 50, tol: float = 1e-6) -> dict:
555     """Self-consistent composition of the target layer from fitted reference-line areas.
556 
557     For every group ``g`` the measured reference-line area is modelled as
558 
559         A_g  ∝  w_g^t s_g^t + C_g,     s_g^k = (P^k + S^k) eff,
560         C_g  = sum_{k != t} w_g^k s_g^k     (known layers: caps, substrate)
561 
562     so, relative to the reference group ``r``,
563 
564         w_i^t = [ (A_i / A_r) (w_r^t s_r^t + C_r) - C_i ] / s_i^t
565 
566     The target composition (atoms per reference atom, plus ``fixed_atoms``
567     for unmeasured elements) is rebuilt from these mass fractions and the
568     sensitivities recomputed until the ratios change by less than ``tol``.
569     Density and thicknesses are held fixed.  Elements of the target layer
570     that are neither analytes, reference nor fixed are dropped, because the
571     absolute mass fraction of the reference is needed once other layers
572     contribute.  For a one-layer sample ``C = 0`` and this reduces to
573     ``n_i/n_r = (A_i/A_r) (s_r W_r)/(s_i W_i)``.
574 
575     Parameters
576     ----------
577     areas : {label: reference-line area}
578     groups : {label: LineGroup}
579     matrix : MatrixModel
580         Its target layer supplies the starting composition.
581     detector : xrf_fit.Detector
582     reference : str
583         Label of the reference group, e.g. ``"As K"``.
584     analytes : sequence of str
585     fixed_atoms : {symbol: atoms per reference atom}, optional
586 
587     Returns
588     -------
589     dict with keys ``ratios`` ({symbol: n/n_ref} in the target layer),
590     ``layer`` and ``matrix`` (converged), ``k`` ({label: (n_i/n_r)/(A_i/A_r)}),
591     ``enhancement`` and ``absorption`` (target layer, reference lines),
592     ``other_layers`` ({label: fraction of the modelled intensity coming from
593     the other layers}), ``terms`` ({label: (s_t, C)}), ``iterations``,
594     ``history``.
595     """
596     fixed_atoms = dict(fixed_atoms or {})
597     t = matrix.target
598     g_ref = groups[reference]
599     ref_el = g_ref.element
600     z_ref = xraylib.SymbolToAtomicNumber(ref_el)
601     w_ref_at = xraylib.AtomicWeight(z_ref)
602 
603     def terms(mm, g):
604         z, macro, e = _line(g)
605         eff = detector.efficiency(e * 1000.0)
606         s_t = mm.sensitivity(z, macro, e, layer=t) * eff
```

```
        c = sum(lay.mass_fractions.get(z, 0.0) * mm.sensitivity(z, macro, e, layer=k)
                for k, lay in enumerate(mm.stack) if k != t and z in lay.mass_fractions) * eff
        return s_t, c

    atoms0 = matrix.layer.atoms
    n0 = atoms0.get(ref_el, 1.0)
    ratios = {el: n / n0 for el, n in atoms0.items()}
    history = [dict(ratios)]
    mm = matrix
    it = 0
    for it in range(1, max_iter + 1):
        w = mm.layer.mass_fractions
        w_r = w.get(z_ref, 0.0)
        if w_r <= 0:
            raise ValueError(f"reference element {ref_el} is not in the target layer.")
        s_r, c_r = terms(mm, g_ref)
        i_ref = w_r * s_r + c_r
        new = {ref_el: 1.0}
        new.update(fixed_atoms)
        for lab in analytes:
            g = groups[lab]
            s_i, c_i = terms(mm, g)
            w_i = (areas[lab] / areas[reference] * i_ref - c_i) / s_i
            if w_i < 0:
                logger.warning("%s: the other layers account for more than the measured "
                               "intensity; clipped to 0.", lab)
                w_i = 0.0
            w_at = xraylib.AtomicWeight(xraylib.SymbolToAtomicNumber(g.element))
            new[g.element] = float((w_i / w_at) / (w_r / w_ref_at))
        change = max(abs(new.get(el, 0.0) - ratios.get(el, 0.0)) for el in set(new) | set(ratios))
        ratios = new
        history.append(dict(ratios))
        mm = mm.with_layer(mm.layer.with_composition(ratios))
        if change < tol:
            break
    else:
        logger.warning("quantify_layer did not converge in %d iterations", max_iter)

    enh, absf, other, k_fac, all_terms = {}, {}, {}, {}, {}
    w = mm.layer.mass_fractions
    for lab in [reference, *analytes]:
        g = groups[lab]
        z, macro, e = _line(g)
        enh[lab] = mm.enhancement(z, macro, e)
        absf[lab] = mm.absorption_factor(z, macro, e)
        s_t, c = terms(mm, g)
        all_terms[lab] = (s_t, c)
        tot = w.get(z, 0.0) * s_t + c
        other[lab] = c / tot if tot > 0 else 0.0
        if lab != reference:
            k_fac[lab] = ratios[g.element] / (areas[lab] / areas[reference])
    return dict(ratios=ratios, layer=mm.layer, matrix=mm, k=k_fac, enhancement=enh,
                absorption=absf, other_layers=other, terms=all_terms,
                reference=reference, iterations=it, history=history)
```

4.6.3 `atom_ratio`

```
atom_ratio(quant, label, area, area_ref, err=None, err_ref=None)
```

Applies the converged sensitivities of `quantify_layer` to other areas—typically every pixel of a map—with the matrix held fixed: $n_i/n_r = [(R\,I_r - C_i)/s_i/W_i]/(w_r/W_r)$, $R = A_i/A_r$, $I_r = w_r s_r + C_r$. Errors of the two areas are propagated as uncorrelated. Returns `(ratio, error)`.

xrf_matrix.py, lines 663–695

```
def atom_ratio(quant: dict, label: str, area, area_ref, err=None, err_ref=None):
    """Per-pixel atom ratio ``n_i / n_ref`` in the target layer, using the
    converged sensitivities of :func:`quantify_layer` (matrix held fixed).

    ``n_i/n_r = [(R I_r - C_i)/s_i / W_i] / (w_r / W_r)`` with ``R = A_i/A_r``,
    ``I_r = w_r s_r + C_r``.  Reduces to :func:`xrf_fit.number_ratio` for a
```

```
    one-layer sample.  Errors are propagated from the two areas, uncorrelated.

    Returns
    -------
    ratio, ratio_err (None without errors)
    """
    ref = quant["reference"]
    lay = quant["layer"]
    z_ref = xraylib.SymbolToAtomicNumber(_ref_symbol(quant))
    w_r = lay.mass_fractions[z_ref]
    s_r, c_r = quant["terms"][ref]
    s_i, c_i = quant["terms"][label]
    el = _label_symbol(label)
    w_at_i = xraylib.AtomicWeight(xraylib.SymbolToAtomicNumber(el))
    w_at_r = xraylib.AtomicWeight(z_ref)
    a = np.asarray(area, dtype=float)
    b = np.asarray(area_ref, dtype=float)
    i_r = w_r * s_r + c_r
    scale = w_at_r / (w_at_i * w_r * s_i)
    with np.errstate(divide="ignore", invalid="ignore"):
        R = a / b
        ratio = (R * i_r - c_i) * scale
        err_out = None
        if err is not None and err_ref is not None:
            dR = np.abs(R) * np.sqrt((np.asarray(err) / a) ** 2 + (np.asarray(err_ref) / b) ** 2)
            err_out = dR * i_r * scale
    return ratio, err_out
```

xrf_matrix.py, lines 698–699

```
def _label_symbol(label: str) -> str:
    return label.split()[0]
```

xrf_matrix.py, lines 702–703

```
def _ref_symbol(quant: dict) -> str:
    return _label_symbol(quant["reference"])
```

5 `xrf_fit`: the spectrum model and the fits

5.1 Overview: the constrained model

An energy-dispersive XRF spectrum is a sum of peaks on a continuum. Fitting every peak with free position, width and height is unstable as soon as peaks overlap, and produces numbers that cannot be compared between elements. `xrf_fit` instead builds a *constrained* model in which almost everything except one amplitude per element and shell is computed:

$$y(c) = B(c) + \sum_{g} A_g \sum_{l \in g} r_l \, \tau_l(d) \Big[\big(1 - \textstyle\sum_e f_{le}\big)\, R\big(c - c_l; \sigma_l\big) + \sum_e f_{le}\, R\big(c - c_{le}; \sigma_{le}\big) \Big] + Y_{\text{scatter}}(c) + \tau_{\text{pp}}\, P(c), \tag{15}$$

where

- c is the MCA channel, and $E(c) = Gc + O$ the energy calibration;
- $B(c)$ is a polynomial background (or the exponential of a polynomial);
- g runs over *line groups*, one per (element, shell family): `"Cu K"`, `"Pb L"`, `"Pb M"`;
- A_g, the only free amplitude of the group, is the integrated number of counts of its most intense line (the *reference line*);
- r_l is the intensity of line l relative to the reference line ($r = 1$ for it), computed for the excitation, the detector and, optionally, the sample matrix;
- $\tau_l(d)$ rescales r_l when the sample–detector air path d is fitted;
- $c_l = (E_l - O)/G$ is the position of the line, from its tabulated energy; σ_l its Gaussian width from the detector resolution law;
- f_{le} and c_{le} are the fraction and position of its sensor escape peaks;

- R is the unit-area detector response (Gaussian or Voigt, optionally with a low-energy tail and a step);
- Y_{scatter} holds the elastic and Compton peaks under monochromatic excitation, and $\tau_{\mathrm{pp}}P(c)$ the pile-up spectrum.

An N-element fit has N amplitudes plus a handful of shared parameters, instead of several per peak. Peak positions are never free individually: they move only through the calibration. This is the approach of the fundamental-parameter fitting codes such as PyMca [20], written here for white-beam excitation and for maps.

The module also provides the statistical treatment of map data (normalisation, averaging, uncertainties), the per-pixel linear solve that turns the average-spectrum fit into maps with error maps, the tracking of detector-resolution changes during a scan, and diagnostic plots.

xrf_fit.py, lines 1–137

```
"""
Module: xrf_fit.py

Constrained fitting of synchrotron X-ray fluorescence spectra (BM32, ESRF).

The model
---------
A spectrum is described as

    y(c) = B(c) + sum_g  A_g * sum_{l in g} r_l * V(E(c) - E_l ; sigma(E_l), gamma)

where

* ``c``        is the MCA channel and ``E(c)`` the energy calibration,
* ``B(c)``     is a polynomial background,
* ``g``        runs over *line groups* — one group per (element, shell family),
                 e.g. ("Nd", "L"), ("Cu", "K"),
* ``A_g``      is the single free amplitude of group ``g``: the integrated area of
                 the group's **most intense** line,
* ``r_l``      is the tabulated intensity of line ``l`` **relative to that most
                 intense line** (so ``r = 1`` for the reference line),
* ``V``        is a unit-area Voigt profile,
* ``sigma(E)`` follows the detector resolution law (two global parameters),
* ``gamma``    is one global Lorentzian width.

So an N-element fit has ``N + n_bkg + 3`` free parameters instead of four per
line.  Peak positions are never free: they come from the tabulated line energies
and the calibration.

Relative intensities: why not RadRate
-------------------------------------
``xraylib.RadRate`` is normalised **within a subshell** — the L1, L2 and L3 rates
each sum to 1 independently — so it carries no information about the relative
ionisation of those subshells.  Using it across an L series is wrong: for Ir it
predicts Lb1/La1 = 1.13, where the Kissel partial fluorescence cross-sections
give 0.65 and the measured ratio on the NIO295 map is 0.77.

This module therefore takes relative intensities from
``xraylib.CS_FluorLine_Kissel(Z, line, E0)``, which includes subshell ionisation
cross-sections, fluorescence yields and Coster-Kronig transfer.  For a K series
the two approaches agree, since all K lines share one initial vacancy.

``E0`` is a single *effective* excitation energy.  Under white-beam illumination
the physically correct weighting is the flux-weighted integral
``int sigma(E) dPhi/dE dE`` (pass ``incident_spectrum`` to
:func:`element_line_groups`; the per-0.1 %BW -> per-keV conversion is applied
internally).  With a scalar ``E0``, it must exceed the highest relevant
absorption edge (e.g. the Ir L1 edge at 13.42 keV) or the high-subshell lines
are silently given zero intensity.

Detection efficiency (air path, entrance window, sensor absorption) is applied
when a :class:`Detector` is passed to :func:`element_line_groups`; its defaults
describe the BM32 KETEK VITUS H7 CH.  :func:`number_ratio` then converts fitted
areas into atom-number ratios.

Matrix effects (self-absorption and secondary fluorescence in a layer or bulk
```

```
sample) come from :mod:`xrf_matrix`: pass a ``MatrixModel`` to
:func:`element_line_groups`.

With monochromatic excitation, :class:`ScatterSource` adds the elastic peak and
the Compton peak (impulse approximation with the scatterer's Compton profile,
averaged over the range of scattering angles the detector collects).

Sensor escape peaks (``escape=True``) and pile-up sum peaks
(``pileup_template``, from :func:`pileup_spectra`) are optional parts of
:class:`XRFModel`; the escape peaks add no free parameter, pile-up adds one
(the pulse-pair resolving time).

What this module does *not* do yet
----------------------------------
* The detector response is a Voigt by default; a low-energy tail and a step
  (Hypermet-type, :func:`peak_shape`) are optional (``Detector.tail_*`` /
  ``fit_tails``).
* No Laue (Bragg) peaks from the sample in the fluorescence spectrum.
* The energy calibration is an input of map fits.  It can be fitted on a
  reference spectrum (``fit_calibration`` in :func:`fit_average_spectrum`);
  :mod:`xrf_calibration` wraps that into a calibration procedure for
  reference samples.

Authors: Samuel Tardif, with Claude (Anthropic).
"""

from __future__ import annotations

import logging
import re
from dataclasses import dataclass, field
from typing import Dict, Iterable, List, Mapping, Optional, Sequence, Tuple

import numpy as np
import xraylib
from scipy.integrate import trapezoid
from scipy.optimize import curve_fit
from scipy.special import erfc, erfcx, voigt_profile
from scipy.signal import fftconvolve

from xrf_beamline import AIR_FORMULA, air_density, per_unit_energy
from xrf_matrix import MatrixModel

logger = logging.getLogger(__name__)

# xraylib raises ValueError for lines/edges that do not exist for a given Z;
# silence its stderr chatter and handle the exceptions instead.
xraylib.SetErrorMessages(0)

__all__ = [
    "Calibration",
    "Detector",
    "Line",
    "LineGroup",
    "XRFModel",
    "FitResult",
    "element_line_groups",
    "normalise_spectra",
    "average_spectrum",
    "counting_error",
    "snip_background",
    "fit_average_spectrum",
    "plot_fit",
    "identify_peaks",
    "MapResult",
    "design_matrix",
    "fit_map",
    "number_ratio",
    "pileup_spectra",
    "average_pileup_template",
    "width_window",
    "peak_width_profile",
    "detect_steps",
    "segment_labels",
    "fit_segments",
```

```
    "pixel_parameters",
    "select_pixels",
    "plot_pixel_fits",
    "ScatterSource",
    "peak_shape",
]
```

5.2 Energy calibration

5.2.1 `Calibration`

```
Calibration(gain=10.032, offset=-31.263)
```

Linear calibration $E\,[\mathrm{eV}] = G\,c + O$ (a frozen dataclass). The defaults were measured on Ga/As K lines with the BM32 XIA configuration of 2026; `xrf_calibration` measures it on reference samples.

`gain`
: G, eV per channel.

`offset`
: O, energy of channel 0 in eV.

Methods `energy(channel)` and `channel(energy_ev)` (not rounded) convert in either direction, element-wise; `channel_range(emin_ev, emax_ev, n_channels)` returns the half-open channel range $[\lfloor c(E_{\min})\rfloor, \lceil c(E_{\max})\rceil + 1)$ clipped to the spectrum.

xrf_fit.py, lines 140–160

```
# ============================================================================
# Energy calibration
# ============================================================================

@dataclass(frozen=True)
class Calibration:
    """Linear MCA channel <-> energy calibration, ``E[eV] = gain * channel + offset``.

    The BM32 XIA default below was determined from Ga/As K lines.  To measure
    it on a reference sample, see :mod:`xrf_calibration`.

    Parameters
    ----------
    gain : float
        eV per channel.
    offset : float
        Energy of channel 0, in eV.
    """

    gain: float = 10.032
    offset: float = -31.263
```

xrf_fit.py, lines 162–164

```
    def energy(self, channel: np.ndarray | float) -> np.ndarray | float:
        """Channel -> energy (eV)."""
        return self.gain * np.asarray(channel, dtype=float) + self.offset
```

xrf_fit.py, lines 166–168

```
    def channel(self, energy_ev: np.ndarray | float) -> np.ndarray | float:
        """Energy (eV) -> channel (not rounded)."""
        return (np.asarray(energy_ev, dtype=float) - self.offset) / self.gain
```

xrf_fit.py, lines 170–174

```
    def channel_range(self, emin_ev: float, emax_ev: float, n_channels: int) -> Tuple[int, int]:
        """Half-open channel range ``[lo, hi)`` covering an energy window."""
        lo = max(0, int(np.floor(self.channel(emin_ev))))
        hi = min(n_channels, int(np.ceil(self.channel(emax_ev))) + 1)
```

```
174         return lo, hi
```

5.3 The detector

5.3.1 `Detector`

```
Detector(material="Si", noise_ev=30.0, fano=0.12, gamma_ev=5.0, sensor_thickness_um=450.0,
window_material="C", window_density_g_cm3=2.2, window_thickness_um=1.0, air_path_cm=20.0,
air_temperature_c=20.0, air_pressure_hpa=1013.25, dead_layer_um=0.0, tail_fraction=0.0,
tail_slope=1.5, step_fraction=0.0)
```

The energy resolution, detection efficiency and line shape of a solid-state detector. The defaults describe the BM32 KETEK VITUS H7 silicon drift detector [21]: 450 µm of Si behind a 1 µm carbon window, about 20 cm from the sample.

`material`
: sensor, `"Si"` or `"Ge"`: sets the pair-creation energy ε (3.85 and 2.96 eV) and the density.

`noise_ev`
: electronic-noise contribution N to σ.

`fano`
: Fano factor F.

`gamma_ev`
: Lorentzian half width Γ of the Voigt profile.

`sensor_thickness_um`
: active thickness d_s.

`window_*`
: entrance-window material, density and thickness.

`air_path_cm, air_temperature_c, air_pressure_hpa`
: sample–window air path and ambient conditions.

`dead_layer_um`
: inactive Si at the entrance.

`tail_fraction, tail_slope, step_fraction`
: line-shape parameters, see `peak_shape`.

Resolution The Gaussian width combines electronic noise and the statistics of charge creation [22, 23]:

$$\sigma^2(E) = N^2 + F\varepsilon E, \qquad \mathrm{FWHM} = 2\sqrt{2\ln 2}\,\sigma = 2.3548\,\sigma. \tag{16}$$

The defaults ($N = 30$ eV, $F = 0.12$) give 142 eV FWHM at 5.9 keV (the class docstring quotes "∼135 eV", the middle of the 125–150 eV range of the BM32 detector; the resolution is fitted in practice).

Hypotheses and limits

Only the product $F\varepsilon$ enters Eq. (16). The value of ε for Si used here, 3.85 eV, is above the measured 3.66 eV at room temperature [24]; since F is fitted, this only rescales the fitted Fano factor and not the widths. The Lorentzian width Γ is physically the natural width of the level (a few eV); a fitted value above ∼30 eV signals that a tail term is needed.

xrf_fit.py, lines 177–263

```
177 # ============================================================================
178 # Detector response
179 # ============================================================================
180
```

```
#: Mean energy to create one electron-hole pair, in eV.
PAIR_ENERGY_EV = {"Si": 3.85, "Ge": 2.96}

#: Sensor densities, g/cm^3, for the detection-efficiency model.
SENSOR_DENSITY_G_CM3 = {"Si": 2.33, "Ge": 5.323}


@dataclass
class Detector:
    """Energy-resolution model of a solid-state detector.

    The Gaussian width combines energy-independent electronic noise with the
    Fano-limited statistical term::

        sigma(E)^2 = noise_ev^2 + fano * pair_energy * E
        FWHM(E)    = 2.3548 * sigma(E)

    Defaults correspond to ~135 eV FWHM at 5.9 keV on a Si detector, i.e. the
    middle of the 125-150 eV range quoted for the BM32 XIA.

    Parameters
    ----------
    material : {"Si", "Ge"}
        Sets the electron-hole pair creation energy.
    noise_ev : float
        Electronic-noise contribution to sigma, in eV.
    fano : float
        Fano factor (~0.12 for Si, ~0.06-0.13 for Ge).
    gamma_ev : float
        Lorentzian HWHM of the Voigt profile, in eV.  Physically this is the
        natural level width (a few eV for an L line); in practice it also
        absorbs part of the low-energy tail, so treat a fitted value above
        ~30 eV as a sign that a proper tail term is needed.

    Detection efficiency
    --------------------
    The remaining fields describe the path from sample to sensor, used by
    :meth:`efficiency`.  Defaults are the BM32 KETEK VITUS H7 CH SDD:
    450 um Si sensor, 1 um carbon ("CH" graphene) entrance window, and
    ~20 cm of air between sample and detector.

    sensor_thickness_um : float
        Active sensor thickness ("absorption depth" on the datasheet).
    window_material, window_density_g_cm3, window_thickness_um
        Entrance window.  At 1 um, carbon is transparent above ~2 keV.
    air_path_cm : float
        Sample-to-window air path.  **This dominates the efficiency of soft
        lines**: 20 cm of air transmits ~4 % at In La (3.29 keV), and each
        extra cm costs ~15 % there, against ~0.3 % at Ga Ka.
    air_temperature_c, air_pressure_hpa : float
        Ambient conditions for the air density (local pressure, ~990 hPa at
        the ESRF, changes the 20 cm optical depth at 3.3 keV by ~8 %).
    dead_layer_um : float
        Inactive Si at the entrance; unknown for this unit, default 0.

    Line shape (see :func:`peak_shape`)
    -----------------------------------
    Incomplete charge collection adds a low-energy tail and a flat step to
    every peak.  Zero (the default) gives a pure Voigt.

    tail_fraction : float
        Fraction of each line's area in the exponential low-energy tail.
    tail_slope : float
        Decay length of the tail, in units of the line's Gaussian sigma.
    step_fraction : float
        Height of the step below each line, as a fraction of the peak height.
    """

    material: str = "Si"
    noise_ev: float = 30.0
    fano: float = 0.12
    gamma_ev: float = 5.0
    sensor_thickness_um: float = 450.0
    window_material: str = "C"
    window_density_g_cm3: float = 2.2
```

```
    window_thickness_um: float = 1.0
    air_path_cm: float = 20.0
    air_temperature_c: float = 20.0
    air_pressure_hpa: float = 1013.25
    dead_layer_um: float = 0.0
    tail_fraction: float = 0.0
    tail_slope: float = 1.5
    step_fraction: float = 0.0
```

5.3.2 `pair_energy_ev`

`pair_energy_ev` (property)

ε for the sensor material; `ValueError` for other materials.

xrf_fit.py, lines 265–270

```
    @property
    def pair_energy_ev(self) -> float:
        try:
            return PAIR_ENERGY_EV[self.material]
        except KeyError:
            raise ValueError(f"material must be one of {list(PAIR_ENERGY_EV)}, got {self.material!r}")
```

5.3.3 `Detector.efficiency`

`Detector.efficiency(energy_ev)`

Probability that a photon emitted towards the detector is recorded in the full-energy region:

$$\epsilon(E) = \underbrace{\mathrm{e}^{-\mu_{\mathrm{air}}(E)\rho_{\mathrm{air}}d_{\mathrm{air}}}}_{\text{air}}\ \underbrace{\mathrm{e}^{-\mu_{w}(E)\rho_{w}t_{w}}}_{\text{window}}\ \underbrace{\mathrm{e}^{-\mu_{\mathrm{Si}}(E)\rho_{\mathrm{Si}}t_{\mathrm{dead}}}}_{\text{dead layer}}\ \underbrace{\left[1-\mathrm{e}^{-\tau_{\mathrm{Si}}(E)\rho_{\mathrm{Si}}d_{s}}\right]}_{\text{absorbed}}. \tag{17}$$

Attenuation along the path uses total cross-sections (any interaction removes the photon); absorption in the sensor uses the photoelectric cross-section τ only, since a scattered photon does not deposit its full energy. Escape peaks are not subtracted: they redistribute counts within the recorded spectrum and belong to the line shape. Returns a float for a scalar input, else an array; energies where xraylib has no data give 0.

Hypotheses and limits

Normal incidence on a uniform sensor, collimator and grid ignored, single air-path length. The air path dominates the efficiency of soft lines: 20 cm of air transmit about 4 % at In Lα (3.29 keV), and each extra centimetre costs about 15 % there, against 0.3 % at Ga Kα.

xrf_fit.py, lines 272–303

```
    def efficiency(self, energy_ev: np.ndarray | float) -> np.ndarray | float:
        """Probability that a photon emitted towards the detector is recorded
        in the full-energy peak region.

        ``T_air * T_window * T_dead * (1 - exp(-mu_photo * d_sensor))``

        Attenuation along the path uses the total cross-section (any
        interaction removes the photon); absorption in the sensor uses the
        **photoelectric** cross-section only, since a Compton- or Rayleigh-
        scattered photon does not deposit its full energy.  Escape peaks are
        not subtracted here: they are a redistribution within the recorded
        spectrum and belong to the line shape.
        """
        scalar = np.ndim(energy_ev) == 0
        e_kev = np.atleast_1d(np.asarray(energy_ev, dtype=float)) / 1000.0
        rho_air = air_density(self.air_temperature_c, self.air_pressure_hpa)
        rho_sensor = SENSOR_DENSITY_G_CM3[self.material]
```

```
        z_sensor = xraylib.SymbolToAtomicNumber(self.material)
        out = np.zeros_like(e_kev)
        for i, ek in enumerate(e_kev):
            ek = float(ek)
            try:
                t_air = np.exp(-xraylib.CS_Total_CP(AIR_FORMULA, ek) * rho_air * self.air_path_cm)
                t_win = np.exp(-xraylib.CS_Total_CP(self.window_material, ek)
                               * self.window_density_g_cm3 * self.window_thickness_um * 1e-4)
                t_dead = np.exp(-xraylib.CS_Total(z_sensor, ek) * rho_sensor * self.dead_layer_um * 1e-4)
                absorbed = 1.0 - np.exp(-xraylib.CS_Photo(z_sensor, ek) * rho_sensor
                                        * self.sensor_thickness_um * 1e-4)
            except ValueError:
                continue
            out[i] = t_air * t_win * t_dead * absorbed
        return float(out[0]) if scalar else out
```

5.3.4 `Detector.escape_lines`

```
Detector.escape_lines(energy_ev)
```

Sensor escape peaks of a line. A photon absorbed in the K shell of the sensor can be followed by a sensor K X-ray that leaves the crystal, so the event is recorded at $E - E_X$. For normal incidence on a semi-infinite sensor [25],

$$f_X = \tfrac{1}{2}\,\omega_K\,\frac{\tau_K}{\tau}\,p_X\left[1 - \frac{\mu_X}{\mu_E}\ln\!\left(1 + \frac{\mu_E}{\mu_X}\right)\right], \tag{18}$$

with ω_K the K fluorescence yield of the sensor, τ_K/τ the K-shell share of photoabsorption at E, p_X the radiative branching of the escaping line (Kα or Kβ), μ_E the photoabsorption coefficient at E and μ_X the attenuation of the escaping X-ray. Returns `[(escape energy in eV, fraction), ...]`, empty below the sensor's K edge. Typical values: 0.1 % for Ga/As K in Si, about 1 % just above the Si K edge.

xrf_fit.py, lines 305–339

```
    def escape_lines(self, energy_ev: float) -> List[Tuple[float, float]]:
        """Escape peaks of a line at ``energy_ev``: ``[(escape energy eV, fraction)]``.

        A photon absorbed in the sensor's K shell can be followed by a sensor
        K X-ray that leaves the crystal, so the event is recorded at
        ``E - E_X``.  For normal incidence on a semi-infinite sensor
        (Reed & Ware 1972)::

            f_X = 1/2 * omega_K * (tau_K / tau) * p_X * [1 - (mu_X/mu_E) ln(1 + mu_E/mu_X)]

        with ``tau_K/tau`` the K-shell share of photoabsorption at E,
        ``p_X`` the radiative branching of the escaping line (K-alpha,
        K-beta), ``mu_E`` the photoabsorption coefficient at E and ``mu_X``
        the attenuation of the escaping X-ray.  ~0.1 % for Ga/As K in Si,
        ~1 % just above the Si K edge.
        """
        z = xraylib.SymbolToAtomicNumber(self.material)
        e_kev = energy_ev / 1000.0
        if e_kev <= xraylib.EdgeEnergy(z, xraylib.K_SHELL):
            return []
        omega = xraylib.FluorYield(z, xraylib.K_SHELL)
        tau = xraylib.CS_Photo(z, e_kev)
        tau_k = xraylib.CS_Photo_Partial(z, xraylib.K_SHELL, e_kev)
        out = []
        for macro in (xraylib.KA_LINE, xraylib.KB_LINE):
            try:
                e_x = xraylib.LineEnergy(z, macro)
                p_x = xraylib.RadRate(z, macro)
            except ValueError:
                continue
            ratio = xraylib.CS_Total(z, e_x) / tau
            frac = 0.5 * omega * (tau_k / tau) * p_x * (1.0 - ratio * np.log1p(1.0 / ratio))
            if frac > 0:
                out.append((energy_ev - 1000.0 * e_x, float(frac)))
```

```
339         return out
```

5.3.5 sigma_ev, fwhm_ev, from_fwhm

`sigma_ev(energy_ev, noise_ev=None, fano=None)`, `fwhm_ev(energy_ev, **kw)`, `from_fwhm(fwhm_ev, at_energy_ev, material="Si", fano=0.12, gamma_ev=5.0, **kw)` (class method)

$\sigma(E)$ and FWHM from Eq. (16), optionally with other noise and Fano values (used by the fit). `from_fwhm` solves Eq. (16) for N given a quoted resolution ("135 eV at Mn Kα") and a Fano factor, and raises `ValueError` if the FWHM is below the Fano limit; extra keywords set the efficiency fields.

xrf_fit.py, lines 341–347

```
341     def sigma_ev(self, energy_ev: np.ndarray | float,
342                  noise_ev: Optional[float] = None,
343                  fano: Optional[float] = None) -> np.ndarray | float:
344         """Gaussian sigma (eV) at one or more energies."""
345         n = self.noise_ev if noise_ev is None else noise_ev
346         f = self.fano if fano is None else fano
347         return np.sqrt(n ** 2 + f * self.pair_energy_ev * np.asarray(energy_ev, dtype=float))
```

xrf_fit.py, lines 349–351

```
349     def fwhm_ev(self, energy_ev: np.ndarray | float, **kw) -> np.ndarray | float:
350         """Gaussian FWHM (eV) at one or more energies."""
351         return 2.3548 * self.sigma_ev(energy_ev, **kw)
```

xrf_fit.py, lines 353–372

```
353     @classmethod
354     def from_fwhm(cls, fwhm_ev: float, at_energy_ev: float, material: str = "Si",
355                   fano: float = 0.12, gamma_ev: float = 5.0, **kw) -> "Detector":
356         """Build a detector whose FWHM equals ``fwhm_ev`` at ``at_energy_ev``.
357
358         Solves for the electronic-noise term given a Fano factor, which is the
359         usual way a quoted resolution figure ("135 eV at Mn Ka") is turned into
360         a resolution law.  Extra keyword arguments set the detection-efficiency
361         fields (``air_path_cm``, ``window_thickness_um`` …).
362         """
363         sigma = fwhm_ev / 2.3548
364         var_fano = fano * PAIR_ENERGY_EV[material] * at_energy_ev
365         var_noise = sigma ** 2 - var_fano
366         if var_noise <= 0:
367             raise ValueError(
368                 f"FWHM {fwhm_ev} eV at {at_energy_ev} eV is below the Fano limit "
369                 f"({2.3548 * np.sqrt(var_fano):.1f} eV) for {material} with F={fano}."
370             )
371         return cls(material=material, noise_ev=float(np.sqrt(var_noise)), fano=fano,
372                    gamma_ev=gamma_ev, **kw)
```

5.4 Fluorescence lines and line groups

5.4.1 Line, LineGroup

`Line(name, energy_ev, rel)`, `LineGroup(element, family, lines, ref_strength=nan)`

A `Line` is an IUPAC name without the `_LINE` suffix, a tabulated energy and an intensity relative to the reference line of its group. A `LineGroup` is a set of lines of one element and one shell family (`"K"`, `"L"`, `"M"`) sharing one amplitude; `lines[0]` is the reference line. `ref_strength` keeps the absolute strength of the reference line—the normalisation that `rel` divides out: cross-section (cm^2/g) at the excitation energy, or flux-weighted cross-section, or matrix sensitivity, times the detection efficiency if a detector was given. It converts fitted areas into atom ratios. Properties: `label` (`"Cu K"`), `reference`, and `total_rel` ($\sum_l r_l$, converting the reference-line area into the area of the whole modelled group).

xrf_fit.py, lines 375–396

```
# ============================================================================
# Fluorescence lines
# ============================================================================

@dataclass(frozen=True)
class Line:
    """One fluorescence line.

    Attributes
    ----------
    name : str
        xraylib macro name without the ``_LINE`` suffix, e.g. ``"LA1"``.
    energy_ev : float
        Tabulated line energy in eV.
    rel : float
        Intensity relative to the most intense line of its group (reference
        line has ``rel == 1``).
    """

    name: str
    energy_ev: float
    rel: float
```

xrf_fit.py, lines 399–418

```
@dataclass
class LineGroup:
    """A set of lines sharing one free amplitude.

    One group per (element, shell family).  ``lines[0]`` is the reference line,
    whose integrated area is the fitted parameter.

    ``ref_strength`` keeps the *absolute* strength of the reference line, i.e.
    the normalisation that ``rel`` divides out: the mass fluorescence
    cross-section (cm^2/g) at ``e0_eff_kev`` or, under white-beam excitation,
    ``int sigma(E) dPhi/dE dE``, multiplied by the detection efficiency when a
    detector was supplied.  It is what converts fitted areas into atom-number
    ratios (:func:`number_ratio`).  Groups are only comparable if they were
    built with the same excitation and detector.
    """

    element: str
    family: str          # "K", "L" or "M"
    lines: List[Line]
    ref_strength: float = float("nan")
```

xrf_fit.py, lines 420–422

```
    @property
    def label(self) -> str:
        return f"{self.element} {self.family}"
```

xrf_fit.py, lines 424–426

```
    @property
    def reference(self) -> Line:
        return self.lines[0]
```

xrf_fit.py, lines 428–432

```
    @property
    def total_rel(self) -> float:
        """Sum of relative intensities — converts the fitted reference-line area
        into the total area of the modelled group."""
        return float(sum(l.rel for l in self.lines))
```

5.4.2 `_family_of`, `_all_line_macros`, `_is_iupac_line`

`_family_of(macro_name)`, `_all_line_macros()`, `_is_iupac_line(macro_name)`

xraylib exposes every transition twice—under its Siegbahn name (`KA1`, `LB2`, `LG1`, `LL`) and under its IUPAC name (`KL3`, `L3N5`, `L2N4`, `L3M1`)—plus composite macros (`KA` = KL3 + KL2). Enumerating all of them would count intensity two or three times, so only the IUPAC set is used: it is complete, non-redundant, and has one name per transition. The regular expression demands a full subshell designator on both sides (`L3M5` matches, `LL` does not). `_family_of` gives the family from the initial vacancy; `_all_line_macros` collects every `*_LINE` constant of xraylib.

xrf_fit.py, lines 435–444

```
435 #: Shell family of a line, from the initial-vacancy shell in its xraylib name.
436 def _family_of(macro_name: str) -> Optional[str]:
437     base = macro_name[:-5] if macro_name.endswith("_LINE") else macro_name
438     if base.startswith("K"):
439         return "K"
440     if base.startswith("L"):
441         return "L"
442     if base.startswith("M"):
443         return "M"
444     return None
```

xrf_fit.py, lines 447–449

```
447 def _all_line_macros() -> Dict[str, int]:
448     """All ``*_LINE`` macros exposed by xraylib, keyed by name."""
449     return {n: getattr(xraylib, n) for n in dir(xraylib) if n.endswith("_LINE")}
```

xrf_fit.py, lines 452–469

```
452 _LINE_MACROS = _all_line_macros()
453
454 #: xraylib exposes every transition twice: once under its Siegbahn name
455 #: (``KA1``, ``LB2``, ``LG1``, ``LL`` …) and once under its IUPAC name
456 #: (``KL3``, ``L3N5``, ``L2N4``, ``L3M1`` …), plus composite macros (``KA`` =
457 #: KL3+KL2, ``LA`` = LA1+LA2).  Enumerating all of them would double- or
458 #: triple-count intensity, so we keep only the IUPAC set: it is complete,
459 #: non-redundant, and one name per physical transition.
460 #:
461 #: The pattern demands a full subshell designator on both sides, which is what
462 #: separates IUPAC from Siegbahn: ``L3M5`` matches, ``LL`` (= L3M1) does not,
463 #: because its initial shell lacks a subshell index.
464 _IUPAC_LINE_RE = re.compile(r"^(K|L[1-3]|M[1-5])(K|L[1-3]|M[1-5]|N[1-7]|O[1-7]|P[1-5])$")
465
466
467 def _is_iupac_line(macro_name: str) -> bool:
468     base = macro_name[:-5] if macro_name.endswith("_LINE") else macro_name
469     return bool(_IUPAC_LINE_RE.match(base))
```

5.4.3 `_line_energy_ev`

```
_line_energy_ev(z, macro)
```

Tabulated line energy in eV [4, 26], or `None` if the line does not exist for that element.

xrf_fit.py, lines 472–477

```
472 def _line_energy_ev(z: int, macro: int) -> Optional[float]:
473     try:
474         e = xraylib.LineEnergy(z, macro)
475     except ValueError:
476         return None
477     return e * 1000.0 if e > 0 else None
```

5.4.4 `_line_strength`

```
_line_strength(z, macro, e0_kev, weighting)
```

Unnormalised intensity of a line under monochromatic excitation at E_0:

- `"kissel"`: the partial fluorescence cross-section $Q_l(E_0)$ (cm^2/g), which includes the photoionisation cross-section of each subshell, the Coster–Kronig transfers between subshells and the fluorescence yields [27, 28], and is therefore comparable across subshells;
- `"radrate"`: the radiative rate, normalised *within* each subshell, hence comparable only between lines sharing an initial vacancy. Across an L series it is wrong: for Ir it predicts Lβ_1/Lα_1 = 1.13 where the Kissel cross-sections give 0.65 and the measured ratio on the NIO295 map is 0.77.

Returns 0 below the edge.

xrf_fit.py, lines 520–534

```
520 def _line_strength(z: int, macro: int, e0_kev: float, weighting: str) -> float:
521     """Unnormalised line intensity.
522 
523     ``"kissel"`` uses the partial fluorescence cross-section at ``e0_kev``
524     (cm^2/g), which is comparable across subshells.  ``"radrate"`` uses the
525     radiative rate, which is only comparable *within* a subshell.
526     """
527     try:
528         if weighting == "kissel":
529             return max(0.0, xraylib.CS_FluorLine_Kissel(z, macro, e0_kev))
530         if weighting == "radrate":
531             return max(0.0, xraylib.RadRate(z, macro))
532     except ValueError:
533         return 0.0
534     raise ValueError(f"weighting must be 'kissel' or 'radrate', got {weighting!r}")
```

5.4.5 `_line_strength_spectral`

```
_line_strength_spectral(z, macro, energy_kev, flux)
```

The correct strength under polychromatic excitation:

$$S_l = \int Q_l(E)\,\frac{\mathrm{d}\Phi}{\mathrm{d}E}\,\mathrm{d}E, \tag{19}$$

with the per-0.1 % bandwidth flux converted to a density per keV (Eq. (1); omitting the Jacobian inflated the Ga/As k-factor by about 9 % on BM32). Each subshell is excited only by the part of the spectrum above its own edge. The integral uses the trapezoidal rule, which cannot overshoot at the edge discontinuity (Simpson's rule can); with a grid step h the edge error is of order h times the jump, so a grid of ∼10 eV keeps it below 0.1 %.

xrf_fit.py, lines 480–517

```
480 def _line_strength_spectral(z: int, macro: int,
481                            energy_kev: np.ndarray, flux: np.ndarray) -> float:
482     """Flux-weighted line strength, ``integral sigma_l(E) * dPhi/dE dE``.
483 
484     The correct relative-intensity measure under polychromatic illumination:
485     each subshell is excited only by the part of the spectrum above its own
486     edge, so subshells whose edges sit inside the band are down-weighted
487     relative to what a single effective energy would give.
488 
489     ``flux`` is taken **per 0.1 % bandwidth**, as returned by
490     ``xrf_beamline.Beamline.spectrum``, and converted here to a per-keV
491     density (``/ (1e-3 E)``) before integrating over ``dE``.  Omitting that
492     Jacobian over-weights high energies by a factor E; on BM32 it inflated the
493     Ga/As k-factor by ~9 %.
494 
495     Integrated with the trapezoidal rule on the supplied grid, which cannot
496     overshoot at the absorption edge (Simpson's rule can).  Use a fine grid:
497     the integrand is zero below the edge and jumps discontinuously above it,
498     so the edge error is of order one grid step times the jump.
499 
500     Parameters
501     ----------
502     energy_kev, flux : (n,) arrays
```

```
        Incident spectrum, flux per 0.1 % BW.  Only the shape matters; any
        constant scale cancels in ratios.
    """
    cs = np.zeros_like(energy_kev)
    for i, ek in enumerate(energy_kev):
        try:
            v = xraylib.CS_FluorLine_Kissel(z, macro, float(ek))
        except ValueError:
            continue                        # below the edge
        if v > 0:
            cs[i] = v
    if not np.any(cs):
        return 0.0
    density = per_unit_energy(energy_kev, flux)
    return float(max(0.0, trapezoid(cs * density, x=energy_kev)))
```

5.4.6 `element_line_groups`

```
element_line_groups(elements, emin_ev, emax_ev, e0_eff_kev=20.0, families=("K","L","M"),
min_rel=0.01, weighting="kissel", merge_within_ev=0.0, incident_spectrum=None, detector=None,
matrix=None)
```

Builds the line groups to fit.

`elements`
: symbols or atomic numbers.

`emin_ev, emax_ev`
: only lines inside this window are kept (normally the fit range).

`e0_eff_kev`
: single excitation energy for the cross-sections; it must exceed the highest relevant edge, or high-subshell lines silently get zero intensity.

`families`
: shell families considered.

`min_rel`
: lines weaker than this fraction of the group's reference line are dropped.

`weighting`
: `"kissel"` or `"radrate"` (ignored with a spectrum or a matrix).

`merge_within_ev`
: if positive, lines closer than this are merged into one effective line at their intensity-weighted energy (to shrink the model; the detector cannot resolve them anyway).

`incident_spectrum`
: `(energy_kev, flux_per_0.1%BW)`: strengths become Eq. (19).

`detector`
: multiplies every strength by $\epsilon(E_l)$, so that relative intensities and `ref_strength` describe the *recorded* spectrum.

`matrix`
: an `xrf_matrix.MatrixModel`: strengths include self-absorption (and secondary fluorescence for the lines that can survive `min_rel`); it carries its own spectrum, so `incident_spectrum` and `e0_eff_kev` are ignored. `ref_strength` then means sensitivity per unit mass fraction in the target layer.

Returns one `LineGroup` per (element, family) with at least one line left, in the order of `elements`; lines sorted by decreasing relative intensity.

Method For each element and family, every IUPAC line of that family inside the window gets its strength (cross-section, flux-weighted cross-section, or matrix emission), times the efficiency; the strongest defines the reference; lines below `min_rel` are dropped.

xrf_fit.py, lines 537–677

```
537 def element_line_groups(
538     elements: Sequence[str | int],
539     emin_ev: float,
540     emax_ev: float,
541     e0_eff_kev: float = 20.0,
542     families: Sequence[str] = ("K", "L", "M"),
543     min_rel: float = 0.01,
544     weighting: str = "kissel",
545     merge_within_ev: float = 0.0,
546     incident_spectrum: Optional[Tuple[np.ndarray, np.ndarray]] = None,
547     detector: Optional["Detector"] = None,
548     matrix: Optional["MatrixModel"] = None,
549 ) -> List[LineGroup]:
550     """Build the line groups to fit.
551 
552     Parameters
553     ----------
554     elements
555         Element symbols (``"Nd"``) or atomic numbers (``60``).
556     emin_ev, emax_ev
557         Only lines inside this energy window are kept — normally the fit ROI.
558     e0_eff_kev
559         Effective excitation energy for the Kissel cross-sections.  Must exceed
560         the highest edge you care about; see the module docstring.
561     families
562         Shell families to consider.
563     min_rel
564         Discard lines weaker than this fraction of their group's reference line.
565     weighting
566         ``"kissel"`` (default, correct across subshells) or ``"radrate"``.
567         Ignored when ``incident_spectrum`` is given.
568     incident_spectrum : (energy_kev, flux), optional
569         Incident white-beam spectrum.  When supplied, line strengths become the
570         flux-weighted integrals ``int sigma_l(E) Phi(E) dE`` instead of
571         cross-sections at the single energy ``e0_eff_kev``.  This is the correct
572         treatment for polychromatic illumination and is what changes the L
573         subshell weights of heavy elements, whose edges fall inside the band.
574         ``flux`` is per 0.1 % bandwidth (the 1/E conversion is applied
575         internally) and only its shape matters.  Build one with
576         ``xrf_beamline.BM32_WHITE_BEAM().spectrum(energy_grid)[1]``; a grid
577         step of ~10 eV keeps the edge discretisation error below 0.1 %.
578     detector : Detector, optional
579         If given, every line strength is multiplied by
580         ``detector.efficiency(E_line)``, so the relative intensities and
581         ``LineGroup.ref_strength`` describe the *recorded* spectrum rather
582         than the emitted one.  Needed for K-beta/K-alpha and L-series ratios
583         whose lines differ in efficiency, and for :func:`number_ratio`.
584     matrix : xrf_matrix.MatrixModel, optional
585         Sample-aware strengths: primary fluorescence with self-absorption in
586         the layer (incident and emitted paths, at the model's geometry) plus
587         secondary fluorescence.  Carries its own incident spectrum, so
588         ``incident_spectrum`` and ``e0_eff_kev`` are ignored.  ``ref_strength``
589         then means sensitivity per unit mass fraction in that matrix.  Since
590         absorption and enhancement depend on the composition, use
591         :func:`xrf_matrix.quantify_layer` to iterate it to self-consistency.
592     merge_within_ev
593         If > 0, lines closer than this are merged into one effective line at
594         their intensity-weighted mean energy.  Useful to collapse unresolvable
595         multiplets (the detector FWHM is ~130 eV) and shrink the model.
596 
597     Returns
598     -------
599     list of LineGroup
600         One per (element, family) that has at least one surviving line, ordered
601         as given in ``elements``.  Each group's ``lines`` are sorted by
602         descending relative intensity, so ``lines[0]`` is the reference.
603     """
604     groups: List[LineGroup] = []
```

```
605
606     for item in elements:
607         if isinstance(item, str):
608             try:
609                 z = xraylib.SymbolToAtomicNumber(item.capitalize())
610             except ValueError:
611                 logger.warning("Unknown element symbol %r — skipped.", item)
612                 continue
613         else:
614             z = int(item)
615         symbol = xraylib.AtomicNumberToSymbol(z)
616
617         for fam in families:
618             found: List[Tuple[str, float, float]] = []   # (name, energy, strength)
619
620             for name, macro in _LINE_MACROS.items():
621                 if not _is_iupac_line(name) or _family_of(name) != fam:
622                     continue
623                 e_ev = _line_energy_ev(z, macro)
624                 if e_ev is None or not (emin_ev <= e_ev <= emax_ev):
625                     continue
626                 if matrix is not None:
627                     s = matrix.emission(z, macro, e_ev / 1000.0, secondary=False)
628                 elif incident_spectrum is None:
629                     s = _line_strength(z, macro, e0_eff_kev, weighting)
630                 else:
631                     s = _line_strength_spectral(z, macro, *incident_spectrum)
632                 if detector is not None:
633                     s *= detector.efficiency(e_ev)
634                 if s <= 0:
635                     continue
636                 found.append((name[:-5], e_ev, s))
637
638             if not found:
639                 continue
640
641             if matrix is not None and matrix.secondary_enabled:
642                 # secondary fluorescence only for lines that can survive min_rel
643                 s_max_p = max(s for _, _, s in found)
644                 updated = []
645                 for n, e, s in found:
646                     if s >= 0.5 * min_rel * s_max_p:
647                         s = matrix.emission(z, _LINE_MACROS[n + "_LINE"], e / 1000.0,
648                                             secondary=True)
649                         if detector is not None:
650                             s *= detector.efficiency(e)
651                     updated.append((n, e, s))
652                 found = updated
653
654             if merge_within_ev > 0:
655                 found = _merge_close_lines(found, merge_within_ev)
656
657             s_max = max(s for _, _, s in found)
658             lines = [Line(name=n, energy_ev=e, rel=s / s_max)
659                      for n, e, s in found if s / s_max >= min_rel]
660             lines.sort(key=lambda l: -l.rel)
661
662             if lines:
663                 ref_strength = float(s_max)
664                 if matrix is not None and "+" not in lines[0].name:
665                     # per unit mass fraction in the target layer (the emission
666                     # weighting above includes the stack composition)
667                     e_ref = lines[0].energy_ev
668                     ref_strength = matrix.sensitivity(z, _LINE_MACROS[lines[0].name + "_LINE"],
669                                                       e_ref / 1000.0)
670                     if detector is not None:
671                         ref_strength *= detector.efficiency(e_ref)
672                 groups.append(LineGroup(element=symbol, family=fam, lines=lines,
673                                         ref_strength=float(ref_strength)))
674                 logger.info("%s %s: %d lines, reference %s at %.0f eV",
675                             symbol, fam, len(lines), lines[0].name, lines[0].energy_ev)
676
677     return groups
```

5.4.7 _merge_close_lines, _collapse

`_merge_close_lines(found, tol_ev), _collapse(bucket)`

Lines sorted by energy are grouped while consecutive lines are within `tol_ev`; each group becomes one line at the strength-weighted mean energy, with the summed strength and the name of its strongest member plus the number of merged lines (`"L3M5+1"`).

xrf_fit.py, lines 680–693

```
680 def _merge_close_lines(found: List[Tuple[str, float, float]],
681                       tol_ev: float) -> List[Tuple[str, float, float]]:
682     """Collapse lines within ``tol_ev`` into intensity-weighted effective lines."""
683     found = sorted(found, key=lambda t: t[1])
684     merged: List[Tuple[str, float, float]] = []
685     bucket = [found[0]]
686     for entry in found[1:]:
687         if entry[1] - bucket[-1][1] <= tol_ev:
688             bucket.append(entry)
689         else:
690             merged.append(_collapse(bucket))
691             bucket = [entry]
692     merged.append(_collapse(bucket))
693     return merged
```

xrf_fit.py, lines 696–704

```
696 def _collapse(bucket: List[Tuple[str, float, float]]) -> Tuple[str, float, float]:
697     s_tot = sum(s for _, _, s in bucket)
698     if s_tot <= 0:
699         return bucket[0]
700     e_eff = sum(e * s for _, e, s in bucket) / s_tot
701     name = bucket[int(np.argmax([s for _, _, s in bucket]))][0]
702     if len(bucket) > 1:
703         name = f"{name}+{len(bucket) - 1}"
704     return (name, e_eff, s_tot)
```

5.5 Normalisation, averaging and counting statistics

5.5.1 normalise_spectra

`normalise_spectra(spectra, monitor=None, live_time=None, normalise_monitor=True)`

Corrects raw MCA spectra for the incident flux and the detector dead time,

$$y_i = \frac{x_i}{s_i}, \qquad s_i = \frac{M_i}{\langle M \rangle}\, t_{\mathrm{live},i}, \tag{20}$$

with M_i the monitor of pixel i (divided by its mean if `normalise_monitor`) and t_{live} the live time reported by the XIA electronics (`xia_det0_live_time`). Dividing by the live time turns recorded counts into an incoming rate. On BM32 white-beam maps the dead-time fraction varies strongly with position (0.33–0.79 on the NIO295 map, a factor 1.5–4.8 in the correction) because it follows the total scattering power, so it does not cancel in elemental ratios.

Returns `(corrected, scale)`; pixels with a non-positive or non-finite scale are NaN.

xrf_fit.py, lines 707–771

```
707 # ============================================================================
708 # Normalisation and averaging
709 # ============================================================================
710
711 def normalise_spectra(
712     spectra: np.ndarray,
713     monitor: Optional[np.ndarray] = None,
714     live_time: Optional[np.ndarray] = None,
715     normalise_monitor: bool = True,
716 ) -> Tuple[np.ndarray, np.ndarray]:
```

```
    """Correct raw MCA spectra for incident flux and detector dead time.

    Both corrections are multiplicative per pixel::

        y_corrected = y_raw / (monitor_rel * live_time)

    ``live_time`` is the *active* counting time reported by the XIA
    (``xia_det0_live_time``), so dividing by it converts recorded counts into a
    true incoming rate.  This is the correction that matters most on BM32 white-beam
    maps: the dead-time fraction is strongly position dependent (0.33-0.79 on the
    NIO295 map, i.e. a factor 1.5-4.8 in the correction) because it tracks total
    scattering power, so it does **not** cancel in an elemental ratio.

    ``monitor`` is divided by its own mean when ``normalise_monitor`` is True, so
    the output keeps the magnitude of counts-per-live-second rather than becoming
    counts per monitor count.

    Parameters
    ----------
    spectra : (n_pixels, n_channels) array
    monitor : (n_pixels,) array, optional
    live_time : (n_pixels,) array, optional
        In seconds.
    normalise_monitor : bool
        Divide the monitor by its mean before use.

    Returns
    -------
    corrected : (n_pixels, n_channels) float array
        Pixels with a non-positive or non-finite scale are set to NaN.
    scale : (n_pixels,) float array
        The divisor actually applied.
    """
    spectra = np.asarray(spectra, dtype=float)
    n = spectra.shape[0]
    scale = np.ones(n, dtype=float)

    if monitor is not None:
        mon = np.asarray(monitor, dtype=float)
        if normalise_monitor:
            good = np.isfinite(mon) & (mon > 0)
            if not good.any():
                raise ValueError("monitor has no positive finite values.")
            mon = mon / mon[good].mean()
        scale *= mon

    if live_time is not None:
        scale *= np.asarray(live_time, dtype=float)

    bad = ~np.isfinite(scale) | (scale <= 0)
    if bad.any():
        logger.warning("%d/%d pixels have an invalid normalisation and are set to NaN.",
                       int(bad.sum()), n)
    safe = np.where(bad, np.nan, scale)
    return spectra / safe[:, None], scale
```

5.5.2 `_pixel_rows`

```
_pixel_rows(spectra, name)
```

Reshapes any array whose last axis is the channel axis—a single spectrum, a list of pixels, a gridded map—into $(n_{\text{pixels}}, n_{\text{channels}})$.

xrf_fit.py, lines 774–784

```
def _pixel_rows(spectra: np.ndarray, name: str = "spectra") -> np.ndarray:
    """Spectra as a (n_pixels, n_channels) float array, whatever the scan shape.

    The last axis is the channel axis; every leading axis is a pixel axis, so a
    single spectrum (n_channels,), a list of pixels (n_pixels, n_channels) and a
    gridded map (n_slow, n_fast, n_channels) are all accepted.
    """
```

```
    arr = np.asarray(spectra, dtype=float)
    if arr.ndim == 0 or arr.shape[-1] == 0:
        raise ValueError(f"{name} must have a channel axis; got shape {arr.shape}.")
    return arr.reshape(-1, arr.shape[-1])
```

5.5.3 average_spectrum

```
average_spectrum(corrected)
```

The mean of the valid pixels (those without NaN) and the standard error of that mean, $s/\sqrt{n}$, per channel. The standard error contains both counting noise and real pixel-to-pixel variation. With a single spectrum (`ct` scan) the standard error is undefined and returned as NaN; use `counting_error` for the fit weights.

Returns `(mean, sem, n_used)`.

xrf_fit.py, lines 787–824

```
def average_spectrum(corrected: np.ndarray) -> Tuple[np.ndarray, np.ndarray, int]:
    """Map-averaged spectrum and the standard error of that mean.

    The per-channel standard error across pixels is the right uncertainty for
    fitting an *average* spectrum: it propagates both counting statistics and
    genuine pixel-to-pixel variation, and needs no assumption about how the
    normalisation transformed the Poisson variance.

    Works for any scan shape: a single-point measurement (``ct``), a line scan
    or a map, gridded or not (see ``_pixel_rows``).

    Parameters
    ----------
    corrected : (n_channels,), (n_pixels, n_channels) or (..., n_channels) array
        Normalised spectra, e.g. from :func:`normalise_spectra`.

    Returns
    -------
    mean, sem : (n_channels,) arrays
        With a single valid spectrum, ``mean`` is that spectrum and ``sem`` is
        NaN: there is no pixel-to-pixel spread to estimate. Use
        :func:`counting_error` for the fit weights, as for a map.
    n_used : int
        Number of pixels contributing (pixels containing NaN are dropped).
    """
    data = _pixel_rows(corrected, "corrected")
    data = data[np.isfinite(data).all(axis=1)]
    n = data.shape[0]
    if n == 0:
        raise ValueError("No valid pixel: every spectrum contains NaN or inf.")
    mean = data.mean(axis=0)
    if n == 1:
        logger.info("Single spectrum: the standard error across pixels is undefined (NaN); "
                    "use counting_error() for the uncertainty.")
        sem = np.full_like(mean, np.nan)
    else:
        sem = data.std(axis=0, ddof=1) / np.sqrt(n)
    return mean, sem, int(n)
```

5.5.4 counting_error

```
counting_error(raw_spectra, scale)
```

The Poisson uncertainty of the normalised average. With $\bar{y} = \frac{1}{n}\sum_i x_i/s_i$ and $\mathrm{Var}(x_i) = x_i$,

$$\sigma_{\bar{y}} = \frac{1}{n}\sqrt{\sum_i \frac{x_i}{s_i^2}}. \tag{21}$$

This is the right weight for fitting the *average* spectrum: pixel-to-pixel variation is signal, not noise, as far as the average is concerned, and on a heterogeneous map the standard error can exceed this by an order of magnitude. Accepts any scan shape; `scale` can be a scalar.

xrf_fit.py, lines 827–862

```
827 def counting_error(raw_spectra: np.ndarray, scale: np.ndarray) -> np.ndarray:
828     """Poisson uncertainty of the normalised map-averaged spectrum.
829 
830     With ``y_avg = (1/N) * sum_i y_i / s_i`` and Poisson raw counts
831     (``Var(y_i) = y_i``), the variance of the average is
832     ``(1/N^2) * sum_i y_i / s_i^2``.
833 
834     This is the appropriate weight when fitting the *average* spectrum, and it is
835     not the same as the standard error returned by :func:`average_spectrum`: the
836     latter also contains genuine pixel-to-pixel variation, which is signal rather
837     than noise as far as the average is concerned.  On a heterogeneous map the
838     SEM can exceed this by an order of magnitude.
839 
840     Parameters
841     ----------
842     raw_spectra : (n_channels,), (n_pixels, n_channels) or (..., n_channels) array
843         **Raw** counts, before normalisation.
844     scale : scalar or array with one value per pixel
845         The divisor applied by :func:`normalise_spectra`.
846 
847     Returns
848     -------
849     (n_channels,) array
850         1-sigma uncertainty, in the same units as the normalised average.
851     """
852     raw = _pixel_rows(raw_spectra, "raw_spectra")
853     s = np.asarray(scale, dtype=float).reshape(-1)
854     if s.size == 1:
855         s = np.full(raw.shape[0], s[0])
856     if s.size != raw.shape[0]:
857         raise ValueError(f"scale has {s.size} values for {raw.shape[0]} spectra.")
858     good = np.isfinite(s) & (s > 0)
859     if not good.any():
860         raise ValueError("No pixel has a valid normalisation scale.")
861     n = int(good.sum())
862     return np.sqrt((raw[good] / s[good, None] ** 2).sum(axis=0)) / n
```

5.6 Pile-up

5.6.1 `pileup_spectra`

```
pileup_spectra(raw_spectra, calibration, real_time=1.0, chunk=512)
```

Per-pixel pile-up spectra per second of resolving time. For random arrivals, two events closer than the pulse-pair resolving time τ are recorded as one event at $E_1 + E_2$. With x the measured spectrum, recorded in the real time t, the number of such pairs at sum energy E is [29]

$$N_{\mathrm{pp}}(E) = \frac{\tau}{t}\,(x \otimes x)(E), \tag{22}$$

where $\otimes$ is the self-convolution over ordered pairs; the total, $\tau N^2/t = N\,r\,\tau$ with r the count rate, is the expected number of pairs. The function returns $(x \otimes x)/t$ on the MCA channel grid, so multiplying by τ in seconds gives counts.

Method The convolution is computed with FFTs (padded to a power of two above $2n$, 30), in batches of `chunk` pixels. The self-pair term is made unbiased by subtracting x_c at index $2c$ ($x_c(x_c-1)$ pairs instead of x_c^2), which matters at low counts. Index n of the convolution has energy $Gn + 2O$ (the offset counts twice), so it is mapped back to channel $c = n + O/G$ by linear interpolation.

Hypotheses and limits

Random (Poisson) arrivals and a single resolving time; pile-up of three or more events is neglected. Pile-up removes parent events with a probability independent of their energy, so it does not bias line *ratios*; it adds sum peaks that other components would otherwise absorb.

xrf_fit.py, lines 865–923

```
865 def pileup_spectra(raw_spectra: np.ndarray, calibration: Calibration,
866                     real_time: np.ndarray | float = 1.0, chunk: int = 512) -> np.ndarray:
867     """Per-pixel pile-up spectra per second of resolving time.
868 
869     For random arrivals, two events closer than the pulse-pair resolving time
870     ``tau`` are recorded as one event at ``E1 + E2``.  With ``N`` recorded
871     counts in a real time ``t``, the number of such pairs with sum energy ``E``
872     is::
873 
874         N_pp(E) = (tau / t) * (x ⊗ x)(E)
875 
876     where ``x`` is the measured spectrum and ⊗```` the self-convolution over
877     ordered pairs.  This returns ``(x ⊗ x) / t`` mapped back onto the MCA
878     channel grid, so multiplying by ``tau`` (seconds) gives counts.
879 
880     Details: the energy offset is accounted for (a sum of two channel energies
881     carries the offset twice), and the self-pair term is made unbiased
882     (``x_c (x_c - 1)`` instead of ``x_c^2``), which matters at low counts.
883 
884     Pile-up removes the two parent events from their peaks with a probability
885     independent of their energy, so it does not bias line *ratios*; what it
886     does is put sum peaks in the spectrum, which other components would
887     otherwise absorb.
888 
889     Parameters
890     ----------
891     raw_spectra : (n_pixels, n_channels) array
892         Raw counts over the **full** MCA range.
893     calibration : Calibration
894     real_time : float or (n_pixels,) array
895         Real (elapsed) acquisition time per pixel, s
896         (``xia_det0_elapsed_time`` on BM32).
897     chunk : int
898         Pixels per FFT batch, to bound memory.
899 
900     Returns
901     -------
902     (n_pixels, n_channels) array
903     """
904     raw = np.atleast_2d(np.asarray(raw_spectra, dtype=float))
905     n_pix, n_ch = raw.shape
906     rt = np.broadcast_to(np.asarray(real_time, dtype=float), (n_pix,))
907     nfft = 1 << int(np.ceil(np.log2(2 * n_ch)))
908     # energy of conv index n is gain*n + 2*offset  ->  channel c = n + offset/gain
909     src = np.arange(n_ch) - calibration.offset / calibration.gain
910     n0 = np.floor(src).astype(int)
911     fr = src - n0
912     valid = (n0 >= 0) & (n0 + 1 < 2 * n_ch)
913     out = np.zeros((n_pix, n_ch))
914     for s in range(0, n_pix, chunk):
915         x = raw[s:s + chunk]
916         X = np.fft.rfft(x, nfft, axis=1)
917         conv = np.fft.irfft(X * X, nfft, axis=1)[:, :2 * n_ch]
918         conv[:, 0:2 * n_ch:2] -= x                    # unbiased self-pairs
919         np.clip(conv, 0.0, None, out=conv)            # FFT round-off
920         out[s:s + chunk, valid] = (conv[:, n0[valid]] * (1.0 - fr[valid])
921                                    + conv[:, n0[valid] + 1] * fr[valid])
922         out[s:s + chunk] /= rt[s:s + chunk, None]
923     return out
```

5.6.2 average_pileup_template

```
average_pileup_template(pileup, scale)
```

Pile-up template for the normalised average spectrum, per microsecond of resolving time: $10^{-6}\,\langle P_i/s_i\rangle$. It must be built from the per-pixel spectra, since pile-up goes as the square of the local rate; the fitted amplitude is then τ in µs.

xrf_fit.py, lines 926–937

```
926 def average_pileup_template(pileup: np.ndarray, scale: np.ndarray) -> np.ndarray:
927     """Pile-up template for the map-averaged spectrum, per µs of resolving time.
928 
929     ``mean_i(P_i / s_i) * 1e-6`` with ``P_i`` from :func:`pileup_spectra` and
930     ``s_i`` the normalisation divisor: the same units as the averaged,
931     normalised spectrum.  The fitted amplitude is then ``tau`` in µs.  Using
932     the per-pixel spectra (not the average) matters: pile-up goes as the
933     *square* of the local rate.
934     """
935     s = np.asarray(scale, dtype=float)
936     good = np.isfinite(s) & (s > 0)
937     return (pileup[good] / s[good, None]).mean(axis=0) * 1e-6
```

5.7 Background estimate

5.7.1 snip_background

```
snip_background(y, width=40, iterations=24, smooth=True)
```

The Statistics-sensitive Non-linear Iterative Peak-clipping algorithm [31]. The spectrum (optionally smoothed over 3 channels) is transformed by the log-log-square-root operator $v = \ln[\ln(\sqrt{y+1}+1)+1]$, which compresses peaks and makes the clipping statistics-aware; then each point is replaced by $v_c \leftarrow \min\!\big(v_c, \tfrac{1}{2}(v_{c-w}+v_{c+w})\big)$ for windows $w = 1,\dots,W$ and then decreasing again for `iterations` passes (decreasing windows smooth the result, 32); finally the transform is inverted. `width` should exceed the half-width of the widest peak. Used by the coarse calibration; the fits themselves model the background.

xrf_fit.py, lines 940–984

```
940 def snip_background(y: np.ndarray, width: int = 40, iterations: int = 24,
941                    smooth: bool = True) -> np.ndarray:
942     """SNIP (Statistics-sensitive Non-linear Iterative Peak-clipping) background.
943 
944     The standard XRF continuum estimator (Ryan et al. 1988): iteratively clips
945     each point to the mean of its neighbours at increasing then decreasing
946     separation, which erodes peaks while following the slowly varying continuum.
947     Operates on the log-log-square-root transformed spectrum so that the clipping
948     is statistics-aware.
949 
950     Parameters
951     ----------
952     y : (n,) array
953         Spectrum, assumed non-negative.
954     width : int
955         Maximum half-window in channels; should exceed the widest peak.
956     iterations : int
957         Number of decreasing-window passes after the increasing ones.
958     smooth : bool
959         Apply a 3-point moving average first, which stabilises the transform on
960         low-count channels.
961 
962     Returns
963     -------
964     (n,) array
965         Estimated background, same length as ``y``.
966     """
967     y = np.asarray(y, dtype=float).copy()
968     if smooth:
```

```
969         y = np.convolve(y, np.ones(3) / 3.0, mode="same")
970     y = np.clip(y, 0.0, None)
971
972     # LLS transform
973     v = np.log(np.log(np.sqrt(y + 1.0) + 1.0) + 1.0)
974
975     windows = list(range(1, width + 1)) + list(range(width, 0, -1))[:iterations]
976     for w in windows:
977         shifted_lo = np.roll(v, w)
978         shifted_hi = np.roll(v, -w)
979         shifted_lo[:w] = v[:w]
980         shifted_hi[-w:] = v[-w:]
981         v = np.minimum(v, 0.5 * (shifted_lo + shifted_hi))
982
983     # inverse LLS
984     return (np.exp(np.exp(v) - 1.0) - 1.0) ** 2 - 1.0
```

5.8 Line shape

5.8.1 `peak_shape`

```
peak_shape(x, sigma, gamma=0.0, tail_fraction=0.0, tail_slope=1.5, step_fraction=0.0)
```

The unit-area detector response to one line, plus its step, of the Hypermet type [33, 34]:

$$R(x) = (1-t)\,V(x;\sigma,\Gamma) + \frac{t}{2\beta}\,\exp\!\left(\frac{x}{\beta}+\frac{\sigma^2}{2\beta^2}\right)\operatorname{erfc}\!\left(\frac{x}{\sqrt{2}\sigma}+\frac{\sigma}{\sqrt{2}\beta}\right) + \frac{s}{2\sqrt{2\pi}\sigma}\,\operatorname{erfc}\!\left(\frac{x}{\sqrt{2}\sigma}\right), \quad (23)$$

where x is the distance to the line centre (in channels in the model), V is the Voigt profile (`scipy.special.voigt_profile`, Γ the Lorentzian half width) or the Gaussian when $\Gamma = 0$, t the tail fraction, β = `tail_slope`$\times\sigma$ the decay length of the tail, and s the step height relative to the Gaussian peak height $1/(\sqrt{2\pi}\sigma)$. The tail term is the convolution of a one-sided exponential $\mathrm{e}^{x/\beta}/\beta$ ($x<0$) with the Gaussian, and has unit area; the step is the convolution of a flat shelf with the Gaussian. Both describe incomplete charge collection: part of the charge of some events is lost near the entrance window or the sensor edges, so the event is recorded below the full energy.

Numerics The tail term overflows when evaluated directly: the exponential diverges where erfc underflows. With $z = (x/\sigma + \sigma/\beta)/\sqrt{2}$ and the scaled complementary error function $\operatorname{erfcx}(z) = \mathrm{e}^{z^2}\operatorname{erfc}(z)$, the identity $x/\beta + \sigma^2/2\beta^2 - z^2 = -x^2/2\sigma^2$ gives

$$\exp\!\left(\frac{x}{\beta}+\frac{\sigma^2}{2\beta^2}\right)\operatorname{erfc}(z) = \begin{cases} \mathrm{e}^{-x^2/2\sigma^2}\,\operatorname{erfcx}(z), & z > 0, \\ \exp\!\left(\frac{x}{\beta}+\frac{\sigma^2}{2\beta^2}\right)\operatorname{erfc}(z), & z \le 0, \end{cases} \quad (24)$$

both of which are finite (the exponent in the second case is capped at 50).

Hypotheses and limits

One tail and one step shared by all lines, in units of σ; real detectors show a tail fraction that decreases with energy and may need a second, long tail. The step is a fraction of the peak *height*, so its area per unit energy scales with $1/\sigma$.

xrf_fit.py, lines 987–1016

```
987 # ============================================================================
988 # The model
989 # ============================================================================
990
991 def peak_shape(x: np.ndarray, sigma: float, gamma: float = 0.0, tail_fraction: float = 0.0,
992                tail_slope: float = 1.5, step_fraction: float = 0.0) -> np.ndarray:
993     """Unit-area detector response to one line (plus its step), Hypermet-type.
994
995     ``(1 - t) * Voigt(x; sigma, gamma) + t * T(x) + s * h * erfc(x / (sqrt(2) sigma)) / 2``
996
```

```
    with ``T`` an exponential low-energy tail of decay length
    ``beta = tail_slope * sigma`` convolved with the Gaussian (unit area),
    ``h = 1 / (sqrt(2 pi) sigma)`` the Gaussian peak height, ``t`` the
    tail fraction and ``s`` the step height relative to the peak.  ``x`` is
    the distance to the line centre in any unit, the same as ``sigma``.
    """
    x = np.asarray(x, dtype=float)
    out = (1.0 - tail_fraction) * voigt_profile(x, sigma, gamma) if gamma > 0 else \
        (1.0 - tail_fraction) * np.exp(-0.5 * (x / sigma) ** 2) / (np.sqrt(2.0 * np.pi) * sigma)
    if tail_fraction > 0:
        beta = tail_slope * sigma
        z = (x / sigma + sigma / beta) / np.sqrt(2.0)
        # exp(x/beta + sigma^2/2beta^2) erfc(z), written to avoid overflow on both sides
        tail = np.where(z > 0, np.exp(-0.5 * (x / sigma) ** 2) * erfcx(np.maximum(z, 0.0)),
                        np.exp(np.minimum(x / beta + 0.5 * (sigma / beta) ** 2, 50.0))
                        * erfc(np.minimum(z, 0.0)))
        out = out + tail_fraction * tail / (2.0 * beta)
    if step_fraction > 0:
        out = out + step_fraction * erfc(x / (np.sqrt(2.0) * sigma)) / (2.0 * np.sqrt(2.0 * np.pi) * sigma)
    return out
```

5.9 Scatter peaks under monochromatic excitation

Under monochromatic excitation the spectrum also contains the photons scattered by the sample: elastically (Rayleigh) at the excitation energy, and inelastically (Compton) at lower energy. With a white beam the scattered radiation is a continuum and belongs to the background.

Compton energy. A photon of energy E_0 scattered by a free electron at rest through the angle θ has the energy [35]

$$E_C = \frac{E_0}{1 + \dfrac{E_0}{m_e c^2}(1 - \cos\theta)}. \tag{25}$$

Impulse approximation. Electrons are bound and moving, which broadens the Compton line. In the impulse approximation [36, 37] the energy distribution of photons scattered at angle θ is

$$\frac{\mathrm{d}^2\sigma}{\mathrm{d}\Omega\,\mathrm{d}E} \propto J(p_z)\,\frac{\mathrm{d}p_z}{\mathrm{d}E}, \qquad p_z = \frac{E_0 E(1-\cos\theta)/m_e c^2 - (E_0 - E)}{\sqrt{E_0^2 + E^2 - 2E_0 E\cos\theta}}, \tag{26}$$

where p_z is the projection of the electron momentum on the scattering vector, in units of $m_e c$ in Eq. (26) (multiplied by $1/\alpha = 137.036$ to obtain atomic units), and $J(p_z)$ is the Compton profile of the scatterer: the sum of the tabulated Hartree–Fock profiles of its atoms [5], available in xraylib as `ComptonProfile`. Outer electrons give a sharp core, inner shells broad wings. The peak is centred at $p_z = 0$, i.e. at E_C.

Range of scattering angles. A detector close to the sample (2 mm on the training spectrum of Section 10) collects photons scattered over several tens of degrees. Since Eq. (25) is non-linear in the angle, the Compton peak becomes broad and asymmetric; the model averages it over an angle range that is fitted.

5.9.1 `_angle_grid`

```
_angle_grid(angle_deg, spread_deg, n=15)
```

Scattering angles sampled uniformly over $\theta \pm \Delta\theta$ (15 points), clipped to 1–179°; a single angle when the spread is below 0.05°.

xrf_fit.py, lines 1019–1024

```
def _angle_grid(angle_deg: float, spread_deg: float, n: int = 15) -> np.ndarray:
```

```
    """Scattering angles sampled uniformly over ``angle +/- spread`` (one angle
    without spread), kept inside (1, 179) degrees."""
    if spread_deg <= 0.05:
        return np.array([angle_deg])
    return np.clip(angle_deg + spread_deg * np.linspace(-1.0, 1.0, n), 1.0, 179.0)
```

5.9.2 `ScatterSource`

```
ScatterSource(energies_ev, weights=(1.0,), angle_deg=90.0, scatterer=None)
```

The monochromatic excitation, for the elastic and Compton peaks.

`energies_ev, weights`
: excitation lines and their relative intensities (normalised internally), e.g. from `tube_lines("Mo")`.

`angle_deg`
: scattering angle between the incident beam and the direction to the detector: the starting value of the fitted Compton angle.

`scatterer`
: chemical formula of the scattering medium (`"H2O"`): the Compton peak then has the impulse-approximation shape; without it, the Compton peak is a Gaussian (averaged over the angle range).

Methods `compton_energy_ev(e_ev, angle_deg)` is Eq. (25); `profile_table()` tabulates $J(p_z)$ of one formula unit for $0 \le p_z \le 60$ atomic units (3001 points), summing the atoms of the formula with their multiplicities; `compton_pz` is Eq. (26) in atomic units.

Hypotheses and limits

- The prefactor of the impulse-approximation cross-section (Klein–Nishina factor and E/E_0 terms, 37, 38) is taken as constant across the peak, and the angular distribution as uniform over $\theta \pm \Delta\theta$ (no Klein–Nishina, solid angle or polarisation weighting). The free angle and spread absorb these approximations.
- The elastic peak has a free amplitude; its intensity is not computed from atomic form factors.
- The scattering medium is the given formula (the matrix), not the full sample; the profile of trace elements is negligible.

xrf_fit.py, lines 1027–1058

```
@dataclass
class ScatterSource:
    """Monochromatic excitation, for the elastic and Compton scatter peaks.

    Parameters
    ----------
    energies_ev : sequence of float
        Excitation line energies (e.g. Mo K-alpha1 and K-alpha2 of a
        monochromatised tube).
    weights : sequence of float
        Relative intensities of those lines (normalised internally).
    angle_deg : float
        Scattering angle (between the incident beam and the direction to the
        detector), the starting value of the fitted Compton angle.
    scatterer : str, optional
        Chemical formula of the scattering medium (e.g. ``"H2O"``).  The
        Compton peak then has the impulse-approximation shape, from the
        tabulated Compton profiles ``J(p_z)`` of its atoms (xraylib): a sharp
        core from the outer electrons and broad wings from the inner shells.
        Without it, the Compton peak is a Gaussian.

    Start the fit from a calibration within a few channels of the truth at
    the scatter peaks (e.g. :func:`xrf_calibration.calibration_from_peaks`
    on a fluorescence line and the elastic peak): from a start ~20 channels
    off, the optimiser can stop in a poorer minimum where the Compton
    angle spread, its extra width and the detector tails trade off.
```

```
    """

    energies_ev: Sequence[float]
    weights: Sequence[float] = (1.0,)
    angle_deg: float = 90.0
    scatterer: Optional[str] = None
```

xrf_fit.py, lines 1060–1068

```
    def __post_init__(self):
        e = np.atleast_1d(np.asarray(self.energies_ev, dtype=float))
        w = np.atleast_1d(np.asarray(self.weights, dtype=float))
        if w.size == 1 and e.size > 1:
            w = np.full(e.size, w[0])
        if w.size != e.size:
            raise ValueError("energies_ev and weights must have the same length.")
        self.energies_ev = tuple(e)
        self.weights = tuple(w / w.sum())
```

xrf_fit.py, lines 1070–1073

```
    @staticmethod
    def compton_energy_ev(e_ev: float | np.ndarray, angle_deg: float) -> float | np.ndarray:
        """Compton-scattered photon energy (free electron at rest)."""
        return e_ev / (1.0 + e_ev / 510998.95 * (1.0 - np.cos(np.radians(angle_deg))))
```

xrf_fit.py, lines 1075–1084

```
    def profile_table(self) -> Optional[Tuple[np.ndarray, np.ndarray]]:
        """``(p_z, J)`` of the scatterer, ``p_z >= 0`` in atomic units, ``J``
        summed over the atoms of one formula unit; None without a scatterer."""
        if not self.scatterer:
            return None
        comp = xraylib.CompoundParser(self.scatterer)
        pz = np.linspace(0.0, 60.0, 3001)
        j = sum(n * np.array([xraylib.ComptonProfile(z, float(q)) for q in pz])
                for z, n in zip(comp["Elements"], comp["nAtoms"]))
        return pz, j
```

xrf_fit.py, lines 1086–1094

```
    @staticmethod
    def compton_pz(e_ev: np.ndarray, e0_ev: float, angle_deg: float) -> np.ndarray:
        """Electron momentum component ``p_z`` (atomic units) that scatters a
        photon of ``e0_ev`` into ``e_ev`` at ``angle_deg`` (impulse approximation)."""
        m = 510998.95
        c = np.cos(np.radians(angle_deg))
        e = np.asarray(e_ev, dtype=float)
        q = np.sqrt(np.maximum(e0_ev ** 2 + e ** 2 - 2.0 * e0_ev * e * c, 1e-12))
        return 137.036 * (e0_ev * e * (1.0 - c) / m - (e0_ev - e)) / q
```

5.10 `XRFModel`

5.10.1 `XRFModel`

```
XRFModel(groups, calibration, detector, channels, background_order=1, fit_resolution=True,
fit_gamma=True, escape=False, pileup_template=None, fit_pileup=True, pileup_tau_us=0.2,
fit_calibration=False, fit_air_path=False, fit_tails=False, scatter=None,
background_kind="poly")
```

Eq. (15) as a callable with the signature that `curve_fit` expects, `model(channels, *params)`. Table 3 lists the parameters, in their order in the parameter vector; `param_names` holds the names.

`groups`
: from `element_line_groups` (may be empty with a scatter source).

`calibration`
: fixed calibration, or the starting point when it is fitted.

`detector`
: resolution, efficiency and line shape; supplies the starting (or fixed) values of the shared parameters.

`channels`
: channel grid of the fit range.

`background_order`
: polynomial order (`None`: no background term).

`background_kind`
: `"poly"`: the background is the polynomial; `"exp"`: its exponential, positive by construction and closer to a scatter continuum over a wide range (non-linear in its coefficients).

`fit_resolution, fit_gamma`
: free N, F and Γ.

`escape`
: add the escape peaks of every line, tied to its amplitude (no extra parameter): the group amplitude then counts parent and escape, i.e. every photon absorbed in the sensor, which is what `Detector.efficiency` describes.

`pileup_template, fit_pileup, pileup_tau_us`
: pile-up spectrum per µs (full MCA range), with τ free or fixed. The sum peaks of the main lines usually lie above the analysis range: fit τ once over an extended range and fix it in the analysis.

`fit_calibration`
: `True` (gain and offset), `"offset"` (energy shift only) or `False`.

`fit_air_path`
: free air path d. The relative intensities were computed at d_0 = `detector.air_path_cm`; at d each line is rescaled by $\tau_l = \exp[-(\mu_l - \mu_{\mathrm{ref}})(d - d_0)]$, with μ the linear attenuation of air at the line and at the group's reference line. The information comes from intensity ratios within groups (Kα/Kβ, Lα/Lβ/Lγ), independent of the sample composition; the amplitudes absorb the rest.

`fit_tails`
: free tail and step, shared by all lines.

`scatter`
: a `ScatterSource`: adds the elastic and Compton peaks.

xrf_fit.py, lines 1097–1193

```
1097 class XRFModel:
1098     """Constrained Voigt model of an XRF spectrum.
1099 
1100     Free parameters, in order:
1101 
1102     ====================  ========================================
1103     ``bkg_0 .. bkg_n``    polynomial background coefficients
1104     ``A_<group>``         reference-line area, one per line group
1105     ``noise_ev``          electronic-noise sigma term  (if fitted)
1106     ``fano``              Fano factor                  (if fitted)
1107     ``gamma_ev``          global Lorentzian HWHM        (if fitted)
1108     ``cal_gain``          eV per channel                (if fitted)
1109     ``cal_offset_ev``     energy of channel 0           (if fitted)
1110     ``air_path_cm``       sample-detector air path      (if fitted)
1111     ``tail_fraction``,    detector tail and step        (if fit_tails)
1112     ``tail_slope``,
1113     ``step_fraction``
1114     ``A_elastic``,        scatter peaks                 (with scatter)
1115     ``elastic_sigma_ev``,
1116     ``A_compton``,
```

```
    ``compton_angle_deg``,
    ``compton_angle_spread_deg``,
    ``compton_sigma_ev``,
    ``compton_tail_fraction``,
    ``compton_tail_slope``
    ``pileup_tau_us``     pulse-pair resolving time     (if fitted)
    ==================  ======================================

    The background polynomial is evaluated in a centred, scaled channel
    coordinate so that its coefficients stay well conditioned.

    Parameters
    ----------
    groups : list of LineGroup
    calibration : Calibration
    detector : Detector
        Supplies the initial values (and the frozen values, if not fitted) of
        the resolution parameters.
    channels : (n,) array
        The channel grid of the fit ROI.
    background_kind : {"poly", "exp"}
        ``"poly"`` (default): the background is the polynomial.  ``"exp"``: it
        is ``exp(polynomial)``, positive by construction and closer to the shape
        of a scatter continuum over a wide energy range (the fit is then not
        linear in the background terms, so :func:`fit_map` needs ``"poly"``).
    background_order : int
        Polynomial order; 0 is a constant, 1 linear.  Use ``None`` for no
        background term (e.g. after SNIP stripping).
    fit_resolution : bool
        Free ``noise_ev`` and ``fano``.
    fit_gamma : bool
        Free ``gamma_ev``.
    escape : bool
        Add the sensor escape peaks of every line (:meth:`Detector.escape_lines`),
        tied to the line's amplitude: no extra free parameter.  The group
        amplitude then counts parent + escape, i.e. every photon absorbed in
        the sensor, which is what ``Detector.efficiency`` describes.
    pileup_template : (n_mca_channels,) array, optional
        Pile-up spectrum per microsecond of pulse-pair resolving time, in the
        units of the fitted data (see :func:`average_pileup_template`).
    fit_pileup : bool
        With a template: free ``pileup_tau_us`` (appended last) if True,
        otherwise hold it at ``pileup_tau_us``.  The sum peaks of the main
        lines usually lie above the analysis ROI, so the practical recipe is
        to fit tau once over an extended range and fix it in the analysis fit.
    pileup_tau_us : float
        Fixed (or starting) pulse-pair resolving time, µs.
    fit_calibration : bool or "offset"
        Free the energy calibration: ``True`` fits gain and offset (the peak
        positions of all groups then move together, tied to the tabulated
        line energies), ``"offset"`` only shifts the energy scale (used for
        local position checks).  ``calibration`` is then the starting point;
        :meth:`calibration_of` gives the fitted one.
    fit_air_path : bool
        Free the sample-detector air path.  The groups' relative intensities
        were computed with ``detector.air_path_cm`` (= d0); for a path ``d``
        each line is rescaled by ``exp(-(mu_l - mu_ref) (d - d0))``, with
        ``mu`` the air attenuation at the line and at the group's reference
        line.  The information comes from intensity ratios *within* groups
        (K-alpha/K-beta, L-alpha/L-beta/L-gamma), which do not depend on the
        sample composition.  The reference-line amplitudes absorb the rest.
    fit_tails : bool
        Free the detector tail and step (:func:`peak_shape`), shared by all
        lines; otherwise ``detector.tail_*`` / ``step_fraction`` are used.
    scatter : ScatterSource, optional
        Monochromatic excitation: adds the elastic peak (excitation lines with
        the detector response, Gaussian-broadened by ``elastic_sigma_ev`` for
        the bandwidth of the monochromator) and the Compton peak (at the energy of the fitted
        scattering angle, averaged uniformly over the scattering angles
        ``compton_angle_deg +/- compton_angle_spread_deg`` that the detector
        collects (the Compton energy depends non-linearly on the angle, so a
        close detector gives an asymmetric peak), Gaussian-broadened by
        ``compton_sigma_ev``, with its own low-energy tail), both with escape
        peaks when ``escape`` is set.  The elastic peak has the detector
        response of the fluorescence lines.  Two free amplitudes and six
```

```
        shape parameters.
    """
```

xrf_fit.py, lines 1195–1269

```
    def __init__(self, groups: Sequence[LineGroup], calibration: Calibration,
                 detector: Detector, channels: np.ndarray,
                 background_order: Optional[int] = 1,
                 fit_resolution: bool = True, fit_gamma: bool = True,
                 escape: bool = False,
                 pileup_template: Optional[np.ndarray] = None,
                 fit_pileup: bool = True, pileup_tau_us: float = 0.2,
                 fit_calibration: bool | str = False, fit_air_path: bool = False,
                 fit_tails: bool = False, scatter: Optional[ScatterSource] = None,
                 background_kind: str = "poly"):
        if not groups and scatter is None:
            raise ValueError("At least one line group is required.")
        if fit_calibration not in (False, True, "offset"):
            raise ValueError(f"fit_calibration must be False, True or 'offset', got {fit_calibration!r}")
        self.fit_calibration = fit_calibration
        self.fit_air_path = bool(fit_air_path)
        self.fit_tails = bool(fit_tails)
        self.scatter = scatter
        self._jtable = scatter.profile_table() if scatter is not None else None
        self.groups = list(groups)
        self.calibration = calibration
        self.detector = detector
        self.channels = np.asarray(channels, dtype=float)
        self.background_order = background_order
        if background_kind not in ("poly", "exp"):
            raise ValueError(f"background_kind must be 'poly' or 'exp', got {background_kind!r}")
        self.background_kind = background_kind
        self.fit_resolution = bool(fit_resolution)
        self.fit_gamma = bool(fit_gamma)
        self.escape = bool(escape)
        self.pileup = (None if pileup_template is None
                       else np.asarray(pileup_template, dtype=float)[self.channels.astype(int)])
        self.fit_pileup = bool(fit_pileup) and self.pileup is not None
        self.fixed_pileup_tau_us = float(pileup_tau_us)
        # escape companions per line, computed once
        self._escapes = {(gi, li): (detector.escape_lines(l.energy_ev) if self.escape else [])
                         for gi, g in enumerate(self.groups) for li, l in enumerate(g.lines)}
        # air attenuation (1/cm) of each line relative to its group's reference line
        self._dmu = {}
        if self.fit_air_path:
            rho = air_density(detector.air_temperature_c, detector.air_pressure_hpa)
            mu = lambda e_ev: xraylib.CS_Total_CP(AIR_FORMULA, e_ev / 1000.0) * rho
            self._dmu = {(gi, li): mu(l.energy_ev) - mu(g.reference.energy_ev)
                         for gi, g in enumerate(self.groups) for li, l in enumerate(g.lines)}

        self.energies = calibration.energy(self.channels)

        # Centred/scaled abscissa for the background polynomial.
        c0, c1 = self.channels[0], self.channels[-1]
        self._u = 2.0 * (self.channels - c0) / max(c1 - c0, 1.0) - 1.0

        self.n_bkg = 0 if background_order is None else background_order + 1

        names = [f"bkg_{i}" for i in range(self.n_bkg)]
        names += [f"A_{g.label.replace(' ', '')}" for g in self.groups]
        if self.fit_resolution:
            names += ["noise_ev", "fano"]
        if self.fit_gamma:
            names += ["gamma_ev"]
        if self.fit_calibration is True:
            names += ["cal_gain", "cal_offset_ev"]
        elif self.fit_calibration == "offset":
            names += ["cal_offset_ev"]
        if self.fit_air_path:
            names += ["air_path_cm"]
        if self.fit_tails:
            names += ["tail_fraction", "tail_slope", "step_fraction"]
        if self.scatter is not None:
            names += ["A_elastic", "elastic_sigma_ev", "A_compton", "compton_angle_deg",
```

```
                      "compton_angle_spread_deg", "compton_sigma_ev", "compton_tail_fraction",
                      "compton_tail_slope"]
        if self.fit_pileup:
            names += ["pileup_tau_us"]
        self.param_names: List[str] = names
        self._index = {n: i for i, n in enumerate(names)}
```

The constructor precomputes the escape lines of every line, the air attenuation differences $\mu_l - \mu_{\mathrm{ref}}$, the Compton profile table, the energies of the channel grid and the scaled coordinate $u = 2(c - c_0)/(c_1 - c_0) - 1 \in [-1, 1]$ in which the background polynomial is evaluated (powers of u stay of order 1, which keeps the fit well conditioned).

5.10.2 `_unpack`, `calibration_of`, `air_path_of`, `pileup_tau_us`, `pileup_profile`, `tails_of`

Accessors that read a parameter vector: background coefficients, amplitudes and resolution parameters (fitted values, or the detector's when frozen); the calibration, air path, resolving time and line-shape parameters, fitted or fixed. `pileup_profile` is $\tau_{\mathrm{pp}} P(c)$ on the fit range.

xrf_fit.py, lines 1273–1285

```
    def _unpack(self, params: Sequence[float]):
        i = 0
        bkg = np.asarray(params[i:i + self.n_bkg], dtype=float); i += self.n_bkg
        amps = np.asarray(params[i:i + len(self.groups)], dtype=float); i += len(self.groups)
        if self.fit_resolution:
            noise, fano = float(params[i]), float(params[i + 1]); i += 2
        else:
            noise, fano = self.detector.noise_ev, self.detector.fano
        if self.fit_gamma:
            gamma = float(params[i]); i += 1
        else:
            gamma = self.detector.gamma_ev
        return bkg, amps, noise, fano, gamma
```

xrf_fit.py, lines 1287–1295

```
    def calibration_of(self, params: Sequence[float]) -> Calibration:
        """Energy calibration for a parameter vector: the fitted one when
        ``fit_calibration`` is set, otherwise the fixed input."""
        if not self.fit_calibration:
            return self.calibration
        offset = float(params[self._index["cal_offset_ev"]])
        if self.fit_calibration is True:
            return Calibration(gain=float(params[self._index["cal_gain"]]), offset=offset)
        return Calibration(gain=self.calibration.gain, offset=offset)
```

xrf_fit.py, lines 1297–1301

```
    def air_path_of(self, params: Sequence[float]) -> float:
        """Sample-detector air path (cm): fitted, or ``detector.air_path_cm``."""
        if not self.fit_air_path:
            return self.detector.air_path_cm
        return float(params[self._index["air_path_cm"]])
```

xrf_fit.py, lines 1303–1307

```
    def pileup_tau_us(self, params: Sequence[float]) -> float:
        """Pulse-pair resolving time (µs): fitted, fixed, or 0 without pile-up."""
        if self.pileup is None:
            return 0.0
        return float(params[-1]) if self.fit_pileup else self.fixed_pileup_tau_us
```

xrf_fit.py, lines 1309–1313

```
    def pileup_profile(self, params: Sequence[float]) -> np.ndarray:
        """Modelled pile-up contribution over the ROI."""
        if self.pileup is None:
            return np.zeros_like(self.channels)
        return self.pileup_tau_us(params) * self.pileup
```

xrf_fit.py, lines 1324–1330

```
1324    def tails_of(self, params: Sequence[float]) -> Tuple[float, float, float]:
1325        """(tail_fraction, tail_slope, step_fraction): fitted, or the detector's."""
1326        if not self.fit_tails:
1327            d = self.detector
1328            return d.tail_fraction, d.tail_slope, d.step_fraction
1329        return tuple(float(params[self._index[n]])
1330                     for n in ("tail_fraction", "tail_slope", "step_fraction"))
```

5.10.3 `background, _shape`

```
background(params), _shape(centre_ch, sigma_ch, gamma_ch, tails)
```

$B = \sum_k b_k u^k$, or $\exp(\sum_k b_k u^k)$ with the exponent clipped to ± 50. `_shape` evaluates `peak_shape` on the channel grid.

xrf_fit.py, lines 1317–1322

```
1317    def background(self, params: Sequence[float]) -> np.ndarray:
1318        bkg, *_ = self._unpack(params)
1319        if self.n_bkg == 0:
1320            return np.zeros_like(self.channels)
1321        poly = sum(c * self._u ** k for k, c in enumerate(bkg))
1322        return np.exp(np.clip(poly, -50.0, 50.0)) if self.background_kind == "exp" else poly
```

xrf_fit.py, lines 1332–1333

```
1332    def _shape(self, centre_ch: float, sigma_ch: float, gamma_ch: float, tails) -> np.ndarray:
1333        return peak_shape(self.channels - centre_ch, sigma_ch, gamma_ch, *tails)
```

5.10.4 `group_profile`

```
group_profile(index, params, sigma_scale=1.0)
```

The counts of one group, Eq. (15) without background: for each line, the parent peak with weight $A_g r_l \tau_l (1 - \sum_e f_{le})$ and each escape peak with weight $A_g r_l \tau_l f_{le}$. Profiles are evaluated in *channel* space, with widths σ/G in channels: since the profile has unit area over its argument, the amplitude is directly the number of counts summed over channels. An escape peak has the resolution of its own (deposited) energy. `sigma_scale` multiplies every Gaussian width; it is used for the width-correction term of `fit_map`.

xrf_fit.py, lines 1421–1451

```
1421    def group_profile(self, index: int, params: Sequence[float],
1422                      sigma_scale: float = 1.0) -> np.ndarray:
1423        """Modelled counts of one line group (background excluded).
1424
1425        Profiles are evaluated in *channel* space: ``voigt_profile`` integrates
1426        to 1 over its argument, so with widths expressed in channels the
1427        amplitude is directly the integrated number of counts summed over
1428        channels — the same quantity the data carries.  ``sigma_scale``
1429        multiplies every Gaussian width (used for the width-correction term).
1430        """
1431        _, amps, noise, fano, gamma = self._unpack(params)
1432        g = self.groups[index]
1433        cal = self.calibration_of(params)
1434        gain = cal.gain
1435        gamma_ch = gamma / gain
1436        d_air = self.air_path_of(params) - self.detector.air_path_cm
1437        tails = self.tails_of(params)
1438        out = np.zeros_like(self.channels)
1439        for li, line in enumerate(g.lines):
1440            escapes = self._escapes[(index, li)]
1441            parent = 1.0 - sum(f for _, f in escapes)
1442            rel = line.rel * np.exp(-self._dmu[(index, li)] * d_air) if self.fit_air_path else line.rel
1443            sigma_ch = sigma_scale * self.detector.sigma_ev(line.energy_ev, noise_ev=noise, fano=fano) / gain
```

```
            centre_ch = cal.channel(line.energy_ev)
            out += (amps[index] * rel * parent) * self._shape(centre_ch, sigma_ch, gamma_ch, tails)
            for e_esc, f in escapes:
                # the charge cloud corresponds to the deposited energy, so the
                # escape peak has the resolution of E - E_X
                s_esc = sigma_scale * self.detector.sigma_ev(e_esc, noise_ev=noise, fano=fano) / gain
                out += (amps[index] * rel * f) * self._shape(cal.channel(e_esc), s_esc, gamma_ch, tails)
        return out
```

5.10.5 `scatter_profile`, `_compton_kernel`, `_compton_ia`

`scatter_profile(params, which="both")`, `_compton_kernel(x, sigma, v, step)`, `_compton_ia(cal, e0, v, noise, fano, tails)`

The scatter peaks, for each excitation line with its weight:

- **elastic**: the detector response at E_0 with the width $\sqrt{\sigma^2(E_0) + \sigma_{\mathrm{el}}^2}$ (the extra width stands for the bandwidth of the monochromator), with its escape peaks;
- **Compton**, impulse approximation (with a scatterer): `_compton_ia` evaluates $J(|p_z|)\,|\mathrm{d}p_z/\mathrm{d}E|$ on a channel grid extended 3 keV below the fit range (so that the wings are not cut), summed over the angle grid, set to zero above E_0 and normalised to unit area. It is convolved by FFT with the Compton kernel—a Gaussian of width $\sqrt{\sigma^2(E_C) + \sigma_C^2}$ with its own low-energy tail (`compton_tail_fraction`, `compton_tail_slope`)—to which the detector step is added as a shelf proportional to the number of counts above each channel. The escape peaks are shifted copies of this base profile;
- **Compton**, Gaussian (without scatterer): the kernel at $E_C(\theta)$ for each angle of the grid, averaged, with escape peaks.

Returns zeros without a scatter source.

xrf_fit.py, lines 1335–1375

```
    def scatter_profile(self, params: Sequence[float], which: str = "both") -> np.ndarray:
        """Elastic and/or Compton contribution (``which`` = "elastic",
        "compton" or "both"); zero without a scatter source."""
        out = np.zeros_like(self.channels)
        if self.scatter is None:
            return out
        _, _, noise, fano, gamma = self._unpack(params)
        cal = self.calibration_of(params)
        gain = cal.gain
        gamma_ch = gamma / gain
        tails = self.tails_of(params)
        v = {n: float(params[self._index[n]]) for n in
             ("A_elastic", "elastic_sigma_ev", "A_compton", "compton_angle_deg",
              "compton_angle_spread_deg", "compton_sigma_ev", "compton_tail_fraction",
              "compton_tail_slope")}
        for e0, w in zip(self.scatter.energies_ev, self.scatter.weights):
            if which in ("elastic", "both"):
                peaks = [(e0, 1.0, 0.0)]
                peaks += [(e, f, 0.0) for e, f in (self.detector.escape_lines(e0) if self.escape else [])]
                parent = 1.0 - sum(f for _, f, _ in peaks[1:])
                for k, (e, f, extra) in enumerate(peaks):
                    s_ch = np.hypot(self.detector.sigma_ev(e, noise_ev=noise, fano=fano),
                                    v["elastic_sigma_ev"]) / gain
                    out += v["A_elastic"] * w * (parent if k == 0 else f) * self._shape(
                        cal.channel(e), s_ch, gamma_ch, tails)
            if which in ("compton", "both") and self._jtable is not None:
                out += v["A_compton"] * w * self._compton_ia(cal, e0, v, noise, fano, tails)
            elif which in ("compton", "both"):
                angles = _angle_grid(v["compton_angle_deg"], v["compton_angle_spread_deg"])
                for a in angles:
                    ec = float(ScatterSource.compton_energy_ev(e0, a))
                    peaks = [(ec, 1.0)]
                    peaks += [(e, f) for e, f in (self.detector.escape_lines(ec) if self.escape else [])]
                    parent = 1.0 - sum(f for _, f in peaks[1:])
                    for k, (e, f) in enumerate(peaks):
                        s_ch = np.hypot(self.detector.sigma_ev(e, noise_ev=noise, fano=fano),
```

```
                                        v["compton_sigma_ev"]) / gain
                        x = self.channels - cal.channel(e)
                        out += (v["A_compton"] * w * (parent if k == 0 else f) / len(angles)
                                * self._compton_kernel(x, s_ch, v, tails[2]))
        return out
```

xrf_fit.py, lines 1377–1380

```
    @staticmethod
    def _compton_kernel(x: np.ndarray, sigma: float, v: dict, step: float) -> np.ndarray:
        """Unit-area Compton response: Gaussian with a low-energy tail."""
        return peak_shape(x, sigma, 0.0, v["compton_tail_fraction"], v["compton_tail_slope"], step)
```

xrf_fit.py, lines 1382–1419

```
    def _compton_ia(self, cal: Calibration, e0: float, v: dict, noise: float, fano: float,
                    tails) -> np.ndarray:
        """Unit-area Compton peak of one excitation line, impulse approximation:
        ``J(p_z(E)) |dp_z/dE|`` on the channel grid, convolved with the detector
        response (Gaussian of the resolution at the Compton energy combined with
        ``compton_sigma_ev``, low-energy tail), plus the step and escape peaks."""
        pz_tab, j_tab = self._jtable
        gain = cal.gain
        # evaluate on a grid extending 3 keV below the ROI so that the wings are
        # not cut, in channel steps
        n_ext = int(3000.0 / gain)
        ch = np.arange(self.channels[0] - n_ext, self.channels[-1] + 1.0)
        e = cal.energy(ch)
        angle = v["compton_angle_deg"]
        prof = np.zeros_like(e)
        for a in _angle_grid(angle, v["compton_angle_spread_deg"]):
            pz = ScatterSource.compton_pz(e, e0, a)
            prof += np.interp(np.abs(pz), pz_tab, j_tab, right=0.0) * np.abs(np.gradient(pz, e))
        prof[e >= e0] = 0.0
        total = prof.sum()
        if total <= 0:
            return np.zeros_like(self.channels)
        prof /= total
        ec = float(ScatterSource.compton_energy_ev(e0, angle))
        sig = np.hypot(self.detector.sigma_ev(ec, noise_ev=noise, fano=fano), v["compton_sigma_ev"]) / gain
        half = prof.size
        k = np.arange(-half, half + 1, dtype=float)
        kern = self._compton_kernel(k, sig, v, 0.0)
        conv = fftconvolve(prof, kern, mode="full")[half:half + prof.size]
        step = tails[2] / (np.sqrt(2.0 * np.pi) * sig) * (
            np.cumsum(prof[::-1])[::-1] - prof)                 # counts above each channel
        base = conv + step
        resp = base.copy()
        if self.escape:
            for e_esc, f in self.detector.escape_lines(ec):
                shift = (ec - e_esc) / gain        # escape peaks sit E_X lower
                resp += f * (np.interp(ch + shift, ch, base, left=0.0, right=0.0) - base)
        return resp[n_ext:]
```

5.10.6 `__call__`

```
__call__(channels, *params)
```

The total model: background, pile-up, scatter, and every group.

xrf_fit.py, lines 1453–1458

```
    def __call__(self, channels: np.ndarray, *params: float) -> np.ndarray:
        """Total model, in the signature ``curve_fit`` expects."""
        total = self.background(params) + self.pileup_profile(params) + self.scatter_profile(params)
        for i in range(len(self.groups)):
            total = total + self.group_profile(i, params)
        return total
```

5.10.7 `initial_guess`

```
initial_guess(y)
```

Starting values and bounds from the data. The background starts at the 5th percentile of the spectrum (its logarithm for `"exp"`), other coefficients at 0. Each amplitude is seeded from the observed height above that baseline at the strongest line of the group inside the range, divided by its relative intensity and converted to an area with the modelled width, $A = h\,\sigma\sqrt{2\pi}$. Bounds: amplitudes ≥ 0; noise 1–200 eV; Fano 0.01–1; Γ 0–100 eV; gain ±20 %; offset ±50 channels; air path 0–200 cm; tail fraction 0–0.6, slope 0.2–20 σ, step 0–0.1; Compton angle 10–175°, spread 0–80°, extra widths up to 0.5 keV (elastic) and 2 keV (Compton); resolving time 0–50 µs.

Hypotheses and limits

The non-linear parameters (calibration, Compton angle) need a good start: a calibration off by tens of channels can lead to a wrong local minimum. Start from a calibration fitted on known peaks (`xrf_calibration.calibration_from_peaks` or `calibrate`).

xrf_fit.py, lines 1462–1540

```
1462    def initial_guess(self, y: np.ndarray) -> Tuple[List[float], Tuple[List[float], List[float]]]:
1463        """Initial parameters and bounds, estimated from the data.
1464
1465        Each group's amplitude is seeded from the observed height at its
1466        reference line, converted to an area with the modelled width; the
1467        background is seeded from the lower envelope of the ROI.
1468        """
1469        y = np.asarray(y, dtype=float)
1470        base = float(np.percentile(y, 5))
1471
1472        p0: List[float] = []
1473        lo: List[float] = []
1474        hi: List[float] = []
1475
1476        if self.n_bkg:
1477            c0 = np.log(max(base, 1e-6)) if self.background_kind == "exp" else base
1478            p0 += [c0] + [0.0] * (self.n_bkg - 1)
1479            lo += [-np.inf] * self.n_bkg
1480            hi += [np.inf] * self.n_bkg
1481
1482        span = max(float(y.max()) - base, 1.0)
1483        c_lo, c_hi = self.channels[0], self.channels[-1]
1484        for g in self.groups:
1485            # seed from the strongest line inside the ROI (the reference line,
1486            # unless the ROI cuts it off), scaled back to the reference line
1487            inside = [l for l in g.lines if c_lo <= self.calibration.channel(l.energy_ev) <= c_hi]
1488            line = max(inside, key=lambda l: l.rel) if inside else g.reference
1489            ch = self.calibration.channel(line.energy_ev)
1490            j = int(np.clip(np.searchsorted(self.channels, ch), 0, len(self.channels) - 1))
1491            height = max(float(y[j]) - base, 0.01 * span) / line.rel
1492            sigma_ch = float(self.detector.sigma_ev(line.energy_ev)) / self.calibration.gain
1493            # Gaussian peak height -> integrated area, in channel units.
1494            p0.append(height * sigma_ch * np.sqrt(2.0 * np.pi))
1495            lo.append(0.0)
1496            hi.append(np.inf)
1497
1498        if self.fit_resolution:
1499            p0 += [self.detector.noise_ev, self.detector.fano]
1500            lo += [1.0, 0.01]
1501            hi += [200.0, 1.0]
1502        if self.fit_gamma:
1503            p0 += [self.detector.gamma_ev]
1504            lo += [0.0]
1505            hi += [100.0]
1506        if self.fit_calibration:
1507            g0, o0 = self.calibration.gain, self.calibration.offset
1508            if self.fit_calibration is True:
1509                p0 += [g0]
1510                lo += [0.8 * g0]
```

```
                hi += [1.2 * g0]
            p0 += [o0]                      # +/- 50 channels around the start
            lo += [o0 - 50.0 * g0]
            hi += [o0 + 50.0 * g0]
        if self.fit_air_path:
            p0 += [self.detector.air_path_cm]
            lo += [0.0]
            hi += [200.0]
        if self.fit_tails:
            d = self.detector
            p0 += [max(d.tail_fraction, 0.05), d.tail_slope, max(d.step_fraction, 1e-3)]
            lo += [0.0, 0.2, 0.0]
            hi += [0.6, 20.0, 0.1]
        if self.scatter is not None:
            def area_at(e_ev, sigma_ev):
                ch = self.calibration.channel(e_ev)
                j = int(np.clip(np.searchsorted(self.channels, ch), 0, len(self.channels) - 1))
                return max(float(y[j]) - base, 0.01 * span) * sigma_ev / self.calibration.gain * np.sqrt(2 * np.pi)
            e_mean = float(np.dot(self.scatter.energies_ev, self.scatter.weights))
            e_c = float(ScatterSource.compton_energy_ev(e_mean, self.scatter.angle_deg))
            p0 += [area_at(e_mean, self.detector.sigma_ev(e_mean)), 20.0, area_at(e_c, 250.0),
                   self.scatter.angle_deg, 20.0, 50.0, 0.1, 2.0]
            lo += [0.0, 0.0, 0.0, 10.0, 0.0, 1.0, 0.0, 0.2]
            hi += [np.inf, 500.0, np.inf, 175.0, 80.0, 2000.0, 0.6, 20.0]
        if self.fit_pileup:
            p0 += [self.fixed_pileup_tau_us]   # µs; pulse-pair resolution is typically 0.1-1 µs
            lo += [0.0]
            hi += [50.0]

        return p0, (lo, hi)
```

5.11 Fitting a spectrum

5.11.1 `FitResult`

The outcome of a fit: the `model`, the parameters `popt`, their errors `perr` and covariance `pcov`, the fitted `channels`, data `y`, errors `yerr` and model `yfit`, the reduced χ^2, and the group `areas` and `area_errors` (reference-line areas). Properties and methods: `calibration` (fitted or fixed), `residual`, `residual_normalised` (in units of the data error), `params()` (`{name: (value, error)}`), `summary()` (text report with the total area of every group).

xrf_fit.py, lines 1543–1561

```
# ============================================================================
# Fitting
# ============================================================================

@dataclass
class FitResult:
    """Outcome of a constrained fit to one spectrum."""

    model: XRFModel
    popt: np.ndarray
    perr: np.ndarray
    channels: np.ndarray
    y: np.ndarray
    yerr: Optional[np.ndarray]
    yfit: np.ndarray
    chi2_red: float
    areas: Dict[str, float] = field(default_factory=dict)
    area_errors: Dict[str, float] = field(default_factory=dict)
    pcov: Optional[np.ndarray] = None
```

xrf_fit.py, lines 1563–1566

```
    @property
    def calibration(self) -> Calibration:
        """Energy calibration of the fit (fitted, or the fixed input)."""
        return self.model.calibration_of(self.popt)
```

xrf_fit.py, lines 1568–1570

```
1568     @property
1569     def residual(self) -> np.ndarray:
1570         return self.y - self.yfit
```

xrf_fit.py, lines 1572–1577

```
1572     @property
1573     def residual_normalised(self) -> np.ndarray:
1574         """Residual in units of the data uncertainty, where available."""
1575         if self.yerr is None:
1576             return self.residual
1577         return self.residual / np.where(self.yerr > 0, self.yerr, np.nan)
```

xrf_fit.py, lines 1579–1582

```
1579     def params(self) -> Dict[str, Tuple[float, float]]:
1580         """``{name: (value, 1-sigma)}`` for every free parameter."""
1581         return {n: (float(v), float(e))
1582                 for n, v, e in zip(self.model.param_names, self.popt, self.perr)}
```

xrf_fit.py, lines 1584–1593

```
1584     def summary(self) -> str:
1585         lines = [f"reduced chi2 = {self.chi2_red:.3f}", "", "free parameters:"]
1586         for name, (v, e) in self.params().items():
1587             lines.append(f"  {name:<20} {v:14.5g} +/- {e:.3g}")
1588         lines += ["", "group areas (reference line / whole group, same units as data x channel):"]
1589         for g in self.model.groups:
1590             a, ae = self.areas[g.label], self.area_errors[g.label]
1591             lines.append(f"  {g.label:<10} {a:12.5g} +/- {ae:<11.3g}"
1592                          f"  total {a * g.total_rel:12.5g}  ({len(g.lines)} lines)")
1593         return "\n".join(lines)
```

5.11.2 `fit_average_spectrum`

```
fit_average_spectrum(y, groups, calibration, detector, roi, yerr=None, background_order=1,
fit_resolution=True, fit_gamma=True, maxfev=20000, escape=False, pileup_template=None,
fit_pileup=True, pileup_tau_us=0.2, fit_calibration=False, fit_air_path=False, fit_tails=False,
scatter=None, background_kind="poly")
```

Fits Eq. (15) to one spectrum (a map average, a single point, a reference spectrum).

`y` spectrum over the full MCA range (sliced to `roi` internally).

`groups, calibration, detector`
as for `XRFModel`.

`roi`
half-open channel range of the fit.

`yerr`
one-sigma errors per channel (e.g. `counting_error`); zero errors are raised to the smallest positive one, which would otherwise make the fit singular.

`maxfev`
limit on function evaluations.

`others`
passed to `XRFModel`.

Method Weighted non-linear least squares,

$$\chi^2(\mathbf{p}) = \sum_c \left(\frac{y_c - y(c;\mathbf{p})}{\sigma_c} \right)^2 , \tag{27}$$

minimised by `scipy.optimize.curve_fit` with bounds, i.e. the trust-region reflective algorithm [1, 2]. With `absolute_sigma=True` the covariance is $(\mathbf{J}^{\mathsf{T}}\mathbf{W}\mathbf{J})^{-1}$ at the minimum, not rescaled by χ^2_ν, so the reported errors are the statistical ones [39]. The reduced $\chi^2_\nu = \chi^2/(n-p)$ measures the quality of the model: values well above 1 mean that the residuals are dominated by model errors, and the statistical errors then underestimate the true uncertainty.

Hypotheses and limits

Gaussian errors. Minimising χ^2 with data-derived weights (Neyman's χ^2) biases the amplitudes low at low counts [40, 41]; map averages and long single measurements have many counts per channel, where the bias is negligible.

Returns a `FitResult`.

xrf_fit.py, lines 1596–1684

```
1596 def fit_average_spectrum(
1597     y: np.ndarray,
1598     groups: Sequence[LineGroup],
1599     calibration: Calibration,
1600     detector: Detector,
1601     roi: Tuple[int, int],
1602     yerr: Optional[np.ndarray] = None,
1603     background_order: Optional[int] = 1,
1604     fit_resolution: bool = True,
1605     fit_gamma: bool = True,
1606     maxfev: int = 20000,
1607     escape: bool = False,
1608     pileup_template: Optional[np.ndarray] = None,
1609     fit_pileup: bool = True,
1610     pileup_tau_us: float = 0.2,
1611     fit_calibration: bool | str = False,
1612     fit_air_path: bool = False,
1613     fit_tails: bool = False,
1614     scatter: Optional[ScatterSource] = None,
1615     background_kind: str = "poly",
1616 ) -> FitResult:
1617     """Fit the constrained Voigt model to a single spectrum.
1618 
1619     ``escape``, ``pileup_template``, ``fit_pileup``, ``pileup_tau_us``,
1620     ``fit_calibration``, ``fit_air_path``, ``fit_tails``, ``scatter`` and
1621     ``background_kind`` are passed to :class:`XRFModel`;
1622     with a fitted calibration, ``FitResult.calibration`` holds the result.
1623 
1624     Parameters
1625     ----------
1626     y : (n_channels,) array
1627         Spectrum over the full MCA range; it is sliced to ``roi`` internally.
1628     groups
1629         From :func:`element_line_groups`.
1630     calibration, detector
1631         Fixed calibration and the detector-response model.
1632     roi : (lo, hi)
1633         Half-open channel range to fit.
1634     yerr : (n_channels,) array, optional
1635         1-sigma uncertainty per channel, e.g. the standard error returned by
1636         :func:`average_spectrum`.  Passed to ``curve_fit`` with
1637         ``absolute_sigma=True`` so that the reported parameter errors are the
1638         real ones rather than being rescaled to force reduced chi2 = 1.
1639     background_order, fit_resolution, fit_gamma
1640         See :class:`XRFModel`.
1641 
1642     Returns
1643     -------
1644     FitResult
1645     """
1646     lo, hi = roi
1647     ch = np.arange(lo, hi, dtype=float)
1648     yy = np.asarray(y, dtype=float)[lo:hi]
1649 
1650     ee = None
```

```
    if yerr is not None:
        ee = np.asarray(yerr, dtype=float)[lo:hi]
        # A zero error would make the fit singular; floor it at the smallest
        # positive value present.
        positive = ee[ee > 0]
        floor = positive.min() if positive.size else 1.0
        ee = np.where(ee > 0, ee, floor)

    model = XRFModel(groups, calibration, detector, ch,
                     background_order=background_order,
                     fit_resolution=fit_resolution, fit_gamma=fit_gamma,
                     escape=escape, pileup_template=pileup_template,
                     fit_pileup=fit_pileup, pileup_tau_us=pileup_tau_us,
                     fit_calibration=fit_calibration, fit_air_path=fit_air_path,
                     fit_tails=fit_tails, scatter=scatter, background_kind=background_kind)
    p0, bounds = model.initial_guess(yy)

    popt, pcov = curve_fit(model, ch, yy, p0=p0, bounds=bounds,
                           sigma=ee, absolute_sigma=ee is not None, maxfev=maxfev)
    perr = np.sqrt(np.diag(pcov))
    yfit = model(ch, *popt)

    dof = max(len(yy) - len(popt), 1)
    if ee is not None:
        chi2 = float(np.sum(((yy - yfit) / ee) ** 2)) / dof
    else:
        chi2 = float(np.sum((yy - yfit) ** 2)) / dof

    i0 = model.n_bkg
    areas = {g.label: float(popt[i0 + i]) for i, g in enumerate(model.groups)}
    aerrs = {g.label: float(perr[i0 + i]) for i, g in enumerate(model.groups)}

    return FitResult(model=model, popt=popt, perr=perr, channels=ch, y=yy, yerr=ee,
                     yfit=yfit, chi2_red=chi2, areas=areas, area_errors=aerrs, pcov=pcov)
```

5.12 Maps: the per-pixel linear solve

Once the shape parameters (resolution, calibration, tails, pile-up time) are fixed by the fit of the average spectrum, Eq. (15) is *linear* in the remaining parameters: the background coefficients and the group amplitudes. Every pixel is then solved exactly and at once, with its own covariance, instead of running a non-linear fit per pixel.

5.12.1 `MapResult`

Per-pixel results: `group_labels`, `areas` and `area_errors` ($n_{\mathrm{pixels}}, n_{\mathrm{groups}}$), `background` coefficients, `chi2_red`, `total_rel`, map `shape`, and optionally `width_change` and `segments`.

Methods `area_map(label, total=False)` and `error_map` reshape a group's areas (or errors) to the map, optionally scaled to the whole series; `snr` is their ratio; `width_change_map` reshapes the width term; `ratio_map(numerator, denominator)` returns the ratio of two groups' areas with the propagated error, $\sigma_r = |r|\sqrt{(\sigma_a/a)^2 + (\sigma_b/b)^2}$; `quality_report` lists, per group, the median area, the median signal-to-noise and the fraction of negative pixels, and flags groups as quantitative (median S/N ≥ 20 by default), detected only (S/N ≥ 3: the map is noise and the average area a lower bound), or not detected.

xrf_fit.py, lines 1687–1725

```
# ============================================================================
# Per-pixel mapping: the linear solve
# ============================================================================

@dataclass
class MapResult:
    """Per-pixel amplitudes from the linear map fit.

    Attributes
    ----------
    group_labels : list of str
```

```
1698            Line-group labels, in column order of ``areas``.
1699        areas, area_errors : (n_pixels, n_groups) arrays
1700            Fitted reference-line area per pixel and its 1-sigma from the per-pixel
1701            covariance.
1702        background : (n_pixels, n_bkg) array
1703        chi2_red : (n_pixels,) array
1704        total_rel : dict
1705            ``{group_label: sum of relative intensities}``, converting a
1706            reference-line area into the group's total modelled area.
1707        shape : tuple
1708            Map shape (n_slow, n_fast), for reshaping.
1709        width_change : (n_pixels,) array or None
1710            With ``width_correction=True``: relative change of the Gaussian width
1711            of every line in the pixel, with respect to the fixed shape
1712            (0.01 = 1 % broader), from the first-order width term.
1713        segments : (n_pixels,) array or None
1714            Segment label of each pixel when per-segment shapes were used.
1715        """
1716
1717        group_labels: List[str]
1718        areas: np.ndarray
1719        area_errors: np.ndarray
1720        background: np.ndarray
1721        chi2_red: np.ndarray
1722        total_rel: Dict[str, float]
1723        shape: Tuple[int, int]
1724        width_change: Optional[np.ndarray] = None
1725        segments: Optional[np.ndarray] = None
```

xrf_fit.py, lines 1727–1731

```
1727    def width_change_map(self) -> np.ndarray:
1728        """2D map of the relative width change (needs ``width_correction=True``)."""
1729        if self.width_change is None:
1730            raise ValueError("The map was fitted without width_correction.")
1731        return self.width_change.reshape(self.shape)
```

xrf_fit.py, lines 1733–1738

```
1733    def area_map(self, label: str, total: bool = False) -> np.ndarray:
1734        """2D map of one group's area. ``total=True`` scales to the whole series."""
1735        a = self.areas[:, self.group_labels.index(label)]
1736        if total:
1737            a = a * self.total_rel[label]
1738        return a.reshape(self.shape)
```

xrf_fit.py, lines 1740–1744

```
1740    def error_map(self, label: str, total: bool = False) -> np.ndarray:
1741        e = self.area_errors[:, self.group_labels.index(label)]
1742        if total:
1743            e = e * self.total_rel[label]
1744        return e.reshape(self.shape)
```

xrf_fit.py, lines 1746–1750

```
1746    def snr(self, label: str) -> np.ndarray:
1747        """Per-pixel signal-to-noise of one group's area."""
1748        a, e = self.area_map(label), self.error_map(label)
1749        with np.errstate(divide="ignore", invalid="ignore"):
1750            return a / e
```

xrf_fit.py, lines 1752–1774

```
1752    def quality_report(self, quantitative_snr: float = 20.0) -> str:
1753        """Per-group median S/N, flagging groups too weak to quantify.
1754
1755        A group whose per-pixel S/N is of order a few can still give a usable
1756        *map-averaged* area, but its per-pixel map is noise and its absolute
1757        value should not be read as a concentration.  Soft L lines are the usual
1758        offenders: they are weak to begin with and heavily attenuated by the
1759        detector window and any air path, so their apparent deficit is an
1760        instrumental effect, not chemistry.
```

```
        """
        lines = [f"{'group':<8}{'median area':>13}{'median S/N':>12}{'neg. px':>9}   status"]
        for lab in self.group_labels:
            a = self.area_map(lab, total=True)
            sn = np.nanmedian(self.snr(lab))
            neg = 100 * np.nanmean(a < 0)
            if sn >= quantitative_snr:
                status = "quantitative"
            elif sn >= 3.0:
                status = "DETECTED ONLY - per-pixel map is noise, area is a lower bound"
            else:
                status = "NOT DETECTED - do not quantify"
            lines.append(f"{lab:<8}{np.nanmedian(a):>13.4g}{sn:>12.1f}{neg:>8.2f}%   {status}")
        return "\n".join(lines)
```

xrf_fit.py, lines 1776–1784

```
    def ratio_map(self, numerator: str, denominator: str,
                  total: bool = True) -> Tuple[np.ndarray, np.ndarray]:
        """Ratio of two groups\' areas, and its propagated 1-sigma error."""
        a, b = self.area_map(numerator, total), self.area_map(denominator, total)
        da, db = self.error_map(numerator, total), self.error_map(denominator, total)
        with np.errstate(divide="ignore", invalid="ignore"):
            r = a / b
            dr = np.abs(r) * np.sqrt((da / a) ** 2 + (db / b) ** 2)
        return r, dr
```

5.12.2 `number_ratio`

```
number_ratio(area_a, area_b, group_a, group_b, err_a=None, err_b=None)
```

Atom-number ratio from two reference-line areas,

$$\frac{n_a}{n_b} = \frac{A_a}{A_b}\,\frac{S_b W_b}{S_a W_a}, \tag{28}$$

with S the groups' `ref_strength` (mass cross-section × excitation × efficiency) and W the atomic weights, which turn mass cross-sections into per-atom ones. Errors are propagated as uncorrelated. Returns `(ratio, ratio_err, k)` with k the factor multiplying A_a/A_b.

Hypotheses and limits

Both groups come from the same `element_line_groups` configuration, and matrix effects are negligible or equal for the two lines (thin sample). Otherwise use `xrf_matrix.quantify_layer`.

xrf_fit.py, lines 1787–1834

```
def number_ratio(area_a, area_b, group_a: LineGroup, group_b: LineGroup,
                 err_a=None, err_b=None):
    """Atom-number ratio ``n_a / n_b`` from two fitted reference-line areas.

    ::

        n_a / n_b = (A_a / A_b) * (S_b * W_b) / (S_a * W_a)

    with ``A`` the fitted **reference-line** area (``FitResult.areas`` or
    ``MapResult.area_map(label)`` with the default ``total=False``), ``S`` the
    group's ``ref_strength`` (mass cross-section x excitation x detection
    efficiency) and ``W`` the atomic weight, which turns the mass
    cross-section (cm^2/g) into a per-atom one.

    Valid when both groups come from the same :func:`element_line_groups` call
    configuration (same excitation, same detector) and matrix effects are
    negligible or equal for the two lines.  Self-absorption and secondary
    fluorescence are **not** included.

    Parameters
    ----------
    area_a, area_b : float or array
```

```
    group_a, group_b : LineGroup
    err_a, err_b : float or array, optional
        1-sigma errors of the areas, propagated as uncorrelated.

    Returns
    -------
    ratio, ratio_err, k
        ``ratio_err`` is None if no errors are given; ``k`` is the
        sensitivity factor multiplying ``A_a / A_b``.
    """
    for g in (group_a, group_b):
        if not np.isfinite(g.ref_strength) or g.ref_strength <= 0:
            raise ValueError(f"{g.label}: ref_strength is not set; rebuild the groups "
                             "with element_line_groups().")
    w_a = xraylib.AtomicWeight(xraylib.SymbolToAtomicNumber(group_a.element))
    w_b = xraylib.AtomicWeight(xraylib.SymbolToAtomicNumber(group_b.element))
    k = (group_b.ref_strength * w_b) / (group_a.ref_strength * w_a)
    a = np.asarray(area_a, dtype=float)
    b = np.asarray(area_b, dtype=float)
    with np.errstate(divide="ignore", invalid="ignore"):
        ratio = k * a / b
        ratio_err = None
        if err_a is not None and err_b is not None:
            ratio_err = np.abs(ratio) * np.sqrt((np.asarray(err_a) / a) ** 2
                                                + (np.asarray(err_b) / b) ** 2)
    return ratio, ratio_err, float(k)
```

5.12.3 `design_matrix`

```
design_matrix(model, shape_params)
```

The columns of the linear model: u^k for the background terms, then the profile of each group with unit amplitude (`group_profile` with all other amplitudes set to 0). Returns an $(n_{\mathrm{channels}}, n_{\mathrm{bkg}} + n_{\mathrm{groups}})$ array.

xrf_fit.py, lines 1837–1865

```
def design_matrix(model: "XRFModel", shape_params: Sequence[float]) -> np.ndarray:
    """Columns of the linear model: background terms, then one per line group.

    With peak positions and widths fixed, the spectrum is a *linear* combination
    of these columns, the coefficients being the background polynomial terms and
    the group reference-line areas.  Building this once turns a per-pixel
    non-linear optimisation into a single stacked linear solve.

    Parameters
    ----------
    model : XRFModel
        Supplies the channel grid, the groups and the background order.
    shape_params : sequence
        A full parameter vector (e.g. ``FitResult.popt``); only its resolution
        parameters are read, the amplitudes in it are ignored.

    Returns
    -------
    (n_channels, n_bkg + n_groups) array
    """
    cols = [model._u ** k for k in range(model.n_bkg)]
    probe = list(shape_params)
    i0 = model.n_bkg
    for j in range(len(model.groups)):
        unit = probe.copy()
        for jj in range(len(model.groups)):
            unit[i0 + jj] = 1.0 if jj == j else 0.0
        cols.append(model.group_profile(j, unit))
    return np.column_stack(cols)
```

5.12.4 fit_map

```
fit_map(raw_spectra, scale, model, shape_params, map_shape, poisson_weights=True,
clip_negative=False, pileup=None, segments=None, width_correction=False, mask=None)
```

Solves the fixed-shape model for every pixel.

`raw_spectra`
raw counts $(n_{\text{pixels}}, n_{\text{channels}})$: the weights are defined on counts, and the normalisation is applied internally.

`scale`
the normalisation divisors of `normalise_spectra`.

`model`
an `XRFModel` for the fit range (no scatter, polynomial background).

`shape_params`
parameter vector of the average fit, or `{segment: vector}`.

`map_shape`
$(n_{\text{slow}}, n_{\text{fast}})$.

`poisson_weights`
weight each channel by $1/\mathrm{Var}(y) = s^2/\max(x, 1)$. Without them the strongest channels dominate every pixel and the errors are meaningless.

`clip_negative`
clip negative amplitudes to zero (off: a negative amplitude is a diagnostic of a mis-specified model).

`pileup`
per-pixel pile-up spectra; with a pile-up term in the model, $\tau P_i/s_i$ is subtracted from each pixel before the solve (a known, pixel-specific component).

`segments`
per-pixel segment labels, with `shape_params` per segment: each segment is solved with its own widths, the exact treatment of an abrupt resolution change.

`width_correction`
add one first-order width term per pixel (below).

`mask`
pixels to solve, e.g. the facet of a crystal (`SurfaceRegion.scan_mask`, section 7); the others are not fitted and are NaN in every map, so that map statistics (medians, histograms, worst pixels) only see the region. Default: all pixels.

Method For each pixel p with design matrix $\mathbf{A}$ and weights $\mathbf{W}_p$, the weighted least-squares solution and its covariance are

$$\hat{\mathbf{c}}_p = (\mathbf{A}^{\mathsf{T}}\mathbf{W}_p\mathbf{A})^{-1}\mathbf{A}^{\mathsf{T}}\mathbf{W}_p\mathbf{y}_p, \qquad \mathrm{Cov}(\hat{\mathbf{c}}_p) = (\mathbf{A}^{\mathsf{T}}\mathbf{W}_p\mathbf{A})^{-1}, \tag{29}$$

assembled for all pixels at once with `einsum` (each system is only $k \times k$) and solved in one batched call; a relative ridge of 10^{-12} times the trace keeps empty pixels from making a system singular (they are set to NaN afterwards). The reduced χ^2 of each pixel uses the same weights.

Width correction If the detector resolution changes slightly during the scan, the fixed widths bias overlapping peaks. The extra column is the derivative of the modelled peaks with respect to a common relative scaling ε of every Gaussian width, at the shape amplitudes $\mathbf{a}$, computed by central differences ($\pm 2\,\%$). A pixel modelled as $\lambda \sum_j a_j g_j[(1+\varepsilon)\sigma] \approx \lambda \sum_j a_j g_j + \lambda\varepsilon\, \partial_\varepsilon(\cdot)$ has a width coefficient $\delta = \lambda\varepsilon$, with λ the projection of the pixel's amplitudes on $\mathbf{a}$; hence $\varepsilon = \delta/\lambda$ is returned as `width_change`. First order: valid for changes of a few per cent, and if the relative amplitudes stay close to those of the average.

xrf_fit.py, lines 1868–2046

```
1868 def fit_map(
1869     raw_spectra: np.ndarray,
1870     scale: np.ndarray,
1871     model: "XRFModel",
1872     shape_params,
1873     map_shape: Tuple[int, int],
1874     poisson_weights: bool = True,
1875     clip_negative: bool = False,
1876     pileup: Optional[np.ndarray] = None,
1877     segments: Optional[np.ndarray] = None,
1878     width_correction: bool = False,
1879     mask: Optional[np.ndarray] = None,
1880 ) -> MapResult:
1881     """Solve the fixed-shape model for every pixel of a map, in one pass.
1882 
1883     The normal equations are assembled and solved for all pixels simultaneously
1884     (``A^T W A`` is only ``k x k`` per pixel, with ``k`` the number of
1885     components), so the cost is two tensor contractions rather than any
1886     iteration.  The per-pixel covariance falls out of the same factorisation,
1887     which is what gives every map an error map for free.
1888 
1889     Parameters
1890     ----------
1891     raw_spectra : (n_pixels, n_channels) array
1892         **Raw** counts.  Required in raw form because Poisson weights are
1893         defined on counts; the normalisation is applied internally.
1894     scale : (n_pixels,) array
1895         Normalisation divisor from :func:`normalise_spectra`.
1896     model : XRFModel
1897         Built for the ROI being fitted.
1898     shape_params : sequence
1899         Parameter vector from the average-spectrum fit, supplying the fixed
1900         resolution parameters.
1901     map_shape : (n_slow, n_fast)
1902     poisson_weights : bool
1903         Weight each channel by ``1 / Var(y)``, ``Var(y) = raw / scale^2``.
1904         Strongly recommended: without it the strongest channels dominate every
1905         pixel and the reported errors are meaningless.
1906     clip_negative : bool
1907         Clip negative fitted amplitudes to zero.  Off by default, because a
1908         negative amplitude is diagnostic of a mis-specified model or an
1909         over-flexible background and silently clipping it hides that.
1910     pileup : (n_pixels, n_channels) array, optional
1911         Per-pixel pile-up spectra from :func:`pileup_spectra` (full MCA
1912         range).  If the model carries a pile-up term, ``tau`` is read from
1913         ``shape_params`` and ``tau * pileup`` is subtracted from each pixel
1914         before the solve (it is a known, pixel-specific component, so it does
1915         not fit into the shared design matrix).  The Poisson weights still use
1916         the raw counts, which include the pile-up events.
1917     segments : (n_pixels,) int array, optional
1918         Segment label of every pixel (e.g. from :func:`segment_labels`).  With
1919         ``shape_params`` given as ``{label: params}`` (e.g. from
1920         :func:`fit_segments`), each segment is solved with its own line widths.
1921         This is the exact treatment of an abrupt change of detector
1922         resolution during the scan.
1923     width_correction : bool
1924         Add one first-order width term per pixel: the derivative of the
1925         modelled peaks with respect to a common scaling of all Gaussian
1926         widths, weighted by the shape amplitudes.  Its coefficient absorbs
1927         small resolution changes (steps or drifts) pixel by pixel, which
1928         otherwise bias the areas of overlapping peaks, and gives
1929         ``MapResult.width_change``.  First order: valid for changes of a few
1930         per cent, and assumes the relative group amplitudes stay close to
1931         those of ``shape_params``.  Can be combined with ``segments``.
1932     mask : (n_pixels,) bool array, optional
1933         Pixels to solve, e.g. the facet of a crystal from
1934         :meth:`xrf_region.SurfaceRegion.scan_mask`.  The others are not
1935         fitted and are NaN in every map, so map statistics (medians,
1936         histograms, worst pixels) only see the region.  Default: all pixels.
1937 
1938     Returns
1939     -------
1940     MapResult
```

```
    """
    if getattr(model, "scatter", None) is not None:
        raise ValueError("fit_map does not handle scatter peaks yet: build the map model "
                         "without `scatter` (fit them on the average spectrum only).")
    if getattr(model, "background_kind", "poly") != "poly":
        raise ValueError("fit_map needs a polynomial background (background_kind='poly').")
    lo, hi = int(model.channels[0]), int(model.channels[-1]) + 1
    raw = np.asarray(raw_spectra, dtype=float)[:, lo:hi]
    s = np.asarray(scale, dtype=float)
    n_pix, n_ch = raw.shape

    good = np.isfinite(s) & (s > 0)
    if mask is not None:
        mask = np.asarray(mask, dtype=bool).reshape(-1)
        if mask.size != n_pix:
            raise ValueError(f"mask has {mask.size} values for {n_pix} pixels.")
        good &= mask
    s_safe = np.where(good, s, 1.0)
    y = raw / s_safe[:, None]

    # one shape per segment (a single shape is one segment covering the map)
    if isinstance(shape_params, Mapping):
        if segments is None:
            raise ValueError("shape_params given per segment: pass `segments` too.")
        seg = np.asarray(segments)
        shapes = {lab: np.asarray(p, dtype=float) for lab, p in shape_params.items()}
        missing = set(np.unique(seg[good])) - set(shapes)
        if missing:
            raise ValueError(f"no shape parameters for segments {sorted(missing)}")
    else:
        seg = np.zeros(n_pix, dtype=int)
        shapes = {0: np.asarray(shape_params, dtype=float)}

    if model.pileup is not None and pileup is None:
        raise ValueError("The model includes pile-up: pass the per-pixel `pileup` spectra.")

    ng, nb = len(model.groups), model.n_bkg
    k = nb + ng + (1 if width_correction else 0)
    coef = np.full((n_pix, k), np.nan)
    err = np.full((n_pix, k), np.nan)
    chi2 = np.full(n_pix, np.nan)
    width_change = np.full(n_pix, np.nan) if width_correction else None

    for lab, params in shapes.items():
        sel = np.flatnonzero((seg == lab) & good)       # invalid or masked pixels: not solved
        if sel.size == 0:
            continue
        ys = y[sel]
        if model.pileup is not None:
            tau_s = model.pileup_tau_us(params) * 1e-6
            ys = ys - tau_s * np.asarray(pileup, dtype=float)[sel, lo:hi] / s_safe[sel, None]

        A = design_matrix(model, params)                        # (n_ch, nb + ng)
        if width_correction:
            h = 0.02
            dcol = sum((model.group_profile(j, params, sigma_scale=1.0 + h)
                        - model.group_profile(j, params, sigma_scale=1.0 - h)) / (2.0 * h)
                       for j in range(ng))
            A = np.column_stack([A, dcol])

        if poisson_weights:
            W = (s_safe[sel, None] ** 2) / np.maximum(raw[sel], 1.0)
        else:
            W = np.ones_like(ys)

        # Stacked normal equations: (A^T W A) c = A^T W y, one k x k system per pixel.
        AtWA = np.einsum("ck,pc,cl->pkl", A, W, A, optimize=True)
        AtWy = np.einsum("ck,pc,pc->pk", A, W, ys, optimize=True)
        # Nudge the diagonal so an all-zero pixel does not make the solve singular;
        # such pixels are flagged NaN below via `good`.
        ridge = 1e-12 * np.trace(AtWA, axis1=1, axis2=2)[:, None, None] * np.eye(k)
        c = np.linalg.solve(AtWA + ridge, AtWy[..., None])[..., 0]
        cov = np.linalg.inv(AtWA + ridge)
        coef[sel] = c
        err[sel] = np.sqrt(np.clip(np.einsum("pkk->pk", cov), 0.0, None))
```

```
        resid = ys - c @ A.T
        chi2[sel] = np.einsum("pc,pc,pc->p", resid, W, resid) / max(n_ch - k, 1)

        if width_correction:
            # pixel model ~ lambda * (shape amplitudes) + delta * dcol, with
            # delta = lambda * eps  ->  eps = delta / lambda; lambda is the
            # least-squares projection of the pixel amplitudes on the shape ones
            a_ref = np.asarray(params[nb:nb + ng], dtype=float)
            lam = (c[:, nb:nb + ng] @ a_ref) / max(float(a_ref @ a_ref), 1e-300)
            with np.errstate(divide="ignore", invalid="ignore"):
                width_change[sel] = c[:, -1] / lam

    coef[~good] = np.nan
    err[~good] = np.nan
    chi2[~good] = np.nan
    if width_correction:
        width_change[~good] = np.nan
    if clip_negative:
        coef[:, nb:nb + ng] = np.clip(coef[:, nb:nb + ng], 0.0, None)

    return MapResult(
        group_labels=[g.label for g in model.groups],
        areas=coef[:, nb:nb + ng],
        area_errors=err[:, nb:nb + ng],
        background=coef[:, :nb],
        chi2_red=chi2,
        total_rel={g.label: g.total_rel for g in model.groups},
        shape=tuple(map_shape),
        width_change=width_change,
        segments=None if len(shapes) == 1 and not isinstance(shape_params, Mapping) else seg,
    )
```

5.13 Resolution stability during a scan

5.13.1 `width_window`

```
width_window(groups, areas, detector, min_frac=0.03, doublet_ev=60.0)
```

Chooses a clean energy window around a strong, isolated line to monitor the resolution. For each group's reference line, the half width is $\min(2.5\,\mathrm{FWHM}, d/2)$ with d the distance to the nearest line of expected area above `min_frac` of the candidate's (lines closer than `doublet_ev`, such as K$\alpha_{1,2}$, belong to the peak); the score is the area times the Gaussian fraction inside the window, $\mathrm{erf}(h/\sqrt{2}\sigma)$, and windows narrower than 1.2 FWHM are rejected. Returns `(emin_ev, emax_ev, label)`.

xrf_fit.py, lines 2049–2085

```
# ============================================================================
# Resolution stability: width profile, step detection, per-segment shapes
# ============================================================================

def width_window(groups: Sequence[LineGroup], areas: Mapping[str, float],
                 detector: Detector, min_frac: float = 0.03,
                 doublet_ev: float = 60.0) -> Tuple[float, float, str]:
    """Pick a clean energy window around a strong, isolated line.

    Candidates are the reference lines of the groups.  The window half-width
    is ``min(2.5 FWHM, half the distance to the nearest interfering line)``.
    A line interferes if its expected area (group area x rel) is at least
    ``min_frac`` of the candidate's and it lies further than ``doublet_ev``
    (unresolvable doublets such as K-alpha1/2 belong to the peak).  The
    candidate with the most counts inside its window wins.

    Returns
    -------
    (emin_ev, emax_ev, label)
    """
    from scipy.special import erf
    all_lines = [(g.label, l.energy_ev, areas.get(g.label, 0.0) * l.rel)
                 for g in groups for l in g.lines]
```

```
    best = None
    for g in groups:
        e0 = g.reference.energy_ev
        a0 = areas.get(g.label, 0.0)
        others = [abs(e - e0) for _, e, a in all_lines
                  if abs(e - e0) > doublet_ev and a >= min_frac * a0]
        d = min(others) if others else np.inf
        fwhm = float(detector.fwhm_ev(e0))
        half = min(2.5 * fwhm, 0.5 * d)
        captured = erf(half / (np.sqrt(2.0) * fwhm / 2.3548))     # Gaussian fraction inside
        score = areas.get(g.label, 0.0) * captured * (half > 1.2 * fwhm)
        if best is None or score > best[0]:
            best = (score, e0 - half, e0 + half, g.label)
    return best[1], best[2], best[3]
```

5.13.2 peak_width_profile

```
peak_width_profile(raw_spectra, calibration, window_ev, bin_size)
```

The centroid and r.m.s. width of a peak along the acquisition order. Consecutive pixels are summed in bins (one map row is a natural bin), a linear background between the window edges is removed, and the first and second moments of the net peak give the centroid $\bar{E}$ and the width w, with the statistical error $w/\sqrt{2n}$ of a Gaussian width estimated from n counts. A resolution change shows as a step in w, a gain or offset change as a step in $\bar{E}$. Independent of the normalisation. Returns `{width_ev, centroid_ev, counts, width_err_ev}`.

xrf_fit.py, lines 2088–2122

```
def peak_width_profile(raw_spectra: np.ndarray, calibration: Calibration,
                       window_ev: Tuple[float, float], bin_size: int) -> Dict[str, np.ndarray]:
    """Peak centroid and rms width along the acquisition order.

    Consecutive pixels (acquisition order, i.e. the flattened scan) are summed
    in bins of ``bin_size`` (one map row = the number of fast-axis points), a
    linear background between the window edges is removed, and the first and
    second moments of the peak inside ``window_ev`` are computed per bin.
    A detector-resolution change shows up as a step in the width; a gain or
    offset change as a step in the centroid.  Insensitive to the per-pixel
    normalisation.

    Returns
    -------
    dict with ``width_ev``, ``centroid_ev``, ``counts`` and ``width_err_ev``
    (statistical, Gaussian approximation), one value per bin.
    """
    raw = np.asarray(raw_spectra, dtype=float)
    n_pix, n_ch = raw.shape
    n_bins = n_pix // bin_size
    binned = raw[:n_bins * bin_size].reshape(n_bins, bin_size, n_ch).sum(axis=1)
    e = calibration.energy(np.arange(n_ch))
    m = (e >= window_ev[0]) & (e <= window_ev[1])
    x = e[m]
    y = binned[:, m]
    edge = max(2, m.sum() // 20)
    left, right = y[:, :edge].mean(axis=1), y[:, -edge:].mean(axis=1)
    bkg = left[:, None] + (right - left)[:, None] * (x - x[0]) / (x[-1] - x[0])
    net = np.clip(y - bkg, 0.0, None)
    n = net.sum(axis=1)
    with np.errstate(divide="ignore", invalid="ignore"):
        c = (net * x).sum(axis=1) / n
        w = np.sqrt((net * (x - c[:, None]) ** 2).sum(axis=1) / n)
        werr = w / np.sqrt(2.0 * n)
    return dict(width_ev=w, centroid_ev=c, counts=n, width_err_ev=werr)
```

5.13.3 detect_steps

```
detect_steps(values, window=5, threshold=6.0, min_step=0.0)
```

Finds abrupt jumps in a series. At each index the difference between the means of the `window` points after and before is divided by its noise, $\sigma\sqrt{2/w}$, where σ is the point-to-point noise estimated robustly from the median absolute deviation of successive differences, $\sigma = 1.4826\,\mathrm{MAD}(\Delta v)/\sqrt{2}$ [42]. Local maxima of this statistic above `threshold`, with a step larger than `min_step`, are returned (a new level starts at each index). Slow drifts are better handled by the width term of `fit_map`.

xrf_fit.py, lines 2125–2157

```
def detect_steps(values: np.ndarray, window: int = 5, threshold: float = 6.0,
                 min_step: float = 0.0) -> List[int]:
    """Indices where a series jumps: a new level starts at each returned index.

    For every index ``i`` the means of the ``window`` points after and before
    it are compared, in units of the point-to-point noise (robust: from the
    median absolute successive difference).  Local maxima of that statistic
    above ``threshold`` and with ``|step| > min_step`` are returned.  Meant
    for abrupt changes; a slow drift is better handled by the width term of
    :func:`fit_map`.
    """
    v = np.asarray(values, dtype=float)
    n = v.size
    good = np.isfinite(v)
    if good.sum() < 2 * window + 1:
        return []
    dv = np.diff(v[good])
    sigma = 1.4826 * np.median(np.abs(dv - np.median(dv))) / np.sqrt(2.0)
    if sigma <= 0:
        sigma = np.std(v[good]) or 1.0
    z = np.zeros(n)
    step = np.zeros(n)
    for i in range(window, n - window + 1):
        a, b = v[i - window:i], v[i:i + window]
        if np.isfinite(a).all() and np.isfinite(b).all():
            step[i] = b.mean() - a.mean()
            z[i] = abs(step[i]) / (sigma * np.sqrt(2.0 / window))
    out = []
    for i in range(window, n - window + 1):
        lo, hi = max(0, i - window), min(n, i + window + 1)
        if z[i] >= threshold and z[i] == z[lo:hi].max() and abs(step[i]) > min_step:
            out.append(i)
    return out
```

5.13.4 `segment_labels`, `fit_segments`

`segment_labels(steps, n_bins, bin_size, n_pixels=None)`, `fit_segments(corrected, raw_spectra, scale, segments, groups, calibration, detector, roi, pileup=None, mask=None, **fit_kw)`

`segment_labels` turns step positions (in bins) into a label per pixel (0, 1, …), the last incomplete bin taking the last label. `fit_segments` fits the average spectrum of each segment, with its own counting error and pile-up template, and returns `{label: FitResult}`; `{label: r.popt}` then goes to `fit_map` with the same segments. With `mask`, only the pixels inside it enter each segment's average, and a segment without any of them is skipped.

xrf_fit.py, lines 2160–2170

```
def segment_labels(steps: Sequence[int], n_bins: int, bin_size: int,
                   n_pixels: Optional[int] = None) -> np.ndarray:
    """Per-pixel segment labels (0, 1, …) from bin indices where segments start."""
    bins = np.zeros(n_bins, dtype=int)
    for s in sorted(steps):
        bins[s:] += 1
    lab = np.repeat(bins, bin_size)
    n_pixels = n_pixels or lab.size
    if lab.size < n_pixels:                       # incomplete last bin
        lab = np.concatenate([lab, np.full(n_pixels - lab.size, bins[-1])])
    return lab[:n_pixels]
```

xrf_fit.py, lines 2173–2207

```
2173 def fit_segments(corrected: np.ndarray, raw_spectra: np.ndarray, scale: np.ndarray,
2174                  segments: np.ndarray, groups: Sequence[LineGroup], calibration: Calibration,
2175                  detector: Detector, roi: Tuple[int, int], pileup: Optional[np.ndarray] = None,
2176                  mask: Optional[np.ndarray] = None, **fit_kw) -> Dict[int, FitResult]:
2177     """Average-spectrum fit of every segment, each with its own resolution.
2178 
2179     The segment's average spectrum, counting error and (if ``pileup`` is
2180     given) pile-up template are formed from its own pixels, and
2181     :func:`fit_average_spectrum` is run with ``fit_kw`` (``yerr`` and
2182     ``pileup_template`` are replaced).  Feed ``{label: r.popt}`` to
2183     :func:`fit_map` together with the same ``segments``.
2184 
2185     With ``mask`` (per-pixel booleans, e.g. the facet of a crystal), only the
2186     pixels inside it enter each segment's average; a segment without any such
2187     pixel is skipped (and needs no shape in :func:`fit_map` with the same mask).
2188     """
2189     fit_kw = dict(fit_kw)
2190     fit_kw.pop("yerr", None)
2191     tmpl_given = fit_kw.pop("pileup_template", None)
2192     out = {}
2193     segments = np.asarray(segments)
2194     keep = np.ones(segments.size, bool) if mask is None else np.asarray(mask, bool).reshape(-1)
2195     for lab in np.unique(segments):
2196         idx = np.flatnonzero((segments == lab) & keep)
2197         if idx.size == 0:
2198             logger.info("segment %s has no pixel inside the mask: skipped", lab)
2199             continue
2200         a_s, _, _ = average_spectrum(corrected[idx])
2201         e_s = counting_error(raw_spectra[idx], scale[idx])
2202         tmpl = tmpl_given
2203         if pileup is not None and tmpl_given is not None:
2204             tmpl = average_pileup_template(pileup[idx], scale[idx])
2205         out[int(lab)] = fit_average_spectrum(a_s, groups, calibration, detector, roi, yerr=e_s,
2206                                              pileup_template=tmpl, **fit_kw)
2207     return out
```

5.14 Diagnostics and plots

5.14.1 `identify_peaks`

```
identify_peaks(y, calibration, roi, prominence_frac=0.01, tol_ev=70.0, min_rate=0.08,
z_range=(12, 83), n_candidates=4)
```

A scouting tool for choosing the element list: peaks are found with `scipy.signal.find_peaks` (prominence above `prominence_frac` of the maximum, at least 8 channels apart), and for each peak the tabulated IUPAC lines within `tol_ev`, of elements in `z_range` and with a radiative rate above `min_rate`, are listed, strongest rate first. An energy coincidence is only a hint: an element is confirmed when its whole line series fits. Returns `[{channel, energy_ev, height, candidates}]`.

xrf_fit.py, lines 2210–2260

```
2210 # ============================================================================
2211 # Diagnostics and plotting
2212 # ============================================================================
2213 
2214 def identify_peaks(
2215     y: np.ndarray,
2216     calibration: Calibration,
2217     roi: Tuple[int, int],
2218     prominence_frac: float = 0.01,
2219     tol_ev: float = 70.0,
2220     min_rate: float = 0.08,
2221     z_range: Tuple[int, int] = (12, 83),
2222     n_candidates: int = 4,
2223 ) -> List[dict]:
2224     """Find peaks and list tabulated lines that could explain them.
2225 
2226     A scouting tool for choosing the element list — it makes no attempt to be a
2227     quantitative identification, and a close energy match is not evidence on its
2228     own.  Candidates are ranked by radiative rate.
```

```
    Returns
    -------
    list of dict
        ``{"channel", "energy_ev", "height", "candidates": [str, ...]}``.
    """
    from scipy.signal import find_peaks

    lo, hi = roi
    seg = np.asarray(y, dtype=float)[lo:hi]
    idx, _ = find_peaks(seg, prominence=np.nanmax(seg) * prominence_frac, distance=8)

    out = []
    for i in idx:
        ch = lo + int(i)
        e0 = float(calibration.energy(ch))
        hits = []
        for z in range(z_range[0], z_range[1] + 1):
            for name, macro in _LINE_MACROS.items():
                if not _is_iupac_line(name):
                    continue
                e = _line_energy_ev(z, macro)
                if e is None or abs(e - e0) > tol_ev:
                    continue
                r = _line_strength(z, macro, 20.0, "radrate")
                if r >= min_rate:
                    hits.append((r, f"{xraylib.AtomicNumberToSymbol(z)}-{name[:-5]}"
                                    f" {e:.0f} eV (r={r:.2f}, d={e - e0:+.0f})"))
        hits.sort(reverse=True)
        out.append({"channel": ch, "energy_ev": e0, "height": float(seg[i]),
                    "candidates": [h[1] for h in hits[:n_candidates]]})
    return out
```

5.14.2 pixel_parameters, select_pixels

`pixel_parameters(model, map_result, shape_params, pixel)`, `select_pixels(map_result, n=10, mode="random", seed=0)`

`pixel_parameters` rebuilds a full parameter vector for one pixel (its background and amplitudes from the map solve, the shape from the average fit or its segment). `select_pixels` picks pixel indices to inspect: `"random"` among finite fits, `"worst"` and `"best"` by reduced χ^2, or `"spread"` along the χ^2 distribution in equal quantiles.

xrf_fit.py, lines 2263–2278

```
def pixel_parameters(model: "XRFModel", map_result: MapResult,
                     shape_params, pixel: int) -> np.ndarray:
    """Reassemble a full parameter vector for one pixel of a map fit.

    Takes the background coefficients and group amplitudes from the per-pixel
    linear solve, and the resolution parameters from the average-spectrum fit
    that defined the fixed shape (``shape_params`` may be ``{segment: params}``
    for a map fitted per segment).  The first-order width term of
    ``width_correction`` is not included.
    """
    if isinstance(shape_params, Mapping):
        shape_params = shape_params[map_result.segments[pixel]]
    p = np.array(shape_params, dtype=float)
    p[:model.n_bkg] = map_result.background[pixel]
    p[model.n_bkg:model.n_bkg + len(model.groups)] = map_result.areas[pixel]
    return p
```

xrf_fit.py, lines 2281–2316

```
def select_pixels(map_result: MapResult, n: int = 10, mode: str = "random",
                  seed: Optional[int] = 0) -> np.ndarray:
    """Choose pixel indices to inspect.

    Parameters
    ----------
```

```
    n : int
        How many pixels.
    mode : {"random", "worst", "best", "spread"}
        ``"random"`` samples uniformly from pixels with a finite fit;
        ``"worst"``/``"best"`` take the extremes of per-pixel reduced chi2
        (``"worst"`` is the one to look at when diagnosing);
        ``"spread"`` walks the chi2 distribution in equal quantiles, giving a
        representative range rather than a typical one.
    seed : int or None
        Seed for ``mode="random"``.

    Returns
    -------
    (n,) int array of flat pixel indices.
    """
    chi2 = map_result.chi2_red
    ok = np.flatnonzero(np.isfinite(chi2))
    if ok.size == 0:
        raise ValueError("No pixel has a finite fit.")
    n = min(n, ok.size)
    if mode == "random":
        return np.random.default_rng(seed).choice(ok, size=n, replace=False)
    order = ok[np.argsort(chi2[ok])]
    if mode == "best":
        return order[:n]
    if mode == "worst":
        return order[::-1][:n]
    if mode == "spread":
        return order[np.linspace(0, order.size - 1, n).astype(int)]
    raise ValueError(f"mode must be random/worst/best/spread, got {mode!r}")
```

5.14.3 plot_pixel_fits

```
plot_pixel_fits(raw_spectra, scale, model, map_result, shape_params, pixels=None, n=10,
mode="random", seed=0, ncols=5, energy_axis=True, logy=True, show_components=True,
figsize_per_panel=(3.2, 2.6), pileup=None)
```

A grid of individual pixel spectra with their fitted model, components shaded and the pixel's χ^2_ν in the title: the direct check that the fixed line ratios hold pixel by pixel and that the background is plausible at the local count level. With pile-up, the per-pixel pile-up spectra replace the map average. Returns `(fig, axes, pixels)`.

xrf_fit.py, lines 2319–2432

```
def plot_pixel_fits(
    raw_spectra: np.ndarray,
    scale: np.ndarray,
    model: "XRFModel",
    map_result: MapResult,
    shape_params: Sequence[float],
    pixels: Optional[Sequence[int]] = None,
    n: int = 10,
    mode: str = "random",
    seed: Optional[int] = 0,
    ncols: int = 5,
    energy_axis: bool = True,
    logy: bool = True,
    show_components: bool = True,
    figsize_per_panel: Tuple[float, float] = (3.2, 2.6),
    pileup: Optional[np.ndarray] = None,
):
    """Panel grid of individual pixel spectra with their fitted model.

    If the model includes pile-up, pass the per-pixel ``pileup`` spectra (as
    for :func:`fit_map`) so each panel shows its own pile-up rather than the
    map average.

    Map-averaged residuals and a median chi2 both hide what a single spectrum
    actually looks like.  This is the direct check: at a glance you see whether
    the fixed line ratios hold pixel by pixel, whether the background is
    plausible at the local count level, and where the weak groups disappear into
```

```
    the noise.

    Parameters
    ----------
    raw_spectra, scale
        As passed to :func:`fit_map`.
    model, map_result, shape_params
        The fitted model, its per-pixel solution, and the shape parameters.
    pixels : sequence of int, optional
        Explicit flat pixel indices.  Overrides ``n``/``mode``/``seed``.
    n, mode, seed
        Passed to :func:`select_pixels` when ``pixels`` is None.
    ncols : int
        Panels per row.
    energy_axis, logy, show_components
        Plot the x axis in keV rather than channels; log the y axis; shade the
        per-group contributions.
    figsize_per_panel : (w, h)
        Size of one panel in inches.

    Returns
    -------
    (fig, axes, pixels)
    """
    import matplotlib.pyplot as plt

    if pixels is None:
        pixels = select_pixels(map_result, n=n, mode=mode, seed=seed)
    pixels = np.asarray(pixels, dtype=int)

    lo, hi = int(model.channels[0]), int(model.channels[-1]) + 1
    raw = np.asarray(raw_spectra, dtype=float)[:, lo:hi]
    s = np.asarray(scale, dtype=float)
    x = model.calibration.energy(model.channels) / 1000.0 if energy_axis else model.channels
    xlabel = "Energy (keV)" if energy_axis else "MCA channel"

    nrows = int(np.ceil(len(pixels) / ncols))
    fig, axes = plt.subplots(nrows, ncols, sharex=True,
                             figsize=(figsize_per_panel[0] * ncols,
                                      figsize_per_panel[1] * nrows))
    axes = np.atleast_1d(axes).ravel()
    cmap = plt.get_cmap("tab10")
    n_fast = map_result.shape[1]

    for ax, pix in zip(axes, pixels):
        y = raw[pix] / s[pix]
        params = pixel_parameters(model, map_result, shape_params, pix)
        bkg = model.background(params)
        total = model(model.channels, *params)
        if model.pileup is not None and pileup is not None:
            total = (total - model.pileup_profile(params)
                     + model.pileup_tau_us(params) * 1e-6 * pileup[pix, lo:hi] / s[pix])

        ax.plot(x, y, color="0.25", lw=0.7, drawstyle="steps-mid", label="data")
        if show_components:
            for j, g in enumerate(model.groups):
                comp = model.group_profile(j, params)
                ax.fill_between(x, bkg, bkg + comp, color=cmap(j % 10), alpha=0.45,
                                lw=0, label=g.label)
        ax.plot(x, bkg, "--", color="0.5", lw=0.8)
        ax.plot(x, total, "r-", lw=1.1, label="fit")

        if logy:
            pos = y[y > 0]
            ax.set_yscale("log")
            if pos.size:
                ax.set_ylim(max(pos.min() * 0.5, 1e-4), y.max() * 3)
        row, col = divmod(int(pix), n_fast)
        ax.set_title(f"({row}, {col})   $\\chi^2_\\nu$ = {map_result.chi2_red[pix]:.2f}",
                     fontsize=8)
        ax.tick_params(labelsize=7)

    for ax in axes[len(pixels):]:
        ax.set_visible(False)
    for ax in axes[max(0, (nrows - 1) * ncols):len(pixels)]:
```

```
2421            ax.set_xlabel(xlabel, fontsize=8)
2422        for r in range(nrows):
2423            if r * ncols < len(pixels):
2424                axes[r * ncols].set_ylabel("counts / live s", fontsize=8)
2425
2426        h, l = axes[0].get_legend_handles_labels()
2427        fig.legend(h, l, loc="lower center", ncol=min(len(l), 7), fontsize=8,
2428                   frameon=False, bbox_to_anchor=(0.5, -0.02))
2429        fig.suptitle(f"Individual pixel fits — {mode} selection of {len(pixels)} "
2430                     f"(slow, fast) positions", y=1.0, fontsize=11)
2431        fig.tight_layout()
2432        return fig, axes, pixels
```

5.14.4 `plot_fit`

```
plot_fit(result, energy_axis=True, logy=False, figsize=(11.0, 7.0), data_label="map-averaged
data", ylabel=...)
```

Two panels: the spectrum with its error band, the background, every group (shaded, with its reference line marked), the scatter peaks if any, and the total model; below, the residual in units of the data error. Returns `(fig, (ax_spectrum, ax_residual))`.

xrf_fit.py, lines 2435–2494

```
2435 def plot_fit(result: FitResult, energy_axis: bool = True, logy: bool = False,
2436              figsize: Tuple[float, float] = (11.0, 7.0),
2437              data_label: str = "map-averaged data",
2438              ylabel: str = "Counts per live second (monitor-normalised)"):
2439     """Two-panel figure: spectrum with per-group components, and the residual.
2440
2441     Returns
2442     -------
2443     (fig, (ax_spectrum, ax_residual))
2444     """
2445     import matplotlib.pyplot as plt
2446
2447     model = result.model
2448     cal = result.calibration
2449     x = cal.energy(result.channels) / 1000.0 if energy_axis else result.channels
2450     xlabel = "Energy (keV)" if energy_axis else "MCA channel"
2451
2452     fig, (ax, axr) = plt.subplots(
2453         2, 1, figsize=figsize, sharex=True,
2454         gridspec_kw={"height_ratios": [3, 1], "hspace": 0.07})
2455
2456     if result.yerr is not None:
2457         ax.fill_between(x, result.y - result.yerr, result.y + result.yerr,
2458                         color="0.75", lw=0, label=r"data $\pm 1\sigma$")
2459     ax.plot(x, result.y, "k-", lw=1.0, label=data_label)
2460
2461     bkg = model.background(result.popt)
2462     ax.plot(x, bkg, "--", color="0.45", lw=1.0, label="background")
2463
2464     cmap = plt.get_cmap("tab10")
2465     for i, g in enumerate(model.groups):
2466         comp = model.group_profile(i, result.popt)
2467         ax.fill_between(x, bkg, bkg + comp, color=cmap(i % 10), alpha=0.45, lw=0,
2468                         label=f"{g.label}  ({len(g.lines)} lines)")
2469         # mark the group's reference line
2470         xref = (g.reference.energy_ev / 1000.0 if energy_axis
2471                 else cal.channel(g.reference.energy_ev))
2472         ax.axvline(xref, color=cmap(i % 10), lw=0.8, ls=":", alpha=0.8)
2473     if getattr(model, "scatter", None) is not None:
2474         for which, col in (("elastic", "0.35"), ("compton", "0.6")):
2475             comp = model.scatter_profile(result.popt, which)
2476             ax.fill_between(x, bkg, bkg + comp, color=col, alpha=0.35, lw=0, label=f"{which} scatter")
2477
2478     ax.plot(x, result.yfit, "r-", lw=1.4, label="total fit")
2479
2480     ax.set_ylabel(ylabel)
2481     if logy:
```

```
        ax.set_yscale("log")
        ax.set_ylim(max(result.y[result.y > 0].min() * 0.5, 1e-3), result.y.max() * 2)
    ax.legend(ncol=2, fontsize=9, framealpha=0.9)
    ax.set_title(f"Constrained Voigt fit — reduced $\\chi^2$ = {result.chi2_red:.2f}")

    r = result.residual_normalised if result.yerr is not None else result.residual
    axr.axhline(0.0, color="0.5", lw=0.8)
    axr.plot(x, r, "k-", lw=0.8)
    axr.set_xlabel(xlabel)
    axr.set_ylabel(r"residual / $\sigma$" if result.yerr is not None else "residual")

    fig.align_ylabels()
    return fig, (ax, axr)
```

6 `xrf_calibration`: energy calibration

6.1 Overview

The channel-to-energy relation $E = Gc + O$ of the detector drifts with the electronics settings (peaking time, gain, number of bins) and must be measured. The module does it on the spectrum of a reference sample in four steps:

1. **coarse match**: the calibrant's tabulated lines are compared with the measured peaks over a grid of gains and offsets; no peak has to be picked or assigned by hand;
2. **global fit**: the constrained model of `xrf_fit` with the gain and offset free, so that every line sits at its tabulated energy with tabulated relative intensities and unresolved multiplets (K$\alpha_{1,2}$, Lβ_{1-4}) are placed correctly instead of being treated as single peaks; the resolution, and optionally the sample–detector air path (from the line ratios within each element), are fitted at the same time;
3. **line checks**: the strongest peaks are refitted one at a time with only an energy shift free; the shifts should scatter around zero within their errors, and a trend with energy reveals a non-linear response;
4. **intensity check**: for a thin-film calibrant of certified loadings, the reference-line areas should follow $A = k\,L\,S$ with L the loading, S the group's `ref_strength` and one overall scale k; this tests the excitation and efficiency models, not the calibration.

`calibration_from_peaks` offers a simpler route when two or more peaks of known energy can be located (e.g. a fluorescence line and the elastic peak of a monochromatic beam).

Hypotheses and limits

- The response is linear in energy over the calibrated range (tested by the line checks).
- Thin-film calibrants (a few atomic layers on a membrane, like the AXO samples) need no matrix correction; the intensity check assumes it.
- The L-line ratios used for the air path depend somewhat on the excitation spectrum, which is the main systematic error of the fitted path.

xrf_calibration.py, lines 1–77

```
"""
Module: xrf_calibration.py

Energy calibration (channel -> energy) of an energy-dispersive detector from
the spectrum of a reference sample.

    from xrf_calibration import CALIBRANTS, calibrate, plot_calibration

    result = calibrate(spectrum, CALIBRANTS["AXO RF C10"], detector,
                       extra_elements=["Ar"], incident_spectrum=(E_kev, phi))
    print(result.summary())
    CALIB = result.calibration          # xrf_fit.Calibration, for the analysis

How it works
```

```
------------
1. **Coarse match.**  The tabulated lines of the calibrant are compared with
   the measured peaks over a grid of gains and offsets around the starting
   guess; the pair that puts the most line intensity on top of peaks wins.
   No peak has to be picked or assigned by hand.
2. **Global fit.**  The constrained model of :mod:`xrf_fit` is fitted with the
   gain and offset free (``fit_calibration=True``): every line of every
   element sits at its tabulated energy, with tabulated relative intensities,
   so unresolved multiplets (K-alpha1/2, L-beta1-4, ...) are placed correctly
   instead of being treated as single peaks.  The detector resolution is
   fitted at the same time, and optionally the sample-detector air path
   (``fit_air_path=True``), from the intensity ratios *within* each element
   (K-alpha/K-beta, L-alpha/L-beta/L-gamma).  Those ratios do not depend on
   the loadings; the L ratios depend somewhat on the excitation spectrum,
   which is the main systematic error of the fitted path.
3. **Line checks.**  The strongest peaks are refitted one at a time in a
   narrow window, with only an energy shift free.  The shifts should scatter
   around zero within their errors; a trend with energy reveals a
   non-linear response that a gain and offset cannot describe.
4. **Intensity check.**  For a thin-film calibrant with certified loadings,
   the reference-line area of each element should be ``A = k * loading * S``,
   with ``S`` the group's ``ref_strength`` (excitation x cross-section x
   detection efficiency, at the fitted air path) and ``k`` one overall scale.
   Comparing the two tests the excitation and efficiency models; it does not
   affect the calibration.

Calibrants
----------
A :class:`Calibrant` lists the certified elements and their loadings.  To add
a new one, add an entry to :data:`CALIBRANTS`.  Thin-film standards (a few
atomic layers on a membrane, like the AXO samples) need no matrix correction,
which is what the intensity check assumes.

Authors: Samuel Tardif, with Claude (Anthropic).
"""

from __future__ import annotations

import dataclasses
import logging
from dataclasses import dataclass, field
from typing import Dict, List, Mapping, Optional, Sequence, Tuple

import numpy as np
import xraylib
from scipy.ndimage import gaussian_filter1d

from scipy.optimize import curve_fit

from xrf_beamline import AIR_FORMULA, air_density
from xrf_fit import (Calibration, Detector, FitResult, LineGroup, element_line_groups,
                     fit_average_spectrum, plot_fit, snip_background)

logger = logging.getLogger(__name__)

#: Siegbahn names of the common lines, for labels (IUPAC name -> Siegbahn).
SIEGBAHN = {"KL3": "Ka1", "KL2": "Ka2", "KM3": "Kb1", "KM2": "Kb3", "KN3": "Kb2",
            "L3M5": "La1", "L3M4": "La2", "L2M4": "Lb1", "L3N5": "Lb2", "L1M3": "Lb3",
            "L1M2": "Lb4", "L2N4": "Lg1", "L1N3": "Lg3", "L3M1": "Ll", "L2M1": "Ln",
            "M5N7": "Ma1", "M4N6": "Mb"}

__all__ = ["Calibrant", "CALIBRANTS", "LineCheck", "CalibrationResult",
           "coarse_calibration", "calibrate", "plot_calibration", "calibration_from_peaks"]
```

6.2 Reference samples and results

6.2.1 `Calibrant`; `elements`, `atom_ratio`

`Calibrant(name, loadings_ng_mm2, description="")`; `elements`, `atom_ratio(element, reference)`

A reference sample: its certified areal densities (ng/mm^2) by element. `elements` lists them; `atom_ratio` gives the certified atom-number ratio, $(L_e/W_e)/(L_r/W_r)$ with W the atomic weights.

xrf_calibration.py, lines 80–100

```
80  # ============================================================================
81  # Reference samples
82  # ============================================================================
83
84  @dataclass(frozen=True)
85  class Calibrant:
86      """A reference sample.
87
88      Parameters
89      ----------
90      name : str
91      loadings_ng_mm2 : mapping
92          Certified areal density of each element, ng/mm^2.  Only the ratios
93          matter here; the elements' lines are what the calibration uses.
94      description : str
95          Free text (supplier, reference, substrate).
96      """
97
98      name: str
99      loadings_ng_mm2: Mapping[str, float]
100     description: str = ""
```

xrf_calibration.py, lines 102–104

```
102     @property
103     def elements(self) -> List[str]:
104         return list(self.loadings_ng_mm2)
```

xrf_calibration.py, lines 106–110

```
106     def atom_ratio(self, element: str, reference: str) -> float:
107         """Certified atom-number ratio ``n_element / n_reference``."""
108         def atoms(el: str) -> float:
109             return self.loadings_ng_mm2[el] / xraylib.AtomicWeight(xraylib.SymbolToAtomicNumber(el))
110         return atoms(element) / atoms(reference)
```

6.2.2 `CALIBRANTS`, `LineCheck`

`CALIBRANTS`, `LineCheck(label, energy_ev, channel, shift_ev, shift_err_ev)`

`CALIBRANTS` holds the known samples; at present the AXO Dresden RF-type C10 standard: Pb 84.9, La 121.4, Pd 23.3, Mo 8.6, Cu 22.2 and Fe 43.9 ng/mm^2 on a 200 nm Si membrane. To add a calibrant, add an entry. A `LineCheck` records one peak measured on its own: `shift_ev` is the energy that the calibration gives the measured peak minus the tabulated energy (positive: the peak sits at a higher channel than predicted). The listing of `LineCheck` starts with `CALIBRANTS`, which precedes it in the file.

xrf_calibration.py, lines 113–142

```
113 #: Known reference samples, by name.
114 CALIBRANTS: Dict[str, Calibrant] = {
115     "AXO RF C10": Calibrant(
116         name="AXO RF C10",
117         loadings_ng_mm2={"Pb": 84.9, "La": 121.4, "Pd": 23.3, "Mo": 8.6, "Cu": 22.2, "Fe": 43.9},
118         description=("AXO Dresden RF-type XRF reference, C10 loading (RF-200-0205-C10-X): "
119                      "a few atomic layers of each element on a 200 nm Si membrane. "
120                      "https://www.axo-dresden.de/en/products/reference-samples/"),
121     ),
122 }
123
124
125 # ============================================================================
126 # Results
127 # ============================================================================
128
129 @dataclass
130 class LineCheck:
131     """Position of one peak measured on its own, relative to the calibration.
```

```
    ``shift_ev`` = energy that the calibration gives the measured peak minus
    the tabulated line energy.  > 0: the peak sits at a higher channel than
    the calibration predicts.
    """

    label: str            # e.g. "Pb LA1"
    energy_ev: float      # tabulated energy of the line
    channel: float        # where the calibration puts it
    shift_ev: float
    shift_err_ev: float
```

6.2.3 `CalibrationResult`

Outcome of `calibrate`: the fitted `calibration`, the 2×2 `covariance` of (gain, offset), the coarse `start`, the global `fit` (`FitResult`), the input `detector` with the fitted resolution and air path, the `calibrant`, the line `checks`, the `intensity` check and `air_path_err`.

Methods `gain_err`, `offset_err`; `energy_error(channel)`, the one-sigma uncertainty of the calibrated energy including the gain–offset correlation,

$$\sigma_E^2(c) = c^2\, V_{GG} + 2c\, V_{GO} + V_{OO}; \tag{30}$$

`check_chi2()`, the mean of (shift/error)^2 over the line checks (about 1 when the response is linear within the precision); `summary()`, a text report; `to_dict()`, a plain record to save as JSON.

xrf_calibration.py, lines 145–180

```
@dataclass
class CalibrationResult:
    """Outcome of :func:`calibrate`.

    Attributes
    ----------
    calibration : Calibration
        The fitted ``E = gain * channel + offset``.
    covariance : (2, 2) array
        Covariance of (gain, offset).
    start : Calibration
        Result of the coarse match (the starting point of the fit).
    fit : FitResult
        The global fit (``plot_fit(result.fit)`` shows it).
    detector : Detector
        The input detector with the fitted resolution (noise, Fano, gamma)
        and, if fitted, air path.
    checks : list of LineCheck
        Individual peak positions (empty if not requested).
    intensity : dict
        Measured vs predicted areas of the certified elements: ``rows``,
        ``scale``, ``chi2_red``, ``air_path_cm``.  Empty without enough
        certified elements in the spectrum.
    air_path_err : float
        1-sigma statistical error of the fitted air path (NaN if not fitted).
    """

    calibration: Calibration
    covariance: np.ndarray
    start: Calibration
    fit: FitResult
    detector: Detector
    calibrant: Calibrant
    checks: List[LineCheck] = field(default_factory=list)
    intensity: dict = field(default_factory=dict)
    air_path_err: float = float("nan")
```

xrf_calibration.py, lines 182–184

```
    @property
    def gain_err(self) -> float:
        return float(np.sqrt(self.covariance[0, 0]))
```

xrf_calibration.py, lines 186–188

```
186     @property
187     def offset_err(self) -> float:
188         return float(np.sqrt(self.covariance[1, 1]))
```

xrf_calibration.py, lines 190–195

```
190     def energy_error(self, channel: np.ndarray | float) -> np.ndarray | float:
191         """1-sigma uncertainty of the calibrated energy at a channel (eV),
192         including the gain-offset correlation."""
193         c = np.asarray(channel, dtype=float)
194         cov = self.covariance
195         return np.sqrt(c ** 2 * cov[0, 0] + 2 * c * cov[0, 1] + cov[1, 1])
```

xrf_calibration.py, lines 197–203

```
197     def check_chi2(self) -> float:
198         """Reduced chi2 of the line shifts against zero: ~1 when the response
199         is linear within the measured precision."""
200         if not self.checks:
201             return float("nan")
202         z = np.array([c.shift_ev / c.shift_err_ev for c in self.checks])
203         return float(np.mean(z ** 2))
```

xrf_calibration.py, lines 205–236

```
205     def summary(self) -> str:
206         c, d = self.calibration, self.detector
207         lines = [
208             f"Calibrant: {self.calibrant.name}",
209             f"E[eV] = {c.gain:.5f} (+/- {self.gain_err:.5f}) * channel "
210             f"{c.offset:+.2f} (+/- {self.offset_err:.2f})",
211             f"  gain-offset correlation {self.covariance[0, 1] / (self.gain_err * self.offset_err):+.3f}; "
212             f"energy uncertainty {self.energy_error(c.channel(5000)):.2f} eV at 5 keV, "
213             f"{self.energy_error(c.channel(15000)):.2f} eV at 15 keV",
214             f"  coarse match was gain {self.start.gain:.4f}, offset {self.start.offset:+.1f} eV",
215             f"global fit: reduced chi2 = {self.fit.chi2_red:.2f}",
216             f"resolution: FWHM {d.fwhm_ev(5900):.0f} eV at 5.9 keV, {d.fwhm_ev(10000):.0f} eV at 10 keV "
217             f"(noise {d.noise_ev:.1f} eV, Fano {d.fano:.3f}, gamma {d.gamma_ev:.1f} eV)",
218             (f"sample-detector air path: {d.air_path_cm:.2f} +/- {self.air_path_err:.2f} cm (fitted, "
219              f"statistical error only)" if np.isfinite(self.air_path_err)
220              else f"sample-detector air path: {d.air_path_cm:.2f} cm (fixed)"),
221         ]
222         if self.checks:
223             lines.append(f"line checks (calibrated energy of the peak - tabulated energy), "
224                          f"chi2 vs 0 = {self.check_chi2():.2f}:")
225             for k in sorted(self.checks, key=lambda k: k.energy_ev):
226                 lines.append(f"  {k.label:<16} {k.energy_ev / 1000:7.3f} keV  ch {k.channel:7.1f}  "
227                              f"{k.shift_ev:+6.1f} +/- {k.shift_err_ev:4.1f} eV")
228         if self.intensity:
229             it = self.intensity
230             lines.append(f"intensity check, area = k x loading x S(E) at {it['air_path_cm']:.1f} cm of air "
231                          f"(tests the excitation/efficiency model), chi2 = {it['chi2_red']:.1f}:")
232             for r in it["rows"]:
233                 note = "" if r["used"] else "   (below 3 sigma, not used)"
234                 lines.append(f"  {r['label']:<6} {r['energy_ev'] / 1000:6.2f} keV  measured / predicted "
235                              f"{r['ratio']:5.2f} +/- {r['ratio_err']:.2f}{note}")
236         return "\n".join(lines)
```

xrf_calibration.py, lines 238–252

```
238     def to_dict(self) -> dict:
239         """Plain-data record, e.g. to save as JSON next to the data."""
240         c, d = self.calibration, self.detector
241         return {
242             "calibrant": self.calibrant.name,
243             "gain_ev_per_channel": c.gain, "gain_err": self.gain_err,
244             "offset_ev": c.offset, "offset_err": self.offset_err,
245             "covariance_gain_offset": self.covariance.tolist(),
246             "chi2_red": self.fit.chi2_red,
247             "detector": {"noise_ev": d.noise_ev, "fano": d.fano, "gamma_ev": d.gamma_ev,
248                          "fwhm_ev_at_5900": float(d.fwhm_ev(5900)), "air_path_cm": d.air_path_cm,
249                          "air_path_err": self.air_path_err},
```

```
            "line_checks": [dataclasses.asdict(k) for k in self.checks],
            "intensity": self.intensity,
        }
```

6.3 Procedure

6.3.1 `coarse_calibration`

```
coarse_calibration(y, groups, gain=10.0, offset=0.0, gain_tolerance=0.1,
offset_tolerance_ev=500.0, smooth_ch=4.0)
```

Grid search of the calibration that best matches the line pattern. The background is removed (SNIP), the spectrum smoothed with a Gaussian of `smooth_ch` channels and square-rooted (so that weak elements count), and each candidate (G, O) is scored by

$$\mathrm{score}(G,O) = \sum_l r_l \sqrt{y_{\mathrm{net}}\left(\frac{E_l - O}{G}\right)}, \tag{31}$$

interpolated at the predicted channel of every line. Gains span $\pm$`gain_tolerance` (801 values) and offsets $\pm$`offset_tolerance_ev` in 2 eV steps; one gain is scanned at a time to bound memory. A warning is issued if the best point lies at the edge of the range. Returns a `Calibration`.

xrf_calibration.py, lines 255–302

```
# ============================================================================
# Procedure
# ============================================================================

def coarse_calibration(
    y: np.ndarray,
    groups: Sequence[LineGroup],
    gain: float = 10.0,
    offset: float = 0.0,
    gain_tolerance: float = 0.1,
    offset_tolerance_ev: float = 500.0,
    smooth_ch: float = 4.0,
) -> Calibration:
    """Best (gain, offset) on a grid, by matching the line pattern.

    The background (SNIP) is removed, the spectrum smoothed and square-rooted
    (so that weak elements count too), and each candidate calibration is
    scored by the sum of that signal at the predicted channels of all lines,
    weighted by their relative intensities.

    Parameters
    ----------
    y : (n_channels,) array
    groups : line groups of the calibrant (:func:`xrf_fit.element_line_groups`)
    gain, offset : starting guess (eV/channel, eV)
    gain_tolerance : searched gain range, as a fraction of ``gain``
    offset_tolerance_ev : searched offset range, +/- eV
    smooth_ch : Gaussian smoothing of the spectrum, channels
    """
    y = np.asarray(y, dtype=float)
    signal = np.sqrt(gaussian_filter1d(np.clip(y - snip_background(y, width=40), 0, None), smooth_ch))
    e = np.array([l.energy_ev for g in groups for l in g.lines])
    w = np.array([l.rel for g in groups for l in g.lines])

    gains = gain * np.linspace(1 - gain_tolerance, 1 + gain_tolerance, 801)
    offsets = offset + np.arange(-offset_tolerance_ev, offset_tolerance_ev + 1e-9, 2.0)
    best = (-np.inf, gain, offset)
    for g in gains:                                # one gain at a time keeps memory small
        ch = (e[None, :] - offsets[:, None]) / g
        score = (np.interp(ch, np.arange(y.size), signal, left=0.0, right=0.0) * w).sum(axis=1)
        j = int(np.argmax(score))
        if score[j] > best[0]:
            best = (float(score[j]), float(g), float(offsets[j]))
    _, g_best, o_best = best
    if abs(g_best / gain - 1) > 0.99 * gain_tolerance or abs(o_best - offset) > 0.99 * offset_tolerance_ev:
```

```
        logger.warning("Coarse calibration at the edge of the search range (gain %.4f, offset %.0f eV): "
                       "widen it or improve the starting guess.", g_best, o_best)
    return Calibration(gain=g_best, offset=o_best)
```

6.3.2 `calibration_from_peaks`

```
calibration_from_peaks(y, peaks, half_width=35)
```

Linear calibration through reference peaks near given channels.

`y` the spectrum.

`peaks`
`[(channel guess, energy in eV), ...]`; for an unresolved doublet give its intensity-weighted energy (e.g. K$\alpha_{1,2}$), since that is where the centroid sits.

`half_width`
half width of the fitting window, channels.

Method Each peak is fitted within `half_width` channels of its guess with a Gaussian on a linear background, Poisson weights; the calibration is the least-squares line through (centroid, energy), exact with two peaks.

Returns `(Calibration, [{channel, channel_err, sigma_ch, energy_ev}])`.

Hypotheses and limits

Each peak is a single Gaussian (or an unresolved multiplet whose centroid is its weighted energy) on a linear background inside the window; a nearby peak inside the window biases the centroid.

xrf_calibration.py, lines 305–340

```
def calibration_from_peaks(y: np.ndarray, peaks: Sequence[Tuple[float, float]],
                           half_width: int = 35) -> Tuple[Calibration, List[dict]]:
    """Linear calibration through reference peaks found near given channels.

    Each peak is located by a Gaussian plus linear-background fit within
    ``half_width`` channels of its starting channel; the calibration is the
    least-squares line (exact with two peaks) through (centroid, energy).

    Parameters
    ----------
    y : (n_channels,) array
    peaks : sequence of (channel guess, energy in eV)
        For an unresolved doublet give its intensity-weighted energy (e.g.
        K-alpha1/K-alpha2), since that is where the centroid sits.

    Returns
    -------
    (Calibration, list of {"channel", "channel_err", "sigma_ch", "energy_ev"})
    """
    y = np.asarray(y, dtype=float)

    def model(x, a, c, s, b0, b1):
        return a * np.exp(-0.5 * ((x - c) / s) ** 2) + b0 + b1 * (x - c)

    found = []
    for c0, e in peaks:
        lo, hi = max(int(c0) - half_width, 0), min(int(c0) + half_width + 1, y.size)
        x, yy = np.arange(lo, hi, dtype=float), y[lo:hi]
        p0 = [float(yy.max() - yy.min()), float(x[np.argmax(yy)]), half_width / 4.0, float(yy.min()), 0.0]
        popt, pcov = curve_fit(model, x, yy, p0=p0, sigma=np.sqrt(np.maximum(yy, 1.0)), maxfev=5000)
        found.append({"channel": float(popt[1]), "channel_err": float(np.sqrt(pcov[1, 1])),
                      "sigma_ch": abs(float(popt[2])), "energy_ev": float(e)})
    ch = np.array([f["channel"] for f in found])
    en = np.array([f["energy_ev"] for f in found])
    gain, offset = np.polyfit(ch, en, 1)
    return Calibration(gain=float(gain), offset=float(offset)), found
```

6.3.3 _line_checks

```
_line_checks(y, yerr, fit, detector, escape, refit_groups=None, min_fraction=0.05,
half_width_fwhm=1.5)
```

Refits the strongest peaks one at a time. Candidate lines are ranked by their fitted area (group area × relative intensity); lines above `min_fraction` of the strongest and more than one FWHM from an already chosen line are kept. Each window spans ±1.5 FWHM and keeps the groups that contribute at least 2 % of the modelled peak signal there (from the global fit). The local fit frees only the calibration offset (`fit_calibration="offset"`), the amplitudes and a linear background, with the resolution of the global fit; the measured peak energy relative to the calibration is minus the change of offset. `refit_groups` replace groups by label (e.g. groups rebuilt at the fitted air path). Failed fits are skipped with a warning.

xrf_calibration.py, lines 343–394

```
343 def _line_checks(y: np.ndarray, yerr: Optional[np.ndarray], fit: FitResult,
344                 detector: Detector, escape: bool,
345                 refit_groups: Optional[Sequence[LineGroup]] = None,
346                 min_fraction: float = 0.05, half_width_fwhm: float = 1.5) -> List[LineCheck]:
347     """Refit the strongest peaks one at a time with only an energy shift free.
348 
349     Each window keeps the groups that contribute at least 2 % of the modelled
350     peak signal there (from the global fit); weaker ones would only add
351     amplitudes the window cannot constrain.  ``refit_groups`` (matched by
352     label) replace the global fit's groups in the refits, e.g. groups rebuilt
353     at a fitted air path.
354     """
355     cal, groups, n = fit.calibration, fit.model.groups, len(y)
356     replace = {g.label: g for g in (refit_groups or [])}
357     profiles = [fit.model.group_profile(i, fit.popt) for i in range(len(groups))]
358     first = int(fit.channels[0])
359     # candidate peaks: fitted area of each line, strongest first
360     peaks = sorted(((fit.areas[g.label] * l.rel, g, l) for g in groups for l in g.lines),
361                    key=lambda t: -t[0])
362     top = peaks[0][0]
363     chosen: List[Tuple[LineGroup, object]] = []
364     for area, g, l in peaks:
365         if area < min_fraction * top:
366             break
367         fwhm = detector.fwhm_ev(l.energy_ev)
368         if all(abs(l.energy_ev - m.energy_ev) > fwhm for _, m in chosen):
369             chosen.append((g, l))
370 
371     checks = []
372     for g, line in chosen:
373         half = half_width_fwhm * detector.fwhm_ev(line.energy_ev)
374         roi = cal.channel_range(line.energy_ev - half, line.energy_ev + half, n)
375         if roi[1] - roi[0] < 8:
376             continue
377         a, b = max(roi[0] - first, 0), max(roi[1] - first, 0)
378         peak = [prof[a:b].max() if b > a else 0.0 for prof in profiles]
379         local = [replace.get(gg.label, gg) for gg, h in zip(groups, peak) if h >= 0.02 * max(peak)]
380         try:
381             r = fit_average_spectrum(y, local, cal, detector, roi, yerr=yerr, background_order=1,
382                                      fit_resolution=False, fit_gamma=False, escape=escape,
383                                      fit_calibration="offset", maxfev=5000)
384         except (RuntimeError, ValueError) as err:
385             logger.warning("Line check %s %s failed: %s", g.element, line.name, err)
386             continue
387         value, error = r.params()["cal_offset_ev"]
388         # a positive shift of the offset moves predicted peaks up; the peak's
389         # measured energy relative to the calibration is minus that
390         name = f"{SIEGBAHN[line.name]} ({line.name})" if line.name in SIEGBAHN else line.name
391         checks.append(LineCheck(label=f"{g.element} {name}", energy_ev=line.energy_ev,
392                                 channel=float(cal.channel(line.energy_ev)),
393                                 shift_ev=-(value - cal.offset), shift_err_ev=error))
394     return checks
```

6.3.4 `_intensity_model`

```
_intensity_model(fit, calibrant, detector, air_path_cm)
```

Compares the reference-line areas of the certified elements (one family per element, the most precise) with $A_e = k\,L_e\,S_e\,\mathrm{e}^{-\mu_{\mathrm{air}}(E_e)(d-d_0)}$, where S_e was computed at the air path d_0 of the detector and d is the fitted one. Only k is fitted, by weighted least squares on the elements detected above 3σ:

$$k = \frac{\sum_e w_e A_e m_e}{\sum_e w_e m_e^2}, \qquad w_e = \sigma_{A_e}^{-2}, \quad m_e = L_e S_e \mathrm{e}^{-\mu(d-d_0)}. \tag{32}$$

Returns the scale, the reduced χ^2 and one row per element (measured/predicted ratio and its error); empty with fewer than two usable elements.

xrf_calibration.py, lines 397–433

```
397 def _intensity_model(fit: FitResult, calibrant: Calibrant, detector: Detector,
398                     air_path_cm: float) -> dict:
399     """Compare the reference-line areas of the certified elements with
400     ``A = k * loading * S(E_ref)``, one family per element (the most precise).
401 
402     ``S`` is the group's ``ref_strength``, computed with
403     ``detector.air_path_cm = d0``; at the air path ``air_path_cm = d`` it
404     scales as ``exp(-mu_air(E_ref) (d - d0))``.  Only ``k`` is fitted, on the
405     elements measured above 3 sigma.
406     """
407     best: Dict[str, LineGroup] = {}
408     snr = {g.label: fit.areas[g.label] / max(fit.area_errors[g.label], 1e-300) for g in fit.model.groups}
409     for g in fit.model.groups:
410         if g.element in calibrant.loadings_ng_mm2 and np.isfinite(g.ref_strength) and g.ref_strength > 0:
411             if g.element not in best or snr[g.label] > snr[best[g.element].label]:
412                 best[g.element] = g
413     groups = [best[el] for el in calibrant.elements if el in best]
414     area = np.array([fit.areas[g.label] for g in groups])
415     err = np.array([fit.area_errors[g.label] for g in groups])
416     used = area > 3 * err
417     if used.sum() < 2:
418         return {}
419 
420     rho = air_density(detector.air_temperature_c, detector.air_pressure_hpa)
421     mu = np.array([xraylib.CS_Total_CP(AIR_FORMULA, g.reference.energy_ev / 1000.0) * rho for g in groups])
422     m = np.array([calibrant.loadings_ng_mm2[g.element] * g.ref_strength for g in groups])
423     m *= np.exp(-mu * (air_path_cm - detector.air_path_cm))
424     w = 1.0 / err[used] ** 2
425     k = float(np.sum(w * area[used] * m[used]) / np.sum(w * m[used] ** 2))
426     pred = k * m
427     chi2 = float(np.sum(((area[used] - pred[used]) / err[used]) ** 2) / max(int(used.sum()) - 1, 1))
428     return {"air_path_cm": float(air_path_cm), "scale": k, "chi2_red": chi2, "rows": [
429         {"label": g.label, "energy_ev": g.reference.energy_ev,
430          "loading_ng_mm2": calibrant.loadings_ng_mm2[g.element],
431          "area": float(a), "area_err": float(e), "predicted": float(pr),
432          "ratio": float(a / pr), "ratio_err": float(e / pr), "used": bool(u)}
433         for g, a, e, pr, u in zip(groups, area, err, pred, used)]}
```

6.3.5 `calibrate`

```
calibrate(y, calibrant, detector, yerr=None, roi_ev=(2500.0, 19500.0), gain=10.0, offset=0.0,
extra_elements=(), families=("K","L","M"), incident_spectrum=None, e0_eff_kev=20.0,
background_order=5, escape=True, min_rel=0.01, coarse=True, check_lines=True,
fit_air_path=False, **coarse_kw)
```

The complete procedure.

`y` spectrum of the calibrant (raw counts or normalised; one measurement or a sum).

`calibrant`
e.g. `CALIBRANTS["AXO RF C10"]`.

`detector`
: starting resolution and efficiency model.

`yerr`
: errors (default Poisson, $\sqrt{\max(y, 1)}$: right for raw counts only).

`roi_ev`
: fitted energy window.

`gain, offset`
: starting guess.

`extra_elements`
: elements present besides the certified ones (Ar from the air, contaminants), with their own amplitudes.

`families, incident_spectrum, e0_eff_kev, min_rel`
: as in `element_line_groups`.

`background_order, escape`
: as in `fit_average_spectrum`.

`coarse`
: run the coarse match (else start from `gain`, `offset`).

`check_lines`
: run the line checks.

`fit_air_path`
: fit the sample–detector air path.

`coarse_kw`
: search ranges for `coarse_calibration`.

Returns a `CalibrationResult`. After the global fit the detector is updated with the fitted resolution and air path; with a fitted air path the groups are rebuilt at that path before the line checks, so that their relative intensities include it.

xrf_calibration.py, lines 436–523

```
436 def calibrate(
437     y: np.ndarray,
438     calibrant: Calibrant,
439     detector: Detector,
440     yerr: Optional[np.ndarray] = None,
441     roi_ev: Tuple[float, float] = (2500.0, 19500.0),
442     gain: float = 10.0,
443     offset: float = 0.0,
444     extra_elements: Sequence[str] = (),
445     families: Sequence[str] = ("K", "L", "M"),
446     incident_spectrum: Optional[Tuple[np.ndarray, np.ndarray]] = None,
447     e0_eff_kev: float = 20.0,
448     background_order: int = 5,
449     escape: bool = True,
450     min_rel: float = 0.01,
451     coarse: bool = True,
452     check_lines: bool = True,
453     fit_air_path: bool = False,
454     **coarse_kw,
455 ) -> CalibrationResult:
456     """Calibrate the channel -> energy relation on a reference spectrum.
457 
458     Parameters
459     ----------
460     y : (n_channels,) array
461         Spectrum of the calibrant (raw counts, or normalised).  A single
462         measurement or a sum / average of several.
463     calibrant : Calibrant
464         For instance ``CALIBRANTS["AXO RF C10"]``.
```

```
    detector : Detector
        Starting resolution and the detection-efficiency model.
    yerr : (n_channels,) array, optional
        1-sigma errors; default Poisson, ``sqrt(max(y, 1))`` (only right for
        raw counts).
    roi_ev : (min, max)
        Energy window fitted, in eV.
    gain, offset : float
        Starting guess, eV/channel and eV.
    extra_elements : sequence of str
        Elements present besides the certified ones: argon from the air path,
        contamination...  They get their own free amplitudes.
    families : shells considered for every element (only lines inside the ROI are kept)
    incident_spectrum, e0_eff_kev
        Excitation, as in :func:`xrf_fit.element_line_groups`.  It sets the
        relative line intensities within each group (and the intensity check).
    background_order : polynomial order of the background over the ROI
    escape : model the sensor escape peaks
    min_rel : drop lines weaker than this fraction of their group's reference
    coarse : run the coarse match first (else start from ``gain``, ``offset``)
    check_lines : measure the strongest peaks individually
    fit_air_path : fit the sample-detector air path (from the line ratios
        within each element), starting from ``detector.air_path_cm``
    **coarse_kw : passed to :func:`coarse_calibration` (search ranges)
    """
    y = np.asarray(y, dtype=float).reshape(-1)
    yerr = np.sqrt(np.maximum(y, 1.0)) if yerr is None else np.asarray(yerr, dtype=float).reshape(-1)
    elements = list(dict.fromkeys(list(calibrant.elements) + list(extra_elements)))
    groups = element_line_groups(elements, *roi_ev, e0_eff_kev=e0_eff_kev, families=families,
                                 min_rel=min_rel, incident_spectrum=incident_spectrum,
                                 detector=detector)
    if not groups:
        raise ValueError(f"No line of {elements} between {roi_ev[0]:.0f} and {roi_ev[1]:.0f} eV.")

    start = (coarse_calibration(y, groups, gain, offset, **coarse_kw) if coarse
             else Calibration(gain=gain, offset=offset))
    roi = start.channel_range(*roi_ev, y.size)
    fit = fit_average_spectrum(y, groups, start, detector, roi, yerr=yerr,
                               background_order=background_order, escape=escape,
                               fit_calibration=True, fit_air_path=fit_air_path)
    p = fit.params()
    i = fit.model.param_names.index("cal_gain")
    covariance = np.array(fit.pcov[i:i + 2, i:i + 2], dtype=float)
    air_path = fit.model.air_path_of(fit.popt)
    fitted_detector = dataclasses.replace(detector, noise_ev=p["noise_ev"][0], fano=p["fano"][0],
                                          gamma_ev=p["gamma_ev"][0], air_path_cm=air_path)

    result = CalibrationResult(calibration=fit.calibration, covariance=covariance, start=start,
                               fit=fit, detector=fitted_detector, calibrant=calibrant,
                               intensity=_intensity_model(fit, calibrant, detector, air_path),
                               air_path_err=p["air_path_cm"][1] if fit_air_path else float("nan"))
    if check_lines:
        # local refits use the fitted response, with the air path fixed: rebuild
        # the groups at that path so their relative intensities include it
        local_groups = (element_line_groups(elements, *roi_ev, e0_eff_kev=e0_eff_kev, families=families,
                                            min_rel=min_rel, incident_spectrum=incident_spectrum,
                                            detector=fitted_detector) if fit_air_path else groups)
        result.checks = _line_checks(y, yerr, fit, fitted_detector, escape, refit_groups=local_groups)
    return result
```

6.3.6 `plot_calibration`

```
plot_calibration(result, logy=True)
```

Two figures: the global fit (`plot_fit`) and the line checks—shift versus energy with error bars and the one-sigma band of the calibration, $\pm\sigma_E(c)$. Returns `(fig_fit, fig_checks)`, the second `None` without line checks.

xrf_calibration.py, lines 526–559

```
def plot_calibration(result: CalibrationResult, logy: bool = True):
```

```
    """Global fit (spectrum and residual) and the line checks.

    Returns
    -------
    (fig_fit, fig_checks) ; ``fig_checks`` is None without line checks.
    """
    import matplotlib.pyplot as plt

    fig_fit, (ax, _) = plot_fit(result.fit, energy_axis=True, logy=logy, data_label="data",
                                ylabel="Intensity (input units)")
    c = result.calibration
    ax.set_title(f"{result.calibrant.name}: E = {c.gain:.4f} x ch {c.offset:+.1f} eV, "
                 f"reduced $\\chi^2$ = {result.fit.chi2_red:.2f}")
    if not result.checks:
        return fig_fit, None

    fig, ax = plt.subplots(figsize=(8, 4))
    k = sorted(result.checks, key=lambda k: k.energy_ev)
    e = np.array([x.energy_ev for x in k]) / 1000
    ax.errorbar(e, [x.shift_ev for x in k], yerr=[x.shift_err_ev for x in k], fmt="o", capsize=3)
    for x, ei in zip(k, e):
        ax.annotate(x.label.split(" (")[0], (ei, x.shift_ev), textcoords="offset points",
                    xytext=(4, 6), fontsize=8)
    grid = np.linspace(min(e.min(), 1.0), max(e.max(), 20.0), 200)
    band = result.energy_error(c.channel(grid * 1000))
    ax.fill_between(grid, -band, band, color="C0", alpha=0.15, lw=0, label="calibration 1$\\sigma$")
    ax.axhline(0, color="0.4", lw=0.8)
    ax.set_xlabel("Line energy (keV)")
    ax.set_ylabel("Calibrated peak energy - tabulated (eV)")
    ax.set_title(f"Individual peak positions ($\\chi^2$ vs 0 = {result.check_chi2():.2f})")
    ax.legend(loc="best", fontsize=8)
    fig.tight_layout()
    return fig_fit, fig
```

7 `xrf_region`: surface region of a map

7.1 Overview

A thin film covers the whole map, and every pixel belongs to the same flat surface. A single crystal glued on the holder does not: the map shows a central flat facet, neighbouring facets at other tilts, the glue and the holder. A facet tilted by δ changes the incidence and take-off angles by up to δ, and with them the absorption paths of every line (section 4); its intensities and line ratios therefore differ from those of the central facet even at the same composition. Only the facet whose orientation is known (on BM32, the plane fitted to align the `sx`/`sy` frame) belongs in the average spectrum and the quantification.

The module describes that facet as a `SurfaceRegion`: polygons that are included, minus polygons that are excluded (an inclusion, a dust grain, a crack). The region gives a boolean mask over the pixels, which the notebook applies to the average spectrum, the pile-up template, the per-segment fits (`fit_segments`) and the map solve (`fit_map`, argument `mask`). The vertices can be typed in, or drawn on the map with `RegionSelector`, and saved to a small JSON file.

Hypotheses and limits

- A pixel belongs to the region when its *centre* (the recorded motor positions of the point) is inside; the beam footprint is not considered, so pixels along an edge mix two facets whichever side they fall on. Drawing the polygon slightly inside the facet avoids them.
- Motor units are the recorded positions of the fast and slow motors of the mesh, read point by point: they do not depend on the grid and remain valid for another map of the same facet in the same frame. They are only meaningful in the frame they were drawn in, so the motor names are stored and checked.
- Points exactly on an edge may fall on either side (`Path.contains_points`). In pixel units with integer vertices, the pixel centres lie on the edges: shift the vertices by half a pixel to make the choice explicit.

xrf_region.py, lines 1–43

```
"""
Surface region of a map: which pixels belong to the flat surface being analysed.

A thin film covers the whole map, so every pixel is used (the default).  A
single crystal glued on a holder does not: the map shows a central flat facet,
neighbouring facets at other tilts (different incidence and take-off angles,
hence different intensities), the glue and the holder.  Only the facet whose
orientation is known (e.g. from the plane fitted to align ``sx``/``sy``) should
enter the average spectrum and the quantification.

A :class:`SurfaceRegion` is one or more polygons that are *included*, minus
optional polygons that are *excluded* (an inclusion, a dust grain, a crack
inside the facet).  A pixel is in the region when its **centre** is inside an
included polygon and inside no excluded one.

Vertices are given either

* in **motor** units: positions of the fast and slow motors of the mesh
  (``sx``, ``sy`` on BM32, in mm), read from the recorded positions of every
  point -- robust against a change of map size or step, and the units in
  which the region is reused on another map of the same facet; or
* in **pixel** units: (column, row) = (fast index, slow index) of the grid.

The region can be typed in (vertices known), or drawn on a map with
:class:`RegionSelector` in a Jupyter notebook (``%matplotlib widget``, ipympl),
and saved to / loaded from a small JSON file.
"""

from __future__ import annotations

import json
import logging
from dataclasses import dataclass, field
from pathlib import Path
from typing import List, Optional, Sequence, Tuple

import numpy as np
from matplotlib.path import Path as MplPath

logger = logging.getLogger(__name__)

Polygon = np.ndarray          # (n_vertices, 2) array of (x, y) = (fast, slow)
_UNITS = ("motor", "pixel")
```

7.2 Polygons and coordinates

7.2.1 `polygon_area`, `points_in_polygon`

`polygon_area(vertices)`, `points_in_polygon(x, y, vertices)`

`_as_polygon` checks and normalises a list of vertices to an $(n, 2)$ array (a repeated closing vertex is dropped, at least three distinct vertices are required). `polygon_area` is the shoelace formula,

$A = \frac{1}{2}\,|\sum_i (x_i y_{i+1} - y_i x_{i+1})|$. `points_in_polygon` returns, for each point, whether it is inside the polygon, from `matplotlib.path.Path.contains_points` (even–odd rule).

xrf_region.py, lines 46–55

```
46 def _as_polygon(vertices) -> Polygon:
47     """Vertices as an (n, 2) float array; at least three distinct vertices."""
48     p = np.asarray(vertices, dtype=float)
49     if p.ndim != 2 or p.shape[1] != 2:
50         raise ValueError(f"A polygon is a list of (x, y) vertices; got shape {p.shape}.")
51     if len(p) > 1 and np.allclose(p[0], p[-1]):
52         p = p[:-1]                                  # closing vertex repeated
53     if len(np.unique(p, axis=0)) < 3:
54         raise ValueError("A polygon needs at least three distinct vertices.")
55     return p
```

xrf_region.py, lines 58–62

```
58 def polygon_area(vertices) -> float:
59     """Area of a simple polygon (shoelace formula), in the units of the vertices squared."""
60     p = _as_polygon(vertices)
61     x, y = p[:, 0], p[:, 1]
62     return 0.5 * abs(float(np.dot(x, np.roll(y, -1)) - np.dot(y, np.roll(x, -1))))
```

xrf_region.py, lines 65–75

```
65 def points_in_polygon(x: np.ndarray, y: np.ndarray, vertices) -> np.ndarray:
66     """Boolean array, True where the point (x, y) lies inside the polygon.
67
68     Uses :meth:`matplotlib.path.Path.contains_points` (even-odd rule, the
69     polygon is closed automatically).  Points exactly on an edge may fall on
70     either side, which is irrelevant for pixel centres.
71     """
72     x, y = np.asarray(x, dtype=float), np.asarray(y, dtype=float)
73     pts = np.column_stack([x.ravel(), y.ravel()])
74     inside = MplPath(_as_polygon(vertices)).contains_points(pts)
75     return inside.reshape(x.shape)
```

7.2.2 `scan_coordinates`

```
scan_coordinates(scan, units="motor")
```

The (x, y) = (fast, slow) coordinates of every pixel of a mesh scan, as flat arrays in acquisition order. `"motor"`: the recorded positions of the two motors (they must have been read, which `BlissMeshScan.read_data` does by default); `"pixel"`: the (column, row) indices on the `scan_shape` grid.

xrf_region.py, lines 78–102

```
78 def scan_coordinates(scan, units: str = "motor") -> Tuple[np.ndarray, np.ndarray]:
79     """Per-pixel (x, y) = (fast, slow) coordinates of a mesh scan, flat arrays.
80
81     Parameters
82     ----------
83     scan : BlissMeshScan (or any object with ``scan_shape``, ``fast_motor_name``,
84         ``slow_motor_name`` and ``counters`` holding the motor positions)
85     units : {"motor", "pixel"}
86         ``"motor"``: recorded positions of the fast and slow motors (both must
87         have been read, which ``BlissMeshScan.read_data`` does by default).
88         ``"pixel"``: (column, row) indices on the ``scan_shape`` grid.
89     """
90     if units not in _UNITS:
91         raise ValueError(f"units must be one of {_UNITS}, got {units!r}")
92     n_slow, n_fast = scan.scan_shape
93     if units == "pixel":
94         row, col = np.indices((n_slow, n_fast))
95         return col.ravel().astype(float), row.ravel().astype(float)
96     try:
97         x = np.asarray(scan.counters[scan.fast_motor_name], dtype=float).ravel()
98         y = np.asarray(scan.counters[scan.slow_motor_name], dtype=float).ravel()
```

```
    except KeyError as exc:
        raise KeyError(f"motor position {exc} not loaded: read it with scan.read_data(), "
                       "or use units='pixel'.") from None
    return x, y
```

7.3 The region

7.3.1 SurfaceRegion

```
SurfaceRegion(include=[], exclude=[], units="motor", motors=None, name="", note="")
```

`include`
: polygons whose pixels are kept; empty means the whole map.

`exclude`
: polygons whose pixels are removed from the included ones.

`units`
: `"motor"` or `"pixel"` (see above).

`motors`
: the (fast, slow) motor names, checked against the scan by `scan_mask`.

`name, note`
: free text, kept in the JSON file.

xrf_region.py, lines 105–144

```
@dataclass
class SurfaceRegion:
    """Polygons defining the analysed surface of a map.

    Parameters
    ----------
    include : list of polygons
        Each polygon is a sequence of (x, y) vertices, x along the fast axis
        and y along the slow axis.  Pixels inside any of them are kept.  An
        empty list means the whole map.
    exclude : list of polygons
        Pixels inside any of these are removed (inclusions, dust, cracks).
    units : {"motor", "pixel"}
        Units of the vertices (see the module docstring).
    motors : (fast, slow) motor names, optional
        Recorded for motor units, and checked against the scan when the mask
        is built (a region drawn in ``sx``/``sy`` makes no sense on a map
        scanned with ``xech``/``yech``).
    name, note : str
        Free text, kept in the JSON file.

    Examples
    --------
    Whole map (thin film)::

        SurfaceRegion()

    Central facet of a crystal, vertices in mm (sx, sy), minus an inclusion::

        SurfaceRegion(include=[[(-3.84, -6.17), (-3.75, -6.17), (-3.74, -6.07), (-3.82, -6.06)]],
                      exclude=[[(-3.80, -6.10), (-3.79, -6.10), (-3.79, -6.09)]],
                      motors=("sx", "sy"), name="NIO295 top facet")
    """

    include: List[Polygon] = field(default_factory=list)
    exclude: List[Polygon] = field(default_factory=list)
    units: str = "motor"
    motors: Optional[Tuple[str, str]] = None
    name: str = ""
    note: str = ""
```

xrf_region.py, lines 146–152

```
146     def __post_init__(self):
147         if self.units not in _UNITS:
148             raise ValueError(f"units must be one of {_UNITS}, got {self.units!r}")
149         self.include = [_as_polygon(p) for p in self.include]
150         self.exclude = [_as_polygon(p) for p in self.exclude]
151         if self.motors is not None:
152             self.motors = tuple(self.motors)
```

7.3.2 constructors

```
SurfaceRegion.from_vertices(vertices, units="motor", exclude=(), **kw),
SurfaceRegion.coerce(region)
```

`from_vertices` makes a region of one polygon, the usual case of one facet. `coerce` accepts what a notebook configuration naturally contains: `None` (the whole map), a `SurfaceRegion`, the path of a JSON file, a dictionary of keyword arguments, or a list of vertices in motor units.

xrf_region.py, lines 155–158

```
155     @classmethod
156     def from_vertices(cls, vertices, units: str = "motor", exclude=(), **kw) -> "SurfaceRegion":
157         """Region made of a single polygon (the usual case of one facet)."""
158         return cls(include=[vertices], exclude=list(exclude), units=units, **kw)
```

xrf_region.py, lines 160–176

```
160     @classmethod
161     def coerce(cls, region) -> "SurfaceRegion":
162         """Accept what a notebook configuration naturally contains.
163 
164         ``None`` -> whole map; a ``SurfaceRegion`` -> itself; a ``str`` or
165         ``Path`` -> :meth:`load`; a list of (x, y) vertices -> one polygon in
166         motor units; a ``dict`` -> keyword arguments of the class.
167         """
168         if region is None:
169             return cls()
170         if isinstance(region, cls):
171             return region
172         if isinstance(region, (str, Path)):
173             return cls.load(region)
174         if isinstance(region, dict):
175             return cls(**region)
176         return cls.from_vertices(region)
```

7.3.3 `mask`, `scan_mask`

```
is_full, mask(x, y), scan_mask(scan), area()
```

`mask` is true for the points inside any included polygon and inside no excluded one; `scan_mask` applies it to the pixels of a scan (a flat $(n_{\text{pixels}},)$ array, all true for the whole map) and warns when the motors differ from those the region was drawn with, or when no pixel is selected. `area` is the included minus the excluded polygon areas (overlaps are not resolved), in mm^2 or pixels.

xrf_region.py, lines 179–182

```
179     @property
180     def is_full(self) -> bool:
181         """True when no polygon is defined: the whole map is used."""
182         return not self.include and not self.exclude
```

xrf_region.py, lines 184–194

```
184     def mask(self, x: np.ndarray, y: np.ndarray) -> np.ndarray:
185         """Boolean array, True for points inside the region (same shape as ``x``)."""
186         x = np.asarray(x, dtype=float)
187         keep = np.ones(x.shape, bool)
```

```
188         if self.include:
189             keep = np.zeros(x.shape, bool)
190             for p in self.include:
191                 keep |= points_in_polygon(x, y, p)
192         for p in self.exclude:
193             keep &= ~points_in_polygon(x, y, p)
194         return keep
```

xrf_region.py, lines 196–212

```
196     def scan_mask(self, scan) -> np.ndarray:
197         """Per-pixel mask of a mesh scan, flat ``(n_pixels,)`` boolean array.
198 
199         Logs a warning when the region selects no pixel, or when it was defined
200         with other motors than those of the scan.
201         """
202         if self.is_full:
203             return np.ones(int(np.prod(scan.scan_shape)), bool)
204         if self.units == "motor" and self.motors is not None:
205             scan_motors = (scan.fast_motor_name, scan.slow_motor_name)
206             if tuple(self.motors) != scan_motors:
207                 logger.warning("region defined in motors %s, scan uses %s",
208                                self.motors, scan_motors)
209         m = self.mask(*scan_coordinates(scan, self.units))
210         if not m.any():
211             logger.warning("the region %r contains no pixel of this scan", self.name)
212         return m
```

xrf_region.py, lines 214–217

```
214     def area(self) -> float:
215         """Included minus excluded polygon areas (overlaps not resolved), units^2."""
216         return (sum(polygon_area(p) for p in self.include)
217                 - sum(polygon_area(p) for p in self.exclude))
```

xrf_region.py, lines 219–227

```
219     def __str__(self) -> str:
220         if self.is_full:
221             return "SurfaceRegion: whole map"
222         u = "mm" if self.units == "motor" else "px"
223         s = (f"SurfaceRegion {self.name!r}: {len(self.include)} included, "
224              f"{len(self.exclude)} excluded polygon(s), {self.units} units")
225         if self.include:
226             s += f", area {self.area():.4g} {u}^2"
227         return s
```

7.3.4 `plot`

```
plot(ax, origin=(0, 0), scale=1.0, include_kw=None, exclude_kw=None)
```

Draws the outlines (included: solid white, excluded: dashed red) at $(\mathbf{v} - \mathbf{o})\,s$, so that they can be laid over a map drawn in other units, e.g. µm from the corner of the map with `origin` the minimum motor positions and $s = 1000$.

xrf_region.py, lines 230–248

```
230     def plot(self, ax, origin: Tuple[float, float] = (0.0, 0.0), scale: float = 1.0,
231              include_kw: Optional[dict] = None, exclude_kw: Optional[dict] = None):
232         """Draw the polygon outlines on ``ax``.
233 
234         Plotted coordinates are ``(vertex - origin) * scale``, so the outline
235         can be laid over a map drawn in other units, e.g. µm from the map
236         corner: ``origin=(x.min(), y.min()), scale=1000``.
237         """
238         from matplotlib.patches import Polygon as MplPolygon
239         ikw = dict(fill=False, edgecolor="w", lw=1.4, ls="-")
240         ikw.update(include_kw or {})
241         ekw = dict(fill=False, edgecolor="r", lw=1.2, ls="--")
242         ekw.update(exclude_kw or {})
```

```
243         o = np.asarray(origin, dtype=float)
244         patches = []
245         for polys, kw in ((self.include, ikw), (self.exclude, ekw)):
246             for p in polys:
247                 patches.append(ax.add_patch(MplPolygon((p - o) * scale, closed=True, **kw)))
248         return patches
```

7.3.5 input/output

`to_dict()`, `save(path, **metadata)`, `SurfaceRegion.load(path)`, `to_code(precision=5)`

The region is saved as JSON (units, motors, vertices; extra metadata such as the scan file are stored alongside and ignored on reading). `to_code` prints a Python expression recreating it, to paste into a notebook configuration.

xrf_region.py, lines 251–255

```
251     def to_dict(self) -> dict:
252         return {"name": self.name, "note": self.note, "units": self.units,
253                 "motors": list(self.motors) if self.motors else None,
254                 "include": [p.tolist() for p in self.include],
255                 "exclude": [p.tolist() for p in self.exclude]}
```

xrf_region.py, lines 257–264

```
257     def save(self, path, **metadata) -> Path:
258         """Write the region to JSON (extra ``metadata`` stored alongside, e.g. scan file)."""
259         path = Path(path)
260         d = self.to_dict()
261         if metadata:
262             d["metadata"] = {k: str(v) for k, v in metadata.items()}
263         path.write_text(json.dumps(d, indent=2))
264         return path
```

xrf_region.py, lines 266–270

```
266     @classmethod
267     def load(cls, path) -> "SurfaceRegion":
268         d = json.loads(Path(path).read_text())
269         d.pop("metadata", None)
270         return cls(**d)
```

xrf_region.py, lines 272–286

```
272     def to_code(self, precision: int = 5) -> str:
273         """Python expression recreating the region, to paste into a notebook."""
274         def fmt(polys):
275             return "[" + ",\n             ".join(
276                 "[" + ", ".join(f"({x:.{precision}f}, {y:.{precision}f})" for x, y in p) + "]"
277                 for p in polys) + "]"
278         args = [f"include={fmt(self.include)}"]
279         if self.exclude:
280             args.append(f"exclude={fmt(self.exclude)}")
281         args.append(f"units={self.units!r}")
282         if self.motors:
283             args.append(f"motors={tuple(self.motors)!r}")
284         if self.name:
285             args.append(f"name={self.name!r}")
286         return "SurfaceRegion(" + ",\n              ".join(args) + ")"
```

7.4 Drawing a region

7.4.1 `map_image`

`map_image(corrected, calibration=None, window_ev=None)`

The per-pixel image to draw on: the sum of the normalised spectra, over an energy window if given. A strong line of the crystal shows the facets best, since their intensity depends on the local angles.

xrf_region.py, lines 289–304

```
289 def map_image(corrected: np.ndarray, calibration=None,
290                window_ev: Optional[Tuple[float, float]] = None) -> np.ndarray:
291     """Per-pixel intensity to draw a region on: the sum of the normalised spectra.
292 
293     ``window_ev=(lo, hi)`` restricts the sum to an energy window (e.g. around
294     a strong line, which shows the facets best); it needs ``calibration``.
295     Non-finite pixels give NaN.
296     """
297     c = np.asarray(corrected, dtype=float)
298     if window_ev is not None:
299         if calibration is None:
300             raise ValueError("window_ev needs a calibration.")
301         lo, hi = (int(round(float(calibration.channel(e)))) for e in window_ev)
302         c = c[:, max(lo, 0):max(hi, lo + 1)]
303     with np.errstate(invalid="ignore"):
304         return c.sum(axis=1)
```

7.4.2 `RegionSelector`

```
RegionSelector(scan, image, units="motor", region=None, name="", log=False, cmap="viridis",
figsize=(7.0, 6.2))
```

An interactive map built on Matplotlib's `PolygonSelector`. It needs an interactive backend: `%matplotlib widget` (ipympl) in Jupyter, where buttons are added with ipywidgets, or a Qt/Tk window in a script. The user clicks the vertices of the facet and closes the polygon on the first vertex; vertices can then be dragged, or the whole polygon with `shift`. **Add facet** (key `a`) adds it to the region, **Add exclusion** (key `x`) to the exclusions, **Undo** (`u`) removes the last one, **Clear all** returns to the whole map. Pixels outside the current region are veiled in grey and the number of selected pixels is shown, so what will be averaged is visible while drawing. `region` and `mask` return the result so far. With a non-interactive backend (inline) the map is still shown and a warning explains how to switch.

xrf_region.py, lines 307–342

```
307 class RegionSelector:
308     """Draw a :class:`SurfaceRegion` on a map, in a Jupyter notebook.
309 
310     Needs an interactive Matplotlib backend: ``%matplotlib widget`` (ipympl)
311     in Jupyter, or Qt/Tk in a script.  Usage::
312 
313         sel = RegionSelector(scan, image)      # shows the map
314         # ... click the vertices of the facet, close the polygon on the first
315         # vertex, adjust by dragging, then press "Add facet" (or key 'a').
316         # Inclusions to remove: draw them and press "Add exclusion" (key 'x').
317         region = sel.region                    # in the next cell
318         region.save("processed/<scan>_region.json")
319 
320     Controls (buttons, or keys with the mouse over the map):
321 
322     * ``a``  add the polygon being drawn to the region (included)
323     * ``x``  add it as an exclusion
324     * ``u``  undo the last added polygon
325     * ``Esc`` restart the polygon being drawn (Matplotlib's own binding)
326     * drag a vertex to move it; ``shift``-drag moves the whole polygon
327 
328     Pixels outside the current region are greyed out, so what will be
329     averaged is visible while drawing.
330 
331     Parameters
332     ----------
333     scan : BlissMeshScan
334     image : (n_pixels,) or (n_slow, n_fast) array
```

```
        What to show, e.g. :func:`map_image`.
    units : {"motor", "pixel"}
        Units in which the vertices are recorded.
    region : SurfaceRegion, optional
        Starting region (e.g. a previous one, to refine).
    log : bool
        Logarithmic colour scale.
    """
```

xrf_region.py, lines 344–395

```
    def __init__(self, scan, image: np.ndarray, units: str = "motor",
                 region: Optional[SurfaceRegion] = None, name: str = "",
                 log: bool = False, cmap: str = "viridis", figsize=(7.0, 6.2)):
        import matplotlib.pyplot as plt
        from matplotlib.colors import LogNorm
        from matplotlib.widgets import PolygonSelector

        self.scan = scan
        self.units = units
        self.shape = tuple(scan.scan_shape)
        self.x, self.y = scan_coordinates(scan, units)
        motors = (scan.fast_motor_name, scan.slow_motor_name) if units == "motor" else None
        region = SurfaceRegion.coerce(region)
        self._include = list(region.include)
        self._exclude = list(region.exclude)
        self._history: List[str] = []
        self._meta = dict(units=units, motors=motors, name=name or region.name,
                          note=region.note)

        backend = plt.get_backend().lower()
        if "inline" in backend or backend == "agg":
            logger.warning("Matplotlib backend %r is not interactive: run `%%matplotlib widget` "
                           "(ipympl) before creating the selector, or give the vertices.",
                           backend)

        img = np.asarray(image, dtype=float).reshape(self.shape)
        X, Y = self.x.reshape(self.shape), self.y.reshape(self.shape)
        with plt.ioff():
            self.fig, self.ax = plt.subplots(figsize=figsize)
        pos = img[np.isfinite(img) & (img > 0)]
        norm = LogNorm(*np.percentile(pos, [1, 99.5])) if log and pos.size else None
        kw = {} if norm else dict(vmin=np.nanpercentile(img, 1), vmax=np.nanpercentile(img, 99.5))
        self.ax.pcolormesh(X, Y, img, shading="nearest", cmap=cmap, norm=norm, **kw)
        self._shade = self.ax.pcolormesh(X, Y, np.zeros(self.shape), shading="nearest",
                                         cmap="Greys", vmin=0, vmax=1.5, alpha=0.0)
        if units == "motor":
            self.ax.set_aspect("equal")
            self.ax.set_xlabel(f"{scan.fast_motor_name} (mm)")
            self.ax.set_ylabel(f"{scan.slow_motor_name} (mm)")
        else:
            self.ax.set_xlabel("fast index (column)")
            self.ax.set_ylabel("slow index (row)")
        self._patches: list = []
        self._title()

        self.selector = PolygonSelector(self.ax, self._on_polygon, useblit=True,
                                        props=dict(color="w", lw=1.5),
                                        handle_props=dict(markersize=5, mfc="w", mec="k"))
        self._current: Optional[np.ndarray] = None
        self.fig.canvas.mpl_connect("key_press_event", self._on_key)
        self._redraw()
        self._show()
```

xrf_region.py, lines 398–420

```
    def _show(self):
        import matplotlib.pyplot as plt
        try:
            import ipywidgets as w
            from IPython.display import display
        except ImportError:                              # plain script: Qt/Tk window
            plt.show(block=False)
            return
```

```
406         if not isinstance(self.fig.canvas, w.DOMWidget):  # inline or other backend
407             display(self.fig)
408             return
409         b_add = w.Button(description="Add facet", button_style="success", icon="plus")
410         b_exc = w.Button(description="Add exclusion", button_style="warning", icon="minus")
411         b_undo = w.Button(description="Undo", icon="undo")
412         b_clear = w.Button(description="Clear all", button_style="danger", icon="trash")
413         b_add.on_click(lambda _: self.add(exclude=False))
414         b_exc.on_click(lambda _: self.add(exclude=True))
415         b_undo.on_click(lambda _: self.undo())
416         b_clear.on_click(lambda _: self.clear())
417         self._status = w.HTML()
418         self._update_status()
419         self.fig.canvas.header_visible = False
420         display(w.VBox([w.HBox([b_add, b_exc, b_undo, b_clear]), self.fig.canvas, self._status]))
```

xrf_region.py, lines 422–424

```
422     def _title(self):
423         self.ax.set_title("click vertices, close on the first one, then 'Add facet' (a) "
424                           "or 'Add exclusion' (x)", fontsize=9, loc="left")
```

xrf_region.py, lines 426–431

```
426     def _update_status(self):
427         if hasattr(self, "_status"):
428             m = self.mask
429             self._status.value = (f"<tt>{len(self._include)} facet polygon(s), "
430                                   f"{len(self._exclude)} exclusion(s): "
431                                   f"{int(m.sum())} / {m.size} pixels selected</tt>")
```

xrf_region.py, lines 433–442

```
433     def _redraw(self):
434         for p in self._patches:
435             p.remove()
436         self._patches = self.region.plot(self.ax) if not self.region.is_full else []
437         m = self.mask.reshape(self.shape)
438         # grey veil on the excluded pixels only (selected ones are masked out)
439         self._shade.set_array(np.ma.masked_array(np.ones(m.size), mask=m.ravel()))
440         self._shade.set_alpha(0.0 if self.region.is_full else 0.6)
441         self._update_status()
442         self.fig.canvas.draw_idle()
```

xrf_region.py, lines 445–446

```
445     def _on_polygon(self, verts):
446         self._current = np.asarray(verts, dtype=float)
```

xrf_region.py, lines 448–454

```
448     def _on_key(self, event):
449         if event.key == "a":
450             self.add(exclude=False)
451         elif event.key == "x":
452             self.add(exclude=True)
453         elif event.key == "u":
454             self.undo()
```

xrf_region.py, lines 457–467

```
457     def add(self, exclude: bool = False):
458         """Add the polygon being drawn to the region (or to the exclusions)."""
459         if self._current is None or len(self._current) < 3:
460             logger.warning("no closed polygon to add: click the vertices and close the "
461                            "polygon on its first vertex")
462             return
463         (self._exclude if exclude else self._include).append(_as_polygon(self._current))
464         self._history.append("exclude" if exclude else "include")
465         self._current = None
466         self.selector.clear()
467         self._redraw()
```

xrf_region.py, lines 469–473

```
469     def undo(self):
470         """Remove the last added polygon."""
471         if self._history:
472             (self._exclude if self._history.pop() == "exclude" else self._include).pop()
473             self._redraw()
```

xrf_region.py, lines 475–480

```
475     def clear(self):
476         """Remove every polygon: back to the whole map."""
477         self._include, self._exclude, self._history = [], [], []
478         self._current = None
479         self.selector.clear()
480         self._redraw()
```

xrf_region.py, lines 483–487

```
483     @property
484     def region(self) -> SurfaceRegion:
485         """The region drawn so far (the whole map if nothing was added)."""
486         return SurfaceRegion(include=list(self._include), exclude=list(self._exclude),
487                              **self._meta)
```

xrf_region.py, lines 489–492

```
489     @property
490     def mask(self) -> np.ndarray:
491         """Flat per-pixel mask of the region drawn so far."""
492         return self.region.mask(self.x, self.y)
```

8 Diffraction-image scripts

Two command-line scripts process the detector frames of a BLISS mesh scan, typically the Laue patterns of a microdiffraction map [43]. Both read the scan through `bliss_hdf5_reader.BlissMeshScan` and parallelise the reading over processes.

8.1 `reduce_image_series`: minimum, maximum and mean images

`python reduce_image_series.py master.h5 -s 1 -o OUTPUT_DIR [--image_suffix .h5] [-c 50] [-w N]` computes, pixel by pixel, the minimum, maximum and mean over every frame of a scan, and saves them in `<master>_scanNNNN_reduced.h5`. The maximum image shows every Laue spot that appears anywhere in the map; the minimum image is the background common to all frames (useful to subtract, and to find hot or dead pixels); the mean is the average pattern.

The reduction is a map–reduce: the file list is split into chunks of `chunk_size` files; each worker returns the minimum, maximum and sum of its chunk and the number of valid frames; the partial results are combined as they arrive (minimum and maximum are associative, sums add), and the mean is the total sum over the number of frames. Sums are accumulated in float64 to avoid integer overflow over thousands of frames.

Known issue

The script iterates over image *files* (`scan.image_path_list`) and `_read_frame_array` returns the *first* frame of each file. This is correct when the detector saves one frame per file; when files hold several frames, only one frame per file is reduced, and the `processed_frames` attribute (the number of files) overstates the frames used. The frame-level access of the reader (`iter_frames`, as in `make_dynamic_mosaic`) would remove this restriction. The scan must also be two-dimensional (`BlissMeshScan`).

reduce_image_series.py, lines 1–28

```
"""
Authors: Gemini and Samuel Tardif
Module: reduce_image_series.py

Description:
    Performs memory-efficient, highly parallelized pixel-wise min, max,
    and mean reductions across independent HDF5 image files from a BLISS scan.
    Integrates with bliss_hdf5_reader_2 to automatically discover frames
    associated with a specific scan number.
    Usage:
    python reduce_image_series.py /data/visitor/ihma907/bm32/20260909/RAW_DATA/epiGan_hex_v2/epiGan_hex_v2_hex8/
      ↪ epiGan_hex_v2_hex8.h5 -s 1 -o /data/bm32/inhouse/STAFF/SAM/ihma907/bm32/20260909/PROCESSED_DATA/
"""

import argparse
import logging
import multiprocessing as mp
from pathlib import Path
from typing import List, Optional, Tuple

import h5py
import numpy as np
from tqdm import tqdm

# Import the BLISS scan reader
from bliss_hdf5_reader import BlissMeshScan

# Configure logging
logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
```

8.1.1 parse_args

```
parse_args()
```

Command-line arguments: the master file, `-s/--scan` (scan number), `-o/--output_dir`, `--image_suffix` (default `.h5`), `-c/--chunk_size` (files per task, default 50), `-w/--workers` (default: all cores).

reduce_image_series.py, lines 31–71

```
def parse_args() -> argparse.Namespace:
    """Parses command-line arguments."""
    parser = argparse.ArgumentParser(
        description="Compute pixel-wise min, max, and mean reductions for a BLISS scan series."
    )
    parser.add_argument(
        "h5_file",
        type=Path,
        help="Path to the master/dataset BLISS HDF5 file (e.g., epiGan_hex_v2_hex8.h5)."
    )
    parser.add_argument(
        "-s", "--scan",
        type=int,
        required=True,
        help="Scan number to process (e.g., 1 for '1.1')."
    )
    parser.add_argument(
        "-o", "--output_dir",
        type=Path,
        required=True,
        help="Output directory to save the reduced image HDF5 file."
    )
    parser.add_argument(
        "--image_suffix",
        type=str,
        default=".h5",
        help="File extension of the detector frames (default: .h5)."
    )
    parser.add_argument(
        "-c", "--chunk_size",
        type=int,
```

```
        default=50,
        help="Number of image files per worker chunk (default: 50)."
    )
    parser.add_argument(
        "-w", "--workers",
        type=int,
        default=None,
        help="Number of CPU cores to utilize (default: all available cores)."
    )
    return parser.parse_args()
```

8.1.2 _read_frame_array

```
_read_frame_array(filepath)
```

Reads the image of a detector file: first the usual Lima/BLISS dataset paths (`/entry_0000/measurement/data`, `.../instrument/eiger4m/data`, `.../CRGIF/eiger4m/data`), else the first dataset of two or more dimensions found under `/entry_0000`; for a 3D dataset the first frame is returned. Raises `KeyError` if no image is found.

reduce_image_series.py, lines 74–106

```
def _read_frame_array(filepath: Path) -> np.ndarray:
    """
    Safely extracts the 2D image array from an independent HDF5 detector file.
    Uses fast direct lookup on common paths before falling back to recursive search.
    """
    with h5py.File(filepath, "r") as h5f:
        common_paths = [
            "/entry_0000/measurement/data",
            "/entry_0000/instrument/eiger4m/data",
            "/entry_0000/CRGIF/eiger4m/data",
        ]
        for path in common_paths:
            if path in h5f:
                ds = h5f[path]
                return ds[0] if ds.ndim == 3 else ds[()]

        # Fallback: traverse tree to find the first 2D/3D dataset
        found_path: List[str] = []

        def _visitor(name: str, obj: h5py.HLObject) -> Optional[bool]:
            if isinstance(obj, h5py.Dataset) and obj.ndim >= 2:
                found_path.append(obj.name)
                return True
            return None

        entry = h5f.get("/entry_0000")
        if entry:
            entry.visititems(_visitor)
            if found_path:
                ds = h5f[found_path[0]]
                return ds[0] if ds.ndim == 3 else ds[()]

    raise KeyError(f"Could not locate a valid image dataset inside {filepath.name}")
```

8.1.3 _worker_process_files

```
_worker_process_files(file_paths)
```

The map step: running minimum, maximum and float64 sum over a list of files, skipping unreadable files with an error message. Returns `(min, max, sum, n_valid)`, with `None` arrays if no file could be read.

reduce_image_series.py, lines 109–145

```
def _worker_process_files(file_paths: List[Path]) -> Tuple[Optional[np.ndarray], Optional[np.
    ↪ ndarray], int]:
```

```
    """
    Map Worker: Reads a chunk of HDF5 files and computes local min, max, and sum.

    Args:
        file_paths: List of file paths to process within this worker.

    Returns:
        Tuple of (local_min, local_max, local_sum, valid_frames_count).
    """
    local_min = None
    local_max = None
    local_sum = None
    valid_frames = 0

    for file_path in file_paths:
        try:
            data = _read_frame_array(file_path)
            data = np.squeeze(data)

            if local_min is None:
                local_min = data.copy()
                local_max = data.copy()
                # Cast to float64 to prevent integer overflow during multi-thousand image sums
                local_sum = data.astype(np.float64)
            else:
                local_min = np.minimum(local_min, data)
                local_max = np.maximum(local_max, data)
                local_sum += data

            valid_frames += 1

        except Exception as err:
            logging.error(f"Error processing {file_path.name}: {err}. Skipping.")
            continue

    return local_min, local_max, local_sum, valid_frames
```

8.1.4 `compute_scan_reductions`

```
compute_scan_reductions(image_files, chunk_size=50, num_workers=None)
```

The reduce step, with a `multiprocessing.Pool` and a progress bar. Returns `(min_image, max_image, mean_image)`; raises if the list is empty or no frame could be read.

reduce_image_series.py, lines 148–200

```
def compute_scan_reductions(
    image_files: List[Path],
    chunk_size: int = 50,
    num_workers: Optional[int] = None
) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    """
    Orchestrates the parallel MapReduce pipeline across all resolved image files.
    """
    total_files = len(image_files)
    if total_files == 0:
        raise ValueError("Image file list is empty.")

    # Partition file list into chunks
    tasks = [
        image_files[i : i + chunk_size]
        for i in range(0, total_files, chunk_size)
    ]

    workers = num_workers or mp.cpu_count()
    logging.info(f"Distributing {total_files} files across {workers} workers (chunk size: {chunk_size})...")

    global_min = None
    global_max = None
    global_sum = None
    total_processed = 0

```

```
    with mp.Pool(processes=workers) as pool:
        for loc_min, loc_max, loc_sum, count in tqdm(
            pool.imap_unordered(_worker_process_files, tasks),
            total=len(tasks),
            desc="Processing chunks"
        ):
            if count == 0 or loc_min is None:
                continue

            if global_min is None:
                global_min = loc_min
                global_max = loc_max
                global_sum = loc_sum
            else:
                global_min = np.minimum(global_min, loc_min)
                global_max = np.maximum(global_max, loc_max)
                global_sum += loc_sum

            total_processed += count

    if total_processed == 0 or global_sum is None:
        raise RuntimeError("No valid frames could be processed. Please check file integrity.")

    logging.info(f"Reduction complete. Successfully processed {total_processed}/{total_files} frames.")
    global_mean = global_sum / total_processed

    return global_min, global_max, global_mean
```

8.1.5 `save_reduced_hdf5`, `main`

`save_reduced_hdf5(output_path, global_min, global_max, global_mean, metadata)`, `main()`

Writes the three images (gzip level 4) and the metadata (master file, scan number and title, number of files) as root attributes. `main` wires the steps together.

reduce_image_series.py, lines 203–224

```
def save_reduced_hdf5(
    output_path: Path,
    global_min: np.ndarray,
    global_max: np.ndarray,
    global_mean: np.ndarray,
    metadata: dict
) -> None:
    """Saves the reduced 2D arrays to an HDF5 file with GZIP compression."""
    output_path.parent.mkdir(parents=True, exist_ok=True)
    logging.info(f"Saving reduced datasets to {output_path}...")

    with h5py.File(output_path, "w") as h5f:
        # Save datasets with GZIP compression level 4 (optimal speed vs compression ratio)
        h5f.create_dataset("min_image", data=global_min, compression="gzip", compression_opts=4)
        h5f.create_dataset("max_image", data=global_max, compression="gzip", compression_opts=4)
        h5f.create_dataset("mean_image", data=global_mean, compression="gzip", compression_opts=4)

        # Store contextual metadata as HDF5 root attributes
        for key, val in metadata.items():
            h5f.attrs[key] = str(val)

    logging.info("Save successful.")
```

reduce_image_series.py, lines 227–262

```
def main() -> None:
    args = parse_args()

    if not args.h5_file.exists():
        raise FileNotFoundError(f"Master HDF5 file not found: {args.h5_file}")

    logging.info(f"Initializing scan {args.scan} from {args.h5_file.name}...")
    scan = BlissMeshScan(
        h5_filepath=args.h5_file,
        scan_number=args.scan,
```

```
        image_suffix=args.image_suffix
    )
    scan.read_data()

    if scan.nb_images == 0:
        raise RuntimeError(f"No image files matching '*{args.image_suffix}' found in {scan.scan_folder}")

    # Compute global reductions
    g_min, g_max, g_mean = compute_scan_reductions(
        image_files=scan.image_path_list,
        chunk_size=args.chunk_size,
        num_workers=args.workers
    )

    # Save to output directory
    output_filename = f"{args.h5_file.stem}_scan{args.scan:04d}_reduced.h5"
    output_filepath = args.output_dir / output_filename

    metadata = {
        "master_h5": str(args.h5_file.resolve()),
        "scan_number": args.scan,
        "scan_title": scan.title,
        "processed_frames": scan.nb_images,
    }

    save_reduced_hdf5(output_filepath, g_min, g_max, g_mean, metadata)
```

reduce_image_series.py, lines 265–266

```
if __name__ == "__main__":
    main()
```

8.2 `make_dynamic_mosaic`: region-of-interest maps

`python make_dynamic_mosaic.py master.h5 -s SCAN -rx X -ry Y [-w HWS] [-o DIR]` follows one region of interest (ROI) of the detector, a square of side $2\,\mathrm{hws}+1$ pixels centred on (X, Y)—usually around one Laue spot—through every frame of a mesh scan. It produces:

- the **mosaic**: the ROIs tiled in the order of the scan grid, a real-space image of how the spot changes across the sample;
- the **entropy** map: the Shannon entropy [44] of the normalised ROI intensity, $H = -\sum_k p_k \ln p_k$ with $p_k = I_k / \sum I$. A sharp spot concentrates the intensity in a few pixels (low H); a diffuse or split spot, or background only, spreads it (high H, at most $\ln N_{\mathrm{pixels}}$ for a flat ROI);
- the **max-to-mean** ratio of each ROI, another sharpness indicator;
- the **centre of mass** of each ROI, $\bar{x} = \sum_k x_k I_k / \sum_k I_k$, as a displacement from its mean over the map: the motion of a Laue spot on the detector follows the local lattice orientation (and the deviatoric strain), so this vector field maps rotations across the sample;
- the **total intensity** of the ROI, normalised by the monitor;
- the sharpest and the most diffuse ROI.

The arrays are saved in `computed_metrics_scanNNNN.h5` and a six-panel figure in physical motor coordinates.

Hypotheses and limits

- The ROI is fixed in detector pixels and must lie entirely inside the frame (a negative start index gives an empty slice); the spot must stay inside it over the whole map.
- No background is subtracted: a flat background raises the entropy and pulls the centre of mass towards the ROI centre.
- Displacements are in detector pixels, not converted into angles or strain.
- Points never recorded (aborted scan) become empty blocks, and their motor positions are extrapolated from a plane fitted to the recorded ones.

make_dynamic_mosaic.py, lines 1–26

```
"""
Module: make_dynamic_mosaic.py

Description:
    Production-ready pipeline to process X-ray diffraction frames from BM32.
    Accepts a master HDF5 file, scan number, and ROI parameters via the CLI.
    Extracts physical motor coordinates directly from the BLISS HDF5 structure
    using bliss_hdf5_reader. Computes spatial metrics (Entropy, Max-to-Mean,
    CoM displacement), stitches a full 2D mosaic, and generates a summary
    figure mapped to the physical motor axes.
"""

import argparse
import multiprocessing as mp
from pathlib import Path
from functools import partial
from typing import Dict, Tuple, List

import h5py
import fabio
import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm

# Import the robust reader class provided by the user
from bliss_hdf5_reader import BlissMeshScan
```

8.2.1 `parse_args`

`parse_args()`

The master file, `-s/--scan`, `-rx/--roi_x`, `-ry/--roi_y`, `-w/--hws` (half window, default 5), `--image_suffix`, `-o/--output_dir`.

make_dynamic_mosaic.py, lines 29–41

```
def parse_args() -> argparse.Namespace:
    """Parses command-line arguments for dynamic ROI and scan selection."""
    parser = argparse.ArgumentParser(
        description="Process X-ray diffraction frames and compute spatial metrics using physical motor coordinates."
    )
    parser.add_argument("h5_file", type=Path, help="Path to the master BLISS HDF5 file.")
    parser.add_argument("-s", "--scan", type=int, required=True, help="Scan number to process (e.g., 1 for '1.1').")
    parser.add_argument("-rx", "--roi_x", type=int, required=True, help="Center X coordinate of the ROI.")
    parser.add_argument("-ry", "--roi_y", type=int, required=True, help="Center Y coordinate of the ROI.")
    parser.add_argument("-w", "--hws", type=int, default=5, help="Half-window size for the ROI (default: 5).")
    parser.add_argument("--image_suffix", type=str, default=".h5", help="Suffix of the image files (default: .h5).")
    parser.add_argument("-o", "--output_dir", type=Path, default=".", help="Output directory for the data.")
    return parser.parse_args()
```

8.2.2 `read_image`, `roi_slices`, `get_image_roi`

`read_image(filepath)`, `roi_slices(roi_x, roi_y, hws)`, `get_image_roi(filepath, roi_x, roi_y, hws)`

Fallback readers for scans whose master file gives no direct access to the frames (TIFF files, or HDF5 files at `/entry_0000/CRGIF/eiger4m/data`, first frame); `roi_slices` returns the (row, column) slices of the ROI.

make_dynamic_mosaic.py, lines 44–53

```
def read_image(filepath: Path) -> np.ndarray:
    """Reads HDF5 or TIFF detector frames."""
    if filepath.suffix == ".h5":
        with h5py.File(filepath, 'r') as h5f:
            # Corrected internal HDF5 path based on eiger4m_0000.h5 structure
            return h5f['/entry_0000/CRGIF/eiger4m/data'][:][0]
    elif filepath.suffix in (".tif", ".tiff"):
```

```
        with fabio.open(filepath) as im:
            return im.data
    raise ValueError(f"Unsupported file format: {filepath}")
```

make_dynamic_mosaic.py, lines 56–58

```
def roi_slices(roi_x: int, roi_y: int, hws: int) -> Tuple[slice, slice]:
    """(row, column) slices of the ROI in a detector frame."""
    return (slice(roi_y - hws, roi_y + hws + 1), slice(roi_x - hws, roi_x + hws + 1))
```

make_dynamic_mosaic.py, lines 61–65

```
def get_image_roi(filepath: Path, roi_x: int, roi_y: int, hws: int) -> np.ndarray:
    """Extracts a Region of Interest from an image file (one frame per file).
    Only used when the master file gives no direct access to the frames."""
    data = read_image(filepath)
    return data[roi_slices(roi_x, roi_y, hws)]
```

8.2.3 `_init_worker`, `_read_roi_batch`, `read_rois`

`_init_worker(scan)`, `_read_roi_batch(indices, roi)`, `read_rois(scan, roi, batch=50)`

Frames are read through the master file with `BlissScan.iter_frames`, which reads only the ROI from disk. The scan object is sent once to each worker process (pool initialiser, global `_WORKER_SCAN`), and each task reads a batch of 50 frames with the file opened once. Returns the ROIs of every recorded frame in acquisition order.

make_dynamic_mosaic.py, lines 68–74

```
# Set once in each worker process, so the scan is not sent with every task.
_WORKER_SCAN = None


def _init_worker(scan: BlissMeshScan) -> None:
    global _WORKER_SCAN
    _WORKER_SCAN = scan
```

make_dynamic_mosaic.py, lines 77–79

```
def _read_roi_batch(indices: List[int], roi: Tuple[slice, slice]) -> List[np.ndarray]:
    """ROIs of a batch of frames, read through the master file opened once."""
    return list(_WORKER_SCAN.iter_frames(indices=indices, roi=roi))
```

make_dynamic_mosaic.py, lines 82–93

```
def read_rois(scan: BlissMeshScan, roi: Tuple[slice, slice], batch: int = 50) -> List[np.ndarray]:
    """ROI of every recorded frame, in acquisition order, read in parallel."""
    n = scan.nb_frames
    batches = [list(range(i, min(i + batch, n))) for i in range(0, n, batch)]
    rois: List[np.ndarray] = []
    with mp.Pool(min(mp.cpu_count(), len(batches)), initializer=_init_worker,
                 initargs=(scan,)) as pool:
        with tqdm(total=n) as bar:
            for out in pool.imap(partial(_read_roi_batch, roi=roi), batches):
                rois.extend(out)
                bar.update(len(out))
    return rois
```

8.2.4 `fill_missing_positions`, `to_scan_grid`

`fill_missing_positions(grid)`, `to_scan_grid(rois, index_grid)`

`fill_missing_positions` replaces the NaN positions of unrecorded points by a plane $a + b\,i + c\,j$ fitted (least squares) to the recorded positions, since plotting functions refuse NaN coordinates. `to_scan_grid` orders the ROIs like the grid through the scan's `index_grid` (which handles back-and-forth meshes), with zero blocks for missing points.

make_dynamic_mosaic.py, lines 96–109

```python
def fill_missing_positions(grid: np.ndarray) -> np.ndarray:
    """Motor positions of points never recorded (aborted scan) are NaN, which
    pcolormesh/quiver refuse. Fill them from a plane fitted to the recorded
    positions (a mesh is linear in row and column index)."""
    missing = ~np.isfinite(grid)
    if not missing.any():
        return grid
    rows, cols = np.indices(grid.shape)
    design = np.column_stack([np.ones(grid.size), rows.ravel(), cols.ravel()])
    ok = ~missing.ravel()
    coef, *_ = np.linalg.lstsq(design[ok], grid.ravel()[ok], rcond=None)
    filled = grid.copy()
    filled[missing] = (design @ coef).reshape(grid.shape)[missing]
    return filled
```

make_dynamic_mosaic.py, lines 112–117

```python
def to_scan_grid(rois: List[np.ndarray], index_grid: np.ndarray) -> List[np.ndarray]:
    """Order the ROIs like the scan grid (row-major), using the scan's index grid.
    Handles back-and-forth meshes; points never recorded (aborted scan) become
    empty (zero) blocks."""
    empty = np.zeros_like(rois[0])
    return [rois[i] if 0 <= i < len(rois) else empty for i in index_grid.ravel()]
```

8.2.5 `compute_pipeline_data`

```
compute_pipeline_data(array_list, grid_shape)
```

All metrics in vectorised form. The mosaic is built by reshaping the stack of blocks to $(n_{\text{rows}}, n_{\text{cols}}, h, w)$, swapping the two middle axes and reshaping to $(n_{\text{rows}}h, n_{\text{cols}}w)$. Empty blocks get zero entropy, max-to-mean and displacement, and are ignored when looking for the sharpest block. Returns a dictionary of arrays.

make_dynamic_mosaic.py, lines 120–184

```python
def compute_pipeline_data(
    array_list: List[np.ndarray], grid_shape: Tuple[int, int]
) -> Dict[str, np.ndarray]:
    """Consolidated computation for metrics, mosaic stitching, and extreme ROI extraction."""
    grid_rows, grid_cols = grid_shape
    stacked_3d = np.array(array_list, dtype=np.float64)
    N, block_rows, block_cols = stacked_3d.shape
    flat_blocks = stacked_3d.reshape(N, -1)

    # 1. Stitch Mosaic
    grid_4d = stacked_3d.reshape(grid_rows, grid_cols, block_rows, block_cols)
    transposed_4d = grid_4d.transpose(0, 2, 1, 3)
    mosaic = transposed_4d.reshape(grid_rows * block_rows, grid_cols * block_cols)

    # 2. Shannon Entropy & Extremes
    block_sums = flat_blocks.sum(axis=1, keepdims=True)
    safe_sums = np.where(block_sums == 0, 1.0, block_sums)
    p = flat_blocks / safe_sums
    p_safe = np.where(p <= 0, 1e-12, p)
    entropy = -np.sum(p * np.log(p_safe), axis=1)
    entropy[block_sums.flatten() == 0] = 0.0

    # Ignore empty frames (entropy == 0.0) when finding the sharpest peak
    valid_entropy = np.where(entropy > 0, entropy, np.inf)
    sharpest_idx = np.argmin(valid_entropy)
    diffuse_idx = np.argmax(entropy)

    # 3. Max-to-Mean
    block_maxes = flat_blocks.max(axis=1)
    block_means = flat_blocks.mean(axis=1)
    safe_means = np.where(block_means == 0, 1.0, block_means)
    max_to_mean = block_maxes / safe_means
    max_to_mean[block_means == 0] = 0.0

```

```
    # Total intensity
    total_intensity = block_sums.reshape(grid_rows, grid_cols)

    # 4. Center of Mass Vector Field
    total_mass = stacked_3d.sum(axis=(1, 2))
    safe_mass = np.where(total_mass == 0, 1.0, total_mass)

    y_coords = np.arange(block_rows).reshape(block_rows, 1)
    x_coords = np.arange(block_cols).reshape(1, block_cols)

    x_com = (stacked_3d * x_coords).sum(axis=(1, 2)) / safe_mass
    y_com = (stacked_3d * y_coords).sum(axis=(1, 2)) / safe_mass

    # Calculate displacement relative to the global mean CoM position
    dx = x_com - np.mean(x_com[total_mass > 0])
    dy = y_com - np.mean(y_com[total_mass > 0])

    # Force empty blocks to have zero displacement
    dx[total_mass == 0] = 0.0
    dy[total_mass == 0] = 0.0

    return {
        "mosaic": mosaic,
        "entropy": entropy.reshape(grid_rows, grid_cols),
        "max_to_mean": max_to_mean.reshape(grid_rows, grid_cols),
        "com_dx": dx.reshape(grid_rows, grid_cols),
        "com_dy": dy.reshape(grid_rows, grid_cols),
        "sharpest_roi": array_list[sharpest_idx],
        "diffuse_roi": array_list[diffuse_idx],
        "total_intensity": total_intensity
    }
```

8.2.6 save_metrics_to_hdf5, plot_and_save_figure

`save_metrics_to_hdf5(data, output_path)`, `plot_and_save_figure(data, x_grid, y_grid, mon_grid, motor_names, output_path)`

The HDF5 archive (gzip), and the figure: mosaic in physical extent, sharpest and most diffuse ROI, entropy map, monitor-normalised intensity, and the centre-of-mass vector field (one arrow in two on dense grids).

make_dynamic_mosaic.py, lines 187–197

```
def save_metrics_to_hdf5(data: Dict[str, np.ndarray], output_path: Path) -> None:
    """Saves computed arrays to an HDF5 file with GZIP compression."""
    with h5py.File(output_path, 'w') as h5f:
        for key, array_data in data.items():
            h5f.create_dataset(
                key,
                data=array_data,
                compression="gzip",
                compression_opts=4
            )
    print(f"Data successfully saved to {output_path}")
```

make_dynamic_mosaic.py, lines 200–279

```
def plot_and_save_figure(
    data: Dict[str, np.ndarray],
    x_grid: np.ndarray,
    y_grid: np.ndarray,
    mon_grid: np.ndarray,
    motor_names: Tuple[str, str],
    output_path: Path
) -> None:
    """Generates and saves a comprehensive summary figure mapped to physical coordinates."""
    fig = plt.figure(figsize=(18, 12))

    x_name, y_name = motor_names
    x_min, x_max = np.nanmin(x_grid), np.nanmax(x_grid)
    y_min, y_max = np.nanmin(y_grid), np.nanmax(y_grid)
```

```
214
215     # Extent bounds for the full mosaic image to map it to physical dimensions
216     extent = [x_min, x_max, y_max, y_min]
217
218     ax_mosaic = plt.subplot(2, 3, 1)
219     ax_sharp = plt.subplot(2, 3, 2)
220     ax_diffuse = plt.subplot(2, 3, 3)
221     ax_entropy = plt.subplot(2, 3, 4)
222     ax_papr = plt.subplot(2, 3, 5)
223     ax_quiver = plt.subplot(2, 3, 6)
224
225     # 1. Full Mosaic (Plotted with physical extent)
226     im_mos = ax_mosaic.imshow(data["mosaic"], cmap='bone', extent=extent, aspect='auto')
227     ax_mosaic.set_title('Full Stitched Mosaic')
228     ax_mosaic.set_xlabel(f"{x_name} (Physical)")
229     ax_mosaic.set_ylabel(f"{y_name} (Physical)")
230     ax_mosaic.invert_yaxis()  # Standardize orientation
231     ax_mosaic.set_aspect("equal")
232     fig.colorbar(im_mos, ax=ax_mosaic, fraction=0.046, pad=0.04)
233
234     # 2. Sharpest ROI (Pixel coordinates are used as it is a single isolated block)
235     im_sharp = ax_sharp.imshow(data["sharpest_roi"], cmap='viridis')
236     ax_sharp.set_title('Sharpest Peak (Min Entropy)')
237     fig.colorbar(im_sharp, ax=ax_sharp, fraction=0.046, pad=0.04)
238
239     # 3. Diffuse ROI
240     im_diff = ax_diffuse.imshow(data["diffuse_roi"], cmap='viridis')
241     ax_diffuse.set_title('Most Diffuse Block (Max Entropy)')
242     fig.colorbar(im_diff, ax=ax_diffuse, fraction=0.046, pad=0.04)
243
244     # 4. Entropy Grid (Mapped via pcolormesh for exact physical alignment)
245     im_ent = ax_entropy.pcolormesh(x_grid, y_grid, data["entropy"], cmap='viridis_r', shading='nearest')
246     ax_entropy.set_title('Entropy Grid Map\n(Dark/Low = Sharper)')
247     ax_entropy.set_xlabel(x_name)
248     ax_entropy.set_ylabel(y_name)
249     fig.colorbar(im_ent, ax=ax_entropy, fraction=0.046, pad=0.04)
250
251     # 5. Total intensity
252     # Prevent division by zero if monitor grid has zeros
253     safe_mon_grid = np.where(mon_grid == 0, 1.0, mon_grid)
254     im_papr = ax_papr.pcolormesh(x_grid, y_grid, data["total_intensity"]/safe_mon_grid, cmap='magma', shading='nearest'
        ↪ )
255     ax_papr.set_title('Total intensity (normalized by monitor)')
256     ax_papr.set_xlabel(x_name)
257     ax_papr.set_ylabel(y_name)
258     fig.colorbar(im_papr, ax=ax_papr, fraction=0.046, pad=0.04)
259
260     # 6. CoM Quiver Plot (Vector field representing shift from the mean)
261     rows, cols = data["com_dx"].shape
262     stride = 2 if cols >= 50 else 1  # Prevents arrow overcrowding on dense grids
263
264     ax_quiver.quiver(
265         x_grid[::stride, ::stride],
266         y_grid[::stride, ::stride],
267         data["com_dx"][::stride, ::stride],
268         data["com_dy"][::stride, ::stride],
269         color='black',
270         angles='xy'
271     )
272     ax_quiver.set_title('CoM Displacement Vector Field\n(Relative to Mean)')
273     ax_quiver.set_xlabel(x_name)
274     ax_quiver.set_ylabel(y_name)
275
276     plt.tight_layout()
277     plt.savefig(output_path, dpi=300, bbox_inches='tight')
278     print(f"Summary figure successfully saved to {output_path}")
279     plt.close()
```

8.2.7 main

```
main()
```

Reads the scan and the monitor, chooses as horizontal axis the motor whose name contains “x” (transposing the grids if it is the slow motor, so that the mosaic is drawn with x horizontal), reads the ROIs (through the master file, or file by file), computes, saves and plots.

make_dynamic_mosaic.py, lines 282–354

```
def main() -> None:
    args = parse_args()

    if not args.h5_file.exists():
        raise FileNotFoundError(f"Master HDF5 file not found: {args.h5_file}")

    print(f"Reading scan {args.scan} metadata from {args.h5_file.name}...")
    scan = BlissMeshScan(
        h5_filepath=args.h5_file,
        scan_number=args.scan,
        image_suffix=args.image_suffix
    )
    scan.read_data(extra_counters=["mon",])
    print(scan.summary())

    if scan.nb_frames == 0:
        raise RuntimeError(f"No images found for scan {args.scan} in {scan.scan_folder}")

    # Determine which motors to use for X and Y plotting based on names
    x_motor = next((m for m in (scan.fast_motor_name, scan.slow_motor_name) if 'x' in m.lower()),
                   scan.fast_motor_name)
    y_motor = scan.slow_motor_name if x_motor == scan.fast_motor_name else scan.fast_motor_name

    x_grid = fill_missing_positions(scan.grids[x_motor])
    y_grid = fill_missing_positions(scan.grids[y_motor])

    # Attempt to use the actual monitor grid; fallback to ones if missing
    if "mon" in scan.grids:
        mon_grid = scan.grids["mon"]
    else:
        print("Warning: 'mon' counter not found. Falling back to unnormalized intensity.")
        mon_grid = np.ones_like(x_grid)

    print(f"Extracting ROIs in parallel from {scan.nb_frames} frames...")
    roi = roi_slices(args.roi_x, args.roi_y, args.hws)
    if scan.frame_sources():
        rois = read_rois(scan, roi)
    else:
        # No virtual dataset in the master file (e.g. TIFF images): one file per frame.
        worker_func = partial(get_image_roi, roi_x=args.roi_x, roi_y=args.roi_y, hws=args.hws)
        with mp.Pool(mp.cpu_count()) as pool:
            rois = list(tqdm(pool.imap(worker_func, scan.image_path_list, chunksize=50),
                             total=scan.nb_images))

    # Blocks in grid order: rows follow the slow motor, columns the fast one.
    blocks = to_scan_grid(rois, scan.index_grid)
    grid_shape = scan.scan_shape
    if x_motor == scan.slow_motor_name:
        # Put y along the rows so the mosaic is drawn with x horizontal.
        n_slow, n_fast = grid_shape
        blocks = [blocks[i * n_fast + j] for j in range(n_fast) for i in range(n_slow)]
        grid_shape = (n_fast, n_slow)
        x_grid, y_grid, mon_grid = x_grid.T, y_grid.T, mon_grid.T

    print("Computing metrics and stitching mosaic...")
    pipeline_data = compute_pipeline_data(blocks, grid_shape)

    print("Saving HDF5 archive...")
    h5_path = args.output_dir / f"computed_metrics_scan{args.scan:04d}.h5"
    save_metrics_to_hdf5(pipeline_data, h5_path)

    print("Generating comprehensive visualization...")
    plot_filename = f"{args.h5_file.stem}_scan{args.scan:04d}_mosaic_X{args.roi_x}_Y{args.roi_y}_W{args.hws}.png"
    plot_path = args.output_dir / plot_filename

    plot_and_save_figure(
        data=pipeline_data,
        x_grid=x_grid,
```

```
        y_grid=y_grid,
        mon_grid=mon_grid,
        motor_names=(x_motor, y_motor),
        output_path=plot_path
    )
```

make_dynamic_mosaic.py, lines 357–358

```
if __name__ == "__main__":
    main()
```

9 Older and superseded tools

These modules predate the current analysis chain. They are kept in the repository for reference and for old data, and documented here with the problems that stop them from running unchanged on a current installation (Python 3.12, NumPy 2).

9.1 `fluo_map_fit`: the first fluorescence map fitter

Fits every spectrum of a BLISS map independently with two "coupled double Gaussians" (Ga and In), a constant background and `curve_fit`, in parallel over the pixels, and plots the two area maps. Each element is a Kα peak plus a Kβ peak at a fixed channel distance and a fixed amplitude ratio, sharing one width:

$$g(x) = A\,\mathrm{e}^{-(x-c)^2/2\sigma^2} + rA\,\mathrm{e}^{-(x-c-\Delta)^2/2\sigma^2}, \qquad \text{area} = A\sigma\sqrt{2\pi}\,(1+r). \tag{33}$$

Known issue

Superseded by `xrf_fit` (Section 5), which replaces every hard-coded number here by tabulated physics. The channel offsets and ratios are hard-coded for a 10 eV/channel calibration (Ga: $\Delta = 101$ channels, $r = 0.13$; the In values are placeholders), as are the fitted channel range (800–1330), the starting centres and their bounds, and the input file in the `__main__` block. The spectra are not normalised (monitor, live time), the widths do not follow the detector resolution law, and Kβ/Kα is not computed for the excitation. The module docstring calls it `constrained_xrf_fitter.py`.

fluo_map_fit.py, lines 1–37

```
# Authors: Gemini and Samuel Tardif

"""
Module: constrained_xrf_fitter.py

Description:
    Reads X-ray fluorescence spectra from a BLISS HDF5 file and performs
    parallelized Gaussian peak fitting. Constrains the K_alpha and K_beta
    lines for each element to fixed relative positions and amplitude ratios
    to stabilize the fit across a 2D spatial map.
"""

import logging
import multiprocessing
from pathlib import Path
from concurrent.futures import ProcessPoolExecutor
from typing import Tuple, Optional

import h5py
import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt

# Configure logging
logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")

```

```
# ============================================================================
# Physics & Detector Configuration
# At ~10 eV/channel, Ga Ka is ~9.25 keV (ch 925), Ga Kb is ~10.26 keV (ch 1026).
# Delta is ~101 channels. Typical Kb/Ka branching ratio is ~0.13.
# Update the Indium values based on the specific L-lines or K-lines in your ROI.
# ============================================================================
GA_DELTA_CH = 101.0       # Channels between Ga Ka and Ga Kb
GA_KB_RATIO = 0.13        # Amplitude ratio of Ga Kb to Ga Ka

IN_DELTA_CH = 45.0        # Placeholder: Update with exact delta for In lines in ROI
IN_KB_RATIO = 0.15        # Placeholder: Update with exact branching ratio for In
```

9.1.1 `coupled_double_gaussian`, `spectrum_model`

`coupled_double_gaussian(x, amp_alpha, cen_alpha, sigma, delta, ratio)`, `spectrum_model(x, bkg, amp_ga, cen_ga, sig_ga, amp_in, cen_in, sig_in)`

The doublet above, and the full model: constant background plus the Ga and In doublets.

fluo_map_fit.py, lines 39–53

```
def coupled_double_gaussian(
    x: np.ndarray,
    amp_alpha: float,
    cen_alpha: float,
    sigma: float,
    delta: float,
    ratio: float
) -> np.ndarray:
    """
    Generates a double Gaussian profile where the secondary peak is rigidly
    constrained relative to the primary peak.
    """
    primary = amp_alpha * np.exp(-((x - cen_alpha) ** 2) / (2 * sigma ** 2))
    secondary = (amp_alpha * ratio) * np.exp(-((x - (cen_alpha + delta)) ** 2) / (2 * sigma ** 2))
    return primary + secondary
```

fluo_map_fit.py, lines 55–70

```
def spectrum_model(
    x: np.ndarray,
    bkg: float,
    amp_ga: float,
    cen_ga: float,
    sig_ga: float,
    amp_in: float,
    cen_in: float,
    sig_in: float
) -> np.ndarray:
    """
    Complete model comprising a constant background and two constrained elements.
    """
    ga_profile = coupled_double_gaussian(x, amp_ga, cen_ga, sig_ga, GA_DELTA_CH, GA_KB_RATIO)
    in_profile = coupled_double_gaussian(x, amp_in, cen_in, sig_in, IN_DELTA_CH, IN_KB_RATIO)
    return bkg + ga_profile + in_profile
```

9.1.2 `fit_single_spectrum`, `process_and_plot`

`fit_single_spectrum(args)`, `process_and_plot(h5_filepath, dataset_path, scan_shape)`

`fit_single_spectrum` fits one pixel, `args = (index, channels, counts)`, and returns `(index, Ga area, In area)` (both `None` if the fit fails). `process_and_plot` loads the MCA dataset, slices the channel range, distributes the pixels over a `ProcessPoolExecutor` (one core left free, chunks of 100), reshapes the areas to `scan_shape` and plots the two maps.

fluo_map_fit.py, lines 72–108

```
def fit_single_spectrum(args: Tuple[int, np.ndarray, np.ndarray]) -> Tuple[int, Optional[float], Optional[float]]:
```

```
    """
    Fits a single XRF spectrum. Designed to be mapped across a ProcessPool.

    Args:
        args: A tuple containing (pixel_index, channel_array, spectrum_counts).

    Returns:
        tuple: (pixel_index, area_ga, area_in). Returns None for areas if the fit fails.
    """
    index, x_data, y_data = args

    # p0: [bkg, amp_ga, cen_ga, sig_ga, amp_in, cen_in, sig_in]
    # Adjust starting centers based on your exact ROI calibration
    p0 = [np.min(y_data), np.max(y_data)*0.5, 925, 10, np.max(y_data)*0.5, 1100, 10]

    # Restrict centers from wandering too far from theoretical positions
    bounds = (
        [0, 0, 915, 1, 0, 1050, 1],
        [np.inf, np.inf, 935, 30, np.inf, 1150, 30]
    )

    try:
        popt, _ = curve_fit(spectrum_model, x_data, y_data, p0=p0, bounds=bounds)

        # Calculate total elemental area (Integral of Ka + Integral of Kb)
        # Area = amp * sigma * sqrt(2*pi)
        ga_area_alpha = popt[1] * popt[3] * np.sqrt(2 * np.pi)
        ga_area_total = ga_area_alpha + (ga_area_alpha * GA_KB_RATIO)

        in_area_alpha = popt[4] * popt[6] * np.sqrt(2 * np.pi)
        in_area_total = in_area_alpha + (in_area_alpha * IN_KB_RATIO)

        return index, ga_area_total, in_area_total

    except RuntimeError:
        return index, None, None
```

fluo_map_fit.py, lines 110–159

```
def process_and_plot(h5_filepath: str, dataset_path: str, scan_shape: Tuple[int, int]) -> None:
    """Loads data, manages the multiprocessing pool, and plots results."""
    h5_path = Path(h5_filepath)

    logging.info(f"Loading dataset {dataset_path} from {h5_path.name}...")
    with h5py.File(h5_path, "r") as f:
        raw_spectra = f[dataset_path][()]

    n_pixels = raw_spectra.shape[0]
    roi_start, roi_end = 800, 1330
    x_data = np.arange(roi_start, roi_end)

    # Slice the ROI prior to multiprocessing to minimize serialization payloads
    tasks = [
        (i, x_data, raw_spectra[i, roi_start:roi_end])
        for i in range(n_pixels)
    ]

    # Reserve one core for OS/scheduler overhead
    max_workers = max(1, multiprocessing.cpu_count() - 1)
    logging.info(f"Initiating ProcessPoolExecutor with {max_workers} workers for {n_pixels} spectra...")

    ga_areas = np.full(n_pixels, np.nan)
    in_areas = np.full(n_pixels, np.nan)

    # Execute fits in parallel with chunking for IPC efficiency
    with ProcessPoolExecutor(max_workers=max_workers) as executor:
        results = executor.map(fit_single_spectrum, tasks, chunksize=100)

        for idx, ga_area, in_area in results:
            if ga_area is not None:
                ga_areas[idx] = ga_area
                in_areas[idx] = in_area

    map_ga = ga_areas.reshape(scan_shape)
```

```
    map_in = in_areas.reshape(scan_shape)

    logging.info("Fitting complete. Rendering plots...")
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

    im1 = ax1.imshow(map_ga, aspect='auto', cmap='plasma')
    ax1.set_title("Gallium Distribution (Total Area)")
    fig.colorbar(im1, ax=ax1)

    im2 = ax2.imshow(map_in, aspect='auto', cmap='plasma')
    ax2.set_title("Indium Distribution (Total Area)")
    fig.colorbar(im2, ax=ax2)

    plt.tight_layout()
    plt.show()
```

fluo_map_fit.py, lines 161–169

```
if __name__ == "__main__":
    # Define file and parameters
    H5_FILE = "InPoSi_v2_center.h5"
    DATASET_PATH = "1.1/measurement/xia_det0"

    # Update grid dimensions based on the amesh metadata title
    SCAN_SHAPE = (81, 81)

    process_and_plot(H5_FILE, DATASET_PATH, SCAN_SHAPE)
```

9.2 `spec_reader`: SPEC data files

SPEC, the control program used on many synchrotron beamlines before BLISS, writes plain-text data files: a file header (`#F` file name, `#E` epoch, `#D` date, `#O` motor names), then for each scan a header (`#S` number and command, `#D` date, `#T` count time, `#G` geometry, `#Q` *hkl*, `#P` motor positions, `#N` number of columns, `#L` column labels, `#C` comments, `#@` MCA and user lines), followed by the data columns. `SpecFile` reads the file header and indexes the byte offset of every scan; `Scan` reads one scan—or concatenates several similar scans—and exposes every counter as an attribute.

Known issue

- Files are opened with mode `'rU'`, which was removed in Python 3.11: on the current environment `SpecFile` raises `ValueError: invalid mode: 'rU'` (not caught, since only `IOError` is). Mode `'r'` gives the same universal-newline behaviour in Python 3.
- `__detcalib__` uses `np.float`, removed in NumPy 1.24 (only reached for `#UDETCALIB` lines).
- The XIA handlers are unreachable: a line `#@XIAFILE path` gives the key `@XIAFILE`, which is not in the dispatch table, and a line `#@ XIAFILE path` reaches `__special__` but splits into an empty first word. Both end as "unprocessed lines" (checked on the current code), so `xianame`, `xiacalib` and `xiaroi` are never set.

9.2.1 `SpecFile`

```
SpecFile(spec_file, verbose=False)
```

Reads the file header up to the first `#S` line, dispatching each line to a handler by its key (`__param__` maps the key letters to methods), and records in `scan_dict` the byte position of every `#S` line, so that a scan can later be read with a single `seek`. The key is the text after `#`, with trailing digits removed except for user keys starting with `U` (so `#O0`, `#O1` both go to the `O` handler).

spec_reader.py, lines 1–8

```
# SPEC reader
# 2016-11-23
# Sam Tardif (samuel.tardif@gmail.com)
# Python3 compatibility by Nils Blanc

```

```
6  import numpy as np
7  import os.path
8  import string
```

spec_reader.py, lines 11–38

```
11 class SpecFile:
12   """
13
14   Simple class to read SPEC file. The main purposes are :
15   (i) read the header and initiate the relevant properties
16   (ii) build a dictionary of the scan number and their binary position in the file
17   The latter allows for faster subsequent scan reading
18
19   Definition:
20   -----------
21   SpecFile(spec_file, verbose = False)
22    > spec_file : string
23    > scan_numbers : integer or as tuple/list/array of integer
24
25   Attributes (typical):
26   -----------
27   comments........all comments
28   date............scan start datestamp
29   file............string of the SPEC file name
30   scan_dict.......dictionary {scan number : binary position in file}
31
32   Examples:
33   --------
34   # read the specfile
35   In : sf = SpecFile('./lineup0.dat')
36
37   """
38   # dictionary definitions for handling the spec identifiers
```

spec_reader.py, lines 39–61

```
39   def __param__(self):
40     return  { 'S' : self.__scanline__,
41               'D' : self.__dating__,
42               'T' : self.__counting__,
43               'G' : self.__configurating__,
44               'Q' : self.__hkl__,
45               'O' : self.__motorslabeling__,
46               'o' : self.__motorslabelingnospace__,
47               'P' : self.__positioning__,
48               'M' : self.__marccdpath__,
49               'N' : self.__speccol__,
50               'L' : self.__counterslabeling__,
51               'J' : self.__allcounterslabeling__,
52               'j' : self.__allcounterslabelingnospace__,
53               'C' : self.__commenting__,
54               'E' : self.__initialepoch__,
55               'F' : self.__specfilenaming__,
56               '@' : self.__special__,
57               'XIAFILE' : self.__xiafilenaming__,
58               'XIACALIB' : self.__xiacalibrating__,
59               'XIAROI' : self.__xiaroidefining__,
60               'ULIMA_mpx4' : self.__limampx4path__,
61               'UDETCALIB' : self.__detcalib__}
```

spec_reader.py, lines 145–151

```
145  def __readSpecLine__(self,l, verbose=False):
146    try:
147      SpecKey = l.split()[0][1:]
148      if SpecKey[0] != "U": SpecKey = SpecKey.rstrip(string.digits) # remove trailing digits for SpecKeys other than
       ↪ those starting with "U" (User defined?)
149      self.__param__()[SpecKey](l)
150    except KeyError:
151      if verbose : print("unprocessed line (SpecKey {}): ".format(SpecKey) + l)
```

spec_reader.py, lines 154–195

```
154   def __init__(self, spec_file, verbose = False):
155     # init a bunch of stuff that will also be used by the children class Scan
156     self.file = spec_file
157     self.__motorslabels__ = "" # list of all motors in the experiment
158     self.__motorslabelsnospace__ = "" # list of all motors in the experiment
159     self.__positions__ = "" # list the values of all motors
160     self.__counters__ = "" # list of all counters in the experiment
161     self.__countersnospace__ = "" # list of all counters in the experiment
162     self.__config__ = "" # list the values of the UB matrix config
163     self.comments = ""
164     self.scan_dict={}  # dictionary to store the position in the file of the scans
165     try:
166       with open(spec_file,'rU') as f:  # the U mode indicates universal line break, essential for accurate counting
167         # first read the file header (mostly comments and motors definition)
168         # up to the first scan (identified by a line starting with "#S"
169         reading_header = True
170         while reading_header:
171           position_in_file = f.tell() # get the position AHEAD of the scan first line
172           l = f.readline()
173           if len(l) > 1: # not an empty line
174             if l[:2] == '#S':
175               reading_header = False
176               scan_number = int(l.split()[1])
177               if verbose:
178                   print("after reading the header, found scan {} at location {}".format(scan_number,position_in_file))
179               self.scan_dict[scan_number] = position_in_file
180             else:
181               self.__readSpecLine__(l, verbose=verbose)
182         reading_file = True  # change this switch when the end of file is reached (i.e. read an empty string)
183         while reading_file:
184           position_in_file = f.tell() # get the position AHEAD of the scan first line
185           l = f.readline()
186           if len(l) > 1: # not an empty line
187             if l[:2] == '#S':
188               scan_number = int(l.split()[1])
189               if verbose:
190                   print("found scan {} at location {}".format(scan_number,position_in_file))
191               self.scan_dict[scan_number] = position_in_file
192           if l == "":
193             reading_file = False
194     except IOError:
195       print("could not find the file {}".format(spec_file))
```

9.2.2 Line handlers

One method per key, each parsing its line into attributes: `__scanline__` (`#S`: `number`, `type`, `args`, `command`), `__dating__` (`#D`), `__counting__` (`#T`: `ct`, `ct_units`), `__configurating__` (`#G`), `__hkl__` (`#Q`: `Qo`), `__motorslabeling__` and `__motorslabelingnospace__` (`#O`, `#o`), `__positioning__` (`#P`), `__speccol__` (`#N`: `N`), `__counterslabeling__` (`#L`: `counters`), `__allcounterslabeling__` and `__allcounterslabelingnospace__` (`#J`, `#j`), `__specfilenaming__` (`#F`), `__commenting__` (`#C`), `__initialepoch__` (`#E`), `__marccdpath__` (`#M`), `__limampx4path__` (`#ULIMA_mpx4`), `__detcalib__` (`#UDETCALIB`: detector centre, pixels per degree, distances), `__special__` (`#@`: dispatches on the following word), `__xiafilenaming__`, `__xiacalibrating__`, `__xiaroidefining__` (XIA MCA file, calibration and ROIs).

spec_reader.py, lines 63–67

```
63   def __scanline__(self, l):
64     self.number = int(l.split()[1])
65     self.type = l.split()[2]
66     self.args = l.split()[3:]
67     self.command = l[3+len(l.split()[1])+2:].rstrip()
```

spec_reader.py, lines 69–70

```
69   def __dating__(self, l):
70     self.date = " ".join(l.split()[1:])
```

spec_reader.py, lines 72–74

```
72    def __counting__(self, l):
73      self.ct = float(l.split()[1])
74      self.ct_units = l.split()[2]
```

spec_reader.py, lines 76–77

```
76    def __configurating__(self, l):
77      self.__config__ = self.__config__ + " ".join(l.split()[1:]) + " "
```

spec_reader.py, lines 79–80

```
79    def __hkl__(self, l):
80      self.Qo = l.split()[1:]
```

spec_reader.py, lines 82–83

```
82    def __motorslabeling__(self, l):
83      self.__motorslabels__ = self.__motorslabels__ + " ".join(l.split()[1:]) + " "
```

spec_reader.py, lines 85–86

```
85    def __motorslabelingnospace__(self, l):
86      self.__motorslabelsnospace__ = self.__motorslabelsnospace__ + " ".join(l.split()[1:]) + " "
```

spec_reader.py, lines 88–89

```
88    def __positioning__(self, l):
89      self.__positions__ = self.__positions__ + " ".join(l.split()[1:]) + " "
```

spec_reader.py, lines 91–92

```
91    def __speccol__(self, l):
92      self.N = int(" ".join(l.split()[1:]))
```

spec_reader.py, lines 94–96

```
94    def __counterslabeling__(self, l):
95      # column labels
96      self.counters = l.split()[1:]
```

spec_reader.py, lines 98–100

```
98    def __allcounterslabeling__(self, l):
99      # all counters in the experiment
100     self.__counters__ = self.__counters__  + " ".join(l.split()[1:]) + " "
```

spec_reader.py, lines 102–104

```
102   def __allcounterslabelingnospace__(self, l):
103     # all counters in the experiment
104     self.__countersnospace__ = self.__countersnospace__  + " ".join(l.split()[1:]) + " "
```

spec_reader.py, lines 106–107

```
106   def __specfilenaming__(self, l):
107     self.specfilename = l.split()[1:]
```

spec_reader.py, lines 109–110

```
109   def __commenting__(self, l):
110     self.comments = self.comments + " ".join(l.split()[1:])
```

spec_reader.py, lines 112–113

```
112   def __initialepoch__(self, l):
113     self.Epoch0 = int(l.split()[1])
```

spec_reader.py, lines 115–116

```
115   def __marccdpath__(self, l):
116     self.M = l.split()[1:]
```

spec_reader.py, lines 118–119

```
118   def __limampx4path__(self, l):
119     self.limampx4path = os.path.join(*l.split()[1:][0].split(os.path.sep))  #l.split()[1:][0]
```

spec_reader.py, lines 121–129

```
121   def __detcalib__(self, l):
122     detcalib = l.split(' ')[1].split(',')
123 #    cen_pix_x=352.585,cen_pix_y=139.262,pixperdeg=315.152,det_distance_CC=0.993,det_distance_COM=0.992,timestamp
        ↪ =2017-11-10T11:54:52.621448
124     self.detcalib_cen_pix_x = np.float(detcalib[0].split('=')[1])
125     self.detcalib_cen_pix_y = np.float(detcalib[1].split('=')[1])
126     self.detcalib_pixperdeg = np.float(detcalib[2].split('=')[1])
127     self.detcalib_det_distance_CC = np.float(detcalib[3].split('=')[1])
128     self.detcalib_det_distance_COM = np.float(detcalib[4].split('=')[1])
129     self.detcalib_timestamp = detcalib[5].split('=')[1]
```

spec_reader.py, lines 131–132

```
131   def __special__(self, l):
132     self.__param__()[l[2:].split(' ')[0]](l)
```

spec_reader.py, lines 134–136

```
134   def __xiafilenaming__(self, l):
135     self.xianame = l.split()[1:][0]
136     self.xiaroi = dict()
```

spec_reader.py, lines 138–139

```
138   def __xiacalibrating__(self, l):
139     self.xiacalib = l.split()[1:][0]
```

spec_reader.py, lines 141–142

```
141   def __xiaroidefining__(self, l):
142     self.xiaroi[l.split()[1]] = [int(l.split()[2]),int(l.split()[3]),int(l.split()[4]),int(l.split()[5]),int(l.split()
        ↪ [6])]
```

9.2.3 `Scan`

```
Scan(spec_file, scan_numbers, verbose=False)
```

Reads one scan, or several concatenated, from a `SpecFile` (or a file name).

`spec_file`
: a `SpecFile`, or a path (then indexed first).

`scan_numbers`
: one scan number, or a sequence of them.

`verbose`
: print the lines that are read or not processed.

Method The first scan's header is parsed with the handlers; data lines are read until a blank line or a `#` line other than `#C`. Each further scan is appended if it is "similar": same scan type, same first argument (the scanned motor) and, for commands with more than five arguments, the same fourth argument (for an `ascan`, the number of intervals); otherwise a message is printed and the scan is skipped. Every column becomes an attribute named after its counter label, `motors` maps the motor names to their start positions (using the `#o` names when motor names contain spaces), and with an `Epoch` column the start and end times, duration and time per point are set. A mismatch between `#N` and the number of labels is reported.

spec_reader.py, lines 199–258

```
class Scan(SpecFile):
  """
  Simple class to read extract scans from SPEC files. All the parameters of the scan and the data are read
  and stored as attributes.

  Definition:
  -----------
  Scan(spec_file, scan_numbers, verbose = False)
   > spec_file as SpecFile object or string (in this case a SpecFile object will be instanced)
   > scan_numbers as integer or as tuple/list/array of integer

  Attributes:
  -----------
  <countername>...data in counter <countername> (see counters for description)
  number.........scan number of the first scan in the list
  scan_numbers....list of all the scans included
  type...........scan type
  args...........scan arguments (motor start stop npoints counting)
  date...........scan start datestamp
  ct.............scan counting time per point
  Qo.............H K L position at start of scan
  M..............MarCCD image file path
  N..............number of counters
  counters........list of counters
  motors.........dictionary of motors and their initial position
  comments........all comments
  tstart, tend....starting and finishing time
  duration........duration in s
  time_per_point..duration per point

  Examples:
  --------
  # read the specfile
  In : sf = SpecFile('./lineup0.dat')

  # read the scan
  In : scan = Scan(sf, 265)

  # read a series of scans
  In : scan = Scan(sf, (265,266,270))
  In : scan = Scan(sf, arange(265,270))

  # learn about the motors
  In : scan.motors
  Out:
  {'Chi': -144.8913,
   'Phi': 176.9778,
   ...
   'xpr3z': -8.8}

  # extract the info on a particular motor
  In : scan.motors['tth']
  Out: 35.042

  # plot two counters vs each others
  In : plot(scan.th,scan.det/scan.IC1)




  """
```

spec_reader.py, lines 262–380

```
  def __init__(self, spec_file, scan_numbers, verbose = False):
    if type(spec_file) == str:
      spec_file = SpecFile(spec_file)
    self.file = spec_file.file
    try :
      len_scan_number = len(scan_numbers)  # it is a list of scan
    except TypeError:
      scan_numbers = [scan_numbers,]  # it is a simple scan, we make a len 0 list
    scan_number = scan_numbers[0] #for all intents and purposes
    # recover the names of the motors from the header of the SPEC file (i.e. the SpecFile instance)
```

```
    self.__motorslabels__ = spec_file.__motorslabels__ # list of all motors in the experiment
    self.__motorslabelsnospace__ = spec_file.__motorslabelsnospace__ # list of all motors in the experiment
    # prepare the scan-specific attributes
    self.__positions__ = "" # list the values of all motors
    self.__config__ = "" # list the values of the UB matrix config
    self.__counters__ = "" # list of all counters in the experiment
    self.__countersnospace__ = "" # list of all counters in the experiment
    self.scan_numbers = scan_numbers
    self.comments = ""


    with open(self.file,'rU') as f:
      # read the first (and possibly only) scan in the list
      # now try to find the scan
      f.seek(spec_file.scan_dict[scan_number])
      l = f.readline()
      if verbose  : print("reading scan " + l)

      # read the scan header
      while l[0] == '#':
        self.__readSpecLine__(l, verbose=verbose)
        l = f.readline()

      # finally read the data (comments at the end are also read and added to the comment attribute)
      data = [list(map(float,l.split()))]
      l = f.readline()
      while l != '\n' and l != '':
        if l[0] == '#' and l[1] != 'C':
          break
        if l[0] == '#':
            self.__readSpecLine__(l, verbose=verbose)
        else :
            data.append(list(map(float,l.split())))
        l = f.readline()



      # now get the data for each scan in the list
      if len(scan_numbers) > 0:
        for scan_number in scan_numbers[1:]:
          # now try to find the scan
          f.seek(spec_file.scan_dict[scan_number])
          l = f.readline()
          if verbose  : print("reading scan " + l)

          # check that we actually concatenate similar scans !
          similar_scan  = (l.split()[2] == self.type)
          if len(self.args) > 1 :
            try:
              similar_scan = similar_scan * (l.split()[3]==self.args[0])
            except ValueError:
              similar_scan = False
          if len(self.args) > 5 :
            try:
              similar_scan = similar_scan * (l.split()[6]==self.args[3])
            except ValueError:
              similar_scan = False


          if similar_scan:
            # read pass the scan header
            while l[0] == '#':
              l = f.readline()

            # finally read the data (comments at the end are also read and added to the comment attribute)
            data.append(list(map(float,l.split())))
            l = f.readline()
            while l != '\n' and l != '':
              if l[0] == '#' and l[1] != 'C':
                break
              if l[0] == '#':
                self.__readSpecLine__(l, verbose=verbose)
              else :
                data.append(list(map(float,l.split())))
                l = f.readline()
```

```
347           else :
348             print("not all scans are the same type")
349
350       # set the data as attributes with the counter name
351       for i in range(len(self.counters)):
352         #print(i,range(len(self.counters)),self, self.counters[i], np.asarray(data).shape, np.asarray(data), data)
353         setattr(self, self.counters[i], np.asarray(data)[:,i])
354
355
356       # make the motors/positions dictionary
357       # usual case
358       if len(self.__motorslabels__.split()) == len(self.__positions__.split()) :
359         self.motors = dict(zip(self.__motorslabels__.split(), list(map(float,self.__positions__.split()))))
360       # when some motors names have spaces and there is a second line (small o) to describe them
361       elif len(self.__motorslabelsnospace__.split()) == len(self.__positions__.split()) :
362         self.motors = dict(zip(self.__motorslabelsnospace__.split(), list(map(float,self.__positions__.split()))))
363
364
365       #TEST : attribute-like dictionary
366       # removed due to conflicts when a motor was also a counter
367       #    for motor in self.motors:
368       #                setattr(self, motor, self.motors[motor])
369
370
371       # small sanity check, sometimes N is diffrent from the actual number of columns
372       # which is known to trouble GUIs like Newplot and PyMCA
373       if self.N != len(self.counters):
374         print("Watch out! There are %i counters in the scan but SPEC knows only N = %i !!"%(len(self.counters),self.N))
375
376       if hasattr(self,'Epoch'):
377         self.tstart = self.Epoch[0]
378         self.tend   = self.Epoch[-1]
379         self.duration = self.tend - self.tstart
380         self.time_per_point = self.duration/len(self.Epoch)
```

spec_reader.py, lines 381–382

```
381 #class Scan2D(Scan):
382   #def __init__(self, spec_file, scan_number_list, verbose = verbose):
```

9.3 `xray_tools`: wavelengths, Bragg angles and flipping ratios

Small tools from the SPEC era, including the analysis of X-ray phase retarder (XPR) scans: a diamond crystal rocked near a Bragg reflection turns the linear polarisation of the beam into circular polarisation of opposite helicities on either side of the reflection [45], and the *flipping ratio* of a signal measured at symmetric offsets is the helicity asymmetry (e.g. an XMCD signal).

Known issue

- `xprplot_th` computes `midpoint = n/2`, a float in Python 3, then slices with it: `TypeError`. Integer division (`//`) is needed.
- Only `matplotlib` is imported, but `matplotlib.pyplot` is used: it fails unless `pyplot` was imported elsewhere first.
- The module does not import on the current environment: `from numpy.linalg.linalg import *` raises `ModuleNotFoundError` with NumPy 2.5 (the submodule was made private in NumPy 2.0; `from numpy.linalg import norm` is the replacement). `from numpy import *` also puts the whole NumPy namespace into the module.
- Scans are read with `spec_reader`, which cannot open files on Python $\geq$ 3.11.

xray_tools.py, lines 1–8

```
1 # X-ray tools
2 # Sam Tardif
3 # samuel.tardif@gmail.com
4
5 from numpy import *
6 from numpy.linalg.linalg import *
```

```
7  import spec_reader as sr
8  import matplotlib
```

9.3.1 `E2L, L2E`

`E2L(energy), L2E(wavelength)`

$\lambda = hc/E$ and $E = hc/\lambda$, in metres and electronvolts, with $h = 4.135667516 \times 10^{-15}$ eV s and $c = 2.99792458 \times 10^{8}$ m/s (CODATA 2010, 46): $hc = 1.2398 \times 10^{-6}$ eV m.

xray_tools.py, lines 10–18

```
10 def E2L(energy):
11   """
12   Calculates the wavelength (in m) corresponding to the energy (in eV)
13   using h = 4.135667516E-15 eV.s
14   and   c = 2.99792458E8 m/s
15
16   Numerical values from P.J. Mohr, B.N. Taylor, and D.B. Newell (2011), "The 2010 CODATA Recommended Values of the
        ↪ Fundamental Physical Constants" (Web Version 6.0). This database was developed by J. Baker, M. Douma, and S.
        ↪ Kotochigova. Available: http://physics.nist.gov/constants [Thursday, 02-Jun-2011 21:00:12 EDT]. National
        ↪ Institute of Standards and Technology, Gaithersburg, MD 20899.
17   """
18   return 4.135667516E-15*2.99792458E8/energy
```

xray_tools.py, lines 21–29

```
21 def L2E(wavelength):
22   """
23   Calculates the energy (in eV) corresponding to the wavelength (in m)
24   using h = 4.135667516E-15 eV.s
25   and   c = 2.99792458E8 m/s
26
27   Numerical values from P.J. Mohr, B.N. Taylor, and D.B. Newell (2011), "The 2010 CODATA Recommended Values of the
        ↪ Fundamental Physical Constants" (Web Version 6.0). This database was developed by J. Baker, M. Douma, and S.
        ↪ Kotochigova. Available: http://physics.nist.gov/constants [Thursday, 02-Jun-2011 21:00:12 EDT]. National
        ↪ Institute of Standards and Technology, Gaithersburg, MD 20899.
28   """
29   return 4.135667516E-15*2.99792458E8/wavelength
```

9.3.2 `tth`

`tth(H, E, a)`

Bragg angle 2θ (degrees) of the reflection $\mathbf{H} = (h, k, l)$ of a cubic crystal of lattice parameter a (m) at energy E (eV): $d = a/|\mathbf{H}|$ and $2\theta = 2\arcsin(\lambda/2d)$ [47]. Example: (0,0,10) at 10 871 eV with $a = 10.015$ Å gives 69.42°.

xray_tools.py, lines 32–39

```
32 def tth(H,E,a):
33   """
34   Calculates the 2theta angle (degree) for the reflection vector H (r.l.u.) at energy E (eV) for a cubic crystal of
        ↪ lattice parameter a (m)
35   Example:
36   >>> tth((0,0,10),10871,10.015E-10)
37   >>> 69.416502777784046
38   """
39   return 2*arcsin(E2L(E)/(2*a/norm(H)))*180./pi
```

9.3.3 `uncertainty_fr`

`uncertainty_fr(p, n, po, no)`

Flipping ratio and its uncertainty from the detector counts p, n and monitor counts p_o, n_o for the two helicities:

$$\mathrm{FR} = \frac{P-N}{P+N}, \quad P = \frac{p}{p_o},\ N = \frac{n}{n_o}; \qquad \delta\mathrm{FR} = \sum_{x\in\{p,n,p_o,n_o\}} \left|\frac{\partial\,\mathrm{FR}}{\partial x}\right| \sqrt{x}, \tag{34}$$

with, for instance, $\partial\mathrm{FR}/\partial p = 2N/[p_o(P+N)^2]$. Returns `(dfr, fr)`.

Hypotheses and limits

Poisson statistics for all four counts. The contributions are added linearly, which is an upper bound; independent errors would add in quadrature [39], giving a value up to two times smaller.

xray_tools.py, lines 42–64

```
42 def uncertainty_fr(p,n,po,no):
43   """
44   Returns the flipping ratio (fr) quantity, defined as
45     (+)-(-)/(+)+(-)
46   as well as the Poisson-based uncertainty (dfr)
47
48   p  = detector intensity for (+) orientation
49   n  = detector intensity for (-) orientation
50   po = monitor intensity for (+) orientation
51   no = monitor intensity for (-) orientation
52   """
53
54   dp, dn, dpo, dno = sqrt(p), sqrt(n), sqrt(po), sqrt(no)
55   #dp, dn, dpo, dno = sqrt(p), sqrt(n), po.std(), no.std() # Non-Poisson statistics
56
57   dfr = no/po/n*2./(no/po*p/n+1.)**2*dp +\
58         n/p/no*2./(po/no*n/p+1.)**2*dpo +\
59         po/no/p*2./(po/no*n/p+1.)**2*dn +\
60             p/po/n*2./(no/po*p/n+1)**2*dno
61
62   fr = (p/po-n/no)/(p/po+n/no)
63
64   return dfr, fr
```

9.3.4 `xprplot_th`

```
xprplot_th(spec_file, scan_number, detector, monitor, centered=True, do_plot=True, save=False,
auto_close=False)
```

Processes an XPR angle scan measured symmetrically about the centre of the phase-retarder Bragg peak: the second half of the scan gives one helicity (p, p_o) and the first half, reversed, the other (n, n_o), at the same offsets; the centre point is skipped for an odd number of points. Computes the flipping ratio at each offset, optionally plots and saves it (PDF, EMF, PNG). Returns `(offsets, fr, dfr)`. Non-centred scans are not implemented.

xray_tools.py, lines 67–136

```
67 def xprplot_th(spec_file, scan_number, detector, monitor, centered=True, do_plot=True, save=False, auto_close=False):
68   """
69   Process the XPR theta scans by determining the center of the XPR Bragg peak and calculating the flipping ratio
       ↪ symmetrically.
70   Optional: plot the results
71
72   'centered'   : True if the scan was measured symmetrically to the XPR Bragg peak center  # False case not implemented
       ↪  yet
73   'do_plot'    : True if the results are to be plotted
74   'save'       : True if the plots are to be saved (unused if 'do_plot' = False). Default name is "<spec_file>_scan<
       ↪ scan_number>_<detector>"  (.png, .pdf and .Semf)
75   'auto_close' : True if the plot windows should be closed automatically, useful for batch processing (unused if '
       ↪ do_plot' = False)
76   """
77
```

```
78    s = sr.Scan(spec_file,scan_number)
79    xprth = getattr(s,s.counters[0]) # depending on the xpr used (1 or 2), x axis is xpr1th or xpr2th
80
81    if centered:
82      midpoint = xprth.shape[0]/2
83      if xprth.shape[0]%2 == 1 : # odd number of points in the scan
84        x  = xprth[midpoint+1:]-xprth[midpoint]
85        n  = getattr(s,detector)[:midpoint][::-1]
86        p  = getattr(s,detector)[midpoint+1:]
87        no = getattr(s,monitor)[:midpoint][::-1]
88        po = getattr(s,monitor)[midpoint+1:]
89        dfr, fr = uncertainty_fr(p,n,po,no)
90        #print 'midpoint is %f'%(midpoint)
91
92      else: # even number of points in the scan (center point not measured)
93        x  = xprth[midpoint:]-(xprth[0]+xprth[-1])/2.
94        n  = getattr(s,detector)[:midpoint][::-1]
95        p  = getattr(s,detector)[midpoint:]
96        no = getattr(s,monitor)[:midpoint][::-1]
97        po = getattr(s,monitor)[midpoint:]
98        dfr, fr = uncertainty_fr(p,n,po,no)
99
100
101   if do_plot:
102     initial_font_size = matplotlib.rcParams.get('font.size')
103     matplotlib.rcParams.update({'font.size': 16})
104     matplotlib.pyplot.figure()
105     matplotlib.pyplot.errorbar(x,fr,yerr=dfr,fmt='bo-')
106     matplotlib.pyplot.xlabel('xpr offset (degree)')
107     matplotlib.pyplot.ylabel('Flipping ratio (detector = %s, monitor = %s)'%(detector, monitor))
108     matplotlib.pyplot.title(spec_file + ' scan #%i'%(scan_number))
109     matplotlib.pyplot.hlines(0,x.min(),x.max())
110     matplotlib.pyplot.grid()
111     matplotlib.pyplot.show()
112     if save:
113       matplotlib.pyplot.savefig(spec_file + '_scan%i_%s.pdf'%(scan_number,detector))
114       matplotlib.pyplot.savefig(spec_file + '_scan%i_%s.emf'%(scan_number,detector))
115       matplotlib.pyplot.savefig(spec_file + '_scan%i_%s.png'%(scan_number,detector),dpi=150)
116     if auto_close:
117       matplotlib.pyplot.close()
118     matplotlib.rcParams.update({'font.size': initial_font_size})
119
120     # if do_plot:
121     # figure()
122     # errobar(x,fr,yerr=dfr,fmt='bo-')
123     # xlabel('xpr offset (degree)')
124     # ylabel('Flipping ratio (detector = %s, monitor = %s)'%(detector, monitor))
125     # title(spec_file + ' scan #%i'%(scan_number))
126     # hlines(0,x.min(),x.max())
127     # grid()
128     # show()
129     # if save:
130       # savefig(spec_file + '_scan%i_%s.pdf'%(scan_number,detector))
131       # savefig(spec_file + '_scan%i_%s.emf'%(scan_number,detector))
132       # savefig(spec_file + '_scan%i_%s.png'%(scan_number,detector),dpi=150)
133     # if auto_close:
134       # close()
135
136   return x,fr,dfr
```

9.3.5 `XprScan`, `XprScan.fr`

`XprScan`, `XprScan.fr(detector, monitor, ...)`

A `spec_reader.Scan` with a method that runs `xprplot_th` on itself and stores the offsets, flipping ratio and error as attributes (`xpr_offset`, `<detector>_fr`, `<detector>_dfr`).

xray_tools.py, lines 139–144

```
139 class XprScan(sr.Scan):
140   """
141   XprScan class to include the flipping ratio and the theoretical error.
```

```
142   Attributes are inherited from the `spec_reader.Scan` class.
143   Flipping ratio and error can calculated and added for any counter using the `XprScan.fr` method.
144   """
```

xray_tools.py, lines 146–155

```
146   def fr(self, detector, monitor, centered=True, do_plot=False, save=False, auto_close=False):
147     """
148     Method to calculate the flipping ratio and estimated error for any counter (detector), given a monitor.
149     Calculated flipping ratio and error are added as new attributes `XprScan.<detector>_fr` and `XprScan.<detector>_dfr
        ↪ ` respectively.
150     The xpr offset axis is also added as `XprScan.xpr_offset`
151     """
152     xpr_offset, fr,  dfr  = xprplot_th(self.file, self.number, detector, monitor, centered=centered, do_plot=do_plot,
        ↪ save=save, auto_close=auto_close)
153     setattr(self, 'xpr_offset', xpr_offset)
154     setattr(self, detector+'_fr', fr)
155     setattr(self, detector+'_dfr', dfr)
```

9.4 `LiveFourierTransform`: webcam Fourier transform

A demonstration script (2011, from a Matlab original by B. R. Pauw) that grabs webcam frames, computes the two-dimensional Fourier transform of a region of interest and displays, side by side, the real-space image, the logarithm of the intensity $|F|^2$, of the modulus $|F|$, and the phase of the transform, each scaled to 8 bits. It illustrates that a far-field (Fraunhofer) diffraction pattern is the squared modulus of the Fourier transform of the object [48], and that the phase—lost in a diffraction measurement—carries much of the information. The transform uses the FFT [30] with the zero frequency shifted to the centre; the optional "kill centre lines" replaces the central row and column (which hold the strong contributions of the image borders) by their neighbours to increase the dynamic range.

Known issue

Python 2 code (`print` statement, integer division `h/2` for indexing) using the OpenCV 1.x Python bindings (`opencv`, `highgui`) and the old `Image` module of PIL; `numpy` and its functions (`angle`, `log10`, `ones`, `uint8`, `transpose`, `append`) are used without being imported. It does not run on a current installation and is kept as a historical record. Because it is not valid Python 3, the listing shows the whole file.

LiveFourierTransform.py, lines 1–154

```
1  # Live Fourier Transform
2  # original Matlab code by Brian R. Pauw (http://www.lookingatnothing.com/)
3  # python code by Samuel K.Z. Tardif
4  # 2011-09-22
5
6  import pygame
7  import Image
8  from pygame.locals import *
9  import opencv
10 import sys
11 from opencv import highgui
12 import time
13
14
15
16 def get_image(camera):
17   """
18   Grabs a frame from the camera
19   and returns it as a PIL image
20   """
21   im = highgui.cvQueryFrame(camera)
22   im = opencv.cvGetMat(im)
23   return opencv.adaptors.Ipl2PIL(im)  #convert Ipl image to PIL image
24
25
```

```
26
27  def logfft(im_data,kill_center_lines=True):
28    """
29    Computes the FFT of the image, shift the origin at the center
30    and returns the log of the intensity (absolute value squared)
31    scaled on 8 bits
32    """
33    imft = numpy.fft.fftshift(numpy.fft.fft2(im_data))
34    ft_I = abs(imft)**2
35    ft_m = abs(imft)
36    ft_a = angle(imft)
37
38    if kill_center_lines:
39      # kill the center lines for higher dynamic range
40      # by copying the next row/column
41      h,w=ft_I.shape
42      ft_I[h/2,:]=ft_I[h/2+1,:]
43      ft_I[:,w/2]=ft_I[:,w/2+1]
44      ft_m[h/2,:]=ft_m[h/2+1,:]
45      ft_m[:,w/2]=ft_m[:,w/2+1]
46
47    #logscale
48    ft_I = log10(ft_I+ones(imft.shape))
49    ft_m = log10(ft_m+ones(imft.shape))
50
51    #over 8 bits
52    log10Imax = ft_I.max()
53    ft_I = (255*ft_I/log10Imax).astype(uint8)
54    ft_m = (255*ft_m/log10(sqrt(10**log10Imax))).astype(uint8)
55    ft_a = (255*(ft_a+pi)/(2*pi)).astype(uint8)
56
57    return ft_I, ft_m, ft_a
58
59
60
61  # frame per second
62  fps = 30.0
63
64
65  # define the ROI of the CCD to be used
66  # the CCD full size is 480x640 by default
67  roi_h,roi_w = 300,300
68  roi_h_offset,roi_w_offset = 90,120
69
70  # fft parameter
71  kill_center_lines=False
72
73  # start the stream
74  camera = highgui.cvCreateCameraCapture(0)
75
76
77  # start the display
78  pygame.init()
79  window = pygame.display.set_mode((roi_w*2,roi_h*2))
80  pygame.display.set_caption("TL: Real space | TR: Intensity | BL: Modulus | BR: Angle")
81  screen = pygame.display.get_surface()
82
83
84  i=0
85  starttime=time.time()
86  stop=False
87  while not(stop):
88
89    # loop control (stops on key down)
90    events = pygame.event.get()
91    for event in events:
92      if event.type == QUIT or event.type == KEYDOWN:
93        stop=True
94
95
96    # grabs the image from the camera and converts it to grayscale
97    im = get_image(camera)
98    im = im.convert(mode = 'L')
99
100   # get the data as an 8 bits array and do the FFT
```

```
  im_data=numpy.asarray(im.getdata()).reshape((480,640)).astype(uint8)
  h,w=im_data.shape
  im_data=im_data[roi_h_offset:roi_h_offset+roi_h,roi_w_offset:roi_w_offset+roi_w]
  ft_I, ft_m, ft_a=logfft(im_data,kill_center_lines)

  # stitch the real space image and the FT
  newshape = (im_data.shape[1]*2,im_data.shape[0]*1)
  im_top = transpose(append(transpose(im_data),transpose(ft_I)).reshape(newshape))
  im_bot = transpose(append(transpose(ft_m),transpose(ft_a)).reshape(newshape))
  newshape = (im_data.shape[0]*2,im_data.shape[1]*2)
  im_all = append(im_top, im_bot).reshape(newshape)
  im_mod=Image.fromarray(im_all)

  # KLUDGE : convert back to RGB for display
  im_mod=im_mod.convert('RGB')

  # insert modified image in stream
  pg_img = pygame.image.frombuffer(im_mod.tostring(), im_mod.size, im_mod.mode)
  screen.blit(pg_img, (0,0))
  pygame.display.flip()
  pygame.time.delay(int(1000 * 1.0/fps))
  i+=1
  print "current frame rate = %2.2f fps"%(i/(time.time()-starttime))



# cleaning
pygame.quit()
highgui.cvReleaseCapture(camera)




#  Webcam properties
#  #    name
#  0    CV_CAP_PROP_POS_MSEC
#  1    CV_CAP_PROP_POS_FRAMES
#  2    CV_CAP_PROP_POS_AVI_RATIO
#  3    CV_CAP_PROP_FRAME_WIDTH
#  4    CV_CAP_PROP_FRAME_HEIGHT
#  5    CV_CAP_PROP_FPS
#  6    CV_CAP_PROP_FOURCC
#  7    CV_CAP_PROP_FRAME_COUNT
#  8    CV_CAP_PROP_FORMAT
#  9    CV_CAP_PROP_MODE
# 10    CV_CAP_PROP_BRIGHTNESS
# 11    CV_CAP_PROP_CONTRAST
# 12    CV_CAP_PROP_SATURATION
# 13    CV_CAP_PROP_HUE
# 14    CV_CAP_PROP_GAIN
# 15    CV_CAP_PROP_EXPOSURE
# 16    CV_CAP_PROP_CONVERT_RGB
# 17    CV_CAP_PROP_WHITE_BALANCE
# 18    CV_CAP_PROP_RECTIFICATION
```

10 Notebooks

The notebooks chain the modules into complete analyses. They are not documented function by function: each is organised in numbered sections with explanations, and its settings are gathered in its first cells. Table 5 summarises what each one does and which functions it relies on.

Results files. The notebooks write their results to `processed/`: fit records (JSON), average spectra and maps (NPZ), and figures. The fit records hold the calibration, the detector parameters, every fitted parameter with its error and the group areas, so that later notebooks can start from them.

Table 3: Parameters of `XRFModel`, in order. Optional blocks appear only when the corresponding option is set.

Name	Present if	Meaning
`bkg_0` … `bkg_n`	`background_order` not `None`	background coefficients in $u \in [-1, 1]$
`A_<group>`	always	reference-line area of each group (counts summed over channels)
`noise_ev`, `fano`	`fit_resolution`	resolution law, Eq. (16)
`gamma_ev`	`fit_gamma`	Lorentzian half width
`cal_gain`, `cal_offset_ev`	`fit_calibration`	calibration (only the offset with `"offset"`)
`air_path_cm`	`fit_air_path`	sample–detector air path
`tail_fraction`, `tail_slope`, `step_fraction`	`fit_tails`	detector line shape, Eq. (23)
`A_elastic`, `elastic_sigma_ev`	`scatter`	elastic area and extra Gaussian width (monochromator bandwidth)
`A_compton`, `compton_angle_deg`, `compton_angle_spread_deg`	`scatter`	Compton area, mean angle and half range
`compton_sigma_ev`, `compton_tail_fraction`, `compton_tail_slope`	`scatter`	extra Gaussian width and low-energy tail of the Compton peak
`pileup_tau_us`	template and `fit_pileup`	pulse-pair resolving time (µs)

Table 4: IUPAC and Siegbahn names of the common lines (`SIEGBAHN`, used for labels).

IUPAC	Siegbahn	IUPAC	Siegbahn	IUPAC	Siegbahn	IUPAC	Siegbahn
KL3	Kα_1	KL2	Kα_2	KM3	Kβ_1	KM2	Kβ_3
KN3	Kβ_2	L3M5	Lα_1	L3M4	Lα_2	L2M4	Lβ_1
L3N5	Lβ_2	L1M3	Lβ_3	L1M2	Lβ_4	L2N4	Lγ_1
L1N3	Lγ_3	L3M1	Lℓ	L2M1	Lη	M5N7	Mα_1
M4N6	Mβ						

Table 5: Notebooks of the repository.

Notebook	Content and main functions
`fluo_analysis_clean.ipynb`	The white-beam workflow on a BLISS mesh scan: `BlissScan`; normalisation (`normalise_spectra`); surface region (the whole map for a film, a facet of a crystal: `SurfaceRegion`, `RegionSelector`); map-averaged spectrum and counting error; peak scouting (`identify_peaks`); BM32 beam from the scan (`bm32_from_h5`), detector and sample matrix (`MatrixModel`); internal energy calibration starting from the C10 calibration; pile-up and escape peaks; line groups and constrained fit (`element_line_groups`, `fit_average_spectrum`); sensitivity to the element list; resolution stability (`width_window`, `peak_width_profile`, `detect_steps`, `fit_segments`); per-pixel maps (`fit_map`) with quality report, individual pixel fits, quantitative ratios (`quantify_layer`, `atom_ratio`) and closure tests.
`fluo_analysis.ipynb`	Earlier version of the same workflow, kept with its outputs. It does not include the internal calibration step.
`xrf_energy_calibration.ipynb`	Energy calibration on the AXO RF C10 reference (`calibrate`, `plot_calibration`), with the beamline settings read from the scan, the fitted air path, line checks and intensity check, comparison with the calibration in use, and a JSON record that the analysis notebooks read as their starting calibration.
`xrf_mono_spectrum.ipynb`	A spectrum excited by a monochromatised Mo tube (the PyMca training spectrum `XRFSpectrum.mca`, not distributed with the repository; see the PyMca tutorial): calibration from two peaks (`calibration_from_peaks`), peak scouting, first-pass residuals revealing Tl and Pb, full fit up to the elastic peak with detector tails and the impulse-approximation Compton peak (`ScatterSource`, `fit_tails`), rejected candidates, and concentrations from the Co internal standard.

References

[1] M. A. Branch, T. F. Coleman, and Y. Li. A subspace, interior, and conjugate gradient method for large-scale bound-constrained minimization problems. *SIAM Journal on Scientific Computing*, 21:1–23, 1999. doi: 10.1137/S1064827595289108.

[2] P. Virtanen, R. Gommers, T. E. Oliphant, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.

[3] A. Brunetti, M. Sanchez del Rio, B. Golosio, A. Simionovici, and A. Somogyi. A library for X-ray–matter interaction cross sections for X-ray fluorescence applications. *Spectrochimica Acta Part B*, 59:1725–1731, 2004. doi: 10.1016/j.sab.2004.03.014.

[4] T. Schoonjans, A. Brunetti, B. Golosio, M. Sanchez del Rio, V. A. Solé, C. Ferrero, and L. Vincze. The xraylib library for X-ray–matter interactions. Recent developments. *Spectrochimica Acta Part B*, 66:776–784, 2011. doi: 10.1016/j.sab.2011.09.011.

[5] F. Biggs, L. B. Mendelsohn, and J. B. Mann. Hartree–Fock Compton profiles for the elements. *Atomic Data and Nuclear Data Tables*, 16:201–309, 1975. doi: 10.1016/0092-640X(75)90030-3.

[6] A. Collette. *Python and HDF5*. O'Reilly, Sebastopol, CA, 2013.

[7] M. Könnecke, F. A. Akeroyd, H. J. Bernstein, et al. The NeXus data format. *Journal of Applied Crystallography*, 48:301–305, 2015. doi: 10.1107/S1600576714027575.

[8] J. Schwinger. On the classical radiation of accelerated electrons. *Physical Review*, 75:1912–1925, 1949. doi: 10.1103/PhysRev.75.1912.

[9] K.-J. Kim. Characteristics of synchrotron radiation. In *Physics of Particle Accelerators*, volume 184 of *AIP Conference Proceedings*, pages 565–632. American Institute of Physics, 1989. doi: 10.1063/1.38046.

[10] L. G. Parratt. Surface studies of solids by total reflection of x-rays. *Physical Review*, 95: 359–369, 1954. doi: 10.1103/PhysRev.95.359.

[11] L. Névot and P. Croce. Caractérisation des surfaces par réflexion rasante de rayons X. Application à l'étude du polissage de quelques verres silicates. *Revue de Physique Appliquée*, 15: 761–779, 1980. doi: 10.1051/rphysap:01980001503076100.

[12] J. Als-Nielsen and D. McMorrow. *Elements of Modern X-ray Physics*. Wiley, Chichester, 2nd edition, 2011.

[13] A. Picard, R. S. Davis, M. Gläser, and K. Fujii. Revised formula for the density of moist air (CIPM-2007). *Metrologia*, 45:149–155, 2008. doi: 10.1088/0026-1394/45/2/004.

[14] P. Kirkpatrick and A. V. Baez. Formation of optical images by X-rays. *Journal of the Optical Society of America*, 38:766–774, 1948. doi: 10.1364/JOSA.38.000766.

[15] J. Sherman. The theoretical derivation of fluorescent X-ray intensities from mixtures. *Spectrochimica Acta*, 7:283–306, 1955. doi: 10.1016/0371-1951(55)80041-0.

[16] T. Shiraiwa and N. Fujino. Theoretical calculation of fluorescent X-ray intensities in fluorescent X-ray spectrochemical analysis. *Japanese Journal of Applied Physics*, 5:886–899, 1966. doi: 10.1143/JJAP.5.886.

[17] M. Mantler. X-ray fluorescence analysis of multiple-layer films. *Analytica Chimica Acta*, 188: 25–35, 1986. doi: 10.1016/S0003-2670(00)86025-0.

[18] D. K. G. de Boer. Calculation of X-ray fluorescence intensities from bulk and multilayer samples. *X-Ray Spectrometry*, 19:145–154, 1990. doi: 10.1002/xrs.1300190312.

[19] R. E. Van Grieken and A. A. Markowicz, editors. *Handbook of X-Ray Spectrometry*. Marcel Dekker, New York, 2nd edition, 2002.

[20] V. A. Solé, E. Papillon, M. Cotte, Ph. Walter, and J. Susini. A multiplatform code for the analysis of energy-dispersive X-ray fluorescence spectra. *Spectrochimica Acta Part B*, 62:63–68, 2007. doi: 10.1016/j.sab.2006.12.002.

[21] P. Lechner, S. Eckbauer, R. Hartmann, et al. Silicon drift detectors for high resolution room temperature X-ray spectroscopy. *Nuclear Instruments and Methods in Physics Research A*, 377:346–351, 1996. doi: 10.1016/0168-9002(96)00210-0.

[22] U. Fano. Ionization yield of radiations. II. The fluctuations of the number of ions. *Physical Review*, 72:26–29, 1947. doi: 10.1103/PhysRev.72.26.

[23] G. F. Knoll. *Radiation Detection and Measurement*. Wiley, Hoboken, NJ, 4th edition, 2010.

[24] F. Scholze, H. Rabus, and G. Ulm. Mean energy required to produce an electron-hole pair in silicon for photons of energies between 50 and 1500 eV. *Journal of Applied Physics*, 84: 2926–2939, 1998. doi: 10.1063/1.368398.

[25] S. J. B. Reed and N. G. Ware. Escape peaks and internal fluorescence in X-ray spectra recorded with lithium drifted silicon detectors. *Journal of Physics E: Scientific Instruments*, 5:582–584, 1972. doi: 10.1088/0022-3735/5/6/029.

[26] W. T. Elam, B. D. Ravel, and J. R. Sieber. A new atomic database for X-ray spectroscopic calculations. *Radiation Physics and Chemistry*, 63:121–128, 2002. doi: 10.1016/S0969-806X(01)00227-4.

[27] M. O. Krause. Atomic radiative and radiationless yields for K and L shells. *Journal of Physical and Chemical Reference Data*, 8:307–327, 1979. doi: 10.1063/1.555594.

[28] J. L. Campbell. Fluorescence yields and Coster–Kronig probabilities for the atomic L subshells. *Atomic Data and Nuclear Data Tables*, 85:291–315, 2003. doi: 10.1016/S0092-640X(03)00059-7.

[29] P. J. Statham. Pile-up rejection: limitations and corrections for residual errors in energy-dispersive spectrometers. *X-Ray Spectrometry*, 6:94–103, 1977. doi: 10.1002/xrs.1300060211.

[30] J. W. Cooley and J. W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19:297–301, 1965. doi: 10.1090/S0025-5718-1965-0178586-1.

[31] C. G. Ryan, E. Clayton, W. L. Griffin, S. H. Sie, and D. R. Cousens. SNIP, a statistics-sensitive background treatment for the quantitative analysis of PIXE spectra in geoscience applications. *Nuclear Instruments and Methods in Physics Research B*, 34:396–402, 1988. doi: 10.1016/0168-583X(88)90063-8.

[32] M. Morháč, J. Kliman, V. Matoušek, M. Veselský, and I. Turzo. Background elimination methods for multidimensional coincidence γ-ray spectra. *Nuclear Instruments and Methods in Physics Research A*, 401:113–132, 1997. doi: 10.1016/S0168-9002(97)01023-1.

[33] G. W. Phillips and K. W. Marlow. Automatic analysis of gamma-ray spectra from germanium detectors. *Nuclear Instruments and Methods*, 137:525–536, 1976. doi: 10.1016/0029-554X(76)90472-6.

[34] M. Van Gysel, P. Lemberge, and P. Van Espen. Implementation of a spectrum fitting procedure using a robust peak model. *X-Ray Spectrometry*, 32:434–441, 2003. doi: 10.1002/xrs.665.

[35] A. H. Compton. A quantum theory of the scattering of X-rays by light elements. *Physical Review*, 21:483–502, 1923. doi: 10.1103/PhysRev.21.483.

[36] P. Eisenberger and P. M. Platzman. Compton scattering of X rays from bound electrons. *Physical Review A*, 2:415–423, 1970. doi: 10.1103/PhysRevA.2.415.

[37] R. Ribberfors. Relationship of the relativistic Compton cross section to the momentum distribution of bound electron states. *Physical Review B*, 12:2067–2074, 1975. doi: 10.1103/PhysRevB.12.2067.

[38] O. Klein and Y. Nishina. Über die Streuung von Strahlung durch freie Elektronen nach der neuen relativistischen Quantendynamik von Dirac. *Zeitschrift für Physik*, 52:853–868, 1929. doi: 10.1007/BF01366453.

[39] P. R. Bevington and D. K. Robinson. *Data Reduction and Error Analysis for the Physical Sciences*. McGraw-Hill, New York, 3rd edition, 2003.

[40] S. Baker and R. D. Cousins. Clarification of the use of chi-square and likelihood functions in fits to histograms. *Nuclear Instruments and Methods in Physics Research*, 221:437–442, 1984. doi: 10.1016/0167-5087(84)90016-4.

[41] K. J. Mighell. Parameter estimation in astronomy with Poisson-distributed data. I. The χ^2_γ statistic. *The Astrophysical Journal*, 518:380–393, 1999. doi: 10.1086/307253.

[42] F. R. Hampel. The influence curve and its role in robust estimation. *Journal of the American Statistical Association*, 69:383–393, 1974. doi: 10.1080/01621459.1974.10482962.

[43] O. Ulrich, X. Biquard, P. Bleuet, O. Geaymond, P. Gergaud, J. S. Micha, O. Robach, and F. Rieutord. A new white beam x-ray microdiffraction setup on the BM32 beamline at the European Synchrotron Radiation Facility. *Review of Scientific Instruments*, 82:033908, 2011. doi: 10.1063/1.3555068.

[44] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x.

[45] K. Hirano, K. Izumi, T. Ishikawa, S. Annaka, and S. Kikuta. An X-ray phase plate using Bragg-case diffraction. *Japanese Journal of Applied Physics*, 30:L407–L410, 1991. doi: 10.1143/JJAP.30.L407.

[46] P. J. Mohr, B. N. Taylor, and D. B. Newell. CODATA recommended values of the fundamental physical constants: 2010. *Reviews of Modern Physics*, 84:1527–1605, 2012. doi: 10.1103/RevModPhys.84.1527.

[47] W. H. Bragg and W. L. Bragg. The reflection of X-rays by crystals. *Proceedings of the Royal Society of London A*, 88:428–438, 1913. doi: 10.1098/rspa.1913.0040.

[48] J. W. Goodman. *Introduction to Fourier Optics*. Roberts & Company, Englewood, CO, 3rd edition, 2005.